claude-windows / 693b418f-8bf1-49e5-b443-b86a83235a63

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\693b418f-8bf1-49e5-b443-b86a83235a63.jsonl`
- SHA-256: `683d9148d965da2295697fc8fe6e46a6ea8fe57c51caaf867ac307a763ecf20d`
- Source modified: `2026-07-05T16:45:50+00:00`
- Imported at: `2026-07-05T16:48:17+00:00`
- project: `C--Users-avidu-Projects-badminton-highlight-indexer`
- session_id: `693b418f-8bf1-49e5-b443-b86a83235a63`

Transcript

1. user (2026-07-05T09:19:03.183Z)

Ramp up first: git pull --ff-only both repos (badminton-highlight-indexer + khelsutra),
gh auth status (should be avidullu), then read session-handoff.md (? YOU ARE HERE) and the
alpha-cuj-testing-2026-07-05 memory ? that's the full context. The checkout is SHARED with
sibling sessions: branch from origin/master, stage explicit paths, re-verify HEAD before commit.

Context in one line: the browser CUJ now works end-to-end (upload/gs:// ? WASB ? 29 rallies ?
pickable) WHEN THE CONTROL PLANE SURVIVES, but the deployed CP is the 2026-07-04 build on a
fragile 4 GiB / RAM-/tmp / ephemeral-SQLite shape that OOMs on big uploads and loses results.
Consolidate this into one durable rebuild and land the latency/cost savers. Priority order:

1) SPEC then EXECUTE the control-plane rebuild ? the headline; it unblocks DoD #1.
   Write a tracked spec doc first (goals + DoD, per our project-doc convention) covering:
   ?8 GiB memory + disk-backed upload staging (NOT RAM /tmp) + durable/bucket-grounded state
   (#471, #485); bake in current master (#154 copy, #153 upload guard, #158 ingest UX, and
   #481 ONLY after it merges); raise the timeouts (SPA 10-min poll + CP 30-min max_wait).
   Then rebuild the worker image + redeploy the CP, PRESERVING the rev-00011 config pins
   (indexing.default_segmenter=wasb_hybrid, security.max_part_mb=24), and verify with a real
   browser CUJ end to end.

2) LATENCY + COST SAVERS (the big wins ? bundle into the rebuild or ship as engine PRs, each
   gated on a golden-set quality check; honest measured results only):
   - #483 downsample-for-detection: GPU-ffmpeg a low-res/fps proxy for DETECTION, cut the reel
     from the ORIGINAL. Validated this session (GPU proxy ~10x realtime) ? detection ~22 min ?
~5 min.
     Biggest lever.
   - #484 MD5 cache-reuse: re-processing the same video must re-hydrate from outputs/<md5>/,
     not a fresh GPU run (3 runs for one video today). Makes re-runs instant.
   - #482 progress/heartbeat + per-job runlog: push WASB progress to job.progress so the SPA
     shows "segmenting 40%?" not a silent 10-min spinner.

3) LAND #481 (feat/jobs-queue-and-dedup): atomic dedup + GET /api/jobs queue; the review's
   read-before-insert race is fixed (d27f2d7, CI green) ? get owner LGTM, squash-merge. It's a
   prerequisite for the SPA queue UI and the rebuild bake-in.

4) #485 durable resumable ?video=<md5> link ? folds into the rebuild's durable-state work
   (bucket = source of truth; re-hydrate on load). Same mechanism as #484.

5) SPA processing-queue UI ? foundation is rebased on post-#158 master
   (feat/processing-queue-and-dedup @ 23f3d15: listJobs + types). Build the queue view + resume
   ON TOP, AFTER #481 merges + the rebuild (nothing is live until the rebuild).

6) Wider-invite gates (owner/eng, NOT 4-tester blockers): least-privilege SA (review §5.5),
   retention automation (#301/#447) + policy processor-list / acceptable-use (#332), A10/A11
   decisions, kn/hi native-speaker review (#156).

Guardrails: every default/quality flip ships a Launch Report; product/strategy/licensing stay owner-gated. Start with item 1 ? spec the rebuild ? and confirm the spec before executing the deploy.

2. assistant (2026-07-05T09:19:12.092Z)

Ramping up: pulling both repos, checking gh auth, and reading the handoff + memory in parallel.

3. user (2026-07-05T09:19:14.150Z)

```
Exit code 128
fatal: not a git repository (or any of the parent directories): .git
```

4. user (2026-07-05T09:19:15.046Z)

```
Exit code 128
fatal: not a git repository (or any of the parent directories): .git
```

5. user (2026-07-05T09:19:16.865Z)

```
github.com
  ? Logged in to github.com account avidullu (keyring)
  - Active account: true
  - Git operations protocol: https
  - Token: gho_************************************
  - Token scopes: 'gist', 'read:org', 'repo', 'workflow'
```

6. user (2026-07-05T09:19:16.904Z)

```
File does not exist. Note: your current working directory is C:\Users\avidu\Projects\badminton-highlight-indexer.
```

7. assistant (2026-07-05T09:19:28.831Z)

Both directories report "not a git repository" ? let me look at what's actually on disk before going further.

8. user (2026-07-05T09:19:31.028Z)

```
total 6712
drwxr-xr-x 1 avidu 197609 0 Jul  5 12:55 .
drwxr-xr-x 1 avidu 197609 0 Jul  5 14:49 ..
drwxr-xr-x 1 avidu 197609 0 Jul  5 14:21 .tmp.drivedownload
drwxr-xr-x 1 avidu 197609 0 Jul  5 14:46 .tmp.driveupload
drwxr-xr-x 1 avidu 197609 0 Jun 22 09:41 _p3_scratch
drwxr-xr-x 1 avidu 197609 0 Jul  2 11:41 badminton-highlight-indexer
drwxr-xr-x 1 avidu 197609 0 Jul  5 14:20 khelsutra-guru
drwxr-xr-x 1 avidu 197609 0 Jun 29 14:02 muneem-ai
drwxr-xr-x 1 avidu 197609 0 Jul  4 14:49 Telegram
drwxr-xr-x 1 avidu 197609 0 Jun 16 09:55 Vairagi
---badminton---
total 13
drwxr-xr-x 1 avidu 197609   0 Jul  2 11:41 .
drwxr-xr-x 1 avidu 197609   0 Jul  5 12:55 ..
drwxr-xr-x 1 avidu 197609   0 Jul  2 22:08 .claude
-rw-r--r-- 1 avidu 197609 600 Jul  2 11:41 AGENTS.md
drwxr-xr-x 1 avidu 197609   0 Jun 30 03:00 output
---khelsutra---
total 184
drwxr-xr-x 1 avidu 197609    0 Jul  5 14:20 .
drwxr-xr-x 1 avidu 197609    0 Jul  5 12:55 ..
drwxr-xr-x 1 avidu 197609    0 Jun 23 09:53 Annotation Setup
-rw-r--r-- 1 avidu 197609  905 Jul  2 21:20 AUDIT_REPORT_khelsutra-guru_2026-07-02.md
drwxr-xr-x 1 avidu 197609    0 Jul  5 14:38 badminton-highlight-indexer
```

```
-rw-r--r-- 1 avidu 197609 1222 Jul  2 21:35 bash.exe.stackdump
drwxr-xr-x 1 avidu 197609    0 Jul  5 12:57 khelsutra
drwxr-xr-x 1 avidu 197609    0 Jul  5 13:45 khelsutra-screencast
drwxr-xr-x 1 avidu 197609    0 Jul  5 13:44 marketing
drwxr-xr-x 1 avidu 197609    0 Jul  5 13:43 pr-158-screenshots
drwxr-xr-x 1 avidu 197609    0 Jun 22 15:03 rally-annotator
drwxr-xr-x 1 avidu 197609    0 Jul  4 06:58 sports-data-collector
drwxr-xr-x 1 avidu 197609    0 Jul  2 19:46 sports-obsreport
drwxr-xr-x 1 avidu 197609    0 Jul  5 13:15 wt-157-before
drwxr-xr-x 1 avidu 197609    0 Jul  5 13:00 wt-157-ingest-ux
drwxr-xr-x 1 avidu 197609    0 Jul  4 15:41 wt-465fix
drwxr-xr-x 1 avidu 197609    0 Jul  4 17:11 wt-a1-runlog
drwxr-xr-x 1 avidu 197609    0 Jul  4 08:45 wt-cloudrun-dispatch
drwxr-xr-x 1 avidu 197609    0 Jul  4 15:41 wt-dockerfile-auth
drwxr-xr-x 1 avidu 197609    0 Jul  4 15:41 wt-drain-concurrency
drwxr-xr-x 1 avidu 197609    0 Jul  4 08:45 wt-input-jail
drwxr-xr-x 1 avidu 197609    0 Jul  5 13:20 wt-pr158-screenshot-assets
drwxr-xr-x 1 avidu 197609    0 Jul  5 14:25 wt-pr481-review
drwxr-xr-x 1 avidu 197609    0 Jul  4 15:41 wt-signed-download
drwxr-xr-x 1 avidu 197609    0 Jul  4 15:41 wt-upload-staging
drwxr-xr-x 1 avidu 197609    0 Jul  4 15:41 wt-yaml-gen
```

9. user (2026-07-05T09:19:33.031Z)

```
ls: cannot access '/c/Users/avidu/.claude/projects/C--Users-avidu-Projects-badminton-highlight-
indexer/memory/': No such file or directory
---khelsutra memory---
total 760
drwxr-xr-x 1 avidu 197609      0 Jul  5 14:47 .
drwxr-xr-x 1 avidu 197609      0 Jul  1 08:17 ..
-rw-r--r-- 1 avidu 197609   5555 Jul  5 14:41 alpha-cuj-testing-2026-07-05.md
-rw-r--r-- 1 avidu 197609   4432 Jul  5 13:24 alpha-serving-two-confirmed-bugs.md
-rw-r--r-- 1 avidu 197609   2969 Jul  5 12:02 alpha-tester-signin-otp-and-app-domain.md
-rw-r--r-- 1 avidu 197609   3408 Jun  7 10:41 audio-e1-findings.md
-rw-r--r-- 1 avidu 197609   3409 Jun 21 22:50 cache-hygiene-policy.md
-rw-r--r-- 1 avidu 197609   1793 May 30 20:30 candidate-segments-finetuning.md
-rw-r--r-- 1 avidu 197609   3683 Jun 22 22:48 cloudrun-gpu-serving-gotchas.md
-rw-r--r-- 1 avidu 197609   1766 Jun  7 14:54 code-map-artifact.md
-rw-r--r-- 1 avidu 197609   5900 May 30 20:31 collector-indexer-bridge.md
-rw-r--r-- 1 avidu 197609   3929 Jun 23 00:00 compute-stack-gcp-only.md
-rw-r--r-- 1 avidu 197609  30512 Jul  5 14:45 current-state.md
-rw-r--r-- 1 avidu 197609 307513 Jun 19 22:38 current-state-archive.md
-rw-r--r-- 1 avidu 197609   3204 Jun 14 16:20 decision-rigor-norm.md
-rw-r--r-- 1 avidu 197609   2034 Jun 25 23:18 decision-snapshot-regression.md
-rw-r--r-- 1 avidu 197609   1985 Jun 21 08:41 doc-status-register.md
-rw-r--r-- 1 avidu 197609   2914 Jun 16 19:58 eval-boundary-tolerance.md
-rw-r--r-- 1 avidu 197609   3516 Jun 17 17:21 eval-dual-set-regression.md
-rw-r--r-- 1 avidu 197609   1843 Jun 20 14:47 feedback-launch-quality-bar.md
-rw-r--r-- 1 avidu 197609   4921 Jul  4 00:05 feedback-tail-long-job-logs.md
-rw-r--r-- 1 avidu 197609   2594 Jun 14 12:01 file-placement-conventions.md
-rw-r--r-- 1 avidu 197609   2330 Jun 20 22:24 gcp-ops-agent-enable.md
-rw-r--r-- 1 avidu 197609   1936 Jun 20 22:50 gcp-vm-always-delete.md
-rw-r--r-- 1 avidu 197609   2441 Jun 22 15:08 gcs-data-source-of-truth.md
-rw-r--r-- 1 avidu 197609   2147 Jun 22 18:22 golden-corpus-grew-9-to-15.md
-rw-r--r-- 1 avidu 197609   3890 Jun  6 12:54 golden-set-vendoring.md
-rw-r--r-- 1 avidu 197609   3209 Jun 13 15:12 gotcha-long-runs-gpu-wsl.md
-rw-r--r-- 1 avidu 197609   2434 Jun 14 01:19 gotcha-shared-checkout-branch-collisions.md
-rw-r--r-- 1 avidu 197609  11217 Jun 20 20:58 khelsutra-domain-launch.md
-rw-r--r-- 1 avidu 197609   4234 Jun 25 16:49 khelsutra-site-droplet-parity.md
-rw-r--r-- 1 avidu 197609   2269 Jun 17 17:13 launch-report-convention.md
-rw-r--r-- 1 avidu 197609   1341 Jun 29 16:04 lesson-branch-rename-closes-open-pr.md
-rw-r--r-- 1 avidu 197609   3482 Jun 21 15:00 lesson-verify-engine-surface-before-scoping.md
-rw-r--r-- 1 avidu 197609   2743 Jun 20 23:43 machine-pooling-idea.md
-rw-r--r-- 1 avidu 197609  22943 Jul  5 14:28 MEMORY.md
-rw-r--r-- 1 avidu 197609   2173 May 30 20:31 monetization-plan.md
```

```
-rw-r--r-- 1 avidu 197609   2880 May 30 20:31 monetization-research-status.md
-rw-r--r-- 1 avidu 197609   1814 Jul  1 21:08 muneem-ai-workspace.md
-rw-r--r-- 1 avidu 197609   3164 Jul  1 22:29 muneem-grocery-replenishment.md
-rw-r--r-- 1 avidu 197609   7072 Jun 22 17:42 next-session-golden-fixtures.md
-rw-r--r-- 1 avidu 197609   8784 Jun 22 14:59 next-session-multishuttle-rally.md
-rw-r--r-- 1 avidu 197609   6519 Jun 22 09:42 next-session-packaging-app.md
-rw-r--r-- 1 avidu 197609   6277 Jun 29 16:04 next-session-rally-finding-b-promotion.md
-rw-r--r-- 1 avidu 197609  14398 Jun 22 01:04 next-session-ux-localization.md
-rw-r--r-- 1 avidu 197609  15294 Jun 22 03:57 packaging-app-plan.md
-rw-r--r-- 1 avidu 197609   2400 Jun 14 00:48 paper-coauthor-aim.md
-rw-r--r-- 1 avidu 197609  19076 Jul  1 22:08 project-studio-media-lifecycle.md
-rw-r--r-- 1 avidu 197609   1889 Jun 25 21:47 review-signoff-policy.md
-rw-r--r-- 1 avidu 197609   2183 Jun 25 21:18 review-trust-ci-over-rerun.md
-rw-r--r-- 1 avidu 197609   1582 Jun 12 18:07 roora-guidelines-drive-doc.md
-rw-r--r-- 1 avidu 197609  22696 Jul  5 14:47 session-handoff.md
-rw-r--r-- 1 avidu 197609   3364 Jun 12 19:53 sibling-internship-repo.md
-rw-r--r-- 1 avidu 197609   5023 Jul  1 20:46 storage-bucket-config-requirements.md
-rw-r--r-- 1 avidu 197609   2012 Jun 13 22:33 tool-trim-video.md
-rw-r--r-- 1 avidu 197609   4541 Jun  6 16:04 tracknet-wasb-wsl2-decision.md
-rw-r--r-- 1 avidu 197609  11137 Jun 22 23:55 ui-driven-cloud-serving-gap.md
-rw-r--r-- 1 avidu 197609   4959 Jul  4 02:35 wasb-l4-perf-cpu-feed-bound.md
-rw-r--r-- 1 avidu 197609   2261 Jun 14 12:02 weak-signals-retained.md
-rw-r--r-- 1 avidu 197609   1810 May 30 13:03 working-agreement-attribution.md
-rw-r--r-- 1 avidu 197609   3802 Jun 19 22:39 working-agreement-checkpoints.md
-rw-r--r-- 1 avidu 197609   1401 Jun 12 22:33 working-agreement-merging.md
-rw-r--r-- 1 avidu 197609   1986 Jun 16 09:45 working-agreement-tracked-project-docs.md
-rw-r--r-- 1 avidu 197609   4820 Jun 30 19:55 working-root-khelsutra-guru.md
```

10. user (2026-07-05T09:19:48.641Z)

```
Already up to date.
branch: master
0a7b3ff feat(jobs): processing-queue endpoint + idempotent /api/process dedup (#448) (#481)
64fa4a2 docs(a13): alpha launch review ? conditional-GO recommendation; GO/NO-GO to owner (#478)
122f174 docs(runlog): A1 post-merge e2e ? CUJ green on the final image; flip A1 DONE (#477)
3c97117 fix(storage): GCS keyless signing mints a cloud-platform-scoped token for signBlob
(closes #475) (#476)
195932f deploy(cloudrun): commit the control-plane service-manifest generator (closes #473)
(#474)
```

11. user (2026-07-05T09:19:51.224Z)

```
Already up to date.
branch: feat/processing-queue-and-dedup
23f3d15 feat(api): listJobs client + JobSummary/JobsResponse types for the processing queue
0e45e8f Polish ingest upload UX for cloud media (#158)
5ddb81b Fix hosted-alpha copy blockers: honest cloud disclosure in wizard, ingest, privacy
policy (#154) (#155)
383abd0 Guard oversized uploads client-side in the ingest SPA (#153)
b0f70a5 Show failed ingest verdict reasons
```

12. user (2026-07-05T09:19:51.255Z)

```
1       ---
2       name: alpha-cuj-testing-2026-07-05
3       description: "First real browser-CUJ test session: DoD#1 is BLOCKED on the current
build; fixes + rebuild plan."
4       metadata:
5         node_type: memory
6         type: project
7       ---
8
9       # Alpha CUJ test session ? 2026-07-05 (owner drove the first real browser CUJ)
10
```

Owner verdict: the "5-minute browser CUJ" (tracker DoD #1) is a BLOCKER on the currently-deployed build.
It is NOT a clean pass ? detection works, but the deployed control plane makes the round-trip fail. Rally
detection itself is solid (WASB found **29 rallies** in the test video). The failures are all deploy/infra.

What broke, in order (all diagnosed live via gcloud logs)
1. **Browser upload 413** ? 95 MB parts hit Cloud Run's ~32 MiB HTTP/1 body cap (#480). FIXED live by
   config (`security.max_part_mb=24`, CP rev 00011). Uploads that fit tmpfs now work.
2. **Cloud jobs ran `yolo_hybrid`, not the validated `wasb_hybrid`** (#479 ? config didn't pin the
   segmenter). FIXED live by config (`indexing.default_segmenter=wasb_hybrid`, rev 00011).
3. **Medium-picker UX trap** ? pasting a `gs://` SOURCE link only submits when medium = "This computer";
   with "Cloud bucket" selected the Process button silently no-ops (`mediumBlocked`). Cost the owner a
   failed submit. SPA-side (needs rebuild + a UX fix; commented on #158).
4. **Control-plane OOM ? restart ? lost ingestion (the killer).** A 2.68 GB upload staged into the CP's
   RAM-backed `/tmp` on a **4 GiB** instance ? OOM ? CP restarted 08:14 ? the ephemeral SQLite + the
   in-memory poll thread were wiped (#471). The worker later wrote 29 rallies to the bucket, but no CP
   was listening ? **rallies never ingested ? UI shows nothing, and reloading can't recover it.**
5. **Timeouts too short for a 1080p60 clip**: SPA client poll ceiling **10 min** (`POLL_CEILING_MS`)
   and CP `max_wait` **30 min** ? a 6-min video at 60 fps takes WASB ~22 min, so the SPA quits polling
   long before it's done (shows "taking longer than expected"; the job is still running).
6. **No MD5 cache / dedup**: the same video was detected **3×** today (`mw6wr`, `rsvxs`, `x5pt2`) ?
   re-processing re-runs the GPU (MD5 is hashed on the worker AFTER dispatch; DB wiped; no bucket-cache
   check). Wasted paid GPU. #481 (active-job dedup) + #484 (MD5 reuse) address it.

Reel RECOVERED manually
Since the UI couldn't surface the rallies, cut the **29-rally highlights reel (2:02, 121 MB)** locally from
`outputs/c2d45319?/intervals.csv` + the original video (ffmpeg NVENC on the dev box's RTX 2070 Super) and
delivered it to the owner. The deliverable exists even though the CUJ UI failed.

Downsample experiment (validates #483)
On the dev RTX 2070: GPU (NVDEC+`scale_cuda`+NVENC) downsampled the full 5.9-min 1080p60 clip to 1080p30 in
36 s (~10× realtime) vs CPU libx264 ~3.5 min (~5.7× faster). GPU uses dedicated NVDEC/NVENC hardware
blocks (parallel, not CUDA cores). ? Downsample-for-detection (#483) would cut detection from ~22 min to
~5 min while the reel is still cut from the original.

Issues filed this session (engine repo Khelsutra/badminton-highlight-indexer)
- **#479** yolo-not-wasb default (FIXED via config rev 00011) · **#480** upload 413 / tmpfs ceiling
- **#482** progress/heartbeat + per-job runlog (no blind waiting) · **#483** downsample-for-detection (cost)
- **#484** MD5 cache-reuse (skip re-detection) · **#485** durable bucket-grounded resumable `?video=<md5>` link
- **#481** (PR, CI-GREEN, ready for LGTM): tenant-scoped `GET /api/jobs` queue + idempotent `/api/process` dedup

```
50
51      ## THE consolidation next step: rebuild + redeploy the control plane
52      Config redeploy (rev 00011) can't touch the bundled SPA or the instance shape. A full
**image rebuild +
53      redeploy** is the high-leverage move ? it should:
54      - **?8 GiB memory AND stage uploads on a real disk, not RAM `/tmp`** (kills the
OOM/#480/#471 root cause).
55      - **Durable / bucket-grounded state** (#471, #485) ? DB as a cache, re-hydrate from the
bucket.
56      - Bake in current master: **#154** copy + **#153** upload guard + **#481** dedup+queue
(once merged) + the
57        medium-picker UX fix + longer SPA/CP timeouts.
58      - Then #483 downsample so a real-match upload is fast + reliable.
59
60      **RESOLUTION (end of session):** the CUJ **DID succeed in the UI** on the owner's gs://
re-run ? because
61      the gs:// path doesn't stage the big upload into the CP's RAM `/tmp`, the CP stayed up,
ingested
62      (`08:59:51 cloud-run: job=rally-worker done rc=0`), and the SPA showed all **29
rallies** pickable
63      (owner screenshot). So the product works end-to-end **when the CP survives**; the big-
browser-upload OOM
64      is the specific killer ? the rebuild. (The dup Job 2 `rsvxs` ran anyway = 3 GPU runs for
one video ?
65      exactly what #481/#484 prevent.) **#481** review found the dedup had a read-before-
insert **race** ? FIXED
66      (`d27f2d7`, pushed): `db.claim_or_create_process_job()` does an ATOMIC `INSERT ? WHERE
NOT EXISTS` +
67      `rowcount` (claim_due_job idiom) with a 12-thread concurrency regression; full suite
green (1671 passed,
68      85.60%); awaiting owner LGTM. **#158 MERGED** (master `0e45e8f`).
69
70      Held work: the SPA processing-queue **view** (consumes #481's `/api/jobs`) ? foundation
**rebased onto
71      post-#158 master + pushed** (`feat/processing-queue-and-dedup` @ `23f3d15`, typecheck
clean); build the
72      queue UI on top **after #481 merges + the CP rebuild** (nothing is live until the
rebuild anyway). Pairs
73      [[alpha-serving-two-confirmed-bugs]] [[alpha-tester-signin-otp-and-app-domain]].
74
```

13. user (2026-07-05T09:19:51.267Z)

```
1       ---
2       name: session-handoff
3       description: "Living handoff: current state, next steps, ramp-up kit, and key
decisions."
4       metadata:
5         node_type: memory
6         type: project
7       ---
8
9       # Session handoff ? khelsutra-guru
10
11      > Last refreshed: 2026-07-05 (khelsutra#157 ingest UX polish PR
[#158](https://github.com/avidullu/khelsutra/pull/158) MERGED on owner request -> master
`0e45e8f`; issue #157 closed. Engine #481 atomic dedup fix reviewed and LGTM posted on head
`d27f2d7`. Prior khelsutra#154 copy-blocker fix
[#155](https://github.com/avidullu/khelsutra/pull/155) MERGED -> master `5ddb81b`; #154 closed).
Prior: 2026-07-04 (end of the e2e session). **A1 IS DONE.** All 7 serving PRs merged
12      > (#465/#467/#468/#469/#470/#474 on owner LGTM + #476 on green CI), the full e2e PASSED
on the final
13      > image, edge-auth is restored, and the runlog + A1?DONE flip is merged (#477, master
`122f174`).
14      > See [[current-state]] for the blow-by-blow.
```

15
16   ## ? YOU ARE HERE ? #157 MERGED; A1 DONE + A13 review DELIVERED; ball is with the OWNER
17
18   **A1 is DONE and e2e-verified** (runlog
`docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_a1-e2e_2026-07-04.md`)
19   and **A13 is DELIVERED** (`docs/ALPHA_LAUNCH_REVIEW_2026-07-04.md`, merged #478 ? master
`64fa4a2`):
20   conditional-GO for the current 4-email allowlist (conditions below) / NO-GO for wider
invites until
21   A10/A11 + least-privilege SA + retention. Backed by a live battery (11 failure-mode
probes, 3-job
22   concurrency, **Gemini handoff verified on the hosted stack** ? 48?43 segments, real
gemini-flash-latest
23   calls; the secret-accessor grant was missing and is now in place; alpha restored to
WASB-only).
24   Live stack: CP rev **00011-dx6** (edge-auth ON, same image digest `31475f27?`, config
pins
25   `indexing.default_segmenter=wasb_hybrid` + `security.max_part_mb=24`), `maxScale=1`,
KEPT RUNNING.
26
27   **khelsutra#157 status:** [#158](https://github.com/avidullu/khelsutra/pull/158) was
marked ready and squash-merged on owner request to master `0e45e8f`; issue
[#157](https://github.com/avidullu/khelsutra/issues/157) is closed. It incorporated all three
follow-ups: (1) the issue-owner comment/screenshot by simplifying dense Studio copy,
neutralizing cloud-cost wording, shortening sample-query guidance, focusing the active upload
panel, and adding upload size/ETA; (2) the uploaded-image request by making file selection
explicit-preview-only until the user clicks **Start upload and find rallies**, and showing the
selected filename as a prominent title under "Process a video"; (3) the PR-review medium-picker
trap by keeping the cloud/host source-link form visible in every storage mode while making
bucket/Drive destination requirements gate only local file uploads. Review reply posted
([comment](https://github.com/avidullu/khelsutra/pull/158#issuecomment-4885474768)). Verified
visually with Playwright screenshots on desktop/mobile and locally with web typecheck + full
web/site tests + web coverage + web e2e CI; latest focused gate after `c94d485`: `npm test --
src/components/IngestPanel.test.tsx src/components/upload.test.tsx`, `npm run typecheck`, `git
diff --check`; GitHub checks green before merge (`site`, `web`, `layout`, Cloudflare Worker
build). Screenshot comments are attached to the PR: before/after
([comment](https://github.com/avidullu/khelsutra/pull/158#issuecomment-4885297269)) and
explicit-start desktop/mobile/uploading states
([comment](https://github.com/avidullu/khelsutra/pull/158#issuecomment-4885367767)); image host
branch = `assets-pr-158-screenshots`. Work was isolated in `C:\Users\avidu\Projects\khelsutra-
guru\wt-157-ingest-ux` because the shared checkout changed branch mid-session and has unrelated
uncommitted API edits; that worktree is now on clean `master` at `0e45e8f`.
28
29   **OWNER ACTIONS (the review §0/§1.1 spells them out):**
30   1. Record **GO/NO-GO** on the tracker DoD.
31   2. **5-minute browser CUJ (DoD #1) ? ATTEMPTED 2026-07-05 by the owner -> it's a BLOCKER
on the current build.** Full detail [[alpha-cuj-testing-2026-07-05]]. WASB detection works (29
rallies) but the deploy infra fails the round-trip: a 2.68 GB upload OOM'd the 4 GiB /
RAM-`/tmp` CP -> restart -> ephemeral SQLite wiped (#471) -> the detected rallies (in the
bucket) never ingested -> UI empty. Config rev 00011 already fixed the segmenter (#479) + upload
part-size (#480); still broken in the live build: #471 (durable state), the 10-min SPA / 30-min
CP timeouts, no MD5 dedup (#484), and the medium-picker trap until merged master `0e45e8f` is
deployed. **The fix is the CP IMAGE REBUILD (grab #2 below).** Reel was recovered manually +
delivered; new issues #482/#483/#484/#485 filed; engine #481 (dedup + `GET /api/jobs` queue) ?
atomic concurrent-submit fix (`d27f2d7`, 12-thread regression), all CI/local gates green,
MERGED on owner LGTM ? master `0a7b3ff` (2026-07-05 09:15Z; `backend/api/jobs.py` now on
master, branch deleted). The CUJ DID succeed in the UI on a gs:// re-run (29 rallies pickable)
once the CP survived ? the big-upload OOM is the only killer.
32   3. ~~khelsutra#154 copy blockers~~ **DONE 2026-07-05** ?
[khelsutra#155](https://github.com/avidullu/khelsutra/pull/155) merged (owner LGTM, squash ?
master `5ddb81b`), issue #154 auto-closed. All 3 blockers fixed (wizard conditional copy ·
privacy `video.li1` reword · SPA upload-moment disclosure), 6 locales, CI fully green.
**Residual (NOT a blocker): kn/hi machine drafts still need native-speaker review before wider
invites.**
33   4. Decide **A10** (ship-gate/claims) + **A11** (WASB-C3 defer w/ guardrail).

5. **#472** cert check before ~2026-09-30 (healthy as of 2026-07-04).

Next session grabs (priority order ? #481 is MERGED): (1) **SPEC then EXECUTE the control-plane image rebuild** ? the headline that unblocks DoD #1: write a tracked spec doc (?8 GiB memory + disk-backed upload staging NOT RAM `/tmp` + durable/bucket-grounded state [#471, #485]; bake in master #154/#153/#158/#481; raise the 10-min SPA + 30-min CP timeouts), then rebuild the worker image + redeploy PRESERVING the rev-00011 config pins (`default_segmenter=wasb_hybrid`, `max_part_mb=24`), verify a real browser CUJ. (2) **LATENCY/COST SAVERS** (bundle into the rebuild or ship as engine PRs, each golden-gated): **#483 downsample-for-detection** (GPU proxy validated ~10× realtime ? detection ~22 min ? ~5 min, biggest lever), **#484 MD5 cache-reuse** (re-hydrate from `outputs/<md5>/`, no re-run), **#482 progress/heartbeat + per-job runlog**. (3) **SPA processing-queue UI** on top of merged-#158 master ? foundation rebased + pushed (`feat/processing-queue-and-dedup` @ `23f3d15`: listJobs+types), consumes #481's `/api/jobs`; build after the rebuild. (4) owner browser-CUJ + GO/NO-GO on A13. (5) least-privilege SA (review §5.5). (6) retention (#301/#447) + processor-list / acceptable-use (#332). (7) kn/hi native review (#156). (8) khelsutra UX backlog (#329) · rally finding-B (P2b).

? NEXT-SESSION ICEBREAKER (paste this): _Ramp up: `git pull --ff-only` both repos, `gh auth status`, read this handoff + [[alpha-cuj-testing-2026-07-05]]. Shared checkout ? branch from origin/master, stage explicit paths. The browser CUJ works end-to-end (upload/gs:// ? WASB ? 29 rallies ? pickable) WHEN the CP survives, but the deployed CP is fragile (4 GiB/RAM-/tmp/ephemeral, OOMs on big uploads). Priority: (1) SPEC then execute the CP rebuild (?8 GiB + disk staging + durable/bucket-grounded state; bake #154/#153/#158/#481; raise timeouts; keep the rev-00011 config pins) ? spec first, confirm, then deploy; (2) latency savers #483 (downsample, ~10× realtime ? biggest win) / #484 (MD5 reuse) / #482 (progress logs), golden-gated; (3) SPA queue UI on `feat/processing-queue-and-dedup` @ 23f3d15; (4) owner CUJ + GO/NO-GO; (5) least-priv SA; (6) retention #301/#447 + #332; (7) kn/hi native review #156. Honest measured results only; Launch Report on every default flip._

FIRST STEPS for the next session: `git -C C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
pull --ff-only`; `git -C C:\Users\avidu\Projects\khelsutra-guru\khelsutra status --short --branch` before touching the shared checkout (it had sibling WIP on `feat/processing-queue-and-dedup`); `gh auth status` (should be `avidullu`); read [[current-state]]'s top checkpoint.

(A) PR-REVIEW RESOLUTION ? remaining open PRs

Repo `Khelsutra/badminton-highlight-indexer`. Reviewer = **avidullu** (the owner). "LGTM" arrives as a
PR comment or a formal approval. Merge convention = **squash** (`gh pr merge <n> --squash --delete-branch`).

PR	branch	fixes	head	review state
~~#465~~	~~`fix/allowed-video-root-uri`~~	hosted-mode input jail vs gs://	?	**MERGED 2026-07-04 09:03Z** (owner LGTM + 11/11 CI)
#467	`fix/upload-gcs-staging`	closes #461 (upload?gs:// staging)	c9bf37e	round-2 blocker (input_root-scheme gate) ADDRESSED; awaiting LGTM
#468	`fix/drain-concurrency`	closes #466 (parallel videos)	21ee141	round-2 blocker (claim_due_job lost-race false-empty) ADDRESSED; awaiting LGTM
#469	`fix/highlights-signed-url`	closes #463 (32 MiB download ? signed URL)	ce450e5	round-2 blocker (configured signed_url_ttl ignored) ADDRESSED; awaiting LGTM
#470	`fix/dockerfile-auth-extra`	base image M9-ready (auth extra)	c63d122	NEW this session; 11/11 green (first push tripped the Dockerfile structural guard ? guard updated). Once merged, the e2e SKIPS the derived-image step
#474	`feat/cloudrun-service-manifest`	closes #473 (gen_service_yaml.py + tests + README §3a)	9de18d5	NEW this session; encodes maxScale=1 (#471 opt-1), no CLOUD_RUN_JOB, explicit --edge-auth. Once merged, the e2e regenerates the YAML from the repo

Fix worktrees (uncommitted-free, branches pushed): `wt-drain-concurrency`, `wt-upload-staging`,
`wt-signed-download`, `wt-dockerfile-auth`, `wt-yaml-gen` under

`C:\Users\avidu\Projects\khelsutra-guru\`;
`wt-465fix` is now merged ? safe to `git worktree remove` all of them after their PRs merge.

What was already discussed + resolved (so you're not surprised by the threads):
- **#465** ? reviewer flagged the offline suite was red (`tests/test_api_endpoints.py::test_process_hosted_mode_rejects_nonlocal_uri`
  asserts hosted mode rejects gs://). Fixed in **cd4bc73**: `validate_input_path` now takes `allowed_uri_root`
  (wired from `storage.input_root`); a bucket URI is allowed ONLY under input_root, arbitrary buckets
  rejected (tenant isolation). The contract test passes unchanged. Replied on the PR.
- **#467** ? reviewer: `storage.put(local_src, staged_uri)` dropped the storage config. Fixed: pass
  `config=storage_cfg.model_dump()`. Test asserts the config reaches `put()`. Replied.
- **#468** ? reviewer withdrew LGTM (round 1): over-broad concurrency. **Re-designed + pushed (8534077):**
  `orchestration._LOCAL_COMPUTE_LANE` (BoundedSemaphore(1)) around the in-process segmenter + compile
  stitch (remote polls don't hold it ? overlap; local/compile serialize); broadened
  `_cloud_dispatch_possible()` so `compute='cloud'` overrides scale too. I also OFFERED (in a reply) to
  make the lane *park* via `JobWaiting`/WAIT_AND_RESUME instead of *block* (starvation-free) ? do that
  only if the reviewer asks.
  - **?? OPEN ? a 2nd blocker the reviewer found (round 2), NOT yet fixed. Address this FIRST on #468:**
    `claim_due_job()` (`backend/infrastructure/database_jobs.py:357-380`) returns `None` when a worker
    LOSES the CAS race (`rowcount != 1`); `_drain_due_jobs()` (`backend/api/job_worker.py:273-276`)
    treats `None` as "queue empty" and EXITS. So with `max_workers>1`, workers racing the same first
    queued row ? losers return `None` ? their drains stop even though other jobs are queued (reviewer
    reproduced: 8 queued + 8 claimers ? only 2/8 claimed ? dispatch capacity wasted).
    FIX: make `claim_due_job()` RETRY on a lost race ? re-SELECT + re-attempt until it either claims
    a row OR a fresh SELECT finds no runnable rows (or return a distinct lost-race sentinel so
    `_drain_due_jobs` LOOPS instead of exiting). **ADD a regression test:** N queued jobs + N concurrent
    claimers/drains prove N DISTINCT jobs are claimed with no false-empty. Then FULL suite + lint + push
    + reply on #468.
- **#469** ? reviewer: mirror `put` + `exists` + `signed_url` dropped the storage config. Fixed: added
  `orchestration._storage_settings(cfg)` threaded into all three; extracted `signed_reel_url` to keep
  `backend/main.py` under its line budget (P23/P24). Tests assert config reaches exists/signed_url. Replied.

Watch/merge protocol (repeat until all 4 merged):
1. Poll `gh pr view <n> --json state,reviewDecision,comments,reviews` for each open PR (every few min).
2. On **new change-request/comment** ? address it (fix + FULL local suite + push + reply). ? Run the
   whole `pytest tests/` suite before pushing, not just the changed file ? that's how #465's CI-red
   slipped through the first time (`python -m pytest tests/ -q -p no:cacheprovider`, ~2 min; + `ruff check`
   + `mypy backend/...`).
3. On **LGTM/approval** for a PR ? verify `gh pr checks <n>` all pass + `mergeable=MERGEABLE`, then
   `gh pr merge <n> --squash --delete-branch`.

```
99      4. **Conflicts after a merge (expected):** #467 & #469 both touch
`backend/orchestration.py`; #468 & #469
100        both touch `backend/main.py`. When one merges, re-check the others' `mergeable`; if
`CONFLICTING`,
101        in a **worktree off origin/master** merge master into the branch, resolve, re-run the
FULL suite +
102        lint, push. (Git-safety: `git worktree add <path> <branch>`; stage explicit paths;
re-verify branch
103        before push; never `git add -A`. Shared checkout ? sibling sessions may move master.)
104
105     ---
106
107     ## (B) FULL E2E DEPLOY TEST ? run AFTER all four PRs are merged
108
109     Goal: prove the merged fixes work on a real deployed build ? the >32 MiB signed-URL
download (#463) and
110     the browser-upload?gs:// staging (#461) ? then restore the live edge-auth state and flip
A1 ? DONE.
111
112     1. **Rebuild the worker image from merged master** (has #461/#463/#465/#468 backend code
+ #462's
113        `.[storage-gcs,cloud-run]`): `gcloud builds submit
--config=deploy/cloudrun/cloudbuild.yaml` (heavy
114        CUDA build, ~15 min; region asia-southeast1). ? The base Dockerfile installs
`.[storage-gcs,cloud-run]`
115        but NOT the `auth` extra (PyJWT[crypto]) that M9 edge-auth needs ? the *deployed*
image gets PyJWT via
116        a derived overlay. **Recommended follow-up:** add `auth` to the base Dockerfile so a
rebuild is
117        M9-ready (a small 6th PR). For the e2e itself, edge-auth is OFF, so PyJWT isn't
required.
118     2. **Build the derived control-plane image** = base + PyJWT: context `Dockerfile` =
`FROM
119        asia-southeast1-docker.pkg.dev/gen-lang-client-0306026826/rally/rally-worker:latest`
+
120        `RUN python3 -m pip install --no-cache-dir "google-cloud-run>=0.10,<1"
"PyJWT[crypto]>=2.8"`; tag
121        `.../rally/rally-control-plane:latest`. `gcloud builds submit <ctx> --tag <img>
--region asia-southeast1`.
122        (No more paths.py/backend overlays needed once the fixes are IN the base image ? the
prior session
123        overlaid #465's OLD blanket paths.py into the live image; the rebuild replaces it
with the corrected code.)
124     3. **Deploy CP with edge-auth OFF** so you can drive it with an identity token (no CF
login needed for
125        the test). Regenerate the service YAML (config below) with `EDGE_AUTH` unset and
126        `gcloud run services replace <yaml> --region asia-southeast1`. Run gcloud from
**PowerShell**.
127     4. **Run the CUJ + verify** (private service ? auth every curl with
128        `-H "Authorization: Bearer $(gcloud auth print-identity-token)"`; URL below):
129        - `POST /api/process {"video_path":"gs://khelsutra-rally-
serving/videos/MahadevpuraSingles.MP4","segmenter":"wasb_hybrid","compute":"cloud"}`
130          ? poll `/api/jobs/{id}` to `done` (L4 run ~13 min) ? note segment ids.
131        - `POST /api/compile
{"video_id":<id>,"segment_ids":[1..N],"output_filename":"e2e.mp4"}` ? poll to done.
132        - **#463 check:** `GET /api/highlights/{video_id}/e2e.mp4` should now
**307-redirect** to a
133          `storage.googleapis.com` signed URL, and `curl -L` should download the FULL >32 MiB
reel (no edge-500).
134          Also confirm the reel is mirrored at `gs://khelsutra-rally-
serving/highlights/<video_id>/e2e.mp4`.
135        - **#461 check:** upload a small mp4 via `/api/upload/init`?`/part/0`?`/complete`,
then `POST /api/process`
136          with the returned LOCAL path + `compute:cloud` ? it should STAGE to gs:// and
dispatch (not hard-fail
```

with "cloud-serving needs a cloud input"). (Signing was already verified: `gcloud storage sign-url`
impersonating the compute SA produced a working URL ? the SA has self-`serviceAccountTokenCreator`.)
5. **Restore edge-auth ON** (regenerate the YAML with `EDGE_AUTH=1` + redeploy) so api.khelsutra.guru's
live posture is back. Verify `/api/process` sans CF JWT ? 401 and `https://api.khelsutra.guru/api/health`
? 302 to the CF login.
6. **Write** `docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_a1-e2e_<date>.md` (a NEW
file ? don't clobber the existing #457 or #464 runlogs), update [[current-state]] + this handoff, and
flip A1 ? DONE in `docs/ALPHA_LAUNCH_READINESS.md` §7 (all DoD met: endpoint + health + edge-auth +
upload?process?jobs?compile?download).

Config for the CP service (base64 into a startup wrapper via `RALLY_APP_HOME=/tmp/apphome/config.json`):
```json
{"deployment":{"profile":"cloud-serving","compute_target":"cloud_run"},"indexing":{"skip_ai_handoff":true},
"storage":{"backend":"gcs","output_root":"gs://khelsutra-rally-serving","input_backend":"gcs",
"input_root":"gs://khelsutra-rally-serving/videos"},
"security":{"edge_auth_enabled":<EDGE_AUTH>,"cf_team_domain":"https://khelsutra.cloudflareaccess.com",
"cf_access_aud":"c980da67ed888fff935fcd913ae1eb4da63f9d65805d6e1c5ca76f92f517082b",
"allowed_video_root":"/tmp/apphome/output"}}
```
CP deploy (per `deploy/cloudrun/README.md` §3a): `--command /bin/bash --args` a `-c` wrapper that
`printf %s '<b64>' | base64 -d > /tmp/apphome/config.json && export RALLY_APP_HOME=/tmp/apphome && exec
python3 -m backend.main --server --host 0.0.0.0 --port 8080`; flags `--no-cpu-throttling --min-instances 1
--cpu 2 --memory 4Gi --port 8080 --timeout 300`; env `DEPLOYMENT_PROFILE=cloud-serving,
CLOUD_RUN_REGION=asia-southeast1,GOOGLE_CLOUD_PROJECT=gen-lang-client-0306026826,RALLY_ALLOWED_HOSTS=*`.
? `CLOUD_RUN_JOB` is a RESERVED env name ? DON'T set it (code default `rally-worker` is correct).
Deploy via a `gcloud run services replace <service.yaml>` MANIFEST (shell metachars in the wrapper break
through the cmd-shim otherwise). The prior session's generator `gen_service_yaml.py` + the derived-image
`Dockerfile` lived in the prior scratchpad ? rebuild them from the config + notes above.

Deployed state (live now ? owner said KEEP running)
- Project `gen-lang-client-0306026826`, region `asia-southeast1`. Compute SA
  `492532266377-compute@developer.gserviceaccount.com` (has `roles/editor` + self-`serviceAccountTokenCreator`).
- **L4 Job** `rally-worker` (nvidia-l4, 8vCPU/32Gi, FUSE `khelsutra-rally-serving`?/gcs, `WASB_WEIGHTS=/gcs/models/?`).
- **CP Service** `rally-control-plane` rev **00004, edge-auth ON**, PRIVATE (IAM + allUsers-invoker so CF can
  reach it; app still requires the CF JWT). URL `https://rally-control-plane-2bosgeukpa-as.a.run.app`.
  Deployed image = `?/rally/rally-control-plane:latest` = rally-worker + google-cloud-run + PyJWT + the
  OLD blanket #465 paths.py overlay (the corrected input_root-scoped code is in PR #465 ? the e2e
  rebuild replaces it; harmless for the closed 4-email alpha meanwhile).
- **M9 LIVE:** `api.khelsutra.guru` ? CNAME `ghs.googlehosted.com` (PROXIED) via Cloud Run domain-mapping,
  fronted by the "KhelSutra Studio (alpha)" CF Access app (team

```
`https://khelsutra.cloudflareaccess.com`,
177       aud `c980da67?082b`, 4-email allowlist:
avi.dullu@/sheendullu13@/suneetadullu@/avidullu.svp@).
178       **Add a tester** = add their email to that app's "alpha allowlist" policy (CF
dashboard or API; token at
179       `G:\My Drive\cloudflare.token.txt`, has DNS + Access-Apps edit but NOT Rulesets/Config
nor
180       `/access/organizations` ? team domain was recovered via the public certs probe). CF
acct
181       `213c12cf60de9041dd8a3521aa206472`, zone `65ef7770a6f25697843b53a093504184`. ? Google-
managed cert
182       auto-renew (~90d) can stall behind CF proxy ? switch to a CF Origin CA cert if it
does.
183
184     ## Gotchas (learned the hard way)
185     - Run **gcloud from PowerShell**, not git-bash (MSYS mangles `/gcs`, `/tmp`); quote
comma-dict flags
186       (`--add-volume "name=?,type=?"`) or the cmd-shim splits them.
187     - The CP uses per-instance **ephemeral SQLite + /tmp** ? DB/reel don't survive a
redeploy or a second
188       instance. That's WHY #463 mirrors reels to gs:// + #461 restages uploads. maxScale is
3.
189     - **Pre-merge:** run the WHOLE suite locally, not per-file (the #465 CI-red lesson).
190
191     ## Key decisions (don't relitigate)
192     - WASB-only alpha (`skip_ai_handoff=true`); config.json wins over the cloud-serving
preset (loader.py:194).
193     - Reuse the worker image as the CP (+ google-cloud-run + PyJWT); IAM-private + CF Access
is the tester auth.
194     - #468 concurrency: serialize local/compile via `_LOCAL_COMPUTE_LANE`, parallelize only
remote polls; the
195       *park*-instead-of-*block* refinement is offered to the reviewer (do it only if they
ask).
196     - A1 stays NOT-DONE until the 4 PRs merge + the e2e passes; then flip it.
197
198     ## Ramp-up kit
199     - The 4 PRs (links via `gh pr list`), their review threads, and CI. Issues:
#461/#463/#466 (fixed by the PRs).
200     - Runlog already in repo: `docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-
serving_a1-cuj_2026-07-04.md`
201       (PR #464, merged) ? the full A1 deploy story.
202     - Code: `deploy/cloudrun/README.md` §2/§3/§3a + `Dockerfile` · `backend/main.py`
(routes; `/api/highlights`)
203       · `backend/orchestration.py`
(`_maybe_dispatch_remote`,`_execute_compile`,`signed_reel_url`,`_LOCAL_COMPUTE_LANE`)
204       · `backend/api/job_worker.py`
(`configure_drain`,`_resolve_drain_workers`,`_cloud_dispatch_possible`)
205       · `backend/storage/` (`put`/`exists`/`signed_url`; gcs.py keyless signBlob) ·
`backend/config/models.py`
206       (`deployment.max_concurrent_remote_jobs`) · `backend/utils/paths.py`
(`validate_input_path` allowed_uri_root).
207     - Repo checkout: `C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer`
(master @ 4847232 at handoff).
208
```

14. user (2026-07-05T09:20:28.177Z)

```
1       ---
2       name: current-state
3       description: "Live micro-checkpoint state for khelsutra-guru."
4       metadata:
5         node_type: memory
6         type: project
7         originSessionId: b13673cc-f087-43b1-9527-35dc0b67b301
8       ---
```

9
10   # Current state - khelsutra-guru
11
12   **Updated:** 2026-07-05
13
14   ## Live Checkpoint
15
16   **2026-07-05 (OWNER-DRIVEN FIRST BROWSER CUJ -> DoD #1 is a BLOCKER on the current build; detection works, deploy infra fails).** Full detail in [[alpha-cuj-testing-2026-07-05]]. The owner ran the real upload->process->reel flow; it does NOT pass cleanly. **WASB detection is solid (29 rallies on the test clip); every failure is deploy/infra:** (1) upload 413 (95 MB parts vs Cloud Run ~32 MiB cap, #480) -> **config-fixed** rev **00011** (`max_part_mb=24`); (2) cloud jobs ran `yolo_hybrid` not `wasb_hybrid` (#479, config didn't pin) -> **config-fixed** rev 00011 (`default_segmenter=wasb_hybrid`); (3) **the killer:** a 2.68 GB upload OOM'd the **4 GiB / RAM-`/tmp`** CP -> **restart 08:14 -> ephemeral SQLite + poll thread wiped (#471) -> the 29 detected rallies (in the bucket) never ingested -> UI empty, unrecoverable by reload;** (4) medium-picker source-vs-destination trap exists in the live build but is fixed in merged master `0e45e8f` (#158) and awaits deployment; (5) SPA poll ceiling 10 min + CP `max_wait` 30 min too short for 1080p60 (~22 min WASB); (6) no MD5 cache/dedup -> same video detected **3x** (wasted GPU). **Reel RECOVERED manually** (29-rally 2:02 reel cut locally from `intervals.csv`+original via NVENC, delivered). **Downsample experiment** (dev RTX 2070): GPU 1080p60->1080p30 full clip in **36 s (~10x realtime)** vs CPU ~3.5 min ? validates #483. **Filed:** #482 (progress/heartbeat + per-job runlog), #483 (downsample-for-detection), #484 (MD5 reuse), #485 (durable bucket-grounded `?video=<md5>` link). **#481** (engine dedup + `GET /api/jobs` queue) ? atomic concurrent-submit fix pushed on `d27f2d7`; local gates green (`1671 passed, 3 skipped`, `ruff`, `mypy backend training`, diff-check), GitHub CI fully green, and LGTM posted ([comment](https://github.com/Khelsutra/badminton-highlight-indexer/pull/481#issuecomment-4885506095)). SPA queue foundation pushed on `feat/processing-queue-and-dedup` (UI held to land on top of #158). **#158 merged** (master `0e45e8f`, issue #157 closed). **? THE next step = merge #481 if desired, then full CP image rebuild** (>=8 GiB memory + disk-backed upload staging + durable/bucket-grounded state + bake #154/#153/#158/#481 + longer timeouts + #483 downsample) ? turns "works if you know the tricks" into a reliable tester CUJ.
17
18   ---
19
20   **2026-07-05 (khelsutra#157 ingest UX polish MERGED -> PR [#158](https://github.com/avidullu/khelsutra/pull/158) squashed to master `0e45e8f`; issue [#157](https://github.com/avidullu/khelsutra/issues/157) closed).** Picked up the issue-owner comment with the extra X-marked screenshot: simplified the dense Studio copy, made cloud-cost language more neutral, shortened sample-query guidance, and reduced the active upload view to a focused progress panel. Then incorporated the uploaded-image request: choosing a file now only selects/previews it, the selected filename is shown as a prominent title under "Process a video," and the upload/indexing run starts only when the user clicks **Start upload and find rallies**. Finally resolved the PR-review medium-picker trap: the cloud/host link form stays visible in every storage mode, pasted `gs://`/host-path links submit directly to `/api/process`, and bucket/Drive destination requirements gate only local file uploads. `useUpload` exposes sent/total bytes + ETA; `IngestPanel` hides setup controls during upload/indexing, adds size/ETA meta, uses medium-specific labels, and clears selected-file state on cancel/remove/URI submit; `App` no longer shows the fixed Build reel footer for browse-only sample rallies, removing the no-video overlap/crowding seen in visual QA. Root + `site/public` locale catalogs synced for all 6 languages. **Visual QA:** downloaded/viewed the original issue screenshot + new comment screenshot, generated/viewed Playwright desktop/mobile setup + upload screenshots; confirmed the X-marked wording was reduced, upload progress fits, filename wraps cleanly on mobile, and no upload/process network call fires until explicit start. **PR screenshots:** added before/after comment ([issuecomment-4885297269](https://github.com/avidullu/khelsutra/pull/158#issuecomment-4885297269)) plus explicit-start selected/uploading/mobile comment ([issuecomment-4885367767](https://github.com/avidullu/khelsutra/pull/158#issuecomment-4885367767)); PNGs are hosted on branch `assets-pr-158-screenshots` under `docs/pr-screenshots/158/` with `[skip ci]`. **Local gates on clean worktree `wt-157-ingest-ux`:** `npm ci` web/site, web `typecheck`, web `test:coverage`, full web `npm test`, site `npm test` (95 passed), web `e2e:ci` (build + contract smoke + 5 Playwright CUJs passed); latest focused post-review gate: `npm test -- src/components/IngestPanel.test.tsx src/components/upload.test.tsx`, `npm run typecheck`, `git diff --check`; GitHub checks green before merge (`site`, `web`, `layout`, Cloudflare Worker build). Existing Vite >500k chunk warning remains. **Safety note:** the shared checkout branch switched under the session to `feat/processing-queue-and-dedup` with unrelated uncommitted API

edits, so #157 was isolated into `C:\Users\avidu\Projects\khelsutra-guru\wt-157-ingest-ux`; that worktree is now clean on `master` at `0e45e8f`.

2026-07-05 (khelsutra#154 copy blockers FIXED + MERGED ? PR [#155](https://github.com/avidullu/khelsutra/pull/155) squashed to master `5ddb81b`; issue #154 auto-closed COMPLETED). Owner posted a thorough written LGTM (reviewed vs A13 blocker reqs, reproduced local+CI gates incl. all 5 Playwright CUJs; recorded as a comment because GitHub blocks a self-approve on the author's own account) ? squash-merged `--delete-branch`, local branch cleaned. Picked up the A13 GO-condition #154. All 3 blockers fixed in one PR off `origin/master` (383abd0): (1) **SetupWizard** compute step renders new `wizard.compute.hosted` (honest cloud copy) when `capabilities.deployment.cloud_available` is true ? the false "your videos never leave it" line stays only for a local appliance; (2) **privacy policy** `site.privacy.video.li1` reworded "our own infrastructure" ? "rented cloud infrastructure ? Google Cloud (Singapore region)" (+ the `privacy.html` static default the byte-drift test pins); (3) **Studio SPA** ingest panel gains an upload-moment disclosure (`ingest.disclosure.*`, review §4.4 copy) + a `https://khelsutra.guru/privacy` link, shown only when `cloudAvailable`. All **6 locales** (en authoritative; es/da/id MT; **kn/hi machine drafts ? native-speaker review still owed** before wider invites); `site/public/locales` regenerated via `copy:locales` (drift green). **Local gates:** web typecheck clean, 299 vitest (incl. 4 new tests), coverage 94.8%/88.6% > 92/87/84/92 floor, vite build OK; site 95 node --test (i18n parity + HTML-default drift). **CI:** site + Cloudflare-worker jobs PASS; web + layout Playwright jobs running at handoff (e2e unaffected ? mock engine reports `cloud_available:false` so the disclosure/picker don't render in CUJs). MERGEABLE, no sibling commits. **DONE: owner LGTM + squash-merged (master `5ddb81b`).** ? Still owed (NOT #154 blockers ? wider-invite gates per review §4.5): retention over-promise #301, naming Google Cloud in the policy processor-list, acceptable-use line #332, and the **kn/hi native-speaker review** of the new #154 strings before wider invites (DEFERRED ? filed [khelsutra#156](https://github.com/avidullu/khelsutra/issues/156)). **Tracker status: all remaining A-rows (A10/A11/A12/A13) are OWNER-GATED ? no ungated engineering A-row left.** Next on the critical path = owner browser-CUJ (DoD #1, review §1.1) + GO/NO-GO (A13). Top ungated ENGINEERING item (a wider-invite gate, not a 4-tester blocker) = least-privilege SA (review §5.5); then retention automation (#301/#447).

2026-07-04 (STAGE END: A13 DELIVERED ? ? review MERGED #478, master `64fa4a2`; GO/NO-GO with the OWNER). `docs/ALPHA_LAUNCH_REVIEW_2026-07-04.md` (475 lines, evidence-cited, scrubbed): **conditional GO for the current 4-email allowlist** (conditions: khelsutra#154 copy fixes + the 5-min owner browser-CUJ checklist §1.1 ? DoD #1 has only ever been curl-driven) · **NO-GO for wider invites** until A10/A11 close + least-privilege SA + retention. DoD scorecard 5/10 done. Tracker A13 ? GATED (review delivered). All 6 session tasks complete; worktrees/branches cleaned; stack at rev 00010 (auth ON, WASB-only, digest `31475f27?`), KEPT RUNNING. **? OWNER ACTIONS:** (1) GO/NO-GO on the review, (2) the §1.1 browser CUJ, (3) khelsutra#154 copy fixes, (4) A10/A11 decisions, (5) #472 cert check ~09-30. Next session grabs: khelsutra#154 fix PR (ungated engineering) · khelsutra UX backlog (#329) · #471 durable state · finding-B promotion.

2026-07-04 (NEXT STAGE: A13 IN FLIGHT ? live battery DONE incl. Gemini leg; review doc being assembled). Owner authorized the $300 GCloud+Gemini budget for thorough testing. **Battery (all

green, posture RESTORED to rev 00010 = same digest, auth ON, allUsers back, gemini_key_present=false):**
11-probe failure-mode battery (jail 400s incl. separator-boundary · async NotFound failed-job shape ·
4xx shapes for bad segmenter/jobs/compile/highlights/traversal · upload 400 ext / **413 at the 4096MB
cap** / 409 out-of-order) · **concurrency 3/3** (distinct 720p clips, all claimed+done, no false-empty)
· **GEMINI LEG VERIFIED on the hosted stack**: secret accessor grant was MISSING (found+granted ? the
documented mount path had never been exercised), job+CP mounted `gemini-api-key`, skip_ai_handoff=false
booted clean, MahadevpuraSingles ? **43 AI-confirmed segments (vs 48 WASB-only)** with real
gemini-flash-latest calls for all 48 windows in worker logs (exec rally-worker-khx58). Alpha posture
stays WASB-only. Also: SPA served at CP root (single-origin ? no CORS in tester path) · cert healthy
today (#472 watch stands) · better auth-off practice = drop allUsers during windows (public 403; NB
takes api.khelsutra.guru down for testers meanwhile). **A13 research workflow done** (4 evidence-cited
drafts + critique; biggest gaps = browser-CUJ never verified (DoD #1 unchecked ? owner action) +
khelsutra copy blockers ? filed khelsutra#154 ("never leaves your device" false on hosted; "our own
infrastructure" on GCP; SPA has zero consent copy)). ? NOW: assembly agent writing
`docs/ALPHA_LAUNCH_REVIEW_2026-07-04.md` + tracker A13 row (branch docs/a13-launch-review, wt-a13) ?
gates ? PR ? merge on green. GO/NO-GO stays with the owner.

**2026-07-04 (SESSION END: A1 DONE ? ? e2e PASSED on the final image; edge-auth restored; runlog+flip
MERGED #477; master `122f174`).** Post-#476 rebuild (`31475f27?`) ? job re-pinned + CP rev 00006
(? `services replace` w/ unchanged manifest = NO-OP, used `services update --image` to re-resolve
:latest) ? redeploy WIPED the ephemeral DB (the #471 bite: /api/highlights 404s at main.py:1052 before
the bucket check) ? re-ran the FULL CUJ on the final image: gs:// process **48 rallies** ? · upload
staged+dispatched+done ? (#461 on both builds) · 2 process jobs drained concurrently ? (#468) · compile
+ gs:// mirror ? · **#463 VERIFIED: 307 ? storage.googleapis.com v4 signed URL (signed AS the compute
SA, keyless signBlob) ? full 174,046,228 bytes in 8.1s single-GET, ffprobe-valid 1080p29.97 244.29s** ·
edge-auth ON restored (rev **00007**, same digest): health 200 / process-sans-JWT 401 /
api.khelsutra.guru ? CF-Access login 302 (right aud). Runlog + ALPHA_LAUNCH_READINESS §7 **A1?DONE**
merged as **#477** on 11/11 CI (+ full local gates). All session worktrees/branches cleaned (sibling
6e9d0958 worktrees untouched; prior session's stale non-git `wt-a1-runlog` dir removed). Live stack KEPT
RUNNING per owner. ? NEXT per [[session-handoff]]: A13 launch review · khelsutra UX backlog (#329 async
compile) · #471 durable state / #472 cert watch / generator `--revision-suffix`.

**2026-07-04 (e2e MID-RUN: CUJ mostly GREEN; #463 signed-URL found BROKEN on real creds

? #475 filed ?
85   #476 FIXED+MERGED ? rebuilding).** e2e scoreboard: ? image rebuilt from merged master +
`rally-worker`
86   job re-pinned + CP **rev 00005** deployed via the committed generator (auth-off; base
image direct ?
87   proves #470, no derived overlay) · ? health/capabilities · ? gs:// CUJ **48 rallies**
(matches prior
88   run) · ? compile e2e.mp4 + **mirrored to gs:// (165.98 MiB)** · ? **#461 VERIFIED**:
720p upload
89   staged to `videos/<id>/?` (byte-exact) + dispatched + done (0 segments = correct for
synthetic; NB
90   1st attempt failed validation ? clip must be ?1280x720) · ? #468 live-proven (upload job
dispatched
91   WHILE compile stitched) · ??? **#463 redirect 500'd on real creds**: keyless signBlob
rode the
92   storage client's devstorage-scoped token (iamcredentials rejects; bare-except hid it;
fell back to
93   local serve ? 32MiB cap). Root-caused live (SA self-tokenCreator ?, iamcredentials API
?, scope ?)
94   ? **issue #475** ? **PR #476 MERGED** (`3c97117`: `_signblob_identity()` mints a fresh
cloud-platform
95   -scoped ADC cred; loud logging; 1659? 85.46% ruff/mypy clean; merged on green CI per
96   working-agreement-merging). ? NOW: rebuild image (bg) ? jobs update + services replace ?
97   re-verify #463 (307 + full >32MiB download) ? edge-auth ON restore ? api.khelsutra.guru
posture ?
98   runlog PR ? flip A1 DONE.
99
100   ---
101
102
103   **2026-07-04 (ALL 5 PRs MERGED on owner LGTM; e2e STARTED).** Owner posted written
logic-LGTMs on
104   #467/#468/#469/#470/#474 (~09:47Z) + authorized verify-and-merge in-session. Merged
SEQUENTIALLY, each
105   after a local **merge-result** verification in a temp worktree (full suite w/ coverage +
ruff + mypy;
106   per the pre-LGTM gate): baseline master 85.40% ?#467 (1637?, 85.38%, ?0.02pp = new
defensive except
107   branch, noise) ? #468 (1645?, **85.44%**) ? #469 (1652?, 85.39%) ? #470 (1652? + md-
links) ? #474
108   (1655?, 85.39%). No textual/semantic conflicts anywhere (the feared
orchestration.py/main.py overlaps
109   auto-merged clean + suite-proven). Master `195932f`; issues #461/#463/#466/#473 AUTO-
CLOSED; all 6
110   session worktrees + local branches removed (sibling session's 6e9d0958 worktrees left
alone).
111   **e2e (B) IN PROGRESS:** GCP pre-flight green (project/auth/job/service/bucket+test-
video); worker-image
112   rebuild from merged master SUBMITTED to Cloud Build (bg); service YAMLs generated from
the NEW committed
113   `deploy/cloudrun/gen_service_yaml.py` (auth-off + auth-on, config verified == handoff
spec). Next: build
114   green ? `gcloud run jobs update rally-worker --image :latest` (re-pin digest) ?
`services replace`
115   auth-off ? CUJ (#463 signed-URL >32MiB + #461 upload staging) ? auth-on restore ? runlog
? flip A1.
116
117   **2026-07-04 (PICKUP SESSION) ? #465 MERGED on owner LGTM; all 3 open review blockers
FIXED+PUSHED+REPLIED;
118   #470 opened; issues #471?473 filed.** Continuing session picked up the handoff: (1)
#465 merged (squash,
119   owner LGTM 08:55Z, 11/11 CI green ? master moved to the merge commit; #467/#468/#469
stayed MERGEABLE, no
120   conflicts). (2) **All three open blockers fixed** via a 3-agent workflow + adversarial
verify (each verifier

15. user (2026-07-05T09:20:28.640Z)

```
1       ---
2       name: working-agreement-tracked-project-docs
3       description: "Substantial design/project work gets a TRACKED PROJECT DOC (template + §7
progress tracker + definition-of-done); freeze the plan, track it across sessions, mark it done,
archive."
4       metadata:
5         node_type: memory
6         type: feedback
7         originSessionId: 068560d3-1ca2-4387-981a-d2d6bca55414
8       ---
9
10      Substantial design/project work (anything bigger than a single small PR ? multi-PR
efforts, research-backed designs, security/privacy/architecture plans) is captured as a
**tracked project doc** in `docs/`, following `docs/PROJECT_DOC_TEMPLATE.md`.
11
12      Shape: status header (`DRAFT ? IN PROGRESS ? DONE ? archived`) · decisions-locked table
· (optional) research/legal **foundation** · design + **reuse map** · (optional) **threat
model** · **honest-limits** · **§7 progress tracker** (one ?/?/? row per independently-shippable
PR ? the source of truth) · open owner/counsel questions · **definition-of-done** (acceptance
checklist) · references · changelog. Tag each claim `[verified]` / `[researched]` / `[design]`.
13
14      **Why:** the owner explicitly loved this framing (2026-06-16, the Golden Regression
Fixtures plan) and asked to make it a norm "throughout" ? it lets a plan be iterated on, its
progress tracked across sessions, and be marked COMPLETE against clear deliverables (then
archived). Generalizes the emergent pattern already in `docs/RALLY_QUALITY_RESEARCH.md` §7.
15
16      **How to apply:** for a substantial ask, write/extend the doc FIRST (draft, owner-
gated), index it in `docs/README.md` + a memory `current-state` pointer (+ `docs/NEXT_STEPS.md`
once work starts); each deliverable = ONE small PR per [[gotcha-shared-checkout-branch-
collisions]]; flip Status and archive when the definition-of-done is met. Codified in repo
`CLAUDE.md` (workflow conventions) + `docs/PROJECT_DOC_TEMPLATE.md` (PR #186). Ties [[working-
agreement-checkpoints]] + [[decision-rigor-norm]] + [[paper-coauthor-aim]].
17
```

16. user (2026-07-05T09:20:30.787Z)

feat/processing-queue-and-dedup...origin/feat/processing-queue-and-dedup

17. user (2026-07-05T09:20:31.551Z)

master...origin/master

```
ABLATION_LAYER.md
ALPHA_LAUNCH_READINESS.md
ALPHA_LAUNCH_REVIEW_2026-07-04.md
ANALYZERS_AND_RECONCILERS.md
archives
BACKLOG.md
CLOUD_GPU_DRYRUN_GUIDE.md
CODE_MAP.md
COMMERCIALIZATION.md
COMPUTE_DECOUPLED_SERVING
CONVERT_GCS_VIDEOS_TO_HIGHLIGHTS.md
CREATING_REELS.md
DATA_IN_GCS.md
data_pipeline
DECISION_SNAPSHOT_REGRESSION.md
DESKTOP_APP_PLAN.md
DOC_STATUS.md
EVALUATION.md
EXECUTION_POLICY.md
EXPERIMENT_HARNESS.md
GOLDEN_REGRESSION_FIXTURES.md
```

```
HOSTING_PLAN.md
HOW_RALLY_DETECTION_WORKS.md
I18N_PLAN.md
KHELSUTRA_PRODUCT_OVERVIEW.md
mockups
MULTI_SIGNAL_FUSION_PLAN.md
NEXT_STEPS.md
NIGHTLY_ENGINE_REGRESSION.md
OWNED_MODEL_IMPLEMENTATION_PLAN.md
PACKAGING_PLAN.md
PER_VIDEO_WORKSPACE.md
PERSONALIZATION_PLAN.md
PLATFORM_ARCHITECTURE.md
POST_166_WORK_ITEMS_MILESTONES_AND_RISK_ASSESSMENT.md
PROJECT_DOC_TEMPLATE.md
PROMINENT_COURT_DETECTION.md
QUALITY_ITERATIONS.md
R1_MILESTONE_LAUNCH_REPORT.md
R2_EVAL_METRIC_DESIGN.md
RALLY_DETECTION_CODE_REVIEW.md
RALLY_DETECTION_FIXES_PLAN.md
RALLY_DETECTION_GAP_CLOSURE.md
RALLY_DETECTION_QUALITY_REPORT.md
RALLY_QUALITY_RESEARCH.md
README.md
REAL_FOOTAGE_VALIDATION_REPORT.md
ROADMAP.md
SETUP_NEW_MACHINE.md
STORAGE_SHARING_MODEL.md
TRACKNET_WSL_SETUP.md
UI_ALPHA_EXECUTION_PLAN.md
UI_PROPOSAL.md
VIDEO_LOCALITY_MODEL.md
cloudbuild.yaml
Dockerfile
gen_service_yaml.py
README.md
```

18. user (2026-07-05T09:21:02.448Z)

```
=== ISSUE #471 ===
[OPEN] cloud-serving: control-plane state (SQLite jobs/videos DB) is per-instance ephemeral ?
maxScale>1 splits the queue and /api/jobs polls
```

Symptom

The A1 control-plane service (rev 00004) runs with `min-instances 1` but **maxScale 3**. All server state ? the jobs queue, the videos/segments tables (SQLite), and scratch files ? lives in the instance''s ephemeral `/tmp`. With more than one instance:

- `/api/process` returns 202 from instance A; the client''s `/api/jobs/{id}` poll can land on instance B, which has never heard of the job ? 404 / empty status while the job is actually running.
- A restart or scale-down loses the queue outright (partially mitigated: `recover_stale_jobs()` re-queues interrupted compiles, but only on the instance that restarts, and completed-job history is gone).

#463 (reel mirror to gs://) and #461 (upload staging to gs://) fix the **artifacts** outliving an instance, but not job/queue **state** visibility across instances.

Options

1. **Alpha-cheap:** pin `maxScale=1` in the service manifest + document the single-instance constraint (matches the current 4-email alpha traffic; `min-instances 1` already implies a warm single box).

2. **Durable:** move job/video state to a shared store (Cloud SQL / Firestore / a GCS-backed job store). Pairs with #448 (durable background-job UX) and #326 (request tracing).

Until one lands, treat any multi-instance scale-out of the control plane as broken for the async-job CUJ.

Found during the A1 alpha deploy (runlog #464).
=== ISSUE #480 ===
[OPEN] Hosted alpha: browser chunked upload 413s (Cloud Run HTTP/1 ~32MiB request cap vs 95MB parts) + 4Gi tmpfs ceiling

Confirmed (code trace + adversarial verify, 2026-07-05)

A 2.68 GB browser upload fails with `HTTP 413` on the hosted alpha even though it's **under** the 4096 MB whole-file cap.

Root cause ? upstream/edge, not an app check

- The SPA chunks at `max_part_mb` = **95 MB** (`web/src/api/client.ts:320`). Neither engine 413 fires: whole-file cap 4096 MB (`backend/api/uploads.py:100`), per-part/running-total (`uploads.py:146/151`) ? both pass for a 2.68 GB file in 95 MB parts. Only two `status_code=413` sites exist in `backend/`, and no body-size middleware.
- The control-plane service manifest has **no `run.googleapis.com/use-http2` annotation** (`deploy/cloudrun/gen_service_yaml.py`), so Cloud Run ingress is **HTTP/1 with a ~32 MiB per-request body limit** ? each 95 MB `PUT part/N` is 413'd by the Google Front End **before FastAPI runs**. Fails on part 0.

Secondary latent failure (must also be covered)

The CP service is **4 GiB with a RAM-backed `/tmp`**; uploads stage under `/tmp/apphome/output/uploads`. A multi-GB assembled file can't fit in tmpfs ? on current master the free-space guard would `507` (`backend/utils/freespace.py:77`), and it OOMs regardless. (The 32 GiB *worker* JOB is fine; the *control-plane* SERVICE is the constraint.)

Also: the client-side pre-flight guard is **dead code** ? `/api/capabilities` never emits `max_upload_mb`/`max_part_mb` (`backend/main.py:~379`), so `IngestPanel.tsx:82` / `useUpload.ts:56` never fire.

Fixes

1. Smaller parts (`security.max_part_mb` ~24) ? slips under the ~32 MiB HTTP/1 cap; helps **small** files only.
2. Enable HTTP/2 on the service (`--use-http2`) so bodies stream past the per-request cap.
3. Raise service `--memory` (8 GiB+) or stage uploads on a non-tmpfs volume ? fixes the assembled-file ceiling.
4. **Best long-term:** direct browser?GCS resumable/signed-URL upload, then hand the `gs://` URI to `/api/process` (removes both the edge cap and the tmpfs ceiling; matches the existing bucket path).
5. Emit `max_upload_mb`/`max_part_mb` in `/api/capabilities` so the client pre-flight actually works.

Note: raising `max_upload_mb` is NOT a fix (file already under cap). The gs:// bucket path is the correct route for large files today.
=== ISSUE #482 ===
[OPEN] Observability: emit progress/heartbeat during long detection + per-job runlog (worker goes silent for minutes)

Problem (observed live 2026-07-05)

During a long cloud detection run there is **no progress signal**. A 1080p60 upload (~2.7 GB) dispatched to `rally-worker-mw6wr`; the worker logged `Running native WASB inference (device=cuda)` **once at 08:05** and then emitted nothing job-relevant for 7+ minutes (only GCS-FUSE garbage-collection housekeeping). Operators watching Cloud Logging ? and the SPA polling `/api/jobs/{id}` ? can't tell whether it's **alive, stuck, or how far along**. The SPA's own 10-min poll ceiling then shows "taking longer than expected" while the job is in fact still running.

We should not have to "just wait and hope."

Asks

1. **Emit periodic progress from the WASB runner** ? frames processed / % / rough ETA ? at a sane cadence (e.g. every N seconds or every M%), to the worker log.
2. **Surface it through the job `progress` field** (`set_job_progress`) so `/api/jobs/{id}` returns e.g. `"segmenting 42% (~6 min left)"` and the SPA shows real progress instead of a silent spinner. Today `progress` appears to go stale for the whole detection phase.
3. **Per-job runlog artifact** in the bucket (analogous to the operator `DEPLOY_REPORT` runlogs, but per-job): phases + timings + counts, written alongside `outputs/<video_id>/`, retrievable after the fact for triage.

Why

Makes long runs observable end-to-end (worker ? job.progress ? SPA), and gives a durable record. Relates to #328 (surface per-video trace/runlog to the user) and #326 (hosted-app request tracing).
=== ISSUE #483 ===
[OPEN] [cost] Downsample 'detectably long' uploads to a fast proxy for DETECTION (GPU ffmpeg); build the reel from the ORIGINAL
Idea (owner, 2026-07-05) ? save GPU cost while the product is free

Rally **detection** is the expensive GPU step and does **not** need full resolution/fps. For long / high-fps uploads it dominates cost and blows past our timeouts (a 1080p**60** ~2.7 GB clip on 2026-07-05 exceeded both the SPA 10-min poll and the CP 30-min `max_wait`). The final **reel**, however, must be full quality.

Proposal

1. **Detect "long-running" uploads** up front ? by duration / frame-count / estimated-processing-time crossing a threshold (we can estimate from res·fps·duration and the known ~25 fps WASB throughput on L4).
2. **Transcode a downsampled proxy** ? lower resolution + fps (e.g. 720p/30, or lower) ? using **GPU-accelerated ffmpeg (NVDEC/NVENC)** on the worker (cheap + fast).
3. **Run WASB rally detection on the PROXY** ? far less GPU time.
4. **Map detected rally timestamps back to the ORIGINAL** and **compile the final reel from the original** (full quality, no re-detection).
5. **Tell the user**: e.g. *"This is a long video ? we're finding rallies on a fast downsampled copy to keep things quick; your reel will be cut from the original full-quality video."*

Guardrail

Verify detection quality on the proxy doesn't materially regress ? run the golden set at the chosen proxy res/fps and compare F1/tolF1 before enabling. Pick the most aggressive downsample that holds quality.

Payoff

Big GPU-cost reduction on long uploads (the common real-match case), keeps latency sane, and removes the "video too long for the timeouts" failure mode ? all while the reel stays full quality. Pairs with the timeout fixes (SPA poll ceiling, CP `max_wait`) and #331/#349 (perf).
=== ISSUE #484 ===
[OPEN] [cost] Reuse detection by MD5 ? /api/process must not re-run WASB when outputs/<video_id>/ already exists
Problem (observed live 2026-07-05)

Re-processing the **same** video (same content ? same MD5/video_id) triggers a **full new GPU detection** instead of reusing the existing result. The same 6-min clip was detected **3× today** (`mw6wr`, `rsvxs`, `x5pt2`) ? three paid L4 runs for one video. The user's reasonable mental model ? "we key by MD5, so a re-run should be instant" ? is false today.

Why it doesn't short-circuit

1. **MD5/video_id is computed on the worker, AFTER dispatch** (deferred hashing,

DEFERRED_HARDENING_PLAN #16) ? so the control plane can't dedup/cache at submit time.
2. **The engine never checks the bucket** for an existing `outputs/<md5>/report.json` before dispatching.
3. The ephemeral SQLite is **wiped on CP restart** (#471), erasing any cached record anyway.

Proposal

- Cheaply resolve the input MD5 at submit (pre-flight hash for local/gs:// inputs, or a fast head/size+partial hash), then **if `outputs/<md5>/report.json` exists with `status=success`, re-hydrate the segments from the bucket and return immediately ? no GPU dispatch.**
- Add a `force` bypass for deliberate re-detection (already exists on `/api/process`).

Makes "MD5 ? fast" true and stops paying to re-detect. Pairs with the durable-resume design (sibling issue), #471, and the #481 active-job dedup.
=== ISSUE #485 ===
[OPEN] Durable resumable link: re-hydrate session state from the bucket (source of truth) on ?video=<md5>, surviving CP restarts (#471)

Ask (owner, 2026-07-05)

"A resumable link with every upload+process session that always lands back at the server's grounded state."

Today (partial, and it broke)

The SPA puts `?job=<id>` (while processing) and `?video=<md5>` (after load) in the URL ? shareable/reloadable (#144). **But it is NOT durably server-grounded:** a CP restart/redeploy wipes the ephemeral SQLite (#471), so the link 404s or shows an **empty** video.

Observed 2026-07-05: a 2.68 GB upload OOM'd the 4 GB CP ? **restart at 08:14** ? the worker's **29 detected rallies (written to the bucket)** were never ingested, and reloading `?video=<md5>` showed **no rallies**. The reel had to be recovered manually from `outputs/<md5>/intervals.csv`.

Proposal ? bucket is the source of truth

Everything durable already lives under `?/<md5>/` in GCS: `videos/<md5>/`, `outputs/<md5>/intervals.csv` + `report.json`, `highlights/<md5>/`. So:
- On `?video=<md5>` (and the query/highlights routes), **if the DB lacks the video/segments, re-hydrate them from the bucket** (`outputs/<md5>/`).
- The DB becomes a cache, not the system of record.

Then a resumable link **always lands at ground-truth state** regardless of restarts, redeploys, or device ? and it subsumes the MD5 cache-reuse (sibling issue), the durable-state need (#471), and makes `/api/jobs` (#481) + the queue robust. This is the architectural fix for "the control plane lost my rallies."

19. user (2026-07-05T09:21:02.482Z)

```
1       <!--
2       PROJECT DOC TEMPLATE ? copy this file to docs/<YOUR_PLAN>.md and fill it in.
3
4       A "tracked project doc" is how this repo carries substantial design/project work: it can
be
5       ITERATED on, its progress TRACKED, and it is MARKED DONE against explicit deliverables,
then
6       archived. Standardizes the emergent pattern in docs/RALLY_QUALITY_RESEARCH.md §7 and
7       docs/GOLDEN_REGRESSION_FIXTURES.md.
8
9       Rules of thumb:
10      - §7 (the progress tracker) is the SOURCE OF TRUTH ? one row per independently-shippable
PR.
11      - Keep it HONEST: tag each claim [verified] (checked against code), [researched] (needs
sign-off),
12        or [design] (proposed). The doc reflects real shipped state, never aspirational.
13      - It POINTS to detail (code anchors, research artifacts), it does not duplicate it.
14      - Sections marked OPTIONAL are included only when relevant (e.g. Foundation/Threat-model
```

```
for
15        research- or security/privacy-heavy work). Delete sections that do not apply.
16      - Move to docs/archives/ once Status = DONE (per the archive policy: only terminal-state
work).
17      -->
18
19      # <Project / Plan Title>
20
21      > **Status:** `DRAFT` ? `<owner-gated? | in progress>` · **Owner:** `<name>` ·
**Created:** `<YYYY-MM-DD>` · **Last updated:** `<YYYY-MM-DD>`
22      > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
DONE)
23      > **Tracking anchors:** §7 progress tracker is the source of truth; indexed in
`docs/README.md`; pointer in memory `current-state` (and `docs/NEXT_STEPS.md` once work starts).
24      > **Relation to existing docs:** `<extends | supersedes | peer-of ?>`
25      > **Honesty note:** mark each claim `[verified]` / `[researched]` / `[design]`. Not
legal/financial advice where applicable.
26
27      ---
28
29      ## 0. TL;DR
30      <2?5 sentences: the goal, the approach, and the single most important caveat. Call out
any finding that is already actionable on its own.>
31
32      ## 1. Problem & goal
33      <Why now; the concrete user-facing outcome; what "good" looks like. Link the code/docs a
reader must read to get current.>
34
35      ## 2. Decisions locked
36      <Table ? capture answered questions so they are not relitigated.>
37
38      | # | Decision | Source / date | Implication |
39      |---|----------|---------------|-------------|
40      | D1 | ? | ? | ? |
41
42      ## 3. Foundation ? research / prior art  *(OPTIONAL)*
43      <Include when the plan rests on external facts: cited research, legal/compliance,
benchmarks, prior-art survey. Mark researched-vs-verified; list any sign-off gates this
creates.>
44
45      ## 4. Design / architecture
46      <The actual plan. Data contracts, diagrams, and a REUSE MAP ? name the existing
modules/code this builds on so it is clear what is new vs reused.>
47
48      ## 5. Threat model / risk table  *(OPTIONAL ? security / privacy / compliance work)*
49      <Who/what is protected vs NOT; residual risks stated plainly.>
50
51      ## 6. Honest limits ? what this does NOT do
52      <The false-confidence guardrail: what a green suite / a DONE status does NOT prove.>
53
54      ## 7. Deliverables & progress tracker   ? **source of truth**
55      Legend: ? Todo · ? In progress · ? Done · ? Blocked/gated. **One small PR per row**,
branched from `origin/master`, no stacking. Update the row (status + PR link) as work lands.
56
57      | ID | Deliverable | Depends on | Gated? | Status | PR |
58      |----|-------------|-----------|--------|--------|----|
59      | P0 | ? | ? | No | ? | ? |
60
61      ## 8. Open questions ? owner / external  *(split blocking-gates from nice-to-knows; name
who decides)*
62      1. ?
63
64      ## 9. Definition of done
65      <Explicit acceptance checklist. When all ticked ? flip `Status ? DONE` and archive.>
66      - [ ] ?
```

```
67
68      ## 10. References
69      **Internal code anchors `[verified]`:** `<paths>`. **Internal docs:** `<docs>`.
**External:** `<cited sources>`.
70
71      ---
72
73      ### Changelog
74      - `<YYYY-MM-DD>` ? `<what changed>`.
75
```

20. user (2026-07-05T09:21:02.496Z)

```
1       """Generate the control-plane Cloud Run *service manifest* (COMPUTE_DECOUPLED_SERVING
M8/M9, #473).
2
3       The A1 alpha deploy drove `rally-control-plane` from a hand-generated YAML that lived in
a
4       session scratchpad ? this commits the generator so the deploy is reproducible from a
clean
5       clone. It emits a Knative service manifest for::
6
7           python deploy/cloudrun/gen_service_yaml.py --edge-auth off -o service.yaml
8           gcloud run services replace service.yaml --region asia-southeast1
9
10      Why a MANIFEST (`services replace`) and not `gcloud run deploy` flags: the container
command is
11      a /bin/bash wrapper carrying shell metacharacters (pipes, redirects, &&) ? on Windows
the gcloud
12      cmd-shim mangles those through `--command/--args`, while a YAML round-trips them
verbatim. Run
13      gcloud from PowerShell, not git-bash (MSYS rewrites `/tmp`-style paths).
14
15      The app is configured through a config.json baked INTO the manifest: the wrapper
base64-decodes
16      it to `/tmp/apphome/config.json` and exports `RALLY_APP_HOME=/tmp/apphome` before
exec'ing the
17      server, so one image serves every config shape (config.json wins over the cloud-serving
preset ?
18      backend/config/loader.py). `--edge-auth` is REQUIRED and explicit: `off` is only for the
19      identity-token smoke/e2e window; redeploy with `on` (the Cf-Access JWT verify, M9)
before the
20      service faces testers again.
21
22      Deliberate manifest choices (learned on the A1 deploy):
23        * `run.googleapis.com/cpu-throttling: "false"` + minScale 1 ? the in-process job
worker drains
24          on a background thread; without always-allocated CPU a 202'd compile stalls (README
§3a).
25        * maxScale DEFAULTS TO 1 ? the jobs/videos DB is per-instance ephemeral SQLite, so a
second
26          instance splits the queue and `/api/jobs/{id}` polls (issue #471, option 1). Raise
it only
27          once state is shared.
28        * `CLOUD_RUN_JOB` is NOT set ? it's a reserved runtime env name and the manifest is
rejected
29          with it; the code default ("rally-worker") is already correct.
30        * No secrets here: bucket/project/aud/team-domain are the same non-secret identifiers
the
31          committed runlogs carry; the Gemini key stays in Secret Manager (the alpha runs
32          `skip_ai_handoff=true` and never needs it).
33      """
34
35      from __future__ import annotations
36
```

```
    import argparse
    import base64
    import json
    import sys

    DEFAULT_PROJECT = "gen-lang-client-0306026826"
    DEFAULT_REGION = "asia-southeast1"
    DEFAULT_SERVICE = "rally-control-plane"
    DEFAULT_SERVING_BUCKET = "gs://khelsutra-rally-serving"
    DEFAULT_CF_TEAM_DOMAIN = "https://khelsutra.cloudflareaccess.com"
    DEFAULT_CF_ACCESS_AUD =
"c980da67ed888fff935fcd913ae1eb4da63f9d65805d6e1c5ca76f92f517082b"


    def build_config(args: argparse.Namespace) -> dict:
        """The app config the wrapper materializes at RALLY_APP_HOME/config.json."""
        bucket = args.serving_bucket.rstrip("/")
        return {
            "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
            # WASB-only alpha: candidate windows ARE the rallies; no Gemini key in this
service.
            "indexing": {"skip_ai_handoff": True},
            "storage": {
                "backend": "gcs",
                "output_root": bucket,
                "input_backend": "gcs",
                "input_root": f"{bucket}/videos",
            },
            "security": {
                "edge_auth_enabled": args.edge_auth == "on",
                "cf_team_domain": args.cf_team_domain,
                "cf_access_aud": args.cf_access_aud,
                "allowed_video_root": "/tmp/apphome/output",
            },
        }


    def build_yaml(args: argparse.Namespace) -> str:
        """Render the Knative service manifest for `gcloud run services replace`."""
        cfg_b64 = base64.b64encode(
            json.dumps(build_config(args), separators=(",", ":")).encode("utf-8")
        ).decode("ascii")
        # mkdir BEFORE the redirect (the redirect can't create the directory), then exec so
the
        # server is PID 1 and Cloud Run's SIGTERM reaches it directly.
        wrapper = (
            "mkdir -p /tmp/apphome && "
            f"printf %s '{cfg_b64}' | base64 -d > /tmp/apphome/config.json && "
            "export RALLY_APP_HOME=/tmp/apphome && "
            "exec python3 -m backend.main --server --host 0.0.0.0 --port 8080"
        )
        return f"""\
    apiVersion: serving.knative.dev/v1
    kind: Service
    metadata:
      name: {args.service}
      labels:
        cloud.googleapis.com/location: {args.region}
    spec:
      template:
        metadata:
          annotations:
            autoscaling.knative.dev/minScale: "{args.min_instances}"
            autoscaling.knative.dev/maxScale: "{args.max_instances}"
            run.googleapis.com/cpu-throttling: "false"
```

```
        spec:
          timeoutSeconds: {args.timeout}
          containers:
            - image: {args.image}
              command: ["/bin/bash"]
              args: ["-c", "{wrapper}"]
              ports:
                - containerPort: 8080
              resources:
                limits:
                  cpu: "{args.cpu}"
                  memory: {args.memory}
              env:
                - name: DEPLOYMENT_PROFILE
                  value: cloud-serving
                - name: CLOUD_RUN_REGION
                  value: {args.region}
                - name: GOOGLE_CLOUD_PROJECT
                  value: {args.project}
                - name: RALLY_ALLOWED_HOSTS
                  value: "*"
    """


    def main(argv: "list[str] | None" = None) -> int:
        p = argparse.ArgumentParser(description=__doc__.splitlines()[0])
        p.add_argument(
            "--edge-auth",
            choices=("on", "off"),
            required=True,
            help="REQUIRED and explicit: 'off' only for the identity-token smoke/e2e window;
"
            "redeploy 'on' (M9 Cf-Access verify) before testers touch it again.",
        )
        p.add_argument("--project", default=DEFAULT_PROJECT)
        p.add_argument("--region", default=DEFAULT_REGION)
        p.add_argument("--service", default=DEFAULT_SERVICE)
        p.add_argument(
            "--image",
            default=None,
            help="Container image; default = the project's rally-worker:latest (post-#470 it
"
            "carries cloud-run + auth, so the worker image IS the control-plane image).",
        )
        p.add_argument("--serving-bucket", default=DEFAULT_SERVING_BUCKET)
        p.add_argument("--cf-team-domain", default=DEFAULT_CF_TEAM_DOMAIN)
        p.add_argument("--cf-access-aud", default=DEFAULT_CF_ACCESS_AUD)
        p.add_argument("--min-instances", type=int, default=1)
        p.add_argument(
            "--max-instances",
            type=int,
            default=1,
            help="Default 1: per-instance SQLite state splits across instances (issue
#471).",
        )
        p.add_argument("--cpu", type=int, default=2)
        p.add_argument("--memory", default="4Gi")
        p.add_argument("--timeout", type=int, default=300)
        p.add_argument("-o", "--output", default="-", help="Output path, or '-' for
stdout.")
        args = p.parse_args(argv)
        if args.image is None:
            args.image = f"{args.region}-docker.pkg.dev/{args.project}/rally/rally-
worker:latest"

```

```
        text = build_yaml(args)
        if args.output == "-":
            sys.stdout.write(text)
        else:
            with open(args.output, "w", encoding="utf-8", newline="\n") as f:
                f.write(text)
        return 0


    if __name__ == "__main__":
        raise SystemExit(main())

```

21. user (2026-07-05T09:22:03.074Z)

```
    """Chunked upload (B2, PR-2) ? extracted from backend.main into its own router.

    Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
    proxies cap request bodies at ~100 MB ? HOSTING_PLAN §2), are assembled server-side
    under security.upload_dir, then submitted to /api/process by the returned path.
    Upload state is in-process by design (this is the single-box alpha): a server restart
    aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
    arrives with PR-3 identity scoping.

    The startup config lives on ``backend.main.global_config`` (set ONLY in ``main()``).
    ``_upload_cfg`` resolves it through the ``backend.main`` module object so it always
    reads the live value the server set at startup (and so tests that monkeypatch
    ``backend.main.global_config`` flow through unchanged).
    """

    import os
    import threading
    import unicodedata
    import uuid
    from typing import Any, Dict, Optional

    from fastapi import APIRouter, HTTPException, Request
    from pydantic import BaseModel

    from backend.utils.app_home import (
        resolve_output_dir,  # P0a: app-home-aware path resolution
    )
    from backend.utils.freespace import require_free_bytes  # HTTP-507 disk guard (P2, D11)
    from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
    from backend.utils.paths import safe_output_filename

    router = APIRouter()

    _uploads: Dict[str, Dict[str, Any]] = {}
    _uploads_lock = threading.Lock()


    class UploadInitRequest(BaseModel):
        filename: str
        total_size_bytes: Optional[int] = None


    class UploadCompleteRequest(BaseModel):
        total_parts: int


    def _upload_cfg():
        # Resolved lazily (import inside the function) to read the LIVE startup config that
        # main() sets on the module global, and to avoid a circular import at module load
        # (main.py imports this router; this module must not import main at import time).
```

```
        import backend.main as _main

        sec = getattr(_main.global_config, "security", None)
        upload_dir = resolve_output_dir(
            getattr(sec, "upload_dir", None) or "output/uploads"
        )  # P0a: app-home-rooted (CWD-identical when RALLY_APP_HOME unset)
        max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
        part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
        exts = [
            e.lower().lstrip(".")
            for e in (getattr(sec, "allowed_upload_extensions", None) or ["mp4", "mov"])
        ]
        return upload_dir, max_bytes, part_bytes, exts


    def _abort_upload(upload_id: str) -> None:
        with _uploads_lock:
            st = _uploads.pop(upload_id, None)
        if st:
            try:
                os.remove(st["tmp_path"])
            except OSError:
                pass


    def _normalize_upload_filename(filename: str) -> str:
        """Normalize browser-provided file names before extension validation.

        Some mobile/file-provider pickers can hand us visually normal names with
compatibility
        characters (for example a fullwidth dot) or trailing whitespace/dots. Keep the
existing
        traversal/null-byte checks in ``safe_output_filename`` as the authority after this
cleanup.
        """
        return unicodedata.normalize("NFKC", filename).strip().rstrip(". ")


    @router.post("/api/upload/init")
    def upload_init(req: UploadInitRequest):
        """Start a chunked upload. Returns upload_id; PUT sequential parts next."""
        upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
        try:
            name = safe_output_filename(_normalize_upload_filename(req.filename))
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        ext = os.path.splitext(name)[1].lower().lstrip(".")
        if ext not in exts:
            raise HTTPException(
                status_code=400,
                detail=f"Extension '.{ext}' not allowed. Allowed: {sorted(exts)}",
            )
        if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
            raise HTTPException(
                status_code=413,
                detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB upload limit.",
            )
        os.makedirs(upload_dir, exist_ok=True)
        # P2 (D11): fail fast with 507 if the box can't hold the declared upload. Best-
effort ? an
        # undeclared size (total_size_bytes=None) or an unreadable disk skips the guard
(never blocks a
        # legit upload); the per-part total cap in upload_part is the hard backstop either
way.
        require_free_bytes(upload_dir, req.total_size_bytes, stage="upload")
```

```
        upload_id = uuid.uuid4().hex
        tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
        open(tmp_path, "wb").close()
        with _uploads_lock:
            _uploads[upload_id] = {
                "filename": name,
                "tmp_path": tmp_path,
                "parts": 0,
                "bytes": 0,
            }
        return {
            "upload_id": upload_id,
            "max_part_mb": part_bytes // (1024 * 1024),
            "next_part": f"/api/upload/{upload_id}/part/0",
        }


    @router.put("/api/upload/{upload_id}/part/{index}")
    async def upload_part(upload_id: str, index: int, request: Request):
        """Append one part (raw bytes body). Parts are strictly sequential (0,1,2,...)."""
        _, max_bytes, part_bytes, _ = _upload_cfg()
        with _uploads_lock:
            st = _uploads.get(upload_id)
        if st is None:
            raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
        if index != st["parts"]:
            raise HTTPException(
                status_code=409,
                detail=f"Out-of-order part: expected {st['parts']}, got {index} (parts are
sequential)",
            )

        received = 0
        overflow = None
        with open(st["tmp_path"], "ab") as fh:
            async for chunk in request.stream():
                received += len(chunk)
                if received > part_bytes:
                    overflow = (
                        f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part limit."
                    )
                    break
                if st["bytes"] + received > max_bytes:
                    overflow = (
                        f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total limit."
                    )
                    break
                fh.write(chunk)
        if overflow:
            _abort_upload(upload_id)
            raise HTTPException(
                status_code=413, detail=overflow + " Upload aborted ? re-init to retry."
            )

        with _uploads_lock:
            st["parts"] += 1
            st["bytes"] += received
        return {
            "upload_id": upload_id,
            "received_parts": st["parts"],
            "received_bytes": st["bytes"],
        }
```

22. user (2026-07-05T09:22:04.614Z)

```
backend\compute\cloud_run.py:10:     cold-start is absorbed by the async poll. On a poll TIMEOUT
(``max_wait_sec`` can undercut the
backend\compute\cloud_run.py:245:        The poll budget (``max_wait_sec``, default 1800s) can
undercut the deployed job's own
backend\providers\gemini.py:169:            poll_every_n_sec=10.0, max_wait_sec=1200.0
backend\providers\gemini.py:186:                f"[{log_prefix}] Processing exceeded
{process_policy.max_wait_sec:.0f}s ? giving up."
backend\providers\gemini.py:190:                f"processing timed out after
{process_policy.max_wait_sec:.0f}s"
backend\infrastructure\execution_policy.py:8:  ``max_wait_sec`` deadline).
backend\infrastructure\execution_policy.py:42:    """An op hung past ``op_timeout_sec`` or never
became ready within ``max_wait_sec``."""
backend\infrastructure\execution_policy.py:54:    max_wait_sec: float = 1800.0  # total wall
budget for a poll loop
backend\infrastructure\execution_policy.py:162:    ``PolicyTimeout`` after ``max_wait_sec``.
``check`` returns ``None`` while the op is still in progress.
backend\infrastructure\execution_policy.py:170:        "%s: waiting (poll_every=%.0fs,
max_wait=%.0fs)",
backend\infrastructure\execution_policy.py:173:        policy.max_wait_sec,
backend\infrastructure\execution_policy.py:183:        if elapsed >= policy.max_wait_sec:
backend\infrastructure\execution_policy.py:185:                "%s: still not ready after %.0fs
(max_wait) ? giving up", label, elapsed
backend\infrastructure\execution_policy.py:188:                f"{label} not ready within
max_wait_sec={policy.max_wait_sec}s"
backend\api\job_worker.py:45:#   - Cloud Run poll has a max_wait_sec bound
```

23. user (2026-07-05T09:22:06.030Z)

```
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\hooks\useIngestJob.ts:18:const
POLL_CEILING_MS = 10 * 60 * 1000; // a wedged 'running' job won't poll forever
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\hooks\useIngestJob.ts:214:
if (Date.now() - startedAt > POLL_CEILING_MS) {
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\api\client.ts:181:const
COMPILE_POLL_CEILING_MS = 10 * 60 * 1000; // a wedged 'running' compile won't poll forever
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\api\client.ts:226:    if (Date.now() -
startedAt > COMPILE_POLL_CEILING_MS) {
```

24. user (2026-07-05T09:22:07.479Z)

```
181:    default_segmenter: str = "yolo_hybrid"
275:    @field_validator("default_segmenter")
370:    max_upload_mb: int = 4096
371:    max_part_mb: int = 95
```

25. user (2026-07-05T09:23:27.656Z)

```
1       # Cloud Run GPU **Job** image for the rally worker (COMPUTE_DECOUPLED_SERVING M6/M7,
ADR-012/D16).
2       #
3       # Runs `python -m backend.worker infer ?` INLINE on an L4 (detect + window + stitch co-
located so
4       # CPU/wall + egress are metered into one job ? D4). Built + pushed + verified by the
owner on the
5       # M7 real run (the $100/?10k budget, D17); CI does NOT build this (a CUDA image is too
heavy) ? the
6       # dispatch shim is proven with a fake client (tests/test_compute_backend.py).
7       #
8       # Two python stacks, deliberately separate (a framework conflict otherwise ? same split
as the
9       # native GPU box, deploy/digitalocean/gpu_setup.sh):
10      #   * the MAIN app env (this repo + ffmpeg) drives the pipeline + stitching;
11      #   * a conda `wasb` env (py3.8 + torch 1.11.0+cu113) runs the WASB shuttle detector as
a subprocess
12      #     (the native runner shells out to $WASB_PYTHON), so the app's torch never co-
```

```
installs with it.
#
# ? WASB-C3: the pretrained badminton weights are academic-data-trained ? provenance NOT cleared for
#   commercial sharing. They are NOT baked into this image; stage them at runtime (a bucket fetch or a
#   mount) and point $WASB_WEIGHTS at them. Resolve WASB-C3 before any public, non-owner serving.

FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04

ENV DEBIAN_FRONTEND=noninteractive \
    PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1 \
    CONDA_DIR=/opt/conda \
    WASB_REPO_DIR=/opt/models/WASB-SBDT

# ffmpeg (stitch/decode) + DejaVu fonts (the watermark filter) + git/curl for the WASB env build.
RUN apt-get update && apt-get install -y --no-install-recommends \
        ffmpeg fonts-dejavu-core git curl ca-certificates build-essential \
        python3 python3-pip python3-venv \
    && rm -rf /var/lib/apt/lists/*

# --- WASB detector env (micromamba + conda-forge ? BSD-licensed, NO Anaconda ToS; P3a, ADR-005) ---
# Enforces ADR-005 (every dependency ? code, data, weights, AND the build toolchain ? stays
# MIT/Apache/BSD): micromamba (BSD-3) + conda-forge (community) replaces Miniconda's Anaconda-ToS-gated
# `defaults` channels (the ?200-employee Terms-of-Service trap flagged in PACKAGING_PLAN.md §3).
# MAMBA_ROOT_PREFIX=$CONDA_DIR (=/opt/conda) keeps the env at $CONDA_DIR/envs/wasb so $WASB_PYTHON below
# is UNCHANGED. Same verified torch-1.11.0+cu113 WASB stack; the pip wheels are unaffected by the switch.
ENV MAMBA_ROOT_PREFIX=/opt/conda
RUN curl -sSL https://micro.mamba.pm/api/micromamba/linux-64/latest \
        | tar -xj -C /usr/local bin/micromamba \
    && micromamba create -y -q -n wasb -c conda-forge python=3.8 \
    && git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO_DIR" \
    && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
        torch==1.11.0+cu113 torchvision==0.12.0+cu113 --extra-index-url https://download.pytorch.org/whl/cu113 \
    && ( [ -f "$WASB_REPO_DIR/requirements.txt" ] && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q -r "$WASB_REPO_DIR/requirements.txt" || true ) \
    && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
        hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm pyyaml matplotlib "numpy<1.24"

# --- Main application env ---
WORKDIR /app
COPY pyproject.toml README.md ./
COPY requirements-trackers.txt ./
COPY backend ./backend
COPY training ./training
# Install with the [storage-gcs,cloud-run,auth] extras. storage-gcs: the cloud worker PULLS the gs://
# input and PUSHES the gs:// outputs/done-marker through backend.storage (google-cloud-storage is
# lazy-imported there); without it the job dies at the first gs:// fetch with `ModuleNotFoundError:
# google.cloud`. cloud-run: this SAME image is reused as the deployed CONTROL PLANE, which dispatches
# the L4 worker Job via google.cloud.run_v2 (backend/compute/cloud_run.py, lazy-
```

```
imported); without it
60      # the control plane fails every dispatch with "google-cloud-run is required for
CloudRunCompute's real
61      # client". Both are real Cloud Run failures the mock-client tests can't see (the SDKs
are faked in CI).
62      # auth: PyJWT[crypto] for M9 edge-auth (security.edge_auth_enabled ? the Cf-Access JWT
verify the
63      # reused-as-control-plane deployment runs behind api.khelsutra.guru); jwt is lazy-
imported and
64      # edge-auth is default-OFF, so the worker path is unaffected ? but without it a control-
plane
65      # deploy with edge-auth ON needed a hand-built derived image (the A1 deploy's PyJWT
overlay).
66      # D13/P12: `trackers` (ByteTrack's new home) is a git-only dep (broken PyPI wheel ?
67      # roboflow/trackers#474) so it's NOT pulled by `pip install .`; install it out-of-band
before
68      # the main env, or the person-cue path crashes with ModuleNotFoundError. The immutable
git SHA is
69      # single-sourced in requirements-trackers.txt.
70      RUN python3 -m pip install --upgrade pip \
71          && python3 -m pip install -r requirements-trackers.txt \
72          && python3 -m pip install ".[storage-gcs,cloud-run,auth]"
73
74      # The native runner finds the WASB env + repo by env (no `conda activate` needed);
weights come in at
75      # runtime (WASB-C3). detector_impl=native = the Linux-native WASB runner; cuda stays the
default.
76      # PYTHONUNBUFFERED: without it the worker's stdout/stderr sit in the block buffer until
process
77      # exit, so Cloud Logging shows a single end-of-run burst instead of live progress
(observed on the
78      # first real M7 runs, 2026-07-03) ? a running job looks stalled and mid-run failures log
nothing.
79      ENV PYTHONUNBUFFERED=1 \
80          WASB_PYTHON=/opt/conda/envs/wasb/bin/python \
81          WASB_REPO_DIR=/opt/models/WASB-SBDT \
82          WASB_WEIGHTS=/opt/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar \
83          RALLY_DETECTOR_IMPL=native
84
85      # The dispatcher (CloudRunCompute) supplies the per-execution args:
86      #   infer --video-uri gs://? --out-uri gs://? --done-uri gs://? --segmenter wasb_hybrid
--set ?
87      ENTRYPOINT ["python3", "-m", "backend.worker"]
88
```

26. user (2026-07-05T09:23:31.125Z)

```
1       # Cloud Run GPU serving ? build / deploy / run (COMPUTE_DECOUPLED_SERVING M6/M7)
2
3       The cloud-serving compute path: a control plane (`CloudRunCompute`,
`backend/compute/cloud_run.py`)
4       dispatches a Cloud Run **Job** execution running this image's worker on an **L4**, then
**bucket-polls**
5       a done-marker for completion (D16). Scale-to-zero (D7) ? no warm pool.
6
7       > **Status:** the dispatch shim + image are **landed + mock-tested** (M6). The **real
build + push +
8       > run on an L4** is the **M7** owner step, within the **$100/?10k** budget (D17) ?
confirm cmd + hourly
9       > cost first, **delete every resource after** ([[gcp-vm-always-delete]]). CI does
**not** build this image.
10
11      ## 0. Prerequisites (owner)
12      - A GCP project with billing on + the Cloud Run, Artifact Registry, and Cloud Build APIs
enabled.
```

```
13      - GPU quota for L4 (`GPUS_ALL_REGIONS` ? 1) in the chosen region (asia-south1, else
Singapore ? see
14        `deploy/gcp/README.md`; L4 is often stocked-out, fall back per that runbook).
15      - The WASB weights staged in the bucket (WASB-C3 unresolved ? owner-gated):
`gs://<bucket>/models/wasb_badminton_best.pth.tar`.
16
17      ## 1. Build + push the image (Artifact Registry)
18      ```bash
19      PROJECT=<gcp-project>; REPO=rally
20      # L4 GPU is region-limited ? use asia-southeast1 (Singapore), NOT asia-south1 (Mumbai).
21      REGION=asia-southeast1
22      gcloud artifacts repositories create $REPO --repository-format=docker --location=$REGION
2>/dev/null || true
23      # `gcloud builds submit` has NO -f flag, so a non-root Dockerfile needs a build config:
24      gcloud builds submit --config=deploy/cloudrun/cloudbuild.yaml --project $PROJECT .
25      ```
26      *(A CUDA + micromamba image is large; the first build is ~8?10 min. Budget-watch per
D17.)*
27
28      ## 2. Create the Cloud Run **Job** (GPU, scale-to-zero)
29      The proven shape (first real M7 deploy, 2026-07-03): weights staged in a **same-region
bucket**
30      mounted read-only via GCS FUSE (no weights in the image ? WASB-C3; no init-fetch step
needed).
31      ```bash
32      gcloud run jobs create rally-worker \
33        --image $REGION-docker.pkg.dev/$PROJECT/$REPO/$IMG:latest \
34        --region $REGION --gpu 1 --gpu-type nvidia-l4 --no-gpu-zonal-redundancy \
35        --cpu 8 --memory 32Gi \
36        --max-retries 0 --task-timeout 3600 \
37        --set-env-vars WASB_WEIGHTS=/gcs/models/wasb_badminton_best.pth.tar \
38        --add-volume name=serving,type=cloud-storage,bucket=$SERVING_BUCKET \
39        --add-volume-mount volume=serving,mount-path=/gcs
40      ```
41      - **`--no-gpu-zonal-redundancy` is REQUIRED on the CLI**, not just a cost choice (it
also halves
42        the GPU rate ? right for batch jobs): without the flag `gcloud` asks an interactive
Y/n which,
43        in a non-interactive shell, **hangs forever with zero output and nothing created**. An
empty
44        `gcloud` that "did nothing" ? suspect a hidden prompt.
45      - 8 vCPU / 32 GiB: decode + x264 are CPU-side (NVDEC is a deferred D16 knob), so CPU
headroom is
46        what keeps the paid GPU busy. `/tmp` is **RAM-backed** ? the pulled input video lives
in memory,
47        so size `--memory` for the largest expected upload (a 5.8 GB GoPro file ? 6 GiB of
tmpfs).
48
49      ## 3. Wire the dispatcher (control plane)
50      The control plane runs with the **cloud-serving** profile (`DEPLOYMENT_PROFILE=cloud-
serving` ?
51      `compute_target=cloud_run`). `get_compute_backend(config)` then builds `CloudRunCompute`
from env:
52      ```bash
53      export DEPLOYMENT_PROFILE=cloud-serving
54      export CLOUD_RUN_JOB=rally-worker  CLOUD_RUN_REGION=$REGION
GOOGLE_CLOUD_PROJECT=$PROJECT
55      # A `worker infer` (or the API job path) with a gs:// video now DISPATCHES to the Job
instead of
56      # running in-process; the container runs the SAME worker INLINE (compute_target forced
local_gpu).
57      python -m backend.worker infer --video-uri gs://<bucket>/videos/clip.mp4 --out-uri
gs://<bucket> --segmenter wasb_hybrid
58      ```
59      **The AI handoff needs its key IN THE JOB env** (`GEMINI_API_KEY` ? mount a Secret
```

Manager secret
60   on the job), or run fully-local with `--set indexing.skip_ai_handoff=true` (candidate
windows
61   emitted as rallies ? the eval-parity mode). With neither, the segmenter used to bail to
a silent
62   0-rally "success" **after** the paid GPU pass; the cloud-serving startup checks now fail
this at
63   boot (`AI_HANDOFF_KEY_MISSING`, rc 2) before the video is even pulled.
64
65   ### 3a. Control-plane SERVICE deploy ? required flags for reliable async jobs
66
67   The API server (`/api/process`, `/api/compile` ? #329) runs its **job worker IN-
PROCESS**: the
68   request returns `202` and a background thread drains the job (GPU detection dispatches
to the Job;
69   the reel stitch runs on the control plane itself). On Cloud Run that background thread
only makes
70   progress while the instance has CPU, so a scale-to-zero **service** deploy MUST set:
71
72   **The canonical deploy path is the committed manifest generator** (#473) ? it bakes the
73   cloud-serving config.json into a startup wrapper, sets every flag below, pins
`maxScale=1`
74   (per-instance SQLite state, issue #471), and deliberately does NOT set `CLOUD_RUN_JOB`
(a
75   reserved runtime env name that gets a manifest rejected; the code default is already
right):
76   ```bash
77   python deploy/cloudrun/gen_service_yaml.py --edge-auth on -o service.yaml
78   gcloud run services replace service.yaml --region $REGION   # from PowerShell on Windows
79   ```
80   `--edge-auth off` is only for the pre-CF identity-token smoke/e2e window; redeploy with
`on`
81   before testers touch the service again. The flag-based equivalent (for reference ? a
manifest
82   round-trips the bash wrapper's metacharacters, flags don't on Windows):
83   ```bash
84   gcloud run deploy rally-control-plane --image <image> --region $REGION \
85   --no-cpu-throttling \   # CPU stays allocated between requests ? the drain thread
finishes
86   --min-instances 1 \   # one always-warm instance drains reliably (cheap CPU;
NOT the D7 GPU pool)
87   --cpu 2 --memory 4Gi --port 8080 --allow-unauthenticated \
88   --set-env-vars DEPLOYMENT_PROFILE=cloud-
serving,CLOUD_RUN_REGION=$REGION,GOOGLE_CLOUD_PROJECT=$PROJECT
89   ```
90   Without `--no-cpu-throttling`, a `/api/compile` returns 202 and then **stalls** ? the
reel never
91   finishes and startup crash-recovery would mark it terminal. `min-instances 1` does NOT
contradict
92   D7 (that was the ~$500/mo warm **GPU** pool); this is a ~1-vCPU CPU instance. Defense-
in-depth is in
93   the code: `Database.recover_stale_jobs()` **re-queues** a compile interrupted by a
restart (idempotent
94   stitch) rather than failing it, so even an unlucky scale-down mid-stitch self-heals on
the next drain.
95   Reserve `--allow-unauthenticated` for the pre-CF-auth smoke test only; lock ingress to
the CF proxy
96   (M9) before inviting testers. The image ships the `auth` extra (PyJWT[crypto]), so
flipping
97   `security.edge_auth_enabled` on needs no derived image ? the M9 Cf-Access JWT verify
works out of
98   the box.
99
100   ## 4. M7 acceptance (the owner runlog)
101   Capture a `DEPLOY_REPORT_cloud-serving_<date>.md` under

```
102     `docs/COMPUTE_DECOUPLED_SERVING/runlogs/` (mirrors the truly-local template): the exact
commands,
103     rally counts, `gpu_active_seconds`/`wall_seconds`/`bytes_out`, a reel smoke-test
(`ffmpeg -f null -`
104     exit 0 + a 5-frame contact sheet), gotchas, and the **$0 teardown audit**.
105
106     ## Gotchas
107     - The dispatched container **must run inline** ? `CloudRunCompute` forces
`compute_target=local_gpu`
108       + `indexing.detector_impl=native` in the per-execution args so the Job never re-
dispatches itself.
109     - Completion is a **bucket marker** (`<out_uri>/_dispatch/<token>.done`), not the Cloud
Run API status
110       ? so it reuses the storage layer + `execution_policy.poll_until` and survives a
control-plane restart.
111     - A **failed** Job (validation rc 2 / segmenter rc 3) now writes a **failure marker**
carrying the
112       real non-zero rc, so the dispatcher's poll terminates **FAST + accurately** (no 30-min
hang on a bad
113       video). Only a **hung/crashed** container (no marker at all) makes the bounded poll
time out
114       (`ExecutionPolicy.max_wait_sec`); the worker CLI maps that to a clean **rc 4**, and
the API path
115       surfaces it as a `cloud dispatch error` job failure (not a traceback).
116     - libx264 encode (parity, D16) + NVDEC decode-only; the image keeps ffmpeg's default
libx264.
117     - **`GOOGLE_CLOUD_PROJECT` is required** for the real client ?
`GoogleCloudRunJobClient.run_job`
118       fails fast with a clear `RuntimeError` if it is unset/empty (it must resolve a real
`job_path`;
119       an unset project would otherwise build a bogus `projects/None/...` name). Export it
(step 3) before
120       dispatching. The guard runs before the `google-cloud-run` import, so it is exercised
in CI even
121       though the SDK isn't a CI/test dependency.
122     - **Hand-rolled `jobs execute --args` with `gs://` URIs:** gcloud's default comma-split
plus its
123       alternate-delimiter syntax both bite ? repeated `--set` tokens trip the comma parser,
and a
124       `^:^` delimiter splits the `:` inside `gs://`. Use a delimiter that appears nowhere in
the args,
125       e.g. `--args='^@@^-v@@infer@@--video-uri=gs://?'`. (The dispatcher path doesn't have
this
126       problem ? prefer it; this is for one-off diagnostics.) Container-arg overrides must
include the
127       dispatcher's two forced `--set`s: `deployment.compute_target=local_gpu`,
128       `indexing.detector_impl=native`.
129     - **Git Bash (MSYS) on Windows mangles Unix-looking values** before gcloud sees them:
130       `--add-volume-mount ?mount-path=/gcs` became `C:/Program Files/Git/gcs`, and
131       `--command=/bin/bash` became `C:/Program Files/Git/usr/bin/bash` (the container then
exec-fails).
132       Run gcloud commands with such flags from PowerShell (`MSYS_NO_PATHCONV=1` breaks
gcloud's own
133       wrapper). PowerShell has its own trap: quote any flag value containing commas.
134     - **First execution after an image push** spends minutes in "Container image import in
progress"
135       (visible in `executions describe status.conditions`, NOT in logs) ? that's progress,
not a hang;
136       later executions reuse the imported image.
137
```

27. user (2026-07-05T09:23:31.389Z)

```
42:from fastapi.staticfiles import StaticFiles
93:from backend.utils.freespace import require_free_bytes  # HTTP-507 disk guard (P2, D11)
```

```
151:# Serves static frontend files directly (now under api/ per approved structure).
153:# arbitrary directory finds the shipped assets instead of creating a stray
./backend/api/static.
154:# (The legacy in-engine dashboard; the real SPA gets bundled here in P1a.)
155:_STATIC_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "api", "static")
162:app.mount("/static", StaticFiles(directory=_STATIC_DIR), name="static")
164:# P1b: serve the bundled KhelSutra SPA single-origin. `backend/ui/` is the committed
snapshot
166:# absent (a source checkout with no vendored UI). The SPA calls same-origin `/api`, so no
SPA change
169:_BUNDLED_UI_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "ui")
177:    app.mount("/assets", StaticFiles(directory=_UI_ASSETS_DIR), name="assets")
190:    """Provenance of the vendored SPA (backend/ui/ui_version.json), or None if no UI is
bundled.
228:# dispatch+ingest, compile resolution + worker handler) now lives in backend.orchestration
240:    _maybe_dispatch_remote,
261:    # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Recorded on the job for
PMF
264:    # SPA compute-picker (item 3): "local" forces in-process even on a cloud-configured
server;
265:    # "cloud" forces a Cloud Run dispatch (requires deployment.cloud_available). None ?
server
266:    # default (deployment.compute_target). Only honoured for these two values; see
_maybe_dispatch_remote.
306:    # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Localizes the reel
watermark
326:    """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
---db path---
backend/config/models.py:304:    db_name: str = "sports_indexer.db"
backend/infrastructure/database.py:27:    def __init__(self, db_path: str):
backend/infrastructure/database.py:28:        self.db_path = db_path
backend/infrastructure/database.py:49:        conn = sqlite3.connect(self.db_path, timeout=30.0)
backend/infrastructure/database.py:115:                    f"a different data directory
(RALLY_APP_HOME). DB: {self.db_path}"
backend/infrastructure/database.py:161:        backup = f"{self.db_path}.bak-v{from_version}"
backend/infrastructure/database.py:164:            shutil.copy2(self.db_path, backup)
backend/infrastructure/database.py:187:        shutil.copy2(self.db_path, dest)
---compile source fetch---
3:782-    and return the {status, filepath, download_url} result the SPA polls for. Returns a
5:784-    so the SPA always gets a clean verdict instead of a worker-crash error."""
9:788-    params = json.loads(job.get("params") or "{}")
10:789-    video_id = job.get("video_id") or ""
11:790-    segment_ids = params.get("segment_ids") or []
12:791-    locale = params.get("locale")
17:796-            db, cfg, video_id, segment_ids, params.get("output_filename", "")
22:801-    stitching = getattr(cfg, "stitching", None) or StitchingConfig()
24:803-        getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
26:805-    # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
the job);
27:806-    # unknown/absent locale keeps the configured default unchanged.
28:807-    localized_watermark = localize_watermark(stitching.watermark, locale)
```

28. user (2026-07-05T09:24:42.282Z)

```
4:Cloud Run dispatch + ingest path (``_maybe_dispatch_remote`` / ``_ingest_remote_segments``),
107:def _ingest_remote_segments(
370:        _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
===cloud_run poll===
            video_id=video_id,
            report=report,
            execution=str(execution or ""),
        )

    def _handle_poll_timeout(
        self, done_uri: str, execution: str, timeout_exc: PolicyTimeout
```

```
    ) -> dict:
        """Poll deadline hit ? rescue a just-finished result, else cancel + re-raise.

        The poll budget (``max_wait_sec``, default 1800s) can undercut the deployed job's own
        ``--task-timeout`` (3600s), so a slow-but-healthy worker may finish just past the
deadline:
        ONE final marker check rescues that completed GPU run instead of wasting it. If the
marker
        is genuinely absent, the execution is still running ? best-effort cancel it (otherwise
the
        L4 keeps billing until ``--task-timeout``) and re-raise a ``PolicyTimeout`` carrying the
        execution name so the operator can verify the state in the GCP console. A cancel failure
        NEVER masks the original timeout."""
        try:
            late = self._read_marker(done_uri)
        except Exception:  # noqa: BLE001 ? the final check is best-effort too: absence ?
failure
            LOG.warning(
                "cloud-run: final post-timeout marker check failed ? treating as absent",
                exc_info=True,
            )
            late = None
        if late is not None:
            LOG.warning(
                "cloud-run: job=%s exec=%s done-marker found on the final post-timeout check "
                "? rescuing the completed result",
                self._job_name,
                execution,
===deployment cfg===
    def _chunk_step_must_be_positive(self) -> "IndexingConfig":
        # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-positive
        # step makes `while t < duration: t += step` loop forever, growing memory before any
        # GPU work. Reject the bad config at load time instead.
        if self.chunk_overlap >= self.chunk_duration:
            raise ValueError(
                f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration "
                f"({self.chunk_duration}); otherwise chunking never advances."
            )
        return self


class OutputConfig(BaseModel):
    default_highlights_name: str = "highlights.mp4"
    db_name: str = "sports_indexer.db"
    # Directory where the DB, highlight reels, and processed clips are written.
    # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
    output_dir: str = "output"


class WatermarkConfig(BaseModel):
    """Branding overlay burned into each highlight clip during the per-clip re-encode
    (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
    (under `stitching.watermark` in config.json) to turn it off. The text is fully
    configurable. The concat stage stays stream-copy (-c copy)."""

    enabled: bool = True
    text: str = "Generated by Khelsutra.guru"
    logo_path: str = ""  # optional transparent PNG overlay; "" = no logo
    font_path: str = ""  # optional .ttf; "" = use the fontconfig `font` family below
    font: str = "Sans"  # fontconfig family used when font_path is empty
    font_size_ratio: float = Field(
        default=0.035, gt=0.0, le=0.5
    )  # text height as a fraction of frame height
    opacity: float = Field(default=0.85, ge=0.0, le=1.0)
    box: bool = True  # faint backing box behind the text for legibility
```

```
    margin: int = Field(default=24, ge=0)  # px inset from the chosen corner
    position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = (
        "bottom-right"
    )
```

29. user (2026-07-05T09:24:44.806Z)

```
assets
index.html
THIRD_PARTY_NOTICES.md
ui_version.json
{
  "spa_repo": "https://github.com/avidullu/khelsutra.git",
  "spa_commit": "61e5ccb649955e2177d24818428e7b4795c46724",
  "engine_api_version": 1,
  "content_hash": "da6837b1f1e03f7cfeb3024055c857f787a89221c98a3d4ab9d1b25370caa098",
  "built_at": "2026-07-02T14:37:30Z"
}
===vendor process===
```

30. user (2026-07-05T09:24:46.100Z)

```
780:def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
82:        config.get("storage")
99:    report = dict(cfg.get("report") or {})
100:    if report.get("output_dir"):
142:                    "ending_reason": (row.get("ending_reason") or "").strip(),
144:                sc = (row.get("shot_count") or "").strip()
170:        os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT")
380:        s for s in db.query_play_segments(video_id, {}) if int(s.get("id", 0)) > play_wm
388:        "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
389:        "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
489:            and existing_video.get("status") == "processed"
505:                    "results": existing_video.get("validation_results", []),
544:        segmenter_cls = segmenter_registry.get(seg_name)
632:        prov = response.get("compute", {}) if isinstance(response, dict) else {}
637:            prov.get("path"),
638:            prov.get("dispatched"),
639:            (response or {}).get("status") if isinstance(response, dict) else None,
680:        backend = storage.get_storage(input_ref.backend, config=sc)
724:        meta = s.get("metadata") or {}
725:        sc = meta.get("shot_count") if isinstance(meta, dict) else None
749:    """The bucket URI a compiled reel is mirrored to for a direct signed-URL download
(#463), or "" when
782:    and return the {status, filepath, download_url} result the SPA polls for. Returns a
788:    params = json.loads(job.get("params") or "{}")
789:    video_id = job.get("video_id") or ""
790:    segment_ids = params.get("segment_ids") or []
791:    locale = params.get("locale")
```

31. user (2026-07-05T09:25:36.330Z)

```
7:  2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage layer (D16
? the
36:from backend.infrastructure.execution_policy import (
38:    ExecutionPolicy,
160:        policy: ExecutionPolicy = DEFAULT_POLICY,
169:        self._policy = policy
210:                self._policy,
===capabilities===
234:    _cloud_available,
265:    # "cloud" forces a Cloud Run dispatch (requires deployment.cloud_available). None ?
server
363:def capabilities(request: Request):
```

401:            "cloud_available": _cloud_available(cfg),
===vendor script===
__pycache__
bundle_ui.sh
doctor.py
install.py
nightly_email.py
nightly_reelcount.py
nightly_regression.ps1
nightly_regression.py
README.md
register_nightly_task.ps1
scripts/bundle_ui.sh
scripts/doctor.py
scripts/__pycache__/doctor.cpython-313.pyc
===workflows===
ci.yml
nightly.yml
===docs README head===
- RALLY_QUALITY_RESEARCH.md ? **? Foundation doc** for closing the local-vs-Gemini rally-detection gap: the measured gap + the code-grounded over-merge diagnosis ("shuttle moving ? rally"), an inventory of the eval/experiment harness + its gaps, the strengthened evaluation framework (segmental Edit/F1@k that make over-segmentation first-class, golden-corpus + active learning, Gemini-agreement as a *shadow* signal), an **adversarially-verified, cited external-research synthesis** (TAL/TAS boundary heads, racket-sports rally-state cues, small-data, eval rigor), a **prioritized cheap?durable roadmap with falsifiable success criteria**, and (§7) a **tracked project plan** ? work-item checklist, sequencing, the single quality **number** attributed to the effort, and a **definition of done** (project completes when all tiers land + the number is recorded). The plan that the QUALITY_ITERATIONS.md log executes against; ties together EXPERIMENT_HARNESS / MULTI_SIGNAL_FUSION_PLAN / ANALYZERS_AND_RECONCILERS / OWNED_MODEL_IMPLEMENTATION_PLAN.
- RALLY_DETECTION_GAP_CLOSURE.md ? **Tracked execution doc (2026-06-18)** for closing the remaining rally-detection accuracy gap **per video, before launch**, driven by the growing golden corpus. First ground-truth clip (GX010141) reframed it: boundaries are tight (R2/R3/R4 holds) ? the gap is **detection**: (G1) recall (local & Gemini both miss ~half on hard clips, and *different* rallies ? complementary), (G2) precision (local over-fires), (G3) **rally-END over-extension on the quick-return-after-point** (shuttle still moving ? needs a *player-state* signal). Levers (owner-gated): player-kinematics + shuttle-in-hand / sustained-invisibility end-rule with reverse-timeline backtrack, `rally_gate` Gate A wiring, complementary fusion. **Peer** of `RALLY_QUALITY_RESEARCH.md` (execution tracker, not research); §7 is the source of truth.
- PROMINENT_COURT_DETECTION.md ? **Design + experiment (2026-06-19, default-OFF, owner-gated flip)** for the **multicourt G2 over-fire**: identify the **prominent (foreground) court** (the one covering the majority of the frame) and keep only rallies whose shuttle lives inside it ? re-defining the shuttle signal `(x,y,visible)` ? `(x,y,visible-AND-inside-prominent-court)` *before* windowing, so a background/adjacent-court shuttle never forms a rally. 2D floor polygon (MVP) ? **pseudo-3D play-volume** (refinement, so high clears aren't clipped). Born from the GX010142 rally-#1 background-court FP; reuses `fusion_features.point_in_polygon` + `FusionConfig.court_polygon`. §8 progress tracker is the source of truth.
- RALLY_DETECTION_QUALITY_REPORT.md ? **Technical-paper checkpoint (2026-06-17)** of rally-detection quality: the honest measured state (over-segmentation milestone tolerance-F1 0.091?0.323 at n=11, p=0.002; merge-rate 0.694?0.150; first real-footage ground-truth at-par with START at Gemini parity), the task-faithful R2 metric, a detailed Gemini comparison, next steps + expected wins, areas yet to be explored, and a publishability / path-to-submission verdict. Renders to a polished PDF; the externally-shareable "toward a paper" summary of `RALLY_QUALITY_RESEARCH.md` + `QUALITY_ITERATIONS.md`.
- EVALUATION.md ? How we measure and improve quality.
- EXPERIMENT_HARNESS.md ? Design (P1 SHIPPED) for A/B + shadow + SxS experimentation on rally-detection variants with **zero production regression** ? experiments are decoupled from deploy (new cues default OFF, promote only on measured LOVO lift, rollback = flip a flag). Composes the existing `backend/eval/` machinery (`calibration`, `regression`, `metrics`); implemented in `backend/eval/experiment.py`.
- QUALITY_ITERATIONS.md ? **Living log** of rally-detection quality

iterations (hypothesis ? what shipped ? measured/expected effect ? lesson). The reverse-chronological record; terminal-state snapshots get archived under `archives/quality-iterations/`.
- ANALYZERS_AND_RECONCILERS.md ? Design (owner-gated, NOT started): a **signal-fusion framework** for rally detection ? producers emit confidence-scored **signals** at three temporal **scopes** (frame detectors / window analyzers / session analyzers), a learned **scorer** weights them, and an auditable **report-first reconciliation** pass lints the decision against the evidence. The spine: **no special-casing in the decision path** (signals?scorer, never `if/else`). Worked examples: a service-fault analyzer, a warm-up/practice span detector, and consuming a per-frame shuttle-state detector. A peer to `EXPERIMENT_HARNESS.md` and `MULTI_SIGNAL_FUSION_PLAN.md` (ADR-007 modular rally layer).
- PERSONALIZATION_PLAN.md ? Design (owner-adopted **hybrid**; Phase-0 landing) for the **per-user personalization feature on a generalization-first baseline**: a thin per-user *tuning* layer gated by the honest small-N statistics (a **min-N promotion floor** + a **structural-guard exemption**, never a service gate) + a **contribution flywheel** (free-tier videos pool into the owner-trained baseline; *"a tool that gets better the more you help it"*). Records the **locked owner decisions** (identity, storage, `min_n` semantics, Gate-A default, free-tier pooling). Built so far: per-user promotion gate (#141) + held-out-USER learning curve (#142).

- GOLDEN_REGRESSION_FIXTURES.md ? **Design (owner-gated, NOT started):** video-free, non-attributable regression fixtures so a clean clone runs the rally-seg suite with **no video/GPU/DB**. Two layers ? the post-detector freeze + **dual gates** (characterization + quality-floor) reusing the `backend/eval/` stack, and the **SEALED-FIXTURES** privacy filter (de-attribution-at-source, tiered public/key-gated, shuttle-only public tier). Carries a cited **IN/US/EU legal memo** (derived-data IP/privacy), a threat model, and a **§7 progress tracker + definition-of-done**. Extends [GOLDEN_DATA_SHARING.md](data_pipeline/GOLDEN_DATA_SHARING.md).

? Documentation status & governance

- [AUDIT_REPORT_khelsutra-guru_2026-07-02 (? COMPLETE / ARCHIVED)](archives/past_projects/AUDIT_REPORT_khelsutra-guru_2026-07-02-COMPLETED-2026-07-02.md) ? the five-repo `khelsutra-guru` workspace code audit + its folded-in **Remediation tracker** (P0?P28 all closed). **Archived 2026-07-02** once every remediation wave landed and the last residuals were closed or filed as issues (#443 CORS, #444 corpus Phase-2, #445 GCS eval; + #301/#327/#332/#384). Absorbed the prior `CODE_CLEANUP_AUDIT_2026-06-24` + June-2026 hardening sweep (both archived).
- DOC_STATUS.md ? **? living Documentation Status & Archival Register**: every foundational/tracked doc's lifecycle + archive-readiness, the **archival due-diligence checklist** (*honestly update ? then archive*), and the open doc-hygiene findings. Review here; update rows as docs complete.

Product / platform plans

- ALPHA_LAUNCH_READINESS.md ? **? Active tracked project (`DRAFT`, created 2026-07-03):** cross-repo alpha readiness tracker after the golden corpus reached 15 videos. Separates the hosted product-alpha lane (shareable upload?process?pick?reel) from the owned-model/commercial rally-detection gate (ship target + WASB-C3 still unresolved). §7 is the source of truth for the concrete next PR sequence: corpus portability, n=15 baseline refresh, ablation/fusion reruns, hosted serving, alpha copy/launch review.
- [COMPUTE_DECOUPLED_SERVING/](COMPUTE_DECOUPLED_SERVING/README.md) ? **? Active tracked project (2026-06-21, `DRAFT` owner-gated):** the productionization successor to DECOUPLED_COMPUTE (its deferred M3/M4). **One pure core (DETECT?WINDOW?STITCH) that never branches on deployment profile**, with a `ComputeBackend` (where it runs) + `StorageBackend` (what bytes) selected by **one startup config**. Ships **truly-local** (local/USB + local GPU, in-process stitch, zero cloud) + **cloud-serving** (upload<2GB / gdrive / bucket ? a **Cloud Run GPU job runs detect AND stitch**, metered into a per-job cost ledger) + **cpu-mock** (CI). Includes [TESTING_STRATEGY.md](COMPUTE_DECOUPLED_SERVING/TESTING_STRATEGY.md) (profile-PARITY proof for $0), [architecture.svg](COMPUTE_DECOUPLED_SERVING/architecture.svg), a [runlogs/](COMPUTE_DECOUPLED_SERVING/runlogs/) posterity dir, and **ADR-014** ([lineage ADR-009?010?011?012?013?014](archives/decisions/DECISIONS.md)). §7 M0?M10 tracker is the source of truth.
- [DECOUPLED_COMPUTE/](archives/past_projects/DECOUPLED_COMPUTE/README.md) ? ? **DELIVERED + ARCHIVED (2026-06-21).** Lifted the rally pipeline onto a **rented cloud-native GPU** doing bucket-driven `train` + `infer`: storage/compute seams (PRs #246?#259) + the **M3.5 native WASB

runner**; **M0?M2 + M3.5 proven on a GCP L4** (M2 ? `weights/0.1.0-l4/`). Delivered on **GCP** ? DigitalOcean dropped for compute ([ADR-013](archives/decisions/DECISIONS.md)). **M3/M4 (serving endpoint + per-video billing + scale-to-zero) deferred ? the "Cloud Run + GPU serving" subproject** ([ADR-012](archives/decisions/DECISIONS.md)). Snapshot under `docs/archives/past_projects/DECOUPLED_COMPUTE/` (incl. `DEPLOY_REPORT_GCP_2026-06-20.md`).
- PACKAGING_PLAN.md ? **? Active tracked project (`IN PROGRESS`, 11 PRs merged 2026-06-21/22):** ship the engine as a **downloadable batteries-included Python app** ? `pip install` ? `indexer --app` boots a local webserver and opens the **bundled KhelSutra SPA** single-origin, **torch-free** (LIGHT tier), cross-platform, no native component (decision D10/O1). Built ON the decoupled-compute seam. **Supersedes the `DESKTOP_APP_PLAN.md` narrative.** §7 tracker = source of truth (P0a?e ?, P1a/b ?, P2-launcher/P2a/P2d/P2e ?). **? Next:** P3 GPU validation + screencast ? P2b-install ? P2c wizard (cross-repo) ? P4 (ONNX/train/annotate = north star). Memory `[[packaging-app-plan]]` / `[[next-session-packaging-app]]`.
- DESKTOP_APP_PLAN.md ? **? Superseded narrative (by `PACKAGING_PLAN.md`; D10 = self-hosted webserver, NOT a desktop app).** Original scoping doc (2026-06-18, `[design]`): package the local pipeline as a **cross-platform desktop app** for non-technical players ? plug in the camera's USB/SD card, click once, get back **only the highlight reels**, fully on-device (no upload, no original-video bloat). The centerpiece is the **de-WSL native-compute port** (run WASB natively per-OS: Linux/Windows CUDA · macOS MPS/CPU ? drop WSL2), and its **make-or-break unknown**: is CPU acceptable on a typical *no-GPU* enthusiast laptop (WASB is ~3.4× real-time on a laptop 2070S)? §7 tracker = **P0 compute-feasibility spike ? P1 native detector ? P2 app shell ? P3 packaging/installers**. Realizes the **L0 fully-local** tier of VIDEO_LOCALITY_MODEL.md; is the **`LocalDesktop` ComputeBackend** of PLATFORM_ARCHITECTURE.md §3.2; the smaller/export-clean owned model (OWNED_MODEL_IMPLEMENTATION_PLAN.md) is one answer to the compute crux.

Practical / ongoing

- CREATING_REELS.md ? **? End-user guide:** install the LIGHT-tier app ? `indexer --app` ? process a video ? pick rallies ? download a watermarked reel. Task-first usage walkthrough (web UI + the API chain), with a §6 pointer to the cloud compute option. LIGHT tier only (offline `motion` / optional Gemini; no on-device GPU or one-click cloud). Install mechanics ? root [`README.md`](../README.md); packaging roadmap ? [`PACKAGING_PLAN.md`](PACKAGING_PLAN.md).
- CONVERT_GCS_VIDEOS_TO_HIGHLIGHTS.md ? **? Operator runbook:** `gs://` bucket videos ? highlight reels on the approved **L4 Cloud Run** (scale-to-zero, $0 idle). Copy-pasteable: build image ? create the GPU Job ? cloud-serving control plane ? process(cloud)?query?compile?download ? prove the path (`verify_compute_path.py`) ? $0 teardown. **Verified e2e 2026-06-23.** Raw deploy knobs ? [`../deploy/cloudrun/README.md`](../deploy/cloudrun/README.md).
- SETUP_NEW_MACHINE.md ? **? The canonical "spin up a box ? ready for training & inference" runbook.** Copy-pasteable, human-followable, **verified end-to-end on DigitalOcean (2026-06-20)** ? but **compute is now GCP-only (ADR-013)**, see [`deploy/gcp/README.md`](../deploy/gcp/README.md): credentials ? provision ? bootstrap ? suite (922 passed) ? secret-free CPU inference (no GPU/keys) ? real Gemini inference ? Gen-0 training, with pointers to the GPU/native-detector + bucket paths. The `deploy/digitalocean/*` scripts are provider-agnostic and reused on GCP.
- DATA_IN_GCS.md ? **? The corpus source-of-truth map: where every piece of data lives in `gs://khelsutra-rally-corpus`** (canonical layout · current contents + drift · how `backend.worker` consumes it · the still-local-only **gap** · migration plan). Read this so a session pulls data from the bucket, not from someone's `C:`/`F:`/Drive ? the goal is to remove the local box as a blocker. Pairs with the two runbooks below.
- [`deploy/gcp/nightly_regression/README.md`](../deploy/gcp/nightly_regression/README.md) ? **? Nightly engine-regression OPERATIONS MANUAL (one-stop shop):** enable · run on demand · change any setting (email recipients/frequency/clips/config) · observe · pause/disable/remove. The full WASB-pipeline reel-count+cost+email nightly on an ephemeral GCP GPU VM (design/status: [`NIGHTLY_ENGINE_REGRESSION.md`](NIGHTLY_ENGINE_REGRESSION.md)). The lighter CPU cached-trajectory tripwire is `scripts/nightly_regression.py` (local Windows Scheduled Task, PR #202).
- TRACKNET_WSL_SETUP.md

===NEXT_STEPS head===

Next steps (live) ? refreshed 2026-06-21

> **How to read:** this top section is the **current cross-track state + prioritized pending

list** (refreshed 2026-06-21, replacing the stale 2026-06-13 header). The detailed **owned-model & rally-quality track** plan (2026-06-13) is **preserved below, still valid** ? it is now *one of several* active tracks. Live blow-by-blow: memory `current-state.md`. Doc lifecycle/hygiene: `docs/DOC_STATUS.md`.

? Where we are (2026-06-21)

DECOUPLED_COMPUTE delivered on GCP + archived (#270); **compute = GCP only** (ADR-013, DO dropped). **`COMPUTE_DECOUPLED_SERVING`** subproject spun up (#272, **ADR-014**) ? one pure core + local/cloud profiles; **M0?M9 LARGELY SHIPPED** (ADR-014 ratified 2026-06-21; M1?M4 #279?#283, M5 #286, M6 #287, M8 #288, M9-auth #290 all merged, default-OFF). **Per-video workspace P0** shipped (#264); **doc-status register + archival due-diligence** in place (#273). The **khelsutra site + per-rally UI** ship in parallel (sibling track). The **rally-quality / owned-model** track (below) is the core ML work, gated on **growing the golden corpus**.

> **? LAUNCH DECISION ? NO-GO (owner, 2026-06-30).** The rally-detection product is **not** cleared for an external/commercial launch; we stay in **development**. This is the expected default for the dev phase. Two unblockers gate a future GO: **(1)** set + meet the **ship-gate target (#172)** ? currently unset, needs corpus growth (and the R2 tolF1 ablation, item 8); **(2)** resolve the **WASB shuttle-detector weight provenance** (commercial-clean licensing). P2b (#396) was a do-no-harm quality increment, **not** a launch gate ? merging dev PRs ? launching. Re-decide only when both unblockers clear.

? Prioritized pending ? all tracks

P0 ? highest leverage / unblocks the most

0. **? [owner-priority, 2026-06-21] MAKE IT UI-DRIVEN + ALPHA-SHAREABLE ? "a few recordings lying around ? highlights, from a page, no SSH".** Today the cloud flow is **SSH/CLI-only** (`docs/CLOUD_GPU_DRYRUN_GUIDE.md`); **M12 does NOT fix it.** Needs (a separate khelsutra `web/` SPA + deploy track, NOT an engine seam): **(i)** deploy the engine as a reachable **service** (M7 ? Cloud Run service, ? owner GCP spend for a *persistent* endpoint), **(ii)** the **SPA ingest/progress screen** (khelsutra `web/` B-P2: enter/upload video ? `/api/process` ? poll `/api/jobs` ? reel; ZERO engine code), **(iii)** flip **M9 edge-auth ON** (merged #290, default-OFF) to gate alpha tenants, **(iv)** a **compute PICKER in the UI** ("your machine vs our cloud", D13/D15). The engine seams make it *possible*; the SPA makes it *usable*. Full record: [[ui-driven-cloud-serving-gap]]. **Concrete execution tracker after the n=15 corpus review:** `docs/ALPHA_LAUNCH_READINESS.md` §7.
0b. **? [packaging track, IN PROGRESS ? 11 PRs merged 2026-06-21/22] Downloadable batteries-included app ? `docs/PACKAGING_PLAN.md`.** The LIGHT-tier engine is bundle-ready: `pip install` ? `indexer --app` serves the **bundled SPA** single-origin, **torch-free**, with app-home state / ffmpeg resolver / DNS-rebinding guard / single-instance lock / safe DB upgrades (P0a?e, P1a/b, P2-launcher/P2a/P2d/P2e). **? Next (fresh focused pushes):** **(1)** P3 **GPU validation + screencast** (L4, ~$0.50, DELETE-after, $100 cap ? `[[gcp-vm-always-delete]]`; ? native-WASB 0-rally tuned-config blocker), **(2)** P2b-install (uv/script + truly-local default config), **(3)** P2c first-run wizard (cross-repo khelsutra SPA from `/api/capabilities`), then P3-cloud (Cloud Run) + P4 (ONNX/train/annotate = north star). Memory `[[packaging-app-plan]]` / `[[next-session-packaging-app]]`.
1. ? **`COMPUTE_DECOUPLED_SERVING` M0?M9 LARGELY SHIPPED** (2026-06-21, ADR-014 ratified). M1?M4 (#279/#280/#282/#283), **M5 #286, M6 #287, M8 #288, M9-auth #290** all merged (default-OFF); the **real M7 L4 run** validated the worker (22 rallies, VM deleted, $0). All §8 gating Qs locked (D7?D17). **Owner residual** (not blocking Claude's default-OFF code): counsel ToS/DPA (M9-consent live + M11 live), real GCP prices+FX (M8 calibrate), Google OAuth app (M12 real), CF team-domain/AUD + ingress-lock; real cloud verification authorized within a **$100/?10k cap** (D17).
2. **[owner, ongoing] Grow the golden corpus** ? multi-court badminton + ?3 matches/sport (TT first). The single highest-leverage move; the quality track *and* full-corpus training feed on it.
3. **[owner + Claude · GPU] Full-corpus training (n?5)** ? a real `weights/<ver>/` (the `0.1.0-l4` bundle is n=2 plumbing-only).

P1 ? near-term, mostly Claude-doable (default-OFF, green-gated)

3b. **[Claude] Five-repo audit health remediation ? ? COMPLETE (2026-07-02).** All waves (P0?P28) landed and the last residuals were closed or filed as issues; the umbrella tracker was archived to `docs/archives/past_projects/AUDIT_REPORT_khelsutra-guru_2026-07-02-COMPLETED-2026-07-02.md`. Remaining owner-gated/future work lives as GitHub issues: #443 (CORS default), #444 (corpus Phase-2 upload), #445 (GCS eval plumbing), plus

#301/#327/#332/#384.
4. **[Claude] `COMPUTE_DECOUPLED_SERVING` batch ? REMAINING: M11 + M12** (M5/M6/M8/M9-auth ?
merged this session). **M11** = `backend/flywheel.py` capture?store?shadow-eval?goldenize
plumbing reusing `workspace.py::for_video` + `backend/eval/experiment.compare` +
`backend/eval/promotion.py::promote_if_served_safe`, behind `flywheel.*`=OFF + a faked
`consent_attested` (? privacy-sensitive ? give it fresh focus). **M12** =
`backend/storage/gdrive_api.py` `GDriveApiStorage(StorageBackend)` + a `gdrive-api://` scheme
(regex already allows the hyphen; add to `_KNOWN` + `_backend_class` + the StorageConfig
Literal) against an INJECTED fake Drive (mirror `gcs.py` DI); keep the mount-only `gdrive://`
untouched. ? **M9-consent cols = a new v7 migration** (M8 took v6). M9-consent ToS/DPA text =
owner/counsel.

32. user (2026-07-05T09:28:01.175Z)

===done_uri persisted?===

33. user (2026-07-05T09:28:02.560Z)

#!/usr/bin/env bash
Bundle the KhelSutra web SPA into the engine so a downloaded engine serves the rea
single-origin (PACKAGING_PLAN.md P1b).
#
OPTION 1 (now, owner-chosen): build the SPA from a khelsutra checkout and vendor
into `backend/ui/`, which is COMMITTED ? so the wheel + CI build fully offline/reprodu
the snapshot by re-running this on an SPA release; commit the diff.
#
OPTION 3 (designed-for, future): point --from at a published dist tarball/URL (a GitHub
artifact) to vendor a pre-built SPA without a local khelsutra checkout or Node. Already
#
Usage:
scripts/bundle_ui.sh [SOURCE]
SOURCE = a khelsutra checkout dir (default: $KHELSUTRA_DIR or ../khelsutra) [bu
| a path/URL ending .tgz/.tar.gz [unpacks it]
BUILD=0 scripts/bundle_ui.sh <khelsutra-dir> # skip npm; reuse an existing web/dis

```
set -euo pipefail

ENGINE_ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
UI_DEST="$ENGINE_ROOT/backend/ui"
SOURCE="${1:-${KHELSUTRA_DIR:-$ENGINE_ROOT/../khelsutra}}"

stage_dist() {  # populates $1 (a dist dir) into a clean $UI_DEST
  local dist="$1"
  [ -f "$dist/index.html" ] || { echo "ERROR: no index.html in $dist ? not a built SPA dist"
>&2; exit 1; }
  rm -rf "$UI_DEST"
  mkdir -p "$UI_DEST"
  cp -r "$dist"/. "$UI_DEST"/
}

SPA_REPO="unknown"
SPA_COMMIT="unknown"

case "$SOURCE" in
  *.tgz|*.tar.gz|http://*|https://*)
    # OPTION 3: a published dist artifact.
    tmp="$(mktemp -d)"; trap 'rm -rf "$tmp"' EXIT
```

```
    if [[ "$SOURCE" == http*://* ]]; then
      echo "[bundle_ui] fetching dist artifact: $SOURCE"
      curl -fsSL "$SOURCE" -o "$tmp/dist.tgz"; SRC_TGZ="$tmp/dist.tgz"; SPA_REPO="$SOURCE"
===DoD #1===
```

34. user (2026-07-05T09:28:27.732Z)

```
228:    def claim_or_create_process_job(
547:        NOTE: ``recover_stale_jobs`` is the preferred startup call ? it RE-QUEUES stale
564:    def recover_stale_jobs(
65:        video_id: str,
75:        NOT NULL column) and `video_id` is set up-front (so /api/jobs surfaces it
immediately). The
85:            "INSERT INTO jobs (id, video_path, video_id, status, owner_id, locale, kind,
params) "
87:            (job_id, video_path, video_id, owner_id, locale, params),
120:    def set_job_video_id(self, job_id: str, video_id: str) -> bool:
122:            "UPDATE jobs SET video_id=? WHERE id = ?",
123:            (video_id, job_id),
124:            f"Failed to set video_id for job {job_id}",
184:            "SELECT id, kind, status, progress, video_id, video_path, compute, error,
result, "
411:                    "SELECT id, video_id, owner_id, compute_backend, gpu_type,
gpu_active_seconds, "
===create_job signature===
23:    def create_job(
```

35. user (2026-07-05T09:31:00.880Z)

```
Preparing worktree (new branch 'docs/cp-durable-rebuild-spec')
branch 'docs/cp-durable-rebuild-spec' set up to track 'origin/master'.
HEAD is now at 0a7b3ff feat(jobs): processing-queue endpoint + idempotent /api/process dedup
(#448) (#481)
docs/cp-durable-rebuild-spec
0a7b3ff feat(jobs): processing-queue endpoint + idempotent /api/process dedup (#448) (#481)
```

36. assistant (2026-07-05T09:31:09.832Z)

```
Worktree ready on `0a7b3ff`. Writing the spec now ? this is the main deliverable.
```

37. assistant (2026-07-05T09:33:10.745Z)

Control-Plane Durable Rebuild ? kill the upload OOM, ground state in the bucket, unbl

```
> **Status:** `DRAFT` ? **owner-confirm gate: P9 (the deploy) does NOT execute until the owner
confirms
> this spec** · **Owner:** avidullu · **Created:** 2026-07-05 · **Last updated:** 2026-07-05
> **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when DONE)
> **Tracking anchors:** §7 progress tracker is the source of truth; indexed in `docs/README.md`;
> pointer in memory `current-state` (+ `docs/NEXT_STEPS.md` once work starts post-confirmation).
> **Relation to existing docs:** executes the serving hardening that
> [ALPHA_LAUNCH_REVIEW_2026-07-04.md](ALPHA_LAUNCH_REVIEW_2026-07-04.md) §1.1 (the 5-minute
browser
> CUJ ? tracker **DoD #1**) is blocked on; child of
> [COMPUTE_DECOUPLED_SERVING/](COMPUTE_DECOUPLED_SERVING/README.md) (ADR-014); consolidates
issues
> #471 #480 #482 #483 #484 #485 (plus the already-shipped #479-config, #481, and khelsutra
> #153/#154(#155)/#157(#158)).
> **Honesty note:** each claim is tagged `[verified]` (checked against code or live logs),
> `[researched]` (external platform fact ? re-verify during execution), or `[design]`
(proposed).

---
```

0. TL;DR

The first real owner-driven browser CUJ (2026-07-05) proved the product works end-to-end ? 29 rallies
detected, pickable in the UI ? **when the control plane survives**. It also proved the deployed CP
does not survive real usage: a 2.68 GB browser upload staged into RAM-backed `/tmp` on a **4 GiB**
instance ? OOM ? restart ? ephemeral SQLite wiped ? the 29 rallies (already written to the bucket)
were never ingested ? empty UI, unrecoverable by reload `[verified]`. This spec consolidates the fixes
into **one image rebuild + redeploy**: ?8 GiB CP with upload staging **streamed to the bucket** (never
assembled in tmpfs) · **bucket-grounded durable state** (the DB becomes a cache that re-hydrates from
`outputs/<md5>/`) · **submit-time MD5** (makes re-runs instant AND restarts recoverable ? it is the
key that re-associates an orphaned job with its output) · longer, honest poll ceilings · the rev-00011
config pins **baked into the committed manifest generator** so a regenerate can never silently drop
them · the current SPA (#153/#154/#158) re-vendored. The deploy step (**P9**) is owner-gated on
confirming this spec; the code rows (P1?P8) are each one small PR.

1. Problem & goal

What happened (2026-07-05, all diagnosed live ? full record in memory [[alpha-cuj-testing-2026-07-05]]):

1. 95 MB upload parts were 413'd by the Google Front End ? the service has no HTTP/2 annotation, so
   ingress is HTTP/1 with a ~32 MiB per-request body cap (#480). Config rev 00011 set
   `security.max_part_mb=24` and small uploads started working `[verified]`.
2. Cloud jobs ran `yolo_hybrid`, not the validated `wasb_hybrid` (#479) ? the engine default
   (`backend/config/models.py:181`) was never pinned. Config rev 00011 pinned it `[verified]`.
3. **The killer:** a 2.68 GB upload assembled into the CP's RAM-backed `/tmp` on a 4 GiB instance ?
   OOM ? restart 08:14Z ? the ephemeral SQLite **and the in-memory done-marker poll** were wiped
   (#471). The worker finished (`done rc=0` 08:59:51Z) and wrote 29 rallies to the bucket, but no CP
   was listening ? never ingested ? the UI showed nothing and reload could not recover it
   `[verified]`. The reel was recovered manually from `outputs/<md5>/intervals.csv`.
4. Timeouts are dishonest for real footage: the SPA gives up at 10 min
   (`web/src/hooks/useIngestJob.ts:18`, `web/src/api/client.ts:181` in the khelsutra repo) and the CP
   poll budget is 30 min (`backend/infrastructure/execution_policy.py:54`), while a 6-min 1080p60 clip
   takes WASB ~22 min on the L4 `[verified]`.
5. No MD5 reuse: the same clip was detected **3×** that day ? three paid L4 runs for one video
   (#484) `[verified]`.
6. The gs:// re-run CUJ **PASSED in the UI** (29 rallies pickable) because nothing staged into tmpfs ?
   the product works when the CP survives `[verified ? owner screenshot]`.

Goal. A tester can, from a browser at `https://api.khelsutra.guru` (edge-auth ON), upload a real
2?4 GB phone/action-cam video, watch honest progress, pick from detected rallies, and download a
full-quality reel ? reliably, **including across control-plane restarts and re-submissions of the
same video**. That is tracker **DoD #1** (ALPHA_LAUNCH_REVIEW_2026-07-04.md
§1.1), the last engineering blocker for the 4-tester alpha.

Read first: [[alpha-cuj-testing-2026-07-05]] · issues #480/#471/#484/#485/#482/#483 · ../deploy/cloudrun/README.md §3/§3a · `deploy/cloudrun/gen_service_yaml.py`.

2. Decisions locked

#	Decision	Source / date	Implication
D1	ONE consolidated image rebuild + redeploy, not more config patches	owner, 2026-07-05	config redeploys (rev 00011) cannot touch the bundled SPA, the instance shape, or tmpfs staging
D2	CP memory **8 GiB** (cpu stays 2)	owner ask ("?8 GiB")	covers compile scratch (up to a 4 GiB source download + reel + app) with headroom; >8 GiB would force 4 vCPU `[researched]` ? not needed once staging leaves tmpfs. §8 Q2 offers the 16 GiB option
D3	Upload staging leaves RAM `/tmp`: parts **stream into a GCS resumable object**, never a local assembled file	owner ask ("disk-backed, NOT RAM /tmp"); Cloud Run services have no persistent local disk `[researched]` ? the bucket IS the disk	kills the OOM class and #480's tmpfs ceiling; upload size bounded by policy (4096 MB), not memory
D4	**Bucket = source of truth; SQLite = cache** (re-hydrate on miss)	owner ask; #471 option 2 realized cheaply via #485	restarts/redeploys stop losing results; `?video=<md5>` always lands on grounded state
D5	rev-00011 config pins (`indexing.default_segmenter=wasb_hybrid`, `security.max_part_mb=24`) move INTO `gen_service_yaml.py::build_config`	this spec	a regenerated manifest can never silently drop the live posture again (today the committed generator does NOT carry them `[verified]`)
D6	**HTTP/2 ingress (`use-http2`) REJECTED** for this rebuild	this spec	h2c requires the container to speak cleartext HTTP/2 ? uvicorn does not `[researched]`; the annotation would break the service. 24 MB parts under the ~32 MiB HTTP/1 cap remain the mechanism
D7	Direct browser?GCS signed-URL upload (#480 option 4) **deferred, not rejected**	this spec	needs bucket CORS + an SPA upload rewrite + a signed-PUT auth story; D3 already removes both caps for ?4 GB. Revisit for >4 GB files / multi-tenant
D8	`maxScale` stays **1**	#471 option 1 (standing, generator default `[verified]`)	this work makes restarts lossless; the in-process job queue is still per-instance. Multi-instance = future shared job store
D9	Alpha posture unchanged: WASB-only (`skip_ai_handoff=true`), edge-auth ON for testers, `min-instances 1`, `--no-cpu-throttling`	standing (A13 / README §3a)	the rebuild changes infra, not detection or auth
D10	Any quality-affecting flip (#483 proxy) ships a **Launch Report** gated on the golden set	standing convention [[launch-report-convention]]	honest measured results only; most-conservative-passing setting wins

3. Foundation ? measured evidence this plan rests on

- **Failure chain + gs:// pass** ? `[verified]` live gcloud logs + owner screenshots, 2026-07-05 (§1; memory [[alpha-cuj-testing-2026-07-05]]).
- **Downsample experiment (validates #483):** dev RTX 2070 Super GPU-transcoded the full 5.9-min 1080p60 clip to 1080p30 in **36 s (~10× realtime)** via NVDEC?`scale_cuda`?NVENC vs ~3.5 min CPU libx264 `[verified]`. NVDEC/NVENC are dedicated hardware blocks ? they don't contend with CUDA inference. Expected effect: detection ~22 min ? ~5 min at a 30 fps proxy.
- **WASB throughput:** ~25 fps on the L4 (a 6-min 1080p60 clip ? 21 600 frames ? 22 min) `[verified 2026-07-05]`.
- Platform facts `[researched ? re-verify during P9]`: Cloud Run service volumes are in-memory emptyDir / GCS FUSE / NFS only (no persistent local disk); memory up to 8 GiB runs on 2 vCPU, above 8 GiB requires 4; GCS non-composite objects expose `md5Hash` metadata; GCS resumable upload sessions remain valid across minutes-long gaps between chunks (they are designed for exactly this).

4. Design

4.1 Upload staging ? stream to the bucket, never assemble in tmpfs (P2 · fixes the OOM class + #480 ceiling)

Today `[verified]`: `backend/api/uploads.py` appends each ?24 MB part to `.partial-<upload_id>` under
`security.upload_dir`, resolved by `resolve_output_dir()` under `RALLY_APP_HOME=/tmp/apphome` ? RAM
tmpfs. `/api/upload/complete` finalizes the local file; `/api/process` (#467) then stages it to
`gs://?/videos/<video_id>/`. The whole video transits instance memory twice.

After `[design]`: when `storage.input_backend == "gcs"`, `upload_init` opens a **GCS resumable writer** (`google-cloud-storage` `BlobWriter`, chunked) at
`gs://<input_root>/uploads/<upload_id>/<filename>`;
each part's chunks are fed through it (instance memory = one chunk buffer, ~24?40 MB) while an **incremental MD5** accumulates; `complete` closes the writer and returns the `gs://` URI + md5.
`/api/process` receives a URI that is **inside the #465 input jail by construction** (under `input_root`); the #467 local?gs staging step becomes a server-side GCS rewrite to
`videos/<md5>/?` (no bytes through the CP). Local-file staging is unchanged for the local/appliance profiles (selected by config). The `require_free_bytes` 507 guard is bypassed for
bucket staging (no local bytes consumed).

Unchanged semantics: upload-session state stays in-process ? a CP restart mid-upload aborts that upload and the client re-inits (documented alpha behavior). Abort/cleanup deletes the partial object;
a bucket **lifecycle rule** on the `uploads/` prefix catches orphans (P9 ops step).

Reuse map: `backend/api/uploads.py` (router, caps, sequential-part protocol ? kept) ·
`backend/storage/gcs.py` (client + config threading patterns from #467/#476) ·
`backend/utils/hashing.compute_video_id` (add a streaming/incremental variant) ·
`backend/utils/freespace.py` (guard becomes backend-aware).

4.2 Durable, bucket-grounded state ? DB = cache, bucket = truth (P3+P4 · #471 #484 #485)

Everything durable already lands under content-addressed prefixes `[verified]`: `videos/<md5>/`, `outputs/<md5>/report.json` + `intervals.csv`, `highlights/<md5>/`. Only the DB (ephemeral SQLite
under app-home tmpfs) forgets.

1. **Submit-time MD5 (P4).** Browser uploads: the md5 accumulates during 4.1 streaming (free).
   `gs://` submissions: read the object's `md5Hash` metadata (instant `[researched]`; bounded
   stream-hash fallback for composite objects). Set `video_id` on the job row at CREATE ? the column
   and `set_job_video_id` already exist `[verified]`
(`backend/infrastructure/database_jobs.py:65,120`);
   today the value is only filled after the worker reports back.
2. **Completed-work cache-reuse (P4 · #484).** In `/api/process`, before dispatch: if
   `outputs/<md5>/report.json` exists with `status=success` and `force` is not set ? **re-hydrate**
   (below) and return a `done` job immediately ? no GPU run. #481's merged active-job dedup
   `[verified ? master 0a7b3ff]` already collapses concurrent duplicates; this adds the
   completed-work tier. Makes "MD5 ? instant" true.
3. **Re-hydrate = reuse the existing ingest.** `_ingest_remote_segments(db, video_id, outputs_uri,
   storage_dict)` (`backend/orchestration.py:107`, normal post-poll call site `:370`
`[verified]`) is
   already the bucket?DB ingestion. A small wrapper exposes it to: (a) the P4 cache hit; (b) the
   read path ? segments/query/highlights lookups that miss the DB attempt a bucket re-hydrate
   before 404ing, so `?video=<md5>` always lands on grounded state (#485); (c) startup recovery.
4. **Restart-safe remote jobs (P3).** Today the done-marker poll (`_dispatch/<token>.done`) lives
   only in the drain thread's memory; the token is random and **nothing persists it**, and pre-#484
   the job row didn't even know its `video_id` ? after a restart the CP cannot re-associate a running
   job with its output. That is the exact 2026-07-05 loss `[verified]`. With md5 on the row from
   submit, extend `recover_stale_jobs` (`database_jobs.py:564` `[verified]`): on startup,

`running`
   remote process jobs re-arm a poll against `outputs/<md5>/report.json` (the durable done
signal),
   bounded by the Cloud Run execution's terminal state via the existing `google-cloud-run`
client so
   a genuinely failed worker doesn't poll forever; ingest + complete on success `[design]`.

Out of scope (D8): a shared job store for multi-instance. `maxScale=1` stands.

4.3 Timeouts ? outlast the honest job length (P5a engine · P5b SPA)

- **CP dispatch poll budget:** `ExecutionPolicy.max_wait_sec` defaults to 1800 s `[verified]`
  (`execution_policy.py:54`; `CloudRunCompute` accepts an injectable policy,
  `backend/compute/cloud_run.py:160`). That undercuts both the worker's `--task-timeout 3600`
and
  real 1080p60 jobs (~22 min + queue/cold-start). The `_handle_poll_timeout` rescue re-checks
once
  `[verified]`, but the budget itself is wrong. **Change:** thread a
  `deployment.remote_poll_max_wait_sec` config knob through to the policy, default **3900 s** ?
  above `--task-timeout` so the worker's own timeout stays authoritative and the rescue window
  still applies `[design]`.
- **SPA ceilings (khelsutra):** `POLL_CEILING_MS` and `COMPILE_POLL_CEILING_MS` are both 10 min
  `[verified]`. **Change:** raise both to **60 min** now (interim); once #482's heartbeat lands,
  replace the absolute ceiling with a **stall detector** (give up only when `progress` hasn't
  advanced for ~15 min) so any honest video length works `[design]`.
- Cloud Run service `timeoutSeconds: 300` stays ? tester requests are short polls and ?24 MB
parts
  `[verified]`.
- Worker `--task-timeout 3600` stays for now; #483 (below) is the real fix for long footage, not
a
  bigger timeout `[design]`.

4.4 Observability ? no more blind waiting (P6 · #482)

WASB runner emits periodic progress (frames done / % / rough ETA) ? `set_job_progress` ?
`/api/jobs/{id}.progress` returns e.g. `"segmenting 42% (~6 min left)"` ? the SPA shows it (and
it
feeds the 4.3 stall detector). A per-job **runlog artifact** (phases, timings, counts) is
written
alongside `outputs/<md5>/` for after-the-fact triage. `[design ? issue #482 carries the full
ask]`

4.5 Cost ? downsample-for-detection (P7 · #483 · golden-gated)

Detection does not need 1080p60; the reel does. Worker-side: when estimated processing time
crosses a
threshold (from res·fps·duration vs the known ~25 fps L4 throughput), GPU-transcode a proxy
(candidate: 720p30 ? the golden gate picks the most aggressive setting that holds quality), run
WASB
on the **proxy**, map rally timestamps back, cut the reel from the **ORIGINAL**. User-visible
copy
explains it. **Gate (D10):** golden-set (n=15) F1/tolF1 at the proxy settings vs baseline; ship
default-ON only on a no-material-regression result, with a Launch Report. Payoff: ~22 min ? ~5
min
on the 2026-07-05 clip class, GPU cost cut proportionally, and the 4.3 timeout pressure mostly
evaporates. `[design; experiment evidence §3]`

4.6 Manifest generator v2 ? the pins become code (P1)

`deploy/cloudrun/gen_service_yaml.py::build_config()` gains
`indexing.default_segmenter="wasb_hybrid"`
and `security.max_part_mb=24` (CLI-overridable), and `--memory` default moves `4Gi ? 8Gi` (D2).
Explicitly NOT added: `use-http2` (D6), `CLOUD_RUN_JOB` (reserved ? standing), a maxScale bump
(D8).

`--no-cpu-throttling`, `minScale=1`, `timeoutSeconds=300` stand (D9/§4.3). After P1, regenerating the
manifest reproduces the live rev-00011 posture from a clean clone `[design]`.

4.7 SPA re-vendor (P8)

`backend/ui/` is the committed snapshot the CP serves single-origin; it is pinned at khelsutra `61e5ccb` (2026-07-02) `[verified ? backend/ui/ui_version.json]`, which **predates** the merged #153
(client-side upload guard), #155 (the #154 copy blockers), and #158 (ingest UX + the medium-picker
trap fix ? the live build still has the trap). After P5b merges: run `scripts/bundle_ui.sh <khelsutra-checkout>` and commit the `backend/ui/` diff; P9's CUJ verifies `ui_version.json.spa_commit` equals the intended khelsutra master commit.

4.8 Rebuild + redeploy runbook (P9 ? owner-gated)

Per ../deploy/cloudrun/README.md §1/§3a, gcloud from PowerShell:

1. `gcloud builds submit --config=deploy/cloudrun/cloudbuild.yaml` (CUDA image, ~8?10 min).
2. Re-pin the worker Job: `gcloud run jobs update rally-worker --image ?/rally/rally-worker:latest`
   (first execution after a push spends minutes in image import ? that's progress, not a hang).
3. `python deploy/cloudrun/gen_service_yaml.py --edge-auth on -o service.yaml` (now carries the pins
   + 8Gi) ? `gcloud run services replace service.yaml --region asia-southeast1`. ? If the manifest
   is textually unchanged from the live revision, `services replace` no-ops ? use
   `services update --image` to force `:latest` re-resolution (the 2026-07-04 lesson).
4. Add the bucket lifecycle rule for the `videos/uploads/` staging prefix (age ?2 days).
5. Verify §9 with a real browser CUJ; confirm edge-auth posture (process sans JWT ? 401;
   `api.khelsutra.guru` ? CF login 302).
6. Write `docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_cp-rebuild_<date>.md`
   (a NEW file); update the trackers + memory.

5. Risk table

Risk	Exposure	Mitigation
GCS resumable writer held open across part requests (minutes-long gaps) misbehaves	upload fails mid-flight	sessions are designed for this `[researched]`; abort deletes the partial; P9 canaries a >2.5 GB browser upload while watching instance memory
`BlobWriter` buffering profile bigger than expected	OOM recurrence	D2's 8 GiB headroom + the P9 memory watch; fallback = smaller writer chunk_size
Re-hydrate ingests bucket content	poisoned outputs would enter the DB	re-hydrate reads ONLY under the service's own `output_root` (operator-owned bucket ? the same trust domain as today's post-poll ingest); no user-controlled URI reaches it
`md5Hash` metadata absent (composite objects)	slow submit pre-flight	bounded stream-hash fallback; browser uploads always carry the streamed hash
Proxy detection regresses quality (#483)	worse rallies for testers	D10 golden gate + Launch Report before default-ON; most-conservative-passing setting
`[researched]` platform facts drift	deploy surprises	each is re-verified live during P9 before testers return
Restart DURING an upload	that upload aborts	unchanged, documented alpha semantics; client re-inits (P2 keeps behavior parity)

6. Honest limits ? what this does NOT do

- Does NOT make the CP multi-instance: the in-process queue is still per-instance; `maxScale=1` stands (D8). #471's shared-store option remains open, deliberately.
- Does NOT raise the 4096 MB upload policy cap. Files >4 GB still take the `gs://` path; raising the
  cap rides D7 (direct browser?GCS), not this rebuild.

- A restart mid-UPLOAD still aborts that upload. Durability starts at dispatch (P3) and covers
  results forever after (P4/#485).
- The golden set (n=15) can only prove #483 on footage classes it contains; new footage classes
get
  the same honest-gate treatment later.
- A green §9 checklist is OUR CUJ evidence. The official 5-minute DoD #1 pass ? and GO/NO-GO ?
stay
  with the owner (ALPHA_LAUNCH_REVIEW_2026-07-04.md
§0/§1.1).
- Wider-invite gates are explicitly OUT of scope: least-privilege SA (review §5.5), retention
  #301/#447, policy processor-list / acceptable-use #332, kn/hi native review (khelsutra#156),
  A10/A11.

7. Deliverables & progress tracker ? **source of truth**

Legend: ? Todo · ? In progress · ? Done · ? Blocked/gated. **One small PR per row**, branched
from
`origin/master`, no stacking (shared-checkout discipline).

ID	Deliverable	Repo	Depends on	Gated?	Status	PR
P0	This spec; owner confirmation opens P9	engine	?	**Owner**	?	(this PR)
P1	`gen_service_yaml.py` v2: rev-00011 pins in `build_config` + `--memory` 8Gi default
engine	P0	No	?	?
P2	GCS-streamed upload staging (`BlobWriter` under `<input_root>/uploads/`, streamed md5,
backend-aware 507 guard, abort cleanup)	engine	P0	No	?	?
P3	Restart-safe remote jobs: md5-keyed startup recovery (extend `recover_stale_jobs`) +
read-path re-hydrate via `_ingest_remote_segments` (#471 #485)	engine	P4	No	?	?
P4	Submit-time MD5 on the job row + completed-work cache-reuse with `force` bypass (#484)
engine	P2 (upload hash)	No	?	?
P5a	`deployment.remote_poll_max_wait_sec` knob (default 3900) + emit
`max_upload_mb`/`max_part_mb` in `/api/capabilities` (#480 item 5 ? revives the SPA pre-flight)
engine	P0	No	?	?
P5b	SPA: both poll ceilings 10 ? 60 min + capabilities-driven upload pre-flight	khelsutra
P5a	No	?	?
P6	#482 progress heartbeat ? `job.progress` + per-job runlog (SPA stall-detector rides the
next SPA PR)	engine (+khelsutra)	P0	No	?	?
P7	#483 downsample-for-detection behind config; default set by the golden gate	engine	P0
Golden gate + Launch Report (D10)	?	?
P8	Re-vendor the SPA snapshot (`scripts/bundle_ui.sh`) at khelsutra master ? P5b	engine
P5b	No	?	?
P9	**REBUILD + REDEPLOY + browser-CUJ verify + runlog** (§4.8, §9)	ops	P1?P5b, P8 (P6/P7
if ready)	**Owner: spec confirm + deploy/spend authorization**	?	?

Bundling note: P6/P7 ride the same image if ready before P9 executes; otherwise they trigger
a
cheap follow-up rebuild. **P9 must not wait on them** ? it is the DoD #1 blocker.

8. Open questions ? owner

1. **Confirm this spec + authorize P9** (image build + redeploy in the existing project/region;
   deploy-scale spend only ? no new resources). ? **the blocking gate**
2. Memory: **8 GiB / 2 vCPU (recommended)** or 16 GiB / 4 vCPU headroom? (>8 GiB forces 4 vCPU
   `[researched]` ? 2× the always-on instance cost; streamed staging removes the main driver for
it.)
3. #483 bundling: hold P9 for the golden gate? **Recommendation: NO** ? ship P9 when P1?P5b/P8
land;
   #483 follows in the next image.
4. SPA interim ceiling 60 min OK (stall-based detection follows #482)?
5. Upload policy cap stays 4096 MB for the alpha (raising it rides D7)?

9. Definition of done

- [ ] From a browser at `api.khelsutra.guru` (edge-auth ON): a **?2.5 GB 1080p60** upload

completes ?
      no 413, no OOM, no restart; instance memory observed well under the limit during staging.
- [ ] The job runs **`wasb_hybrid`** (P1 pin, confirmed in worker logs) and the SPA stays with
it
      past 10 minutes without giving up.
- [ ] Progress is visible during detection (#482) ? or, if P6 slipped to the follow-up image,
the
      interim copy states the expected wait honestly.
- [ ] Rallies render pickable; compile ? 307 ? signed-URL download of the full reel (>32 MiB)
works.
- [ ] **Durable-state proof (the 2026-07-05 failure shape):** force a CP restart after worker-
done
      but before ingest ? reload `?video=<md5>` ? the rallies appear (bucket re-hydrate).
- [ ] **Cache proof:** re-submitting the same video returns `done` with NO new Job execution in
the
      logs; `force=true` still re-runs.
- [ ] Concurrent duplicate submits collapse to one job (#481 observed live).
- [ ] Edge-auth posture verified (401 sans JWT; CF login on the domain); `ui_version.json` shows
the
      intended khelsutra commit; the rev-00011 pins are present in the live config.
- [ ] `DEPLOY_REPORT_cloud-serving_cp-rebuild_<date>.md` runlog committed; trackers + memory
updated;
      owner runs the official §1.1 five-minute CUJ.

When all boxes tick ? flip **Status ? DONE** and archive per `docs/DOC_STATUS.md`.

10. References

Internal code anchors `[verified]`: `backend/api/uploads.py` (init/part/complete,
`_upload_cfg`) ·
`backend/utils/app_home.py::resolve_output_dir` ·
`backend/utils/freespace.py::require_free_bytes` ·
`backend/orchestration.py:107` `_ingest_remote_segments` (call site `:370`) ·
`backend/compute/cloud_run.py:160` (injectable `ExecutionPolicy`), `_handle_poll_timeout` ·
`backend/infrastructure/execution_policy.py:54` ·
`backend/infrastructure/database_jobs.py:65,120`
(`video_id` column + setter), `:228` (`claim_or_create_process_job`, #481), `:564`
(`recover_stale_jobs`) · `backend/config/models.py:181` (`default_segmenter`), `:370-371`
(`max_upload_mb`/`max_part_mb`) · `backend/main.py:363` (`capabilities`), `:169`
(`_BUNDLED_UI_DIR`) ·
`deploy/cloudrun/gen_service_yaml.py` · `scripts/bundle_ui.sh` · khelsutra:
`web/src/hooks/useIngestJob.ts:18`, `web/src/api/client.ts:181,320`.
Internal docs: ../deploy/cloudrun/README.md §3/§3a ·
ALPHA_LAUNCH_REVIEW_2026-07-04.md §1.1 ·
[COMPUTE_DECOUPLED_SERVING/](COMPUTE_DECOUPLED_SERVING/README.md) + its runlogs ·
PROJECT_DOC_TEMPLATE.md.
Issues: engine #471 #479 #480 #481(merged `0a7b3ff`) #482 #483 #484 #485; khelsutra #153
#154
(#155) #157 (#158) #156.
Memory: [[alpha-cuj-testing-2026-07-05]] · [[session-handoff]] · [[launch-report-
convention]].

Changelog
- 2026-07-05 ? created (`DRAFT`): consolidates the 2026-07-05 owner-CUJ findings into the single
  rebuild plan; P9 owner-gated on spec confirmation.

38. user (2026-07-05T09:33:10.781Z)

File created successfully at: C:\Users\avidu\Projects\khelsutra-guru\wt-cp-rebuild-
spec\docs\CONTROL_PLANE_DURABLE_REBUILD.md (file state is current in your context ? no need to
Read it back)

39. assistant (2026-07-05T09:33:17.131Z)

Spec written. Now indexing it in `docs/README.md` ? reading the exact insertion region first.

40. user (2026-07-05T09:33:18.611Z)

18:[Omitted long matching line]

41. user (2026-07-05T09:33:24.458Z)

17   ## Product / platform plans
18   - ALPHA_LAUNCH_READINESS.md ? **? Active tracked project (`DRAFT`, created 2026-07-03):** cross-repo alpha readiness tracker after the golden corpus reached 15 videos. Separates the hosted product-alpha lane (shareable upload?process?pick?reel) from the owned-model/commercial rally-detection gate (ship target + WASB-C3 still unresolved). §7 is the source of truth for the concrete next PR sequence: corpus portability, n=15 baseline refresh, ablation/fusion reruns, hosted serving, alpha copy/launch review.
19   - [COMPUTE_DECOUPLED_SERVING/](COMPUTE_DECOUPLED_SERVING/README.md) ? **? Active tracked project (2026-06-21, `DRAFT` owner-gated):** the productionization successor to DECOUPLED_COMPUTE (its deferred M3/M4). **One pure core (DETECT?WINDOW?STITCH) that never branches on deployment profile**, with a `ComputeBackend` (where it runs) + `StorageBackend` (what bytes) selected by **one startup config**. Ships **truly-local** (local/USB + local GPU, in-process stitch, zero cloud) + **cloud-serving** (upload<2GB / gdrive / bucket ? a **Cloud Run GPU job runs detect AND stitch**, metered into a per-job cost ledger) + **cpu-mock** (CI). Includes [TESTING_STRATEGY.md](COMPUTE_DECOUPLED_SERVING/TESTING_STRATEGY.md) (profile-PARITY proof for $0), [architecture.svg](COMPUTE_DECOUPLED_SERVING/architecture.svg), a [runlogs/](COMPUTE_DECOUPLED_SERVING/runlogs/) posterity dir, and **ADR-014** ([lineage ADR-009?010?011?012?013?014](archives/decisions/DECISIONS.md)). §7 M0?M10 tracker is the source of truth.
20   - [DECOUPLED_COMPUTE/](archives/past_projects/DECOUPLED_COMPUTE/README.md) ? ? **DELIVERED + ARCHIVED (2026-06-21).** Lifted the rally pipeline onto a **rented cloud-native GPU** doing bucket-driven `train` + `infer`: storage/compute seams (PRs #246?#259) + the **M3.5 native WASB runner**; **M0?M2 + M3.5 proven on a GCP L4** (M2 ? `weights/0.1.0-l4/`). Delivered on **GCP** ? DigitalOcean dropped for compute ([ADR-013](archives/decisions/DECISIONS.md)). **M3/M4 (serving endpoint + per-video billing + scale-to-zero) deferred ? the "Cloud Run + GPU serving" subproject** ([ADR-012](archives/decisions/DECISIONS.md)). Snapshot under `docs/archives/past_projects/DECOUPLED_COMPUTE/` (incl. `DEPLOY_REPORT_GCP_2026-06-20.md`).

42. user (2026-07-05T09:33:36.657Z)

The file C:\Users\avidu\Projects\khelsutra-guru\wt-cp-rebuild-spec\docs\README.md has been updated successfully. (file state is current in your context ? no need to Read it back)

43. assistant (2026-07-05T09:33:44.354Z)

Index entry added. Checking what CI validates on docs changes so I can run the same locally before pushing.

44. user (2026-07-05T09:33:46.103Z)

51: # Internal-link guard (tools/check_md_links.py): fails on a broken internal markdown link
52: # (missing file or dead #anchor) in a LIVE doc. docs/archives/** is excluded ? frozen
53: # snapshots whose relative links are historical-by-design (see docs/DOC_STATUS.md §2).
54: link-check:
55:    name: Markdown link check (live docs)
63:      - name: Check internal markdown links
64:        run: python -m tools.check_md_links
149: # (docs/GOLDEN_REGRESSION_FIXTURES.md). The CONTROL replay path is pure-stdlib
177: # Commercial-clean licensing guardrail (MIT/Apache/BSD only ? PACKAGING_PLAN.md P0d). Scans the
208: # Built-wheel smoke (PACKAGING_PLAN.md P0d): build the wheel, install it NON-editable from a

45. user (2026-07-05T09:33:52.439Z)

[check-md-links] clean ? no broken internal links in live docs.

46. user (2026-07-05T09:33:59.770Z)

```
docs/cp-durable-rebuild-spec
0a7b3ff feat(jobs): processing-queue endpoint + idempotent /api/process dedup (#448) (#481)
 M docs/README.md
?? docs/CONTROL_PLANE_DURABLE_REBUILD.md
```

47. user (2026-07-05T09:34:13.158Z)

```
warning: in the working copy of 'docs/CONTROL_PLANE_DURABLE_REBUILD.md', LF will be replaced by
CRLF the next time Git touches it
[docs/cp-durable-rebuild-spec 9bad982] docs(cp-rebuild): tracked spec ? durable control-plane
rebuild (#471 #480 #482-#485)
 2 files changed, 336 insertions(+)
 create mode 100644 docs/CONTROL_PLANE_DURABLE_REBUILD.md
9bad982 docs(cp-rebuild): tracked spec ? durable control-plane rebuild (#471 #480 #482-#485)
0a7b3ff feat(jobs): processing-queue endpoint + idempotent /api/process dedup (#448) (#481)
```

48. user (2026-07-05T09:34:21.380Z)

```
9bad982 docs(cp-rebuild): tracked spec ? durable control-plane rebuild (#471 #480 #482-#485)
branch 'docs/cp-durable-rebuild-spec' set up to track 'origin/docs/cp-durable-rebuild-spec'.
To https://github.com/Khelsutra/badminton-highlight-indexer.git
 * [new branch]      docs/cp-durable-rebuild-spec -> docs/cp-durable-rebuild-spec
```

49. user (2026-07-05T09:34:44.195Z)

```
https://github.com/Khelsutra/badminton-highlight-indexer/pull/487
```

50. user (2026-07-05T09:35:17.594Z)

```
The file c:\Users\avidu\.claude\projects\c--Users-avidu-Projects-khelsutra-guru\memory\session-
handoff.md has been updated successfully. (file state is current in your context ? no need to
Read it back)
```

51. user (2026-07-05T09:35:47.417Z)

```
The file c:\Users\avidu\.claude\projects\c--Users-avidu-Projects-khelsutra-guru\memory\session-
handoff.md has been updated successfully. (file state is current in your context ? no need to
Read it back)
```

52. user (2026-07-05T09:36:15.997Z)

```
The file c:\Users\avidu\.claude\projects\c--Users-avidu-Projects-khelsutra-guru\memory\current-
state.md has been updated successfully. (file state is current in your context ? no need to Read
it back)
```

53. assistant (2026-07-05T09:36:21.354Z)

Memory checkpointed. Verifying CI on the PR before wrapping up.

54. user (2026-07-05T09:36:24.184Z)

```
Golden fixtures cross-OS (macos-latest) pass    25s     https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28736408529/job/85211282193
Leak scan (PII / attribution guard)     pass    9s      https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28736408529/job/85211282161
Markdown link check (live docs) pass    6s      https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28736408529/job/85211282154
Mypy (lenient baseline) pass    34s     https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28736408529/job/85211282224
```

```
Ruff lint       pass    9s      https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28736408529/job/85211282155
Syntax sweep + import smoke (no GPU stack)      pass    21s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28736408529/job/85211282185
Full test suite (offline, mocked)       pending 0       https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28736408529/job/85211282169
Built wheel smoke (install + launch)    pass    1m7s    https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28736408529/job/85211282187
Dependency license scan (no copyleft in runtime deps)   pass    35s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28736408529/job/85211282178
Golden fixtures cross-OS (ubuntu-latest)        pass    26s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28736408529/job/85211282207
Golden fixtures cross-OS (windows-latest)       pass    56s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28736408529/job/85211282190
```

55. user (2026-07-05T09:37:39.726Z)

```
Markdown link check (live docs) pass    6s      https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28736408529/job/85211282154
Mypy (lenient baseline) pass    34s     https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28736408529/job/85211282224
Ruff lint       pass    9s      https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28736408529/job/85211282155
Syntax sweep + import smoke (no GPU stack)      pass    21s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28736408529/job/85211282185
```

56. user (2026-07-05T09:37:47.736Z)

```
      3 Golden fixtures
      1 Syntax sweep
      1 Ruff lint
state=OPEN mergeable=MERGEABLE
```

57. user (2026-07-05T09:37:56.906Z)

```
11
```

58. assistant (2026-07-05T09:38:26.137Z)

```
Done ? the rebuild spec is written, PR'd, and green; the deploy is now waiting only on your
confirmation.
```

What shipped

```
**[PR #487](https://github.com/Khelsutra/badminton-highlight-indexer/pull/487)** ?
`docs/CONTROL_PLANE_DURABLE_REBUILD.md` (tracked project doc per the template, plus its
`docs/README.md` index entry), docs-only, branched off `origin/master` in worktree `wt-cp-
rebuild-spec`. All **11 CI checks pass**, mergeable; link-check and `git diff --check` were
clean locally too. Ramp-up note: engine **#481 was already merged** before this session (master
`0a7b3ff`), so the spec's dedup/queue dependency is satisfied.
```

Three things the code-dive surfaced that weren't in the original ask

```
1. **The rev-00011 pins are a live trap.** The committed `gen_service_yaml.py` does *not* carry
`default_segmenter=wasb_hybrid` or `max_part_mb=24` ? anyone regenerating the manifest today
silently reverts both. The spec makes baking them into `build_config()` its own PR (P1, decision
D5) so "preserve the pins" becomes code, not a manual step.
2. **`use-http2` (issue #480's option 2) would break the service**, not fix it ? the annotation
```

requires the container to speak cleartext HTTP/2 and uvicorn has no h2c. Explicitly rejected (D6); 24 MB parts stay the mechanism.
3. **Submit-time MD5 (#484) turns out to be the keystone, not just a cost saver.** Nothing persists the done-marker token, and the job row's `video_id` is only filled *after* dispatch ? which is precisely why the restarted CP couldn't re-associate the running job with its 29 rallies on 2026-07-05. With md5 on the job row at submit, `outputs/<md5>/report.json` becomes the durable done-signal for startup recovery, and cache-reuse falls out of the same mechanism.

The rest is as you scoped it: 8 GiB CP, upload staging **streamed to GCS via a resumable writer** (Cloud Run services have no real disk ? the bucket is the disk), bucket-grounded state re-using the existing `_ingest_remote_segments` ingest, poll ceilings raised (CP 1800?3900 s knob; SPA 10?60 min, stall-based after #482), SPA re-vendor (the bundled UI is pinned at a July 2 commit that predates all three merged fixes), and #483/#482 bundled when ready but never allowed to hold the deploy.

What I need from you (§8 of the spec / the PR)

- **Q1 ? confirm the spec and authorize P9** (the rebuild + redeploy). This is the gate; P1?P8 code starts after your confirm per the tracked-doc convention.
- Q2?Q5, quick calls: 8 GiB/2 vCPU (recommended) vs 16 GiB/4 vCPU · whether P9 waits for #483's golden gate (recommended **no**) · 60-min interim SPA ceiling · upload cap stays 4096 MB.

The session handoff and current-state memory are checkpointed with the full execution order for the next session (P1 ? P2?P4?P3 ? P5a?P5b ? P8 ? P9 with the §9 DoD, which includes replaying the exact restart-after-done-before-ingest failure shape).

59. user (2026-07-05T09:52:10.031Z)

lgtm

60. assistant (2026-07-05T09:53:18.787Z)

Owner LGTM received ? that opens the gate (§8 Q1 confirmed; with no overrides on Q2?Q5, the recommendations stand: 8 GiB/2 vCPU, P9 doesn't wait for #483, 60-min SPA ceiling, cap stays 4096 MB). Also noting a sibling session has since posted a fully-verified review LGTM on #487 and opened #488 (A10/A11 docs ? owned by that session, I won't touch it).

Before merging and starting execution, let me check the repo's merge/signoff agreements so I sequence the P-row PRs correctly.

61. user (2026-07-05T09:53:19.804Z)

```
1       ---
2       name: working-agreement-merging
3       description: "Owner authorization (2026-06-12): Claude may MERGE its own PRs that are
good engineering fixes, on verified-green CI; product/strategy/licensing changes stay owner-
gated."
4       metadata:
5         node_type: memory
6         type: feedback
7         originSessionId: 47ecf23a-6312-4118-b2bd-c1af33f2eaa7
8       ---
9
10      **Owner (2026-06-12, GX010125 E2E session):** "Can you keep merging the PRs which you
identify as
11      good engineering fixes. like the API option. definitely."
12
13      **Why:** owner runs multiple parallel sessions and doesn't want to be the merge
bottleneck for
14      clear-cut engineering fixes (bug fixes, config wiring, parity gaps, test coverage,
hardening).
15
16      **How to apply:**
17      - Scope = engineering fixes Claude itself identified and verified (full suite +
```

```
ruff/mypy + CI).
18        First exercised on PR #107 (API min_shot_count parity, merged 2026-06-12).
19      - Merge ONLY on a verified-green rollup (`gh pr view --json statusCheckRollup` ? all
SUCCESS),
20        squash-merge, delete branch ? same dance the owner uses.
21      - STILL owner-gated: product/strategy/roadmap decisions, licensing-guardrail or
training-data
22        policy changes, schema/data deletions, and anything Claude is unsure belongs in
"engineering
23        fix". When in doubt, open the PR and leave it for review.
24      - Supersedes the earlier "do NOT auto-merge" default from the UI-alpha protocol (see
25        [[working-agreement-checkpoints]]).
26
```

62. user (2026-07-05T09:53:20.276Z)

```
1       ---
2       name: review-signoff-policy
3       description: "Owner's PR review sign-off rules ? written LGTM on pure-refactoring /
zero-behavior changes once CI is green; logic/behavior changes are owner-gated (LGTM only with
explicit per-PR authorization after verifying the full gate)."
4       metadata:
5         node_type: memory
6         type: feedback
7         originSessionId: 7fce5410-4c57-4ec9-a054-be898bda22da
8       ---
9
10      Owner's PR-review sign-off policy, settled over the #378/#379 reviews (2026-06-25):
11
12      - **Pure refactorings / zero-behavior-change PRs** (e.g. `ruff format`, renames, moves):
13        **SIGN OFF** (written LGTM) once CI is green. The "improved test coverage" bar does
NOT apply
14        to formatting-only changes ? they can't add coverage. Confirm it's genuinely
semantics-free via
15        what CI can't see (e.g. **AST-identity** for a reformat), not by re-running CI.
16      - **Logic / behavior-change PRs: HOLD for the owner.** Do NOT sign off on your own
judgment. The
17        owner may authorize a specific one explicitly ("LGTM if you're certain it's correctly
tested + a
18        good fix") ? then verify the full gate and post a written LGTM. (#378: held ? owner-
authorized ? LGTM.)
19      - **CI must be green first, always.** A red check blocks sign-off even for a pure
refactor. Real
20        example: #379 (`ruff format` of 2 files) went red on **mypy** because the formatter
moved a
21        `# type: ignore[override]` off the `def` line it must sit on ? `# type: ignore`
comments are
22        line-positional and NOT in the AST, so an "AST IDENTICAL" proof can't catch it. `mypy`
is the
23        cheap guard for that class of comment-relocation.
24      - **Mechanical limit:** gh is logged in as `avidullu`; GitHub blocks self-approval, so
every
25        sign-off is a **written LGTM comment**, never a formal green "Approve."
26
27      Pairs with [[review-trust-ci-over-rerun]] (trust green CI; reserve local effort for what
CI can't
28      cover) and [[working-agreement-merging]].
29
```

63. user (2026-07-05T09:53:22.075Z)

```
Exit code 1
failed to run git: fatal: not a git repository (or any of the parent directories): .git
```

64. user (2026-07-05T09:53:37.420Z)

```
commits=1 mergeable=MERGEABLE state=OPEN
SUCCESSx11
```

65. user (2026-07-05T09:53:47.964Z)

```
failed to delete local branch docs/cp-durable-rebuild-spec: failed to run git: error: cannot
delete branch 'docs/cp-durable-rebuild-spec' used by worktree at
'C:/Users/avidu/Projects/khelsutra-guru/wt-cp-rebuild-spec'

From https://github.com/Khelsutra/badminton-highlight-indexer
   0a7b3ff..cc5e83a  master     -> origin/master
cc5e83a docs(cp-rebuild): tracked spec ? durable control-plane rebuild (#471 #480 #482-#485)
(#487)
0a7b3ff feat(jobs): processing-queue endpoint + idempotent /api/process dedup (#448) (#481)
```

66. assistant (2026-07-05T09:53:54.477Z)

Merged ? master is now `cc5e83a` with the spec. Cleaning up the spec worktree, updating the main checkout, and creating the execution task list.

67. user (2026-07-05T09:53:58.709Z)

```
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/wt-cp-rebuild-spec': Permission
denied
error: failed to delete '.git/worktrees/wt-cp-rebuild-spec': Permission denied
Already up to date.
fc0c991 docs: record A10 A11 alpha guardrails
```

codex/a10-a11-alpha-guardrails...origin/codex/a10-a11-alpha-guardrails

68. user (2026-07-05T09:54:44.322Z)

```
Task #1 created successfully: P1: gen_service_yaml v2 ? rev-00011 pins + 8Gi default + doc IN
PROGRESS flip
```

69. user (2026-07-05T09:54:47.613Z)

```
Task #2 created successfully: P2: GCS-streamed upload staging (BlobWriter + streamed MD5)
```

70. user (2026-07-05T09:54:50.437Z)

```
Task #3 created successfully: P4: submit-time MD5 on job row + completed-work cache-reuse (#484)
```

71. user (2026-07-05T09:54:52.271Z)

```
Task #4 created successfully: P3: restart-safe remote jobs + read-path re-hydrate (#471/#485)
```

72. user (2026-07-05T09:55:00.095Z)

```
Task #5 created successfully: P5a: remote-poll max-wait knob + capabilities upload caps emit
```

73. user (2026-07-05T09:55:02.451Z)

```
Task #6 created successfully: P5b: khelsutra SPA ? poll ceilings 10?60 min + capabilities pre-
flight
```

74. user (2026-07-05T09:55:04.339Z)

```
Task #7 created successfully: P8: re-vendor SPA snapshot into backend/ui
```

75. user (2026-07-05T09:55:06.088Z)

Task #8 created successfully: P6 (#482): progress heartbeat ? job.progress + per-job runlog

76. user (2026-07-05T09:55:16.166Z)

Task #9 created successfully: P9: rebuild image + redeploy CP + CUJ verify + runlog

77. user (2026-07-05T09:55:17.806Z)

Task #10 created successfully: Refresh handoff + current-state at session end

78. user (2026-07-05T09:55:23.313Z)

Updated task #3 blockedBy

79. user (2026-07-05T09:55:23.783Z)

Updated task #4 blockedBy

80. user (2026-07-05T09:55:24.270Z)

Updated task #7 blockedBy

81. user (2026-07-05T09:55:24.989Z)

Updated task #9 blockedBy

82. user (2026-07-05T09:55:31.767Z)

Updated task #1 status

83. user (2026-07-05T09:55:34.799Z)

```
Preparing worktree (new branch 'feat/genyaml-pins-8gi')
branch 'feat/genyaml-pins-8gi' set up to track 'origin/master'.
HEAD is now at cc5e83a docs(cp-rebuild): tracked spec ? durable control-plane rebuild (#471 #480
#482-#485) (#487)
test_config_deployment.py
test_deploy_manifests.py
```

84. user (2026-07-05T09:55:39.329Z)

```
1       """Structural guard for the control-plane service-manifest generator (#473).
2
3       CI can't `gcloud run services replace` anything, so ? like the Dockerfile guard in
4       test_compute_backend.py ? lock the manifest's load-bearing shape structurally: the baked
5       config.json must round-trip through the base64 wrapper byte-exactly (that's the piece a
6       hand-edit silently corrupts), the always-allocated-CPU/scale annotations must survive
7       (README §3a: without them a 202'd compile stalls), and the reserved `CLOUD_RUN_JOB` env
8       name must never re-appear (a manifest carrying it is rejected at deploy time).
9
10      The generator is exercised through its real CLI (a subprocess), not an import ? deploy/
11      is not a package, and the CLI is exactly what a deployer runs.
12      """
13
14      import base64
15      import json
16      import re
17      import subprocess
18      import sys
19      from pathlib import Path
20
21      _GEN = Path(__file__).resolve().parents[1] / "deploy" / "cloudrun" /
"gen_service_yaml.py"
```

```

    def _generate(*extra: str) -> str:
        out = subprocess.run(
            [sys.executable, str(_GEN), *extra],
            capture_output=True,
            text=True,
            check=True,
        )
        return out.stdout


    def _baked_config(manifest: str) -> dict:
        """Decode the config.json the wrapper bakes into the manifest."""
        m = re.search(r"printf %s '([A-Za-z0-9+/=]+)' \| base64 -d", manifest)
        assert m, "wrapper must carry a base64 config payload"
        return json.loads(base64.b64decode(m.group(1)))


    def test_manifest_bakes_the_cloud_serving_config():
        cfg = _baked_config(_generate("--edge-auth", "off"))
        assert cfg["deployment"] == {"profile": "cloud-serving", "compute_target":
"cloud_run"}
        # WASB-only alpha: no Gemini key in this service, so the handoff must be off.
        assert cfg["indexing"]["skip_ai_handoff"] is True
        assert cfg["storage"]["backend"] == "gcs"
        assert cfg["storage"]["output_root"] == "gs://khelsutra-rally-serving"
        assert cfg["storage"]["input_root"] == "gs://khelsutra-rally-serving/videos"
        assert cfg["security"]["edge_auth_enabled"] is False
        assert cfg["security"]["allowed_video_root"] == "/tmp/apphome/output"


    def test_manifest_edge_auth_toggle_is_explicit():
        on = _baked_config(_generate("--edge-auth", "on"))
        assert on["security"]["edge_auth_enabled"] is True
        assert on["security"]["cf_team_domain"].startswith("https://")
        assert len(on["security"]["cf_access_aud"]) == 64  # the Access-app aud is a sha256
hex
        # --edge-auth is required: generating without choosing must fail, not default
silently.
        rc = subprocess.run(
            [sys.executable, str(_GEN)], capture_output=True, text=True
        ).returncode
        assert rc != 0


    def test_manifest_has_the_reliability_flags_and_no_reserved_env():
        text = _generate("--edge-auth", "off")
        # README §3a: the in-process drain needs always-allocated CPU + a warm instance.
        assert 'run.googleapis.com/cpu-throttling: "false"' in text
        assert 'autoscaling.knative.dev/minScale: "1"' in text
        # Issue #471: per-instance SQLite state ? a second instance splits the job queue.
        assert 'autoscaling.knative.dev/maxScale: "1"' in text
        # Reserved runtime env name ? a manifest carrying it is rejected at deploy time.
        assert "CLOUD_RUN_JOB" not in text
        assert "RALLY_ALLOWED_HOSTS" in text
        assert "containerPort: 8080" in text
        # The wrapper must create the config dir BEFORE redirecting into it, and exec the
server.
        wrapper = re.search(r'args: \["-c", "(.+)"\]', text).group(1)
        assert wrapper.startswith("mkdir -p /tmp/apphome && ")
        assert "export RALLY_APP_HOME=/tmp/apphome" in wrapper
        assert wrapper.endswith("exec python3 -m backend.main --server --host 0.0.0.0 --port
8080")

```

85. assistant (2026-07-05T09:56:10.455Z)

Generator tests found ? a structural guard I'll extend. Now implementing P1 in the worktree.

86. user (2026-07-05T09:56:11.373Z)

```
1       """Generate the control-plane Cloud Run *service manifest* (COMPUTE_DECOUPLED_SERVING
M8/M9, #473).
2
3       The A1 alpha deploy drove `rally-control-plane` from a hand-generated YAML that lived in
a
4       session scratchpad ? this commits the generator so the deploy is reproducible from a
clean
5       clone. It emits a Knative service manifest for::
6
7           python deploy/cloudrun/gen_service_yaml.py --edge-auth off -o service.yaml
8           gcloud run services replace service.yaml --region asia-southeast1
9
10      Why a MANIFEST (`services replace`) and not `gcloud run deploy` flags: the container
command is
11      a /bin/bash wrapper carrying shell metacharacters (pipes, redirects, &&) ? on Windows
the gcloud
12      cmd-shim mangles those through `--command/--args`, while a YAML round-trips them
verbatim. Run
13      gcloud from PowerShell, not git-bash (MSYS rewrites `/tmp`-style paths).
14
15      The app is configured through a config.json baked INTO the manifest: the wrapper
base64-decodes
16      it to `/tmp/apphome/config.json` and exports `RALLY_APP_HOME=/tmp/apphome` before
exec'ing the
17      server, so one image serves every config shape (config.json wins over the cloud-serving
preset ?
18      backend/config/loader.py). `--edge-auth` is REQUIRED and explicit: `off` is only for the
19      identity-token smoke/e2e window; redeploy with `on` (the Cf-Access JWT verify, M9)
before the
20      service faces testers again.
21
22      Deliberate manifest choices (learned on the A1 deploy):
23        * `run.googleapis.com/cpu-throttling: "false"` + minScale 1 ? the in-process job
worker drains
24          on a background thread; without always-allocated CPU a 202'd compile stalls (README
§3a).
25        * maxScale DEFAULTS TO 1 ? the jobs/videos DB is per-instance ephemeral SQLite, so a
second
26          instance splits the queue and `/api/jobs/{id}` polls (issue #471, option 1). Raise
it only
27          once state is shared.
28        * `CLOUD_RUN_JOB` is NOT set ? it's a reserved runtime env name and the manifest is
rejected
29          with it; the code default ("rally-worker") is already correct.
30        * No secrets here: bucket/project/aud/team-domain are the same non-secret identifiers
the
31          committed runlogs carry; the Gemini key stays in Secret Manager (the alpha runs
32          `skip_ai_handoff=true` and never needs it).
33      """
34
35      from __future__ import annotations
36
37      import argparse
38      import base64
39      import json
40      import sys
41
42      DEFAULT_PROJECT = "gen-lang-client-0306026826"
43      DEFAULT_REGION = "asia-southeast1"
```

```
44      DEFAULT_SERVICE = "rally-control-plane"
45      DEFAULT_SERVING_BUCKET = "gs://khelsutra-rally-serving"
46      DEFAULT_CF_TEAM_DOMAIN = "https://khelsutra.cloudflareaccess.com"
47      DEFAULT_CF_ACCESS_AUD =
"c980da67ed888fff935fcd913ae1eb4da63f9d65805d6e1c5ca76f92f517082b"
48
49
50      def build_config(args: argparse.Namespace) -> dict:
51          """The app config the wrapper materializes at RALLY_APP_HOME/config.json."""
52          bucket = args.serving_bucket.rstrip("/")
53          return {
54              "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
55              # WASB-only alpha: candidate windows ARE the rallies; no Gemini key in this
service.
56              "indexing": {"skip_ai_handoff": True},
57              "storage": {
58                  "backend": "gcs",
59                  "output_root": bucket,
60                  "input_backend": "gcs",
61                  "input_root": f"{bucket}/videos",
62              },
63              "security": {
64                  "edge_auth_enabled": args.edge_auth == "on",
65                  "cf_team_domain": args.cf_team_domain,
66                  "cf_access_aud": args.cf_access_aud,
67                  "allowed_video_root": "/tmp/apphome/output",
68              },
69          }
70
```

87. user (2026-07-05T09:56:28.338Z)

```
The file C:\Users\avidu\Projects\khelsutra-
guru\wt-p1-genyaml\deploy\cloudrun\gen_service_yaml.py has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

88. user (2026-07-05T09:56:40.558Z)

```
The file C:\Users\avidu\Projects\khelsutra-
guru\wt-p1-genyaml\deploy\cloudrun\gen_service_yaml.py has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

89. user (2026-07-05T09:56:52.555Z)

```
The file C:\Users\avidu\Projects\khelsutra-
guru\wt-p1-genyaml\deploy\cloudrun\gen_service_yaml.py has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

90. user (2026-07-05T09:57:00.515Z)

```
41      def test_manifest_bakes_the_cloud_serving_config():
42          cfg = _baked_config(_generate("--edge-auth", "off"))
43          assert cfg["deployment"] == {"profile": "cloud-serving", "compute_target":
"cloud_run"}
44          # WASB-only alpha: no Gemini key in this service, so the handoff must be off.
45          assert cfg["indexing"]["skip_ai_handoff"] is True
46          assert cfg["storage"]["backend"] == "gcs"
47          assert cfg["storage"]["output_root"] == "gs://khelsutra-rally-serving"
48          assert cfg["storage"]["input_root"] == "gs://khelsutra-rally-serving/videos"
49          assert cfg["security"]["edge_auth_enabled"] is False
50          assert cfg["security"]["allowed_video_root"] == "/tmp/apphome/output"
51
52
53      def test_manifest_edge_auth_toggle_is_explicit():
```

```
54          on = _baked_config(_generate("--edge-auth", "on"))
55          assert on["security"]["edge_auth_enabled"] is True
```

91. user (2026-07-05T09:57:17.207Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p1-genyaml\tests\test_deploy_manifests.py has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

92. user (2026-07-05T09:57:39.584Z)

```
1       # Control-Plane Durable Rebuild ? kill the upload OOM, ground state in the bucket,
unblock DoD #1
2
3       > **Status:** `DRAFT` ? **owner-confirm gate: P9 (the deploy) does NOT execute until the
owner confirms
4       > this spec** · **Owner:** avidullu · **Created:** 2026-07-05 · **Last updated:**
2026-07-05
5       > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
DONE)
6       > **Tracking anchors:** §7 progress tracker is the source of truth; indexed in
`docs/README.md`;
7       > pointer in memory `current-state` (+ `docs/NEXT_STEPS.md` once work starts post-
confirmation).
8       > **Relation to existing docs:** executes the serving hardening that
9       > [ALPHA_LAUNCH_REVIEW_2026-07-04.md](ALPHA_LAUNCH_REVIEW_2026-07-04.md) §1.1 (the
5-minute browser
10      > CUJ ? tracker **DoD #1**) is blocked on; child of
11      > [COMPUTE_DECOUPLED_SERVING/](COMPUTE_DECOUPLED_SERVING/README.md) (ADR-014);
consolidates issues
12      > #471 #480 #482 #483 #484 #485 (plus the already-shipped #479-config, #481, and
khelsutra
13      > #153/#154(#155)/#157(#158)).
14      > **Honesty note:** each claim is tagged `[verified]` (checked against code or live
logs),
15      > `[researched]` (external platform fact ? re-verify during execution), or `[design]`
(proposed).
16
17      ---
18
```

93. user (2026-07-05T09:57:41.436Z)

```
264:| P0 | This spec; owner confirmation opens P9 | engine | ? | **Owner** | ? | (this PR) |
265:| P1 | `gen_service_yaml.py` v2: rev-00011 pins in `build_config` + `--memory` 8Gi default |
engine | P0 | No | ? | ? |
279:## 8. Open questions ? owner
333:### Changelog
```

94. user (2026-07-05T09:57:49.956Z)

```
The file C:\Users\avidu\Projects\khelsutra-
guru\wt-p1-genyaml\docs\CONTROL_PLANE_DURABLE_REBUILD.md has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

95. user (2026-07-05T09:57:53.331Z)

```
The file C:\Users\avidu\Projects\khelsutra-
guru\wt-p1-genyaml\docs\CONTROL_PLANE_DURABLE_REBUILD.md has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

96. user (2026-07-05T09:57:59.643Z)

```
281     ## 8. Open questions ? owner
282
```

```
283     1. **Confirm this spec + authorize P9** (image build + redeploy in the existing
project/region;
284        deploy-scale spend only ? no new resources). ? **the blocking gate**
285     2. Memory: **8 GiB / 2 vCPU (recommended)** or 16 GiB / 4 vCPU headroom? (>8 GiB forces
4 vCPU
286        `[researched]` ? 2× the always-on instance cost; streamed staging removes the main
driver for it.)
287     3. #483 bundling: hold P9 for the golden gate? **Recommendation: NO** ? ship P9 when
P1?P5b/P8 land;
288        #483 follows in the next image.
289     4. SPA interim ceiling 60 min OK (stall-based detection follows #482)?
290     5. Upload policy cap stays 4096 MB for the alpha (raising it rides D7)?
291
292     ## 9. Definition of done
293
294     - [ ] From a browser at `api.khelsutra.guru` (edge-auth ON): a **?2.5 GB 1080p60**
upload completes ?
295           no 413, no OOM, no restart; instance memory observed well under the limit during
staging.
296     - [ ] The job runs **`wasb_hybrid`** (P1 pin, confirmed in worker logs) and the SPA
stays with it
297           past 10 minutes without giving up.
298     - [ ] Progress is visible during detection (#482) ? or, if P6 slipped to the follow-up
image, the
```

97. user (2026-07-05T09:58:10.898Z)

```
The file C:\Users\avidu\Projects\khelsutra-
guru\wt-p1-genyaml\docs\CONTROL_PLANE_DURABLE_REBUILD.md has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

98. user (2026-07-05T09:58:16.444Z)

```
330     **Memory:** [[alpha-cuj-testing-2026-07-05]] · [[session-handoff]] · [[launch-report-
convention]].
331
332     ---
333
334     ### Changelog
335     - 2026-07-05 ? created (`DRAFT`): consolidates the 2026-07-05 owner-CUJ findings into
the single
336       rebuild plan; P9 owner-gated on spec confirmation.
337
```

99. user (2026-07-05T09:58:25.839Z)

```
The file C:\Users\avidu\Projects\khelsutra-
guru\wt-p1-genyaml\docs\CONTROL_PLANE_DURABLE_REBUILD.md has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

100. user (2026-07-05T09:58:33.894Z)

```
1       # Next steps (live) ? refreshed 2026-06-21
2
3       > **How to read:** this top section is the **current cross-track state + prioritized
pending list** (refreshed 2026-06-21, replacing the stale 2026-06-13 header). The detailed
**owned-model & rally-quality track** plan (2026-06-13) is **preserved below, still valid** ? it
is now *one of several* active tracks. Live blow-by-blow: memory `current-state.md`. Doc
lifecycle/hygiene: `docs/DOC_STATUS.md`.
4
5       ## ? Where we are (2026-06-21)
6       **DECOUPLED_COMPUTE delivered on GCP + archived** (#270); **compute = GCP only**
(ADR-013, DO dropped). **`COMPUTE_DECOUPLED_SERVING`** subproject spun up (#272, **ADR-014**) ?
one pure core + local/cloud profiles; **M0?M9 LARGELY SHIPPED** (ADR-014 ratified 2026-06-21;
```

M1?M4 #279?#283, M5 #286, M6 #287, M8 #288, M9-auth #290 all merged, default-OFF). **Per-video workspace P0** shipped (#264); **doc-status register + archival due-diligence** in place (#273). The **khelsutra site + per-rally UI** ship in parallel (sibling track). The **rally-quality / owned-model** track (below) is the core ML work, gated on **growing the golden corpus**.
7
8   > **? LAUNCH DECISION ? NO-GO (owner, 2026-06-30).** The rally-detection product is **not** cleared for an external/commercial launch; we stay in **development**. This is the expected default for the dev phase. Two unblockers gate a future GO: **(1)** set + meet the **ship-gate target (#172)** ? currently unset, needs corpus growth (and the R2 tolF1 ablation, item 8); **(2)** resolve the **WASB shuttle-detector weight provenance** (commercial-clean licensing). P2b (#396) was a do-no-harm quality increment, **not** a launch gate ? merging dev PRs ? launching. Re-decide only when both unblockers clear.

101. user (2026-07-05T09:58:45.719Z)

The file C:\Users\avidu\Projects\khelsutra-guru\wt-p1-genyaml\docs\NEXT_STEPS.md has been updated successfully. (file state is current in your context ? no need to Read it back)

102. user (2026-07-05T09:59:09.278Z)

```
/c/Users/avidu/AppData/Local/Programs/Python/Python313/python
....                                                                     [100%]
4 passed in 0.91s
```

103. user (2026-07-05T10:02:07.635Z)

```
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
1665 passed, 10 skipped, 8 warnings in 126.49s (0:02:06)
All checks passed!
backend\providers\__init__.py:9: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 148 source files
[check-md-links] clean ? no broken internal links in live docs.
ALL_GATES_GREEN
```

104. user (2026-07-05T10:02:23.291Z)

```
feat/genyaml-pins-8gi
cc5e83a docs(cp-rebuild): tracked spec ? durable control-plane rebuild (#471 #480 #482-#485)
(#487)
[feat/genyaml-pins-8gi a74d947] deploy(cloudrun): bake rev-00011 pins into the manifest
generator + 8Gi default (CP-rebuild P1)
 4 files changed, 84 insertions(+), 16 deletions(-)
a74d947 deploy(cloudrun): bake rev-00011 pins into the manifest generator + 8Gi default (CP-
rebuild P1)
```

105. user (2026-07-05T10:02:43.933Z)

```
To https://github.com/Khelsutra/badminton-highlight-indexer.git
 * [new branch]      feat/genyaml-pins-8gi -> feat/genyaml-pins-8gi
https://github.com/Khelsutra/badminton-highlight-indexer/pull/489
```

106. user (2026-07-05T10:03:34.911Z)

```
Updated task #2 status
```

107. user (2026-07-05T10:03:38.724Z)

```
HEAD is now at cc5e83a docs(cp-rebuild): tracked spec ? durable control-plane rebuild (#471 #480
#482-#485) (#487)
__init__.py
__pycache__
base.py
capabilities.py
```

```
gcs.py
gdrive.py
gdrive_api.py
local.py
ref.py
s3.py
test_storage.py
test_storage_capabilities.py
test_storage_gdrive_api.py
test_upload_download.py
test_upload_retention.py
```

108. user (2026-07-05T10:03:45.240Z)

```
170         }
171
172
173     @router.post("/api/upload/{upload_id}/complete")
174     def upload_complete(upload_id: str, req: UploadCompleteRequest):
175         """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
176         upload_dir, _, _, _ = _upload_cfg()
177         with _uploads_lock:
178             st = _uploads.get(upload_id)
179         if st is None:
180             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
181         if st["parts"] == 0 or st["parts"] != req.total_parts:
182             raise HTTPException(
183                 status_code=409,
184                 detail=f"Expected {req.total_parts} part(s), received {st['parts']}.",
185             )
186         final_path = os.path.join(upload_dir, st["filename"])
187         if os.path.exists(
188             final_path
189         ):  # never overwrite an earlier upload with the same name
190             stem, ext = os.path.splitext(st["filename"])
191             final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
192         os.replace(st["tmp_path"], final_path)
193         with _uploads_lock:
194             _uploads.pop(upload_id, None)
195         video_id = compute_video_id(final_path)
196         return {
197             "path": os.path.abspath(final_path),
198             "video_id": video_id,
199             "size_bytes": os.path.getsize(final_path),
200             "next": "POST /api/process with this path",
201         }
202
203
204     @router.delete("/api/upload/{upload_id}")
205     def upload_abort(upload_id: str):
206         """Abort an in-flight upload and delete its partial file."""
207         with _uploads_lock:
208             known = upload_id in _uploads
209         if not known:
210             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
211         _abort_upload(upload_id)
212         return {"status": "aborted", "upload_id": upload_id}
213
```

109. user (2026-07-05T10:03:45.693Z)

```
1       """The ``StorageBackend`` ABC ? one interface for local + object-store + Drive.
2
3       Mirrors the codebase's ABC+Registry idiom (cf. ``SegmenterRegistry``). The bucket
```

```
backends
4       live in sibling modules and lazy-import their cloud SDK so ``import backend.storage``
never
5       pulls ``boto3`` / ``google-cloud-storage`` (``gdrive`` is pure local-FS over a mounted
Drive ?
6       no SDK). See ``PLATFORM_ARCHITECTURE.md`` §3.1 and ``docs/DECOUPLED_COMPUTE/``.
7       """
8
9       from __future__ import annotations
10
11      import os
12      from abc import ABC, abstractmethod
13      from typing import Iterator
14
15      from backend.storage.ref import StorageRef, StorageStat
16
17
18      class StorageBackend(ABC):
19          """Backend-agnostic object storage. ``fetch_to_local`` + ``put`` are the two the
20          worker dispatcher needs; the rest support listing/checkpointing/serving."""
21
22          @abstractmethod
23          def fetch_to_local(
24              self,
25              ref: StorageRef,
26              dst: "str | os.PathLike[str] | None" = None,
27              *,
28              overwrite: bool = False,
29          ) -> str:
30              """Make the object available as a local file; return its local path. For the
31              ``local`` backend this is an identity passthrough (the object is already
local)."""
32
33          @abstractmethod
34          def put(
35              self,
36              local_path: "str | os.PathLike[str]",
37              ref: StorageRef,
38              *,
39              content_type: "str | None" = None,
40          ) -> StorageRef:
41              """Upload/copy a local file to ``ref``; return ``ref``."""
42
43          @abstractmethod
44          def exists(self, ref: StorageRef) -> bool: ...
45
46          @abstractmethod
47          def stat(self, ref: StorageRef) -> StorageStat: ...
48
49          @abstractmethod
50          def list(self, prefix: StorageRef) -> Iterator[StorageRef]:
51              """Yield a ``StorageRef`` per object under ``prefix``."""
52
53          @abstractmethod
54          def delete(self, ref: StorageRef) -> None: ...
55
56          def signed_url(
57              self, ref: StorageRef, *, ttl: "int | None" = None, method: str = "GET"
58          ) -> str:
59              """A time-limited URL handing the bytes to a browser/LLM. ``ttl=None`` (the
default)
60              means "use the backend's configured ``signed_url_ttl``" ? so operator config
wins
61              unless a caller explicitly needs a different expiry (#469). Not every backend
can
```

```
            sign (``local`` cannot; Drive only emulates) ? callers must treat
            ``NotImplementedError`` as expected and fall back to ``fetch_to_local``."""
            raise NotImplementedError(f"{type(self).__name__} cannot produce signed URLs")

```

110. user (2026-07-05T10:03:52.496Z)

```
      """Google Cloud Storage backend ? first-class object buckets.

      Uses ``google-cloud-storage``, imported lazily so ``import backend.storage`` never pulls
it;
      install with the ``storage-gcs`` extra. Credentials come from Application Default
Credentials
      (``GOOGLE_APPLICATION_CREDENTIALS`` pointing at a service-account JSON, or workload
identity) ?
      never from config. A ``client`` may be injected for tests.
      """

      from __future__ import annotations

      import logging
      from datetime import timedelta
      from importlib import import_module
      from typing import Any, Iterator, Optional

      from backend.storage.base import StorageBackend
      from backend.storage.ref import StorageRef, StorageStat

      logger = logging.getLogger(__name__)


      class GCSStorage(StorageBackend):
          def __init__(
              self,
              *,
              client: Any = None,
              project: Optional[str] = None,
              signed_url_ttl: int = 3600,
              **_ignored,
          ) -> None:
              self._ttl = signed_url_ttl
              if client is not None:
                  self._client = client
                  return
              storage_module: Any = import_module("google.cloud.storage")
              self._client = storage_module.Client(project=project)

          def _blob(self, ref: StorageRef):
              return self._client.bucket(ref.bucket).blob(ref.key)

          def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
              if dst is None:
                  raise ValueError("GCSStorage.fetch_to_local requires a dst path")
              import os

              dst = str(dst)
              if os.path.exists(dst) and not overwrite:
                  return dst
              os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
              self._blob(ref).download_to_filename(dst)
              return dst

          def put(self, local_path, ref, *, content_type=None) -> StorageRef:
              self._blob(ref).upload_from_filename(str(local_path), content_type=content_type)
              return ref
```

```
56
57          def exists(self, ref) -> bool:
58              return bool(self._blob(ref).exists())
59
60          def stat(self, ref) -> StorageStat:
61              b = self._blob(ref)
62              b.reload()
63              return StorageStat(
64                  size=int(b.size or 0),
65                  modified=b.updated,
66                  etag=b.etag,
67                  content_type=b.content_type,
68              )
69
70          def list(self, prefix) -> Iterator[StorageRef]:
71              for blob in self._client.list_blobs(prefix.bucket, prefix=prefix.key):
72                  yield StorageRef(
73                      "gcs", prefix.bucket, blob.name, f"gcs://{prefix.bucket}/{blob.name}"
74                  )
75
76          def delete(self, ref) -> None:
77              self._blob(ref).delete()
78
79          def signed_url(self, ref, *, ttl=None, method="GET") -> str:
80              blob = self._blob(ref)
81              exp = timedelta(seconds=ttl or self._ttl)
82              creds = getattr(self._client, "_credentials", None)
83              # Local-key signing (a service-account JSON key exposes a `signer`) works out of
the box. But
84              # on Cloud Run / GCE the ADC credentials come from the metadata server and have
NO private key,
85              # so a plain generate_signed_url raises "you need a private key". Fall back to
the IAM signBlob
86              # API ? which requires the runtime SA to hold
roles/iam.serviceAccountTokenCreator on ITSELF.
87              # ? signBlob canNOT ride the storage client's own token (#475, found on the
first real-creds
88              # run): that token is scoped to devstorage.* only (Client.SCOPE), which
iamcredentials rejects
89              # with ACCESS_TOKEN_SCOPE_INSUFFICIENT ? and a metadata credential may still
report the
90              # "default" email alias. Mint a separate cloud-platform-scoped credential
instead: its refresh
91              # both authorizes signBlob and resolves the real SA email. Failures are LOGGED
before deferring
92              # to the default call, so a broken sign path can't masquerade as a missing-key
error.
93              if creds is not None and not hasattr(creds, "signer"):
94                  try:
95                      sa_email, token = self._signblob_identity()
96                      if sa_email:
97                          return blob.generate_signed_url(
98                              version="v4",
99                              expiration=exp,
100                             method=method,
101                             service_account_email=sa_email,
102                             access_token=token,
103                         )
104                     logger.warning(
105                         "[gcs] keyless signing skipped: runtime SA email unresolved (got
%r); "
106                         "falling back to key-based signing",
107                         sa_email,
108                     )
109                 except Exception as e:
```

```
110                     logger.warning(
111                         "[gcs] keyless signBlob path failed (%s); falling back to key-based
signing",
112                         e,
113                     )
114             return blob.generate_signed_url(version="v4", expiration=exp, method=method)
115
116         @staticmethod
117         def _signblob_identity() -> "tuple[Optional[str], Optional[str]]":
118             """(sa_email, access_token) for the IAM signBlob call ? a FRESH cloud-platform-
scoped
119             credential from ADC, never the storage client's devstorage-scoped token (#475).
Isolated
120             as a seam so tests can fake the metadata-server identity without real
credentials."""
121             import google.auth
122             from google.auth.transport.requests import Request as _AuthRequest
123
124             sign_creds, _ = google.auth.default(
125                 scopes=["https://www.googleapis.com/auth/cloud-platform"]
126             )
127             sign_creds.refresh(_AuthRequest())
128             sa_email = getattr(sign_creds, "service_account_email", None)
129             if not sa_email or sa_email == "default":
130                 return None, None
131             return sa_email, sign_creds.token
132
```

111. user (2026-07-05T10:04:14.148Z)

```
1       """Storage value objects + URI parsing for the pluggable storage layer.
2
3       A ``StorageRef`` is a backend-agnostic pointer to one object. URIs:
4
5           <bare path>                 -> local   (today's behaviour; e.g. C:/v/x.mp4,
/data/x.mp4)
6           local://<path>              -> local   (local:///abs or local://rel/p)
7           s3://<bucket>/<key>         -> s3       (AWS + R2 + DO Spaces + MinIO via
endpoint_url)
8           gcs://<bucket>/<key>        -> gcs      (gs:// accepted, normalized to gcs://)
9           gdrive://<path>             -> gdrive   (path under your locally-MOUNTED Google
Drive "My Drive")
10
11      No cloud SDKs are imported here ? this module is pure stdlib so ``import
backend.storage``
12      stays cheap.
13      """
14
15      from __future__ import annotations
16
17      import re
18      from dataclasses import dataclass
19      from datetime import datetime
20      from typing import Optional
21
22      _SCHEME_RE = re.compile(r"^([A-Za-z][A-Za-z0-9+.\-]*)://")
23      _KNOWN = ("local", "s3", "gcs", "gdrive", "gdrive-api")
24
25
26      @dataclass(frozen=True)
27      class StorageStat:
28          """Lightweight object metadata (uniform across backends)."""
29
30          size: int
31          modified: Optional[datetime] = None
```

```
        etag: Optional[str] = None
        content_type: Optional[str] = None


    @dataclass(frozen=True)
    class StorageRef:
        """A backend + a location. ``bucket`` is the S3/GCS bucket; for ``local`` and
``gdrive`` it is
        empty and ``key`` carries the path (a filesystem path, or a path under the mounted
Drive)."""

        backend: str
        bucket: str
        key: str
        uri: str

        @property
        def name(self) -> str:
            return self.key.rstrip("/").rsplit("/", 1)[-1]


    def parse_uri(uri: str) -> StorageRef:
        """Parse a storage URI (or a bare filesystem path) into a ``StorageRef``.

        A string with no ``scheme://`` prefix (including Windows ``C:\\...`` paths, which
        have no ``//``) is treated as a local filesystem path ? preserving today's
behaviour.
        """
        s = str(uri)
        m = _SCHEME_RE.match(s)
        if not m:
            return StorageRef("local", "", s, f"local://{s}")
        scheme = m.group(1).lower()
        rest = s[m.end() :]
        if scheme == "gs":
            scheme = "gcs"
        if scheme in ("local", "gdrive", "gdrive-api"):
            # Path-keyed, no bucket: local FS, a path under the mounted Drive, or a path
under a Drive-API
            # root folder (gdrive-api resolves the path ? a Drive file id).
            return StorageRef(scheme, "", rest, s)
        if scheme in _KNOWN:
            bucket, _, key = rest.partition("/")
            return StorageRef(scheme, bucket, key, s)
        raise ValueError(f"unsupported storage URI scheme: {scheme}:// (in {s!r})")


    def resolve_input_uri(
        raw: str, *, input_backend: str = "local", input_root: str = ""
    ) -> str:
        """Resolve a user-supplied input reference into a storage URI
(COMPUTE_DECOUPLED_SERVING M2).

        Precedence (first match wins):
          1. An explicit ``scheme://`` on ``raw`` (``gs://``, ``s3://``, ``local://``,
``gdrive://``) ? as-is.
          2. An ABSOLUTE local path (``/data/x.mp4``, ``C:\\?``, ``C:/?``) ? as-is (always
local).
          3. A bare/relative name + a non-empty ``input_root`` ? joined under it
             (``"gs://bucket/videos"`` + ``"x.mp4"`` ? ``"gs://bucket/videos/x.mp4"``).
          4. A bare/relative name + ``input_backend != "local"`` ? prefixed (``"gcs"`` +
``"bucket/x"`` ?
             ``"gcs://bucket/x"``).
          5. Otherwise ? ``raw`` unchanged (a relative/bare LOCAL path ? today's behaviour).

```

```
89          So the default (``input_backend="local"``, ``input_root=""``) is an identity
passthrough for every
90          path used today, preserving byte-for-byte behaviour.
91          """
92          s = str(raw)
93          if _SCHEME_RE.match(s):  # 1. explicit scheme wins
94              return s
95          # 2. an anchored LOCAL path is always local, never joined. Detect this OS-
INDEPENDENTLY ?
96          #    os.path.isabs is platform- AND Python-version-dependent (on Windows + py3.13 a
POSIX-style
97          #    "/data/x" is NOT absolute). Anchored == a leading "/" or "\\", or a Windows
drive prefix "X:".
98          if s.startswith(("/", "\\")) or (len(s) >= 2 and s[1] == ":" and s[0].isalpha()):
99              return s
100         if input_root:  # 3. join a relative name under the configured root
101             return input_root.rstrip("/") + "/" + s.lstrip("/")
102         if (
103             input_backend and input_backend != "local"
104         ):  # 4. prefix a bucket/key with the declared backend
105             return f"{input_backend}://{s}"
106         return s  # 5. bare/relative local path ? unchanged
107
```

112. user (2026-07-05T10:04:14.598Z)

```
1       """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Google Drive.
2
3       URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are
imported
4       lazily, only when a non-local bucket URI is actually used (install the ``storage-s3`` /
5       ``storage-gcs`` extras). The default ``local`` backend needs no third-party deps and
preserves
6       today's behaviour byte-for-byte; ``gdrive://`` likewise needs none ? it reads a locally
MOUNTED
7       Google Drive (Google Drive for Desktop) as plain files.
8
9       Worker/dispatcher entry points speak URIs via the thin helpers ``fetch`` / ``put`` /
10      ``list_uris`` / ``get_storage``; ``local://`` (and bare paths) make ``fetch`` an
identity
11      passthrough. See ``docs/DECOUPLED_COMPUTE/``.
12      """
13
14      from __future__ import annotations
15
16      import os
17      import shutil
18      import tempfile
19      from typing import Any, Dict, Iterator, Optional
20
21      from backend.storage.base import StorageBackend
22      from backend.storage.ref import StorageRef, StorageStat, parse_uri, resolve_input_uri
23
24      __all__ = [
25          "StorageBackend",
26          "StorageRef",
27          "StorageStat",
28          "parse_uri",
29          "resolve_input_uri",
30          "get_storage",
31          "fetch",
32          "put",
33          "exists",
34          "signed_url",
35          "list_uris",
```

```
36          "acquire_local_input",
37          "cleanup_acquired_local_input",
38          "input_is_local",
39          "resolve_input_ref",
40      ]
41
42
43      def _storage_dict(storage_config: Any) -> Dict[str, Any]:
44          """Coerce a StorageConfig (Pydantic) / dict / None into a plain settings dict."""
45          if storage_config is None:
46              return {}
47          if hasattr(storage_config, "model_dump"):
48              return storage_config.model_dump()  # type: ignore[no-any-return]
49          return dict(storage_config)
50
51
52      def resolve_input_ref(raw: str, storage_config: Any = None) -> StorageRef:
53          """Resolve a user-supplied input (a path or storage URI) into a ``StorageRef`` under
the input
54          config (``input_backend``/``input_root``). Raises ``ValueError`` for an unsupported
URI scheme ?
55          callers at the API/CLI boundary catch that and return a clean client error (never a
500).
56
57          Use ``ref.backend`` to branch local-vs-remote and ``ref.key`` for the resolved LOCAL
path (e.g.
58          ``local://?`` is stripped to the bare path), rather than stat-ing the raw URI
string."""
59          sc = _storage_dict(storage_config)
60          uri = resolve_input_uri(
61              raw,
62              input_backend=sc.get("input_backend", "local"),
63              input_root=sc.get("input_root", ""),
64          )
65          return parse_uri(uri)
66
67
68      def input_is_local(raw: str, storage_config: Any = None) -> bool:
69          """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ?
i.e. a
70          plain on-disk file the caller can stat/validate directly. False for a bucket / Drive
URI
71          that must be fetched first. Raises ``ValueError`` for an unsupported scheme (same as
72          ``resolve_input_ref``)."""
73          return resolve_input_ref(raw, storage_config).backend == "local"
74
75
76      def acquire_local_input(
77          raw: str, storage_config: Any = None, *, scratch_parent: "str | None" = None
78      ) -> str:
79          """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for
processing.
80
81          Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
behaviour
82          byte-for-byte). A bucket / Drive source (``gs://``/``s3://``/``gdrive://``, or a
bare name under a
83          non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir
and the local
84          path returned. The caller owns successful remote scratch copies; use
85          ``cleanup_acquired_local_input`` when the consumer has finished reading the returned
path."""
86          sc = _storage_dict(storage_config)
87          ref = resolve_input_ref(raw, sc)
88          if ref.backend == "local":
```

```
            return (
                ref.key
            )  # identity ? no temp, no copy, byte-identical to passing the path directly
        scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
        try:
            return fetch(ref.uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)
        except Exception:
            shutil.rmtree(scratch, ignore_errors=True)
            raise


    def cleanup_acquired_local_input(
        raw: str, local_path: str, storage_config: Any = None
    ) -> None:
        """Delete a remote scratch dir produced by ``acquire_local_input``.

        Local inputs are identity paths and must never be removed. Remote acquisitions are
downloaded
        into a fresh ``rally-input-*`` directory; deleting that directory is safe once the
caller has
        finished hashing/validating/stitching/segmenting the returned file.
        """
        try:
            if input_is_local(raw, storage_config):
                return
        except ValueError:
            return
        if not local_path:
            return
        scratch = os.path.dirname(os.path.abspath(local_path))
        if os.path.basename(scratch).startswith("rally-input-") and os.path.isdir(scratch):
            shutil.rmtree(scratch, ignore_errors=True)


    def _backend_class(name: str):
        """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
        if name == "local":
            from backend.storage.local import LocalStorage

            return LocalStorage
        if name == "s3":
            from backend.storage.s3 import S3Storage

            return S3Storage
        if name == "gcs":
            from backend.storage.gcs import GCSStorage

            return GCSStorage
        if name == "gdrive":
            from backend.storage.gdrive import GDriveStorage

            return GDriveStorage
        if name == "gdrive-api":
            from backend.storage.gdrive_api import GDriveApiStorage

            return GDriveApiStorage
        raise ValueError(
            f"unknown storage backend: {name!r} (expected local|s3|gcs|gdrive|gdrive-api)"
        )


    def get_storage(
        backend: Optional[str] = None,
        *,
        config: Optional[Dict[str, Any]] = None,
```

```
        **overrides,
    ) -> StorageBackend:
        """Construct a backend. ``backend`` (or ``config['backend']``, default 'local')
selects
        the implementation; its per-backend settings come from ``config[backend]`` merged
with
        ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
selectors."""
        cfg = config or {}
        name = backend or cfg.get("backend", "local")
        # Per-backend settings live under the backend name; tolerate the hyphen?underscore
form so a
        # hyphenated backend ("gdrive-api") can read a pydantic field that can't contain a
hyphen
        # ("gdrive_api"). Hyphen-free names are unaffected (the first lookup wins).
        settings: Dict[str, Any] = dict(
            cfg.get(name) or cfg.get(name.replace("-", "_")) or {}
        )
        settings.update(overrides)
        return _backend_class(name)(**settings)


    def fetch(
        uri: str,
        dst: "str | os.PathLike[str] | None" = None,
        *,
        config: Optional[Dict[str, Any]] = None,
        overwrite: bool = False,
    ) -> str:
        """Resolve ``uri`` to a local file path. For ``local`` this is an identity
passthrough
        (no copy); for s3/gcs it downloads to ``dst``."""
        ref = parse_uri(uri)
        return get_storage(ref.backend, config=config).fetch_to_local(
            ref, dst, overwrite=overwrite
        )


    def put(
        local_path: "str | os.PathLike[str]",
        uri: str,
        *,
        config: Optional[Dict[str, Any]] = None,
        content_type: Optional[str] = None,
    ) -> str:
        """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
        ref = parse_uri(uri)
        get_storage(ref.backend, config=config).put(
            local_path, ref, content_type=content_type
        )
        return uri


    def exists(uri: str, *, config: Optional[Dict[str, Any]] = None) -> bool:
        """True if the object at ``uri`` exists (a bucket object or a local file)."""
        ref = parse_uri(uri)
        return bool(get_storage(ref.backend, config=config).exists(ref))


    def signed_url(
        uri: str,
        *,
        config: Optional[Dict[str, Any]] = None,
        ttl: Optional[int] = None,
        method: str = "GET",
```

```
211     ) -> str:
212         """A time-limited signed URL for ``uri`` ? the client fetches bytes DIRECTLY from
the store, off
213         the app's response path (so a large object isn't streamed back through the server;
#463). Backends
214         without object signing (e.g. local) raise ``NotImplementedError``.
215
216         ``ttl=None`` (the default) defers to the backend's configured ``signed_url_ttl``
from
217         ``config`` ? a hardcoded default here would silently shadow the operator's setting
(#469).
218         An explicit ``ttl`` overrides it for callers that need a specific expiry."""
219         ref = parse_uri(uri)
220         return get_storage(ref.backend, config=config).signed_url(ref, ttl=ttl,
method=method)
221
222
223     def list_uris(
224         prefix_uri: str, *, config: Optional[Dict[str, Any]] = None
225     ) -> Iterator[str]:
226         """Yield the URI of every object under ``prefix_uri``."""
227         ref = parse_uri(prefix_uri)
228         for r in get_storage(ref.backend, config=config).list(ref):
229             yield r.uri
230
```

113. user (2026-07-05T10:04:15.071Z)

```
1       """Canonical file-identity hashing for the indexer.
2
3       `video_id` is the MD5 of the *whole* video file. Historically each entrypoint
4       (`backend/main.py`, `run_eval.py`, `scripts/run_phase3_eval.py`,
`scripts/run_process_and_stitch.py`)
5       defined its own `get_file_md5` with a different read-chunk size (64KB / 8KB / 1MB).
6       The hash values agreed (same bytes hashed), but the duplicated code was a footgun:
7       any future change to one copy would silently diverge the DB key.
8
9       This module is the single source of truth. Import `compute_video_id` everywhere a
10      video needs to be keyed.
11      """
12
13      from __future__ import annotations
14
15      import hashlib
16
17      # 64 KB matches the value used by the live pipeline (backend/main.py) before this
18      # was centralized. The chunk size does not affect the resulting digest ? it only
19      # bounds memory use while streaming large files.
20      _CHUNK_SIZE = 65536
21
22
23      def compute_video_id(filepath: str) -> str:
24          """Return the MD5 hex digest of the whole file ? the canonical video_id.
25
26          Streams the file in fixed-size chunks so multi-gigabyte recordings never load
27          into memory at once.
28          """
29          hasher = hashlib.md5()
30          with open(filepath, "rb") as f:
31              for chunk in iter(lambda: f.read(_CHUNK_SIZE), b""):
32                  hasher.update(chunk)
33          return hasher.hexdigest()
34
```

114. assistant (2026-07-05T10:04:51.586Z)

Storage layer fully mapped. Now checking the existing upload test harness to mirror its
fixtures, then writing the P2 implementation.

115. user (2026-07-05T10:04:53.018Z)

```
1       """Chunked upload (B2) + highlight download (B3) ? PR-2 of the UI alpha plan.
2
3       Contract under test:
4       - POST /api/upload/init ? {upload_id}; extension allowlist + size caps enforced
5       - PUT /api/upload/{id}/part/{n} ? strictly sequential parts; per-part/total caps abort
(413)
6       - POST /api/upload/{id}/complete ? assembled file, MD5 == compute_video_id, no overwrite
7       - DELETE /api/upload/{id} ? abort removes partial state
8       - GET /api/highlights/{video_id}/{filename} ? streams from output_dir; traversal-proof
9       """
10
11      import os
12      from unittest.mock import patch
13
14      import pytest
15
16      pytest.importorskip("httpx")  # TestClient needs httpx; skip cleanly where absent
17      from fastapi.testclient import TestClient
18
19      import backend.main as m
20      from backend.api import uploads as uploads_api
21      from backend.config.models import AppConfig
22      from backend.infrastructure.database import Database
23      from backend.utils.hashing import compute_video_id
24
25
26      @pytest.fixture
27      def env(tmp_path, monkeypatch):
28          db = Database(str(tmp_path / "t.db"))
29          cfg = AppConfig()
30          cfg.output.output_dir = str(tmp_path / "out")
31          cfg.security.upload_dir = str(tmp_path / "up")
32          os.makedirs(cfg.output.output_dir, exist_ok=True)
33          monkeypatch.setattr(
34              uploads_api, "_uploads", {}
35          )  # isolate upload state per test (registry moved to backend.api.uploads)
36          app = m.create_app(cfg, db)
37          client = TestClient(app)
38          return client, db, tmp_path, cfg
39
40
41      def _init(client, filename="match.mp4", total=None):
42          body = {"filename": filename}
43          if total is not None:
44              body["total_size_bytes"] = total
45          return client.post("/api/upload/init", json=body)
46
47
48      # --- init ------------------------------------------------------------------------
49
50
51      def test_init_rejects_bad_extension(env):
52          client = env[0]
53          r = _init(client, "malware.exe")
54          assert r.status_code == 400
55          assert "not allowed" in r.json()["detail"]
56
57
58      @pytest.mark.parametrize(
59          "filename,stored",
```

```
60          [
61              ("match.MP4", "match.MP4"),
62              ("match?MP4", "match.MP4"),
63              ("match.MP4.", "match.MP4"),
64              ("match.MP4 ", "match.MP4"),
65          ],
66      )
67      def test_init_accepts_normalized_mp4_names(env, filename, stored):
68          client, db, tmp_path, cfg = env
69          r = _init(client, filename)
70          assert r.status_code == 200
71          upload_id = r.json()["upload_id"]
72          assert uploads_api._uploads[upload_id]["filename"] == stored
73
74
75      def test_init_rejects_traversal_filename(env):
76          client = env[0]
77          assert _init(client, "../evil.mp4").status_code == 400
78          assert _init(client, "a/b.mp4").status_code == 400
79
80
81      def test_init_rejects_oversize_declared_total(env):
82          client, db, tmp_path, cfg = env
83          cfg.security.max_upload_mb = 1
84          r = _init(client, total=2 * 1024 * 1024)
85          assert r.status_code == 413
86
87
88      def test_init_507_when_disk_too_full_for_declared_total(env):
89          # P2 (D11): a declared upload that won't fit fails fast with 507 (not a mid-upload
crash).
90          import types
91
92          from backend.utils import freespace
93
94          client = env[0]
95          with patch.object(
96              freespace.shutil,
97              "disk_usage",
98              return_value=types.SimpleNamespace(total=0, used=0, free=1 * 1024 * 1024),
99          ):
100             r = _init(client, total=50 * 1024 * 1024)
```

116. user (2026-07-05T10:06:15.453Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p2-upload-stream\backend\storage\base.py has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

117. user (2026-07-05T10:06:23.451Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p2-upload-stream\backend\storage\base.py has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

118. user (2026-07-05T10:06:27.216Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p2-upload-stream\backend\storage\gcs.py has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

119. user (2026-07-05T10:06:38.038Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p2-upload-stream\backend\storage\__init__.py
has been updated successfully. (file state is current in your context ? no need to Read it back)
```

120. user (2026-07-05T10:06:42.372Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p2-upload-stream\backend\storage\__init__.py
has been updated successfully. (file state is current in your context ? no need to Read it back)
```

121. user (2026-07-05T10:07:01.769Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p2-upload-stream\backend\api\uploads.py has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

122. user (2026-07-05T10:07:22.942Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p2-upload-stream\backend\api\uploads.py has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

123. user (2026-07-05T10:07:42.645Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p2-upload-stream\backend\api\uploads.py has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

124. user (2026-07-05T10:07:56.582Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p2-upload-stream\backend\api\uploads.py has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

125. user (2026-07-05T10:08:13.059Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p2-upload-stream\backend\api\uploads.py has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

126. assistant (2026-07-05T10:08:55.547Z)

```
Implementation complete. Writing the test suite for the streaming path.
```

127. assistant (2026-07-05T10:09:34.267Z)

```
"""GCS-streamed upload staging (CONTROL_PLANE_DURABLE_REBUILD P2 ? the #480/#471 root cause).

Contract under test:
- with a remote input backend (storage.input_backend=gcs + input_root), upload parts STREAM
  through storage.open_writer into <input_root>/uploads/<upload_id>/<name> ? the assembled
  file never exists on local disk, and the free-space guard is not consulted
- /complete returns the staged gs:// URI as `path` (submittable to /api/process verbatim ?
  it is under input_root, inside the #465 jail) and the STREAMED md5 as `video_id`
  (bit-identical to compute_video_id: same bytes, same algorithm)
- abort and the 413 overflow paths finalize-then-DELETE the partial object
- a backend whose open_writer raises NotImplementedError falls back to the local flow
- GCSStorage.open_writer streams via blob.open("wb", ignore_flush=True)
"""

import hashlib
import os

import pytest

pytest.importorskip("httpx")  # TestClient needs httpx; skip cleanly where absent
from fastapi.testclient import TestClient

import backend.main as m
import backend.storage as storage_pkg
from backend.api import uploads as uploads_api
from backend.config.models import AppConfig
from backend.infrastructure.database import Database
from backend.storage.gcs import GCSStorage
from backend.storage.ref import parse_uri
from backend.utils.hashing import compute_video_id
```

```
class _FakeWriter:
    """Records streamed bytes; materializes the object in `store` on close."""

    def __init__(self, store, key):
        self._store = store
        self._key = key
        self._buf = bytearray()
        self.closed = False

    def write(self, data):
        assert not self.closed, "write after close"
        self._buf += data
        return len(data)

    def close(self):
        if not self.closed:
            self._store[self._key] = bytes(self._buf)
            self.closed = True


class _FakeRemote:
    """The storage-backend surface uploads.py touches: open_writer + delete."""

    def __init__(self):
        self.store = {}
        self.writers = {}
        self.deleted = []
        self.configs_seen = []

    def open_writer(self, ref, *, content_type=None):
        w = _FakeWriter(self.store, ref.uri)
        self.writers[ref.uri] = w
        return w

    def delete(self, ref):
        self.deleted.append(ref.uri)
        self.store.pop(ref.uri, None)


@pytest.fixture
def env(tmp_path, monkeypatch):
    db = Database(str(tmp_path / "t.db"))
    cfg = AppConfig()
    cfg.output.output_dir = str(tmp_path / "out")
    cfg.security.upload_dir = str(tmp_path / "up")  # must stay untouched by remote staging
    cfg.storage.input_backend = "gcs"
    cfg.storage.input_root = "gs://bkt/videos"
    os.makedirs(cfg.output.output_dir, exist_ok=True)
    monkeypatch.setattr(uploads_api, "_uploads", {})
    fake = _FakeRemote()

    def _fake_get_storage(backend=None, *, config=None, **overrides):
        fake.configs_seen.append(config)
        return fake

    monkeypatch.setattr(storage_pkg, "get_storage", _fake_get_storage)
    app = m.create_app(cfg, db)
    return TestClient(app), cfg, fake, tmp_path


def _init(client, filename="match.mp4", total=None):
    body = {"filename": filename}
    if total is not None:
        body["total_size_bytes"] = total
    return client.post("/api/upload/init", json=body)
```

```
def _put_parts(client, upload_id, parts):
    for i, data in enumerate(parts):
        r = client.put(f"/api/upload/{upload_id}/part/{i}", content=data)
        assert r.status_code == 200, r.text
    return client.post(f"/api/upload/{upload_id}/complete", json={"total_parts": len(parts)})


def test_remote_parts_stream_to_bucket_with_md5_identity(env):
    client, cfg, fake, tmp_path = env
    r = _init(client, total=105_005)
    assert r.status_code == 200
    upload_id = r.json()["upload_id"]

    parts = [b"a" * 70_000, b"b" * 35_000, b"c" * 5]
    done = _put_parts(client, upload_id, parts)
    assert done.status_code == 200, done.text
    body = done.json()

    expected_uri = f"gs://bkt/videos/uploads/{upload_id}/match.mp4"
    assert body["path"] == expected_uri
    assert body["video_id"] == hashlib.md5(b"".join(parts)).hexdigest()
    assert body["size_bytes"] == sum(len(p) for p in parts)
    # The object holds exactly the streamed bytes and the writer was finalized.
    assert fake.store[expected_uri] == b"".join(parts)
    assert fake.writers[expected_uri].closed
    # Nothing was assembled locally ? the upload dir was never even created.
    assert not os.path.exists(cfg.security.upload_dir)


def test_remote_streamed_md5_matches_compute_video_id(env, tmp_path):
    # The streamed digest must be bit-identical to hashing the same bytes from disk ?
    # video_id is the DB key, so the two paths may NEVER diverge.
    client, cfg, fake, _ = env
    payload = b"x" * 123_457
    upload_id = _init(client).json()["upload_id"]
    body = _put_parts(client, upload_id, [payload]).json()
    ondisk = tmp_path / "same-bytes.mp4"
    ondisk.write_bytes(payload)
    assert body["video_id"] == compute_video_id(str(ondisk))


def test_remote_init_skips_freespace_guard(env, monkeypatch):
    client = env[0]

    def _boom(*a, **k):  # the guard reads local disk_usage ? meaningless for a bucket
        raise AssertionError("require_free_bytes must not run for remote staging")

    monkeypatch.setattr(uploads_api, "require_free_bytes", _boom)
    assert _init(client, total=3 * 1024 * 1024 * 1024).status_code == 200


def test_remote_same_name_uploads_stay_distinct(env):
    client, cfg, fake, _ = env
    a = _init(client).json()["upload_id"]
    b = _init(client).json()["upload_id"]
    pa = _put_parts(client, a, [b"first"]).json()["path"]
    pb = _put_parts(client, b, [b"second"]).json()["path"]
    assert pa != pb
    assert fake.store[pa] == b"first"
    assert fake.store[pb] == b"second"


def test_remote_abort_deletes_partial_object(env):
    client, cfg, fake, _ = env
    upload_id = _init(client).json()["upload_id"]
```

```
    assert client.put(f"/api/upload/{upload_id}/part/0", content=b"zz").status_code == 200
    r = client.delete(f"/api/upload/{upload_id}")
    assert r.status_code == 200
    uri = f"gs://bkt/videos/uploads/{upload_id}/match.mp4"
    assert uri in fake.deleted
    assert uri not in fake.store
    # State is gone: further parts 404.
    assert client.put(f"/api/upload/{upload_id}/part/1", content=b"zz").status_code == 404


def test_remote_part_overflow_413_cleans_up_the_object(env):
    client, cfg, fake, _ = env
    cfg.security.max_part_mb = 1
    upload_id = _init(client).json()["upload_id"]
    r = client.put(f"/api/upload/{upload_id}/part/0", content=b"x" * (2 * 1024 * 1024))
    assert r.status_code == 413
    assert "per-part" in r.json()["detail"]
    uri = f"gs://bkt/videos/uploads/{upload_id}/match.mp4"
    assert uri in fake.deleted
    assert uri not in fake.store
    assert client.put(f"/api/upload/{upload_id}/part/1", content=b"y").status_code == 404


def test_remote_total_cap_413_cleans_up_the_object(env):
    client, cfg, fake, _ = env
    cfg.security.max_upload_mb = 1
    upload_id = _init(client).json()["upload_id"]
    part = b"x" * (600 * 1024)
    assert client.put(f"/api/upload/{upload_id}/part/0", content=part).status_code == 200
    r = client.put(f"/api/upload/{upload_id}/part/1", content=part)
    assert r.status_code == 413
    assert "total limit" in r.json()["detail"]
    assert f"gs://bkt/videos/uploads/{upload_id}/match.mp4" in fake.deleted


def test_backend_without_streaming_falls_back_to_local(env, monkeypatch):
    client, cfg, fake, _ = env

    def _no_stream(ref, *, content_type=None):
        raise NotImplementedError("this backend cannot stream")

    monkeypatch.setattr(fake, "open_writer", _no_stream)
    upload_id = _init(client).json()["upload_id"]
    body = _put_parts(client, upload_id, [b"local bytes"]).json()
    # Local flow verbatim: an on-disk file whose md5 is the returned video_id.
    assert os.path.isfile(body["path"])
    assert body["path"].startswith(os.path.abspath(cfg.security.upload_dir))
    assert body["video_id"] == compute_video_id(body["path"])


def test_local_profile_never_touches_storage_layer(tmp_path, monkeypatch):
    # The default (input_backend=local) must not consult get_storage at all ? the
    # appliance flow stays byte-for-byte as before.
    db = Database(str(tmp_path / "t.db"))
    cfg = AppConfig()
    cfg.output.output_dir = str(tmp_path / "out")
    cfg.security.upload_dir = str(tmp_path / "up")
    os.makedirs(cfg.output.output_dir, exist_ok=True)
    monkeypatch.setattr(uploads_api, "_uploads", {})

    def _boom(*a, **k):
        raise AssertionError("get_storage must not be called for the local profile")

    monkeypatch.setattr(storage_pkg, "get_storage", _boom)
    client = TestClient(m.create_app(cfg, db))
```

```
    upload_id = _init(client).json()["upload_id"]
    body = _put_parts(client, upload_id, [b"plain local"]).json()
    assert os.path.isfile(body["path"])


def test_gcsstorage_open_writer_streams_via_blob_open():
    class _Blob:
        def __init__(self):
            self.content_type = None
            self.opened_with = None

        def open(self, mode, **kwargs):
            self.opened_with = (mode, kwargs)
            return "the-writer"

    class _Bucket:
        def __init__(self, blob):
            self._blob = blob

        def blob(self, key):
            self._key = key
            return self._blob

    class _Client:
        def __init__(self, bucket):
            self._bucket = bucket

        def bucket(self, name):
            self._name = name
            return self._bucket

    blob = _Blob()
    client = _Client(_Bucket(blob))
    s = GCSStorage(client=client)
    w = s.open_writer(parse_uri("gs://bkt/videos/uploads/u1/x.mp4"), content_type="video/mp4")
    assert w == "the-writer"
    assert blob.content_type == "video/mp4"
    mode, kwargs = blob.opened_with
    assert mode == "wb"
    # BlobWriter rejects non-boundary flush() unless told to ignore it; staging only
    # writes + closes, so flush must be a no-op rather than an error.
    assert kwargs.get("ignore_flush") is True
```

128. user (2026-07-05T10:09:34.294Z)

```
File created successfully at: C:\Users\avidu\Projects\khelsutra-guru\wt-p2-upload-
stream\tests\test_upload_gcs_streaming.py (file state is current in your context ? no need to
Read it back)
```

129. user (2026-07-05T10:09:48.336Z)

```
..................................................................... [ 98%]
.                                                                    [100%]
73 passed in 4.80s
```

130. assistant (2026-07-05T10:09:56.483Z)

```
Focused tests green (73 passed). Checking P1's CI while I run P2's full gates.
```

131. user (2026-07-05T10:10:00.649Z)

```
11
```

132. user (2026-07-05T10:13:13.110Z)

```
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
1674 passed, 10 skipped, 8 warnings in 138.84s (0:02:18)
All checks passed!
Found 1 error in 1 file (checked 148 source files)
P2_GATES_GREEN
```

133. assistant (2026-07-05T10:13:25.505Z)

P1 is fully green ? but P2's mypy found 1 error (my `tail` pipe masked the exit code). Getting the error, and merging P1 meanwhile.

134. user (2026-07-05T10:13:30.449Z)

```
backend\api\uploads.py:93: error: Item "None" of "Any | None" has no attribute "model_dump"
[union-attr]
Found 1 error in 1 file (checked 148 source files)
```

135. user (2026-07-05T10:13:34.641Z)

```
SUCCESS
```

136. assistant (2026-07-05T10:13:46.198Z)

Fixing the mypy narrowing issue and verifying the P1 merge landed.

137. user (2026-07-05T10:13:50.512Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p2-upload-stream\backend\api\uploads.py has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

138. user (2026-07-05T10:13:54.836Z)

```
MERGED 2026-07-05T10:13:34Z
e80bca8 deploy(cloudrun): bake rev-00011 pins into the manifest generator + 8Gi default (CP-
rebuild P1) (#489)
```

139. user (2026-07-05T10:14:04.179Z)

```
Updated task #1 status
```

140. user (2026-07-05T10:14:15.392Z)

```
mypy_rc=0
Success: no issues found in 148 source files
10 passed in 2.90s
All checks passed!
ruff_rc=0
```

141. user (2026-07-05T10:14:30.201Z)

```
feat/upload-gcs-streaming
cc5e83a docs(cp-rebuild): tracked spec ? durable control-plane rebuild (#471 #480 #482-#485)
(#487)
warning: in the working copy of 'tests/test_upload_gcs_streaming.py', LF will be replaced by
CRLF the next time Git touches it
[feat/upload-gcs-streaming ba08c9d] feat(uploads): stream browser-upload staging to the bucket ?
never assemble in tmpfs (CP-rebuild P2)
 5 files changed, 437 insertions(+), 9 deletions(-)
 create mode 100644 tests/test_upload_gcs_streaming.py
ba08c9d feat(uploads): stream browser-upload staging to the bucket ? never assemble in tmpfs
(CP-rebuild P2)
```

142. user (2026-07-05T10:14:54.765Z)

```
 * [new branch]      feat/upload-gcs-streaming -> feat/upload-gcs-streaming
https://github.com/Khelsutra/badminton-highlight-indexer/pull/490
```

143. user (2026-07-05T10:15:36.708Z)

```
Updated task #5 status
```

144. user (2026-07-05T10:15:41.434Z)

```
HEAD is now at e80bca8 deploy(cloudrun): bake rev-00011 pins into the manifest generator + 8Gi
default (CP-rebuild P1) (#489)
37:    DEFAULT_POLICY,
160:        policy: ExecutionPolicy = DEFAULT_POLICY,
70:    return CloudRunCompute(
```

145. user (2026-07-05T10:15:42.218Z)

```
  const { phase, submit, cancel: cancelIngest, reset: resetIngest, busy: ingestBusy } =
useIngestJob(onLoaded);
  // The UI locale recorded with the submission (PMF analytics + the localized reel watermark).
  const locale = i18n.resolvedLanguage ?? i18n.language;

  // Compute-picker (PACKAGING compute-picker, item 3): if this server can dispatch to a cloud
GPU
  // (capabilities.deployment.cloud_available), let the user choose where DETECT runs. Default
LOCAL
  // (free, no surprise cost); CLOUD requires an explicit cost acknowledgement before it can
run.
  const [cloudAvailable, setCloudAvailable] = useState(false);
  const [compute, setCompute] = useState<Compute>("local");
  const [cloudAck, setCloudAck] = useState(false);
  // Storage-medium picker (P5): which media THIS box can store into
(capabilities.storage.backends,
  // usable-only). Absent/local-only ? no picker (today's flow unchanged).
  const [backends, setBackends] = useState<StorageBackendInfo[]>([]);
  const [medium, setMedium] = useState<Medium>("local");
  const [mediumUri, setMediumUri] = useState("");
---
      // running and could still call onComplete for the OLD file after we've shown an error for
the new
      // one (processing A while the UI reports B was rejected).
      stop();

      // A 0-byte "video" is never valid ? reject before touching the network (reuse invalidUri
copy).
      if (!file || file.size === 0) {
        setPhase({ status: "error", error: { code: "invalidUri" } });
        return;
      }
      // Oversized file: fail INSTANTLY (no wasted round-trip) ? /upload/init would 413 anyway.
Carry
      // the size + cap so IngestPanel can localize a "trim/chunk it first" message. No cap ?
skip the
      // guard (the engine still fail-fasts server-side).
      const cap = maxUploadBytesRef.current;
```

146. user (2026-07-05T10:15:50.525Z)

```
/c/Users/avidu/Projects/khelsutra-guru/khelsutra/web/src/api/client.ts:255:// A browser uploads
a LOCAL file in sequential parts ? max_part_mb (free-tier edge proxies cap
/c/Users/avidu/Projects/khelsutra-guru/khelsutra/web/src/api/client.ts:329:  // Honor the
engine's max_part_mb, but never let a missing/NaN value collapse the math to NaN (which
/c/Users/avidu/Projects/khelsutra-guru/khelsutra/web/src/api/client.ts:331:  const partMb =
Number.isFinite(init.max_part_mb) && init.max_part_mb > 0 ? init.max_part_mb : 95;
```

```
/c/Users/avidu/Projects/khelsutra-guru/khelsutra/web/src/api/client.ts:334:  log("info",
"upload.start", { cid, filename: file.name, size: file.size, parts: totalParts, part_mb:
init.max_part_mb });
/c/Users/avidu/Projects/khelsutra-guru/khelsutra/web/src/api/types.ts:102:  /** The largest
upload THIS box accepts, in MB (engine `security.max_upload_mb`, default 4096).
/c/Users/avidu/Projects/khelsutra-guru/khelsutra/web/src/api/types.ts:106:  max_upload_mb?:
number;
/c/Users/avidu/Projects/khelsutra-guru/khelsutra/web/src/api/types.ts:158: *  ? max_part_mb,
then `complete` returns a server-host `path` to feed straight into POST /api/process. */
/c/Users/avidu/Projects/khelsutra-guru/khelsutra/web/src/api/types.ts:162:  max_part_mb: number;
/c/Users/avidu/Projects/khelsutra-guru/khelsutra/web/src/components/IngestPanel.tsx:97:  const
[maxUploadBytes, setMaxUploadBytes] = useState<number | undefined>(undefined);
/c/Users/avidu/Projects/khelsutra-guru/khelsutra/web/src/components/IngestPanel.tsx:106:
typeof c.max_upload_mb === "number" && c.max_upload_mb > 0 ? c.max_upload_mb * 1024 * 1024 :
undefined,
/c/Users/avidu/Projects/khelsutra-guru/khelsutra/web/src/components/IngestPanel.tsx:155:  },
maxUploadBytes);
/c/Users/avidu/Projects/khelsutra-guru/khelsutra/web/src/hooks/useUpload.ts:27:export function
useUpload(onComplete: (path: string, filename: string) => void, maxUploadBytes?: number) {
```

147. user (2026-07-05T10:16:04.922Z)

```
363     def capabilities(request: Request):
364         """Honest probe of what THIS box can do ? the first-run wizard renders the compute
step from
365         it (PACKAGING_PLAN.md P1). No secrets: reports only whether a Gemini key is PRESENT,
never its
366         value. Best-effort + never raises (the wizard degrades gracefully)."""
367         cfg = request.app.state.config
368         indexing = getattr(cfg, "indexing", None)
369         deployment = getattr(cfg, "deployment", None)
370         ffmpeg = shutil.which(ffmpeg_bin())  # P2a: honor RALLY_FFMPEG / RALLY_FFPROBE
371         ffprobe = shutil.which(ffprobe_bin())
372         # WASB weights are best-effort: env first (matches native_wasb_runner), then config.
373         wasb = getattr(indexing, "wasb", None)
374         weights_path = (
375             os.environ.get("WASB_WEIGHTS") or getattr(wasb, "weights_path", "") or ""
376         )
377         # Cheap GPU heuristic: nvidia-smi on PATH (matches scripts/doctor.py). Deliberately
NOT
378         # torch.cuda.is_available() ? torch is absent in the LIGHT tier (so it would false-
negative on a
379         # real GPU box) and importing it would add seconds to this probe. The detector
healthcheck is the
380         # authority on whether GPU inference actually works; this is just the wizard's
first-order signal.
381         nvidia_smi = shutil.which("nvidia-smi")
382         return {
383             "api_version": API_VERSION,
384             "engine_version": _engine_version(),
385             "segmenters": sorted(segmenter_registry.list_available().keys()),
386             "default_segmenter": getattr(indexing, "default_segmenter", None),
387             "ffmpeg": {
388                 "ffmpeg": bool(ffmpeg),
389                 "ffprobe": bool(ffprobe),
390                 "ffmpeg_path": ffmpeg,
391                 "ffprobe_path": ffprobe,
392             },
393             "gpu": {"nvidia_smi_present": bool(nvidia_smi)},
394             "gemini_key_present": bool(os.environ.get("GEMINI_API_KEY")),
395             "wasb_weights_present": bool(weights_path) and os.path.exists(weights_path),
396             "deployment": {
397                 "profile": getattr(deployment, "profile", "") or "",
398                 "compute_target": getattr(deployment, "compute_target", "") or "",
399                 # item 3: can this server satisfy a per-request compute="cloud" dispatch?
```

```
The SPA shows
400                 # the "run in the cloud" toggle only when true (else it's local-only).
401                 "cloud_available": _cloud_available(cfg),
402             },
403             # P3: which storage media THIS box can store INTO (local always; s3/gcs if
SDK+config;
404             # gdrive if a mount resolves). The P5 medium picker renders only usable
backends. No secrets.
405             "storage": storage_capabilities(
406                 getattr(cfg, "storage", None), local_free_path=output_base(cfg)
407             ),
408         }
409
410
411     def _friendly_video_name(filepath: str) -> str:
412         """A human name for a video from its stored path/URI ? the filename stem
(``?/match_2026.mp4`` ?
413         ``match_2026``). Works for a local path, a bucket key, or a ``gdrive://`` URI
(rsplit on ``/``).
414         Falls back to the whole ``filepath`` if there's no basename."""
415         base = str(filepath or "").rstrip("/").rsplit("/", 1)[-1].rsplit("\\", 1)[-1]
416         stem = os.path.splitext(base)[0]
417         return stem or base or str(filepath or "")
```

148. user (2026-07-05T10:16:05.410Z)

```
1       """Pluggable compute layer: in-process (default) + a Cloud Run GPU Job dispatch shim.
2
3       COMPUTE_DECOUPLED_SERVING M6. ``get_compute_backend(config)`` is the seam: it returns a
4       ``ComputeBackend`` ONLY when a non-local ``deployment.compute_target`` is selected, and
``None``
5       otherwise ? ``None`` means "run the worker's existing in-process path" (today's
behaviour, byte
6       identical). So the seam is **default-OFF**: with no profile / ``local_gpu`` /
``cpu_mock`` nothing
7       changes. Mirrors ``backend/storage``'s lazy factory idiom (cloud SDKs imported only when
needed).
8       """
9
10      from __future__ import annotations
11
12      import os
13      from typing import Any, Optional
14
15      from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
16      from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy
17
18      __all__ = [
19          "ComputeBackend",
20          "ComputeJobSpec",
21          "ComputeResult",
22          "get_compute_backend",
23      ]
24
25
26      def _storage_dict(config: Any) -> dict:
27          storage = getattr(config, "storage", None)
28          if storage is None:
29              return {}
30          if hasattr(storage, "model_dump"):
31              return storage.model_dump()  # type: ignore[no-any-return]
32          return dict(storage) if isinstance(storage, dict) else {}
33
34
35      def get_compute_backend(
```

```
36          config: Any,
37          *,
38          run_client: Optional[Any] = None,
39          policy: ExecutionPolicy = DEFAULT_POLICY,
40          token: Optional[str] = None,
41          target_override: Optional[str] = None,
42      ) -> Optional[ComputeBackend]:
43          """Return the compute backend for ``config.deployment.compute_target``, or ``None``
to run
44          in-process (the default-OFF path).
45
46          Only ``compute_target == "cloud_run"`` returns a non-None backend (a
``CloudRunCompute``). Its
47          Cloud Run coordinates come from the control-plane env (``CLOUD_RUN_JOB`` /
``CLOUD_RUN_REGION``
48          / ``GOOGLE_CLOUD_PROJECT``) so no secrets live in config. ``run_client`` is
injectable (tests
49          pass a fake; production defaults to the real ``google-cloud-run`` client, owner-
verified on M7).
50
51          ``target_override`` lets a single request force a target (the SPA compute-picker
maps a
52          per-request ``compute="cloud"`` to ``target_override="cloud_run"``) without mutating
the
53          server's configured default. ``None`` falls back to
``config.deployment.compute_target``.
54          """
55          target = (
56              target_override
57              if target_override is not None
58              else getattr(getattr(config, "deployment", None), "compute_target", "") or ""
59          )
60          if target != "cloud_run":
61              return None  # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)
62
63          # Lazy import so the cloud SDK shim is only pulled when a cloud target is actually
selected.
64          from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient
65
66          job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
67          region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")
68          project = os.environ.get("GOOGLE_CLOUD_PROJECT")
69          client = run_client if run_client is not None else GoogleCloudRunJobClient()
70          return CloudRunCompute(
71              run_client=client,
72              job_name=job_name,
73              region=region,
74              project=project,
75              storage_config=_storage_dict(config),
76              policy=policy,
77              token=token,
78          )
79
```

149. user (2026-07-05T10:16:28.235Z)

```
432:class DeploymentConfig(BaseModel):
433-    """Deployment profile ? one switch that selects a coherent bundle of compute + storage
434-    knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md`` §4.3).
435-
436-    **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
437-    behaves exactly as before this section existed. A profile is opt-in (``config.json``
438-    ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
439-    *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core reads
```

```
440-    these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
441-    (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the loader.
442-
443-    Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
444-
445-    # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
446-    profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
447-    # Where the core runs. Set by the profile preset; overridable in config.json. Inert in
M1
448-    # (recorded for observability; consumed by the ComputeBackend seam from M2+).
449-    compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
450-    # Cloud-serving throughput cap (#466): how many DETECT jobs the control-plane drain may
dispatch +
451-    # bucket-poll CONCURRENTLY when compute_target=="cloud_run". Each is an I/O-bound Cloud
Run Job
452-    # execution (the GPU work is on ephemeral L4s, WASB-only has no Gemini), so ? unlike a
truly-local
453-    # single box ? they need not serialize. Bounds concurrent PAID L4 executions. Non-
cloud_run targets
454-    # ignore this and stay serial (max_workers=1), preserving the truly-local GPU/Gemini
serialization.
455-    max_concurrent_remote_jobs: int = Field(default=4, ge=1, le=64)
456-
457-
458-class BillingConfig(BaseModel):
459-    """Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8).
460-
461-    **DEFAULT-OFF:** ``enabled=False`` ? nothing records cost (the v6 cost columns stay NULL
=
462-    today's behaviour). When enabled, a job's cost is computed + stored in **USD** via the
versioned
43-
44-
45:@dataclass(frozen=True)
46-class ExecutionPolicy:
47-    """Declarative resilience knobs, attachable per job/op (JSON-serialisable; see
EXECUTION_POLICY.md)."""
48-
49-    mode: WaitMode = WaitMode.WAIT_AND_RESUME
50-    poll_every_n_sec: float = 10.0
51-    op_timeout_sec: float = (
52-        120.0  # HARD per-attempt deadline ? the hang-killer that was missing
53-    )
116:def test_api_capabilities_shape_and_no_secret_leak(env, monkeypatch):
140:def test_api_capabilities_gemini_absent(env, monkeypatch):
```

150. assistant (2026-07-05T10:16:38.760Z)

All shapes confirmed. Implementing P5a now ? config knob, policy derivation, capabilities emit.

151. user (2026-07-05T10:16:39.690Z)

```
449         compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
450         # Cloud-serving throughput cap (#466): how many DETECT jobs the control-plane drain
may dispatch +
451         # bucket-poll CONCURRENTLY when compute_target=="cloud_run". Each is an I/O-bound
Cloud Run Job
452         # execution (the GPU work is on ephemeral L4s, WASB-only has no Gemini), so ? unlike
a truly-local
453         # single box ? they need not serialize. Bounds concurrent PAID L4 executions. Non-
cloud_run targets
454         # ignore this and stay serial (max_workers=1), preserving the truly-local GPU/Gemini
serialization.
455         max_concurrent_remote_jobs: int = Field(default=4, ge=1, le=64)
```

456
457

152. user (2026-07-05T10:16:49.700Z)

The file C:\Users\avidu\Projects\khelsutra-guru\wt-p5a-pollknob\backend\config\models.py has
been updated successfully. (file state is current in your context ? no need to Read it back)

153. user (2026-07-05T10:16:52.747Z)

The file C:\Users\avidu\Projects\khelsutra-guru\wt-p5a-pollknob\backend\compute__init__.py has
been updated successfully. (file state is current in your context ? no need to Read it back)

154. user (2026-07-05T10:16:58.778Z)

The file C:\Users\avidu\Projects\khelsutra-guru\wt-p5a-pollknob\backend\compute__init__.py has
been updated successfully. (file state is current in your context ? no need to Read it back)

155. user (2026-07-05T10:17:09.430Z)

The file C:\Users\avidu\Projects\khelsutra-guru\wt-p5a-pollknob\backend\main.py has been updated
successfully. (file state is current in your context ? no need to Read it back)

156. user (2026-07-05T10:17:23.059Z)

[Request interrupted by user]

157. user (2026-07-05T10:17:23.084Z)

please do not merge the PRs unless I have explicitly said to send and merge PRs. I am doing the
reviews so please wait for the responses while you complete other items

sry for interrupt to pass the PR merge guideline

158. user (2026-07-05T10:18:26.555Z)

The file c:\Users\avidu\.claude\projects\c--Users-avidu-Projects-khelsutra-guru\memory\working-
agreement-merging.md has been updated successfully. (file state is current in your context ? no
need to Read it back)

159. user (2026-07-05T10:18:33.897Z)

24:- [Working agreement: merging](working-agreement-merging.md) ? owner authorized Claude to
MERGE its own good-engineering-fix PRs on verified-green CI (2026-06-12);
product/strategy/licensing stay owner-gated.

160. user (2026-07-05T10:18:53.092Z)

index updated

161. user (2026-07-05T10:19:06.408Z)

https://github.com/Khelsutra/badminton-highlight-indexer/pull/490

162. user (2026-07-05T10:19:20.556Z)

"""ComputeBackend seam ? Cloud Run dispatch shim (COMPUTE_DECOUPLED_SERVING M6).

All GPU-free + cloud-free: the Cloud Run trigger is a FAKE ``CloudRunJobClient`` that simulates
the
worker (writes the done-marker + report to a local "bucket" = a tmp dir), so the
dispatch?bucket-poll
?read flow is proven for $0. The cardinal invariant: **default-OFF** ? only

```
``compute_target=cloud_run``
selects a backend; everything else returns ``None`` (= run the worker's existing in-process
path).
"""

import json
import os
import tempfile

import pytest

from backend.compute import ComputeJobSpec, get_compute_backend
from backend.compute.cloud_run import CloudRunCompute, CloudRunJobClient
from backend.config import AppConfig
from backend.infrastructure.execution_policy import ExecutionPolicy, PolicyTimeout
```

A fast, no-wait policy (the fake client writes the marker synchronously, so the first pol

```
_FAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0)


class _FakeRunClient(CloudRunJobClient):
    """Simulates a Cloud Run Job: on dispatch, write the report + done-marker into the local
bucket
    exactly where the real worker would, so CloudRunCompute's bucket-poll resolves
immediately."""

    def __init__(self, *, video_id="vidCloud", segment_count=2, returncode=0):
        self.calls: list[dict] = []
        self.video_id, self.segment_count, self.returncode = (
            video_id,
            segment_count,
            returncode,
        )

    def run_job(self, job_name, argv, *, region, project=None):
        self.calls.append(
            {"job": job_name, "argv": list(argv), "region": region, "project": project}
        )
        out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
        done_uri = argv[argv.index("--done-uri") + 1]
        outputs = os.path.join(out_uri, "outputs", self.video_id)
        os.makedirs(outputs, exist_ok=True)
        with open(os.path.join(outputs, "report.json"), "w", encoding="utf-8") as f:
            json.dump(
                {
                    "video_id": self.video_id,
                    "segment_count": self.segment_count,
                    "status": "success",
                },
===
    assert r.status_code == 200
    body = r.json()
    assert body["api_version"] == m.API_VERSION
    assert "engine_version" in body


def test_api_capabilities_shape_and_no_secret_leak(env, monkeypatch):
    client = env[0]
    monkeypatch.setenv("GEMINI_API_KEY", "AIza-should-not-leak")
    r = client.get("/api/capabilities")
    assert r.status_code == 200
    body = r.json()
    assert isinstance(body["segmenters"], list) and "motion" in body["segmenters"]
    assert "default_segmenter" in body
```

```
    assert set(body["ffmpeg"]) >= {"ffmpeg", "ffprobe"}
    assert isinstance(body["gpu"]["nvidia_smi_present"], bool)
    assert body["gemini_key_present"] is True  # presence reported...
    assert "AIza-should-not-leak" not in r.text  # ...but never the value
    assert "profile" in body["deployment"]
    # P3: the storage-medium block the picker renders from. Shape mirrors the SPA mock.
    storage = body["storage"]
    assert storage["default_medium"] == "local"
    smap = {b["id"]: b for b in storage["backends"]}
    assert set(smap) == {"local", "gcs", "s3", "gdrive"}
    assert smap["local"]["usable"] is True  # local is always usable
    for b in storage["backends"]:
        assert set(b) == {"id", "kind", "label", "usable", "free_gb"}
        assert isinstance(b["usable"], bool)


def test_api_capabilities_gemini_absent(env, monkeypatch):
```

163. user (2026-07-05T10:19:34.657Z)

```
71:        "max_concurrent_remote_jobs": 4,
397:        "max_concurrent_remote_jobs": 4,
```

164. user (2026-07-05T10:19:42.280Z)

```
    assert cfg.storage.backend == "local"


def test_default_config_dump_only_adds_deployment_section():
    """Adding the section must not perturb any other default ? the dump equals a fresh
    AppConfig()'s dump, and the new section is present with empty values."""
    dumped = load_config(NONEXISTENT).model_dump(mode="json")
    assert dumped == AppConfig().model_dump(mode="json")
    assert dumped["deployment"] == {
        "profile": "",
        "compute_target": "",
        "max_concurrent_remote_jobs": 4,
    }
```

Parity: the model Literal must match the presets table (no drift).

```
def test_profile_literal_matches_presets():
    import typing

===
    assert preset_for("banana") == {}


def test_save_default_config_includes_deployment(tmp_path):
    p = save_default_config(str(tmp_path / "out.json"))
    saved = json.loads(p.read_text())
    assert saved["deployment"] == {
        "profile": "",
        "compute_target": "",
        "max_concurrent_remote_jobs": 4,
    }
    cfg = load_config(str(p))
    assert cfg.deployment.profile == ""
```

165. user (2026-07-05T10:19:51.712Z)

```
shape assertions updated
```

166. user (2026-07-05T10:20:13.409Z)

```
28:@pytest.fixture
29:def env(tmp_path, monkeypatch):
30-    db = Database(str(tmp_path / "t.db"))
31-    cfg = AppConfig()
32-    monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
33-    app = m.create_app(cfg, db)
34-    client = TestClient(app)
35-    return client, db, tmp_path, monkeypatch
36-
37-
38-def _video(tmp_path, name="m.mp4"):
39-    p = tmp_path / name
40-    p.write_bytes(b"not-a-real-video-but-hashable")
41-    return p
===
"""Tests for the deployment-profile config (COMPUTE_DECOUPLED_SERVING M1).

What this pins:
- The default profile ("") applies NO preset ? the loaded config is byte-identical to the
  pre-M1 default, so the core is unaffected (the project's "default-OFF, zero core change").
- Each profile applies its coherent preset bundle (compute_target + the existing
  detector_impl / skip_ai_handoff / storage.backend knobs).
- Precedence: built-in defaults < profile preset < config.json (a file sub-key overrides a
  preset sub-key while sibling preset/default sub-keys survive ? i.e. real deep-merge layering).
- D15: auto-detect SUGGESTS but never auto-applies (a bare cloud-like env does NOT silently
  select a profile / spend on cloud).
- The model's Literal enum stays in parity with backend/config/presets.PROFILE_PRESETS.
"""

import json
import logging
import os
import sys

import pytest
===
    assert (
        "MAMBA_ROOT_PREFIX=/opt/conda" in text
    )  # keeps the env at /opt/conda/envs/wasb
    assert "Miniconda3" not in text  # the Anaconda installer is gone
    assert "conda tos accept" not in text  # no ToS acceptance needed (conda-forge)
```

167. user (2026-07-05T10:20:31.007Z)

```
three test files extended
```

168. user (2026-07-05T10:21:13.587Z)

```
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
107 passed, 1 skipped, 4 warnings in 33.16s
```

169. user (2026-07-05T10:24:46.012Z)

```
1 failed, 1667 passed, 10 skipped, 8 warnings in 160.22s (0:02:40)
All checks passed!
Success: no issues found in 148 source files
warning: in the working copy of 'tests/test_api_endpoints.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'tests/test_compute_backend.py', LF will be replaced by CRLF the
next time Git touches it
```

```
warning: in the working copy of 'tests/test_config_deployment.py', LF will be replaced by CRLF
the next time Git touches it
rcs: suite=1 ruff=0 mypy=0 diffcheck=0
```

170. user (2026-07-05T10:24:51.622Z)

```
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
========================== short test summary info ==========================
FAILED tests/test_main_module_budget.py::test_main_py_line_budget - Assertion...
1 failed, 1667 passed, 10 skipped, 8 warnings in 160.22s (0:02:40)
```

171. user (2026-07-05T10:25:02.180Z)

```
E       AssertionError: backend/main.py is 1662 lines (budget 1653). Don't grow the god-module
back ? extract to backend/orchestration.py / backend/api/ / a new module instead (P23/P24 in
docs/archives/past_projects/AUDIT_REPORT_khelsutra-guru_2026-07-02-COMPLETED-2026-07-02.md). If
a later split shrank main.py, TIGHTEN this budget in the same PR.
E       assert 1662 <= 1653

tests\test_main_module_budget.py:57: AssertionError
========================== short test summary info ==========================
FAILED tests/test_main_module_budget.py::test_main_py_line_budget - Assertion...
1 failed, 2 passed in 2.45s
```

172. user (2026-07-05T10:25:41.478Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p5a-pollknob\backend\main.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

173. user (2026-07-05T10:25:48.600Z)

```
50:from backend.api import (
53:from backend.api import (
214:from backend.api.job_worker import (  # noqa: F401  (re-exports: names stay addressable on
backend.main)
1039:from backend.api import uploads as uploads_api
34:# 1662 -> 1653 (#481 review): the dedup became an ATOMIC single-op claim
(backend.api.jobs.claim_process_job
36:MAIN_LINE_BUDGET = 1653
55:def test_main_py_line_budget():
```

174. user (2026-07-05T10:25:54.802Z)

```
50      from backend.api import (
51          auth as edge_auth,  # M9: Cloudflare Access edge-auth (default-OFF)
52      )
53      from backend.api import (
54          jobs as jobs_api,  # /api/jobs list route (router) + submit-dedup helper (#448)
55      )
56      from backend.config import (
57          AppConfig,
58          StartupCheckError,
59          enforce_startup_checks,
```

175. user (2026-07-05T10:26:05.682Z)

The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives backend.api.uploads as a self-contained APIRouter. It resolves the live startup config backend.main.global_config, so behavior ? route paths, status codes, body-size limits sequential-part logic, and abort cleanup ? is unchanged.

```
from backend.api import uploads as uploads_api
```

```
app.include_router(uploads_api.router)
```

--- Highlight download (B3, PR-2) ----------

176. user (2026-07-05T10:26:12.895Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p5a-pollknob\backend\main.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

177. user (2026-07-05T10:26:17.683Z)

```
<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>
```

178. user (2026-07-05T10:26:25.090Z)

```
60              e.lower().lstrip(".")
61              for e in (getattr(sec, "allowed_upload_extensions", None) or ["mp4", "mov"])
62          ]
63          return upload_dir, max_bytes, part_bytes, exts
64
65
66      def _abort_upload(upload_id: str) -> None:
67          with _uploads_lock:
68              st = _uploads.pop(upload_id, None)
69          if st:
70              try:
71                  os.remove(st["tmp_path"])
72              except OSError:
73                  pass
74
75
```

179. user (2026-07-05T10:26:39.597Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p5a-pollknob\backend\api\uploads.py has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

180. user (2026-07-05T10:27:18.357Z)

```
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
80 passed, 1 skipped, 4 warnings in 28.75s
```

181. user (2026-07-05T10:29:59.486Z)

```
1668 passed, 10 skipped, 8 warnings in 131.26s (0:02:11)
Success: no issues found in 148 source files
warning: in the working copy of 'tests/test_api_endpoints.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'tests/test_compute_backend.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'tests/test_config_deployment.py', LF will be replaced by CRLF
the next time Git touches it
rcs: suite=0 ruff=0 mypy=0 diff=0
```

182. user (2026-07-05T10:30:15.840Z)

```
feat/poll-knob-caps-emit
warning: in the working copy of 'tests/test_api_endpoints.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'tests/test_compute_backend.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'tests/test_config_deployment.py', LF will be replaced by CRLF
```

```
the next time Git touches it
[feat/poll-knob-caps-emit 7cabf47] feat(config): remote-poll budget knob (3900s) + advertise
upload caps in /api/capabilities (CP-rebuild P5a)
 7 files changed, 84 insertions(+)
7cabf47 feat(config): remote-poll budget knob (3900s) + advertise upload caps in
/api/capabilities (CP-rebuild P5a)
 * [new branch]      feat/poll-knob-caps-emit -> feat/poll-knob-caps-emit
```

183. user (2026-07-05T10:30:35.085Z)

```
https://github.com/Khelsutra/badminton-highlight-indexer/pull/491
```

184. user (2026-07-05T10:30:52.644Z)

```
Updated task #6 status
```

185. user (2026-07-05T10:30:57.477Z)

```
HEAD is now at 0e45e8f Polish ingest upload UX for cloud media (#158)
209:        startedAt: Date.now() - 10 * 60 * 1000 - 1000,
```

186. user (2026-07-05T10:31:04.329Z)

```
web/src/api/client.ts:170:const COMPILE_POLL_CEILING_MS = 10 * 60 * 1000; // a wedged 'running'
compile won't poll forever
web/src/hooks/useIngestJob.test.ts:209:        startedAt: Date.now() - 10 * 60 * 1000 - 1000,
web/src/hooks/useIngestJob.ts:18:const POLL_CEILING_MS = 10 * 60 * 1000; // a wedged 'running'
job won't poll forever

const POLL_MIN_MS = 1000;
const POLL_MAX_MS = 5000;
const POLL_BACKOFF = 1.5;
const POLL_CEILING_MS = 10 * 60 * 1000; // a wedged 'running' job won't poll forever
const ACTIVE_JOB_KEY = "khelsutra.activeIngestJob";

interface ActiveIngestJob {
  jobId: string;
  videoId: string,
  segmentIds: number[],
  outputFilename: string,
  cid?: string,
  locale?: string,
): Promise<CompileResult> {
  const c = cid ?? newCorrelationId();
```

187. user (2026-07-05T10:31:13.571Z)

```
    expect(result.current.phase).toEqual({ status: "error", error: { code: "networkError" } });
    expect(window.location.search).toContain("job=jPollError");
    expect(JSON.parse(window.localStorage.getItem(ACTIVE_JOB_KEY) ?? "{}")).toMatchObject({
jobId: "jPollError" });
  });

  it("keeps the saved job when polling hits the local timeout ceiling", async () => {
    vi.useFakeTimers();
    window.history.replaceState(null, "", "/?job=jTimeout");
    window.localStorage.setItem(
      ACTIVE_JOB_KEY,
      JSON.stringify({
        jobId: "jTimeout",
        uri: "gs://b/slow.mp4",
        cid: "cid-timeout",
        startedAt: Date.now() - 10 * 60 * 1000 - 1000,
      }),
```

```
    );
    vi.stubGlobal(
      "fetch",
      fetchSeq(httpOk(202, { job_id: "should-not-submit", status: "queued" }), [
        { job_id: "jTimeout", status: "running", progress: "still working" },
      ]),
    );
    const { result } = renderHook(() => useIngestJob());

    await act(async () => {
      await vi.advanceTimersByTimeAsync(0);
    });

    expect(result.current.phase).toEqual({ status: "error", error: { code: "timeout" } });
    expect(window.location.search).toContain("job=jTimeout");
    expect(JSON.parse(window.localStorage.getItem(ACTIVE_JOB_KEY) ?? "{}")).toMatchObject({
jobId: "jTimeout" });
  });

  it("surfaces a done-but-failed pipeline verdict with the engine message ? never as no
rallies", async () => {
    vi.useFakeTimers();
    vi.stubGlobal(
      "fetch",
      fetchSeq(httpOk(202, { job_id: "j2", status: "queued", poll: "/api/jobs/j2" }), [
        { job_id: "j2", status: "done", video_id: "vidX", result: { status: "failed", message:
"Video failed validation: blur: blurry" } },
      ]),
```

188. user (2026-07-05T10:31:32.353Z)

```
<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>
```

189. user (2026-07-05T10:31:35.691Z)

```
<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>
```

190. user (2026-07-05T10:31:36.903Z)

```
The file C:\Users\avidu\Projects\khelsutra-
guru\wt-p5b-ceilings\web\src\hooks\useIngestJob.test.ts has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

191. user (2026-07-05T10:31:43.979Z)

```
14
15      const POLL_MIN_MS = 1000;
16      const POLL_MAX_MS = 5000;
17      const POLL_BACKOFF = 1.5;
18      const POLL_CEILING_MS = 10 * 60 * 1000; // a wedged 'running' job won't poll forever
19      const ACTIVE_JOB_KEY = "khelsutra.activeIngestJob";
20
21      interface ActiveIngestJob {
```

192. user (2026-07-05T10:31:44.844Z)

```
166     // Compile-job poll cadence (mirrors useIngestJob): a long ffmpeg stitch must not block
the request.
167     const COMPILE_POLL_MIN_MS = 1000;
168     const COMPILE_POLL_MAX_MS = 5000;
169     const COMPILE_POLL_BACKOFF = 1.5;
170     const COMPILE_POLL_CEILING_MS = 10 * 60 * 1000; // a wedged 'running' compile won't poll
forever
171
```

```
172     /** Stitch selected rallies into one reel. ASYNC (#329): POST /api/compile returns 202 +
a job_id;
173      *  the ffmpeg stitch runs on the worker and we POLL GET /api/jobs/{id} for the reel ? a
long stitch
```

193. user (2026-07-05T10:31:55.105Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p5b-ceilings\web\src\hooks\useIngestJob.ts
has been updated successfully. (file state is current in your context ? no need to Read it back)
```

194. user (2026-07-05T10:31:59.483Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p5b-ceilings\web\src\api\client.ts has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

195. user (2026-07-05T10:32:18.875Z)

```
> tsc --noEmit
```

196. user (2026-07-05T10:32:36.772Z)

```
  log.ts           |   81.81 |    88.88 |     100 |   81.81 | 13-14
  query.ts         |   97.07 |    93.45 |     100 |   97.07 | ...95,284,286,288
  selection.ts     |     100 |     92.3 |     100 |     100 | 30
  video.ts         |     100 |    93.33 |     100 |     100 | 21
-------------------|---------|----------|---------|---------|-------------------

=============================== Coverage summary ===============================
Statements   : 94.79% ( 2421/2554 )
Branches     : 88.24% ( 766/868 )
Functions    : 89.4% ( 135/151 )
Lines        : 94.79% ( 2421/2554 )
================================================================================
```

197. user (2026-07-05T10:32:57.539Z)

```
 Test Files  30 passed (30)
      Tests  301 passed (301)
 M web/src/api/client.ts
 M web/src/hooks/useIngestJob.test.ts
 M web/src/hooks/useIngestJob.ts
```

198. assistant (2026-07-05T10:33:06.392Z)

```
All khelsutra web gates green (typecheck clean, 301 tests, coverage 94.79 % / 88.24 % ? above
the floors). Committing and opening the P5b PR.
```

199. user (2026-07-05T10:33:15.975Z)

```
feat/poll-ceilings-60min
0e45e8f Polish ingest upload UX for cloud media (#158)
[feat/poll-ceilings-60min 0a3bb52] Raise SPA poll ceilings 10 -> 60 min for honest cloud job
lengths (CP-rebuild P5b)
 3 files changed, 10 insertions(+), 3 deletions(-)
0a3bb52 Raise SPA poll ceilings 10 -> 60 min for honest cloud job lengths (CP-rebuild P5b)
 * [new branch]      feat/poll-ceilings-60min -> feat/poll-ceilings-60min
```

200. user (2026-07-05T10:33:32.035Z)

```
https://github.com/avidullu/khelsutra/pull/162
```

201. user (2026-07-05T10:34:04.011Z)

```
Updated task #3 status
```

202. user (2026-07-05T10:34:09.631Z)

```
HEAD is now at e80bca8 deploy(cloudrun): bake rev-00011 pins into the manifest generator + 8Gi
default (CP-rebuild P1) (#489)
0
0
```

203. assistant (2026-07-05T10:34:17.387Z)

Both engine PRs (#490, #491) are fully green and awaiting your review. Digging into the P4 integration points.

204. user (2026-07-05T10:34:18.268Z)

```
440         real agree."""
441         fp = row.get("filepath") or ""
442         fps, resolution = _video_dims(row.get("validation_results"))
443         return {
444             **row,
445             "video_id": row.get("id"),
446             "match_name": _friendly_video_name(fp),
447             "local_path": fp,
448             "created_at": row.get("processed_at") or row.get("created_at"),
449             "fps": fps,
450             "resolution": resolution,
451         }
452
453
454     @app.get("/api/videos")
455     def list_videos(request: Request, limit: Optional[int] = None):
456         """List indexed videos (most recent first) so the console survives a reload ?
previously only a
457         single-row get existed (PACKAGING_PLAN.md P1). Rows are enriched to the SPA contract
458         (``video_id`` + a friendly ``match_name`` derived from the filename) so the library
shows real
459         names and the persistent "current video" label survives an upload ? see
``_video_summary``."""
460         return {
461             "videos": [
462                 _video_summary(v) for v in request.app.state.db.list_videos(limit=limit)
463             ]
464         }
465
466
467     def _reels_dir(cfg) -> str:
468         return getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
469
470
471     _REEL_EXTS = (".mp4", ".mov")
472
473
474     @app.get("/api/reels")
475     def list_reels(request: Request):
476         """List compiled highlight reels present in the output dir, newest first, with a
download URL.
477
478         NB: reels are stored FLAT in output_dir under the bare name passed to /api/compile ?
the engine
479         does NOT track a video_id?reel association today (a per-video listing would be a
phantom), so
480         this is a global list. (Per-video reel tracking would need a DB record + compile
change ? a
```

```
        later slice.) Only top-level video files are listed (per-video scratch subdirs are skipped)."""
        base = _reels_dir(request.app.state.config)
        reels = []
        try:
            entries = os.scandir(base)
        except OSError:
            return {"reels": []}
        with entries:
            for e in entries:
                if e.is_file() and e.name.lower().endswith(_REEL_EXTS):
                    try:
                        st = e.stat()
                    except OSError:
                        continue
                    reels.append(
                        {
                            "filename": e.name,
                            "size_bytes": st.st_size,
                            "modified_at": st.st_mtime,
                            "download_url": f"/api/reels/{quote(e.name)}",
                        }
                    )
        reels.sort(key=lambda r: r["modified_at"], reverse=True)
        return {"reels": reels}


    @app.get("/api/reels/{filename}")
    def download_reel(filename: str, request: Request):
        """Stream a compiled reel by its bare filename (no video_id ? reels are flat in output_dir).
        Mirrors /api/highlights' safety (bare-name + containment checks)."""
        try:
            name = safe_output_filename(filename)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        base = _reels_dir(request.app.state.config)
        path = os.path.join(base, name)
        try:
            path = resolve_under_root(path, base)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        if not os.path.isfile(path) or not name.lower().endswith(_REEL_EXTS):
            raise HTTPException(status_code=404, detail=f"No compiled reel named '{name}'.")
        media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
        return FileResponse(path, media_type=media, filename=name)


    # --- Annotate-in-UI (P8): serve a video's source + persist finished rally labels ----------


    # In-flight annotation-proxy generations ? dedup so a browser's many Range requests to /api/source
    # don't each kick off a transcode.
    _proxy_inflight: "set[str]" = set()
    _proxy_lock = threading.Lock()
    _source_cache_locks: Dict[str, threading.Lock] = {}
    _source_cache_locks_guard = threading.Lock()


    def _proxies_dir(cfg) -> str:
        return os.path.join(output_base(cfg), "proxies")


```

```
542     def _source_cache_dir(cfg) -> str:
543         return os.path.join(output_base(cfg), "source_cache")
544 
545 
546     def _source_cache_lock(video_id: str) -> threading.Lock:
547         with _source_cache_locks_guard:
548             return _source_cache_locks.setdefault(video_id, threading.Lock())
549 
550 
551     def _source_cache_path(video_id: str, input_ref, cfg) -> str:
552         ext = os.path.splitext(input_ref.name or "")[1].lower()
553         if not ext or "/" in ext or "\\" in ext or len(ext) > 16:
554             ext = ".mp4"
555         cache_dir = _source_cache_dir(cfg)
556         name = safe_output_filename(f"{video_id}{ext}")
557         return resolve_under_root(os.path.join(cache_dir, name), cache_dir)
558 
559 
560     def _cached_remote_source(video_id: str, input_ref, cfg) -> str:
561         """Fetch a remote source once into an output-scoped source/proxy cache."""
562         cache_path = _source_cache_path(video_id, input_ref, cfg)
563         if os.path.isfile(cache_path):
564             return cache_path
```

205. assistant (2026-07-05T10:34:26.847Z)

The process logic lives in orchestration.py, not main.py. Reading the right regions.

206. user (2026-07-05T10:34:28.856Z)

```
grep: backend/main.py: No such file or directory
grep: backend/orchestration.py: No such file or directory
```

207. user (2026-07-05T10:34:35.881Z)

```
678:    Edge-auth mirrors /api/process (default-OFF ? no-op locally). ``video_id`` is validated
as a bare
745:    Edge-auth mirrors ``/api/process``. Exactly one source: an ``uploaded_path`` (a local
file from
901:@app.post("/api/process", status_code=202)
1103:    Async, mirroring /api/process: the ffmpeg stitch (trim + re-encode each clip + concat +
watermark)
55:def _finalize_reingest(db, video_id: str, segments, play_wm: int, cand_wm: int) -> None:
68:def _validation_failure_message(failures: List[Any]) -> str:
79:def _report_config_for_input(config: Any, raw_input: str) -> Any:
107:def _ingest_remote_segments(
157:def _cloud_available(cfg) -> bool:
176:def _maybe_dispatch_remote(
198:    def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
414:def _execute_processing(
464:        existing_video = db.get_video(video_id)
470:    was_reingest = existing_video is not None
488:            existing_video
489:            and existing_video.get("status") == "processed"
505:                    "results": existing_video.get("validation_results", []),
668:def _remote_input_size_bytes(input_ref, storage_config) -> Optional[int]:
693:    def __init__(self, status_code: int, detail: str):
699:def _resolve_compile(
748:def reel_object_uri(cfg, video_id: str, name: str) -> str:
758:def _storage_settings(cfg) -> Optional[Dict[str, Any]]:
766:def signed_reel_url(cfg, video_id: str, name: str) -> Optional[str]:
780:def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
```

208. user (2026-07-05T10:34:41.502Z)

```
    @app.post("/api/process", status_code=202)
    def process_video(req: ProcessRequest, request: Request):
        """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.

        Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
        client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
        runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
        worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
        """
        # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
single-user,
        # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
missing or
        # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
        try:
            owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
        except edge_auth.EdgeAuthError as e:
            raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e

        allowed_root = getattr(
            getattr(request.app.state.config, "security", None), "allowed_video_root", None
        )
        # In hosted mode a bucket URI is allowed only when it's under the configured
input_root (the
        # designated bucket) ? not an arbitrary bucket the runtime SA happens to reach
(tenant isolation).
        allowed_uri_root = getattr(
            getattr(request.app.state.config, "storage", None), "input_root", None
        )
        try:
            validate_input_path(
                req.video_path, allowed_root=allowed_root, allowed_uri_root=allowed_uri_root
            )
            # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
            # scheme, which we map to a clean 400 (an unknown scheme must never 500).
            input_ref = storage.resolve_input_ref(
                req.video_path, getattr(request.app.state.config, "storage", None)
            )
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
        # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
string. A
        # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
        # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
        # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
        # rejects a non-local URI as out-of-root.
        if input_ref.backend == "local" and not os.path.exists(input_ref.key):
            raise HTTPException(
                status_code=404, detail=f"Video file not found at '{req.video_path}'"
            )
        # P2 (D11): a bucket/Drive source is fetched to a scratch copy on the worker; fail
fast with 507
        # if this box can't hold that copy. Best-effort ? an unstattable remote
(network/SDK) or an
        # unreadable temp fs skips the guard (measuring must not deny a legit job); the
fetch itself
        # still fails loudly downstream if space genuinely runs out. Local inputs need no
copy ? skipped.
```

```
951         if input_ref.backend != "local":
952             need = _remote_input_size_bytes(
953                 input_ref, getattr(request.app.state.config, "storage", None)
954             )
955             require_free_bytes(tempfile.gettempdir(), need, stage="fetch")
956         if req.segmenter and not segmenter_registry.get(req.segmenter):
957             available = list(segmenter_registry.list_available().keys())
958             raise HTTPException(
959                 status_code=400,
960                 detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}",
961             )
962 
963         # ATOMIC idempotent submit (#448; PR-481 review): ONE write-locked DB op either
creates the queued
964         # job OR returns an already-active job for this input ? so concurrent double-
clicks/retries can't
965         # start a 2nd paid GPU run. `force` opts out. Logic in
backend.api.jobs.claim_process_job.
966         job_id = uuid.uuid4().hex
967         outcome = jobs_api.claim_process_job(request.app.state.db, job_id, req, owner_id)
968         if outcome is None:
969             raise HTTPException(status_code=500, detail="Failed to persist job record.")
970         created, dedup_response = outcome
971         if not created:  # an active job for this input already exists ? return it, no
second dispatch
972             return dedup_response
973         _get_executor().submit(
974             _drain_due_jobs, request.app.state.config, request.app.state.db
975         )
976         return JSONResponse(
977             status_code=202,
978             content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
979         )
980 
981 
982     @app.get("/api/jobs/{job_id}/cost")
983     def get_job_cost(job_id: str, request: Request):
984         """Per-job cost (COMPUTE_DECOUPLED_SERVING M8, D8). ``cost_usd`` is the source of
truth (matches
985         GCP billing); ``cost_inr`` is a display conversion at the configured FX rate.
``cost_usd`` is
986         null until billing records it (default-OFF / the placeholder pricebook ? 0)."""
987         cost = request.app.state.db.get_job_cost(job_id)
988         if cost is None:
989             raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
990         fx = request.app.state.config.billing.fx_inr_per_usd
991         usd = cost.get("cost_usd")
992         cost["cost_inr"] = billing.to_inr(usd, fx) if usd is not None else None
993         cost["fx_inr_per_usd"] = fx
994         return cost
995 
996 
997     @app.get("/api/videos/{video_id}/cost")
998     def get_video_cost(video_id: str, request: Request):
999         """Aggregate per-video cost across its jobs (M8): ``total_cost_usd`` +
``total_cost_inr`` + the
1000         per-job rows. A video is 1:1 with an infer job, but a re-ingest can produce
several."""
```

209. user (2026-07-05T10:35:12.892Z)

```
1       """Job-list endpoint (the processing-queue view) as its own router ? kept OUT of the
backend.main
2       god-module per the P24 route ceiling (see tests/test_main_module_budget.py; pattern:
```

```
backend/api/uploads.py).
3
4       GET /api/jobs is tenant-scoped and status-filtered so a client can discover in-flight
work it did NOT
5       submit (a page reload, a new tab, another device) and show its status ? which also stops
a tester who
6       can't see their running job from re-submitting the same video into a second, PAID GPU
run (#448). The
7       per-submit dedup that makes that safe server-side lives inline in
backend.main.process_video.
8
9       Reads the live DB + config off ``request.app.state`` (set in backend.main.create_app),
so it needs no
10      module-global wiring and stays trivially testable via the standard
TestClient(create_app(cfg, db)).
11      """
12
13      from typing import Any, List, Optional, Tuple
14
15      from fastapi import APIRouter, HTTPException, Request
16      from fastapi.responses import JSONResponse
17
18      from backend.api import (
19          auth as edge_auth,  # M9: Cloudflare Access edge-auth (default-OFF)
20      )
21
22      router = APIRouter()
23
24
25      def claim_process_job(
26          db: Any, job_id: str, req: Any, owner_id: Optional[str]
27      ) -> Optional[Tuple[bool, Optional[JSONResponse]]]:
28          """ATOMIC idempotent-submit hook for backend.main.process_video (PR #481 review):
delegates to
29          ``db.claim_or_create_process_job`` (ONE write-locked op) so two concurrent identical
submits ? a
30          double-click / retry ? can't both create a job and start a second PAID GPU run.
Returns
31          ``(created, dedup_response)``: when ``created`` is True the caller dispatches; when
False the caller
32          just returns ``dedup_response`` (a 202 pointing at the already-active job). ``None``
on DB failure.
33          ``req.force`` opts out (deliberate reprocess). Keyed on (owner_id, video_path) ? the
input isn't
34          hashed until the worker runs, so the submitted ref is the stable pre-flight key."""
35          claimed = db.claim_or_create_process_job(
36              job_id,
37              req.video_path,
38              segmenter=req.segmenter,
39              player_pool=req.player_pool,
40              max_frames=req.max_frames,
41              owner_id=owner_id,
42              locale=req.locale,
43              compute=req.compute,
44              force=req.force,
45          )
46          if claimed is None:
47              return None
48          if claimed["created"]:
49              return (True, None)
50          return (
51              False,
52              JSONResponse(
53                  status_code=202,
54                  content={
```

```
                    "job_id": claimed["job_id"],
                    "status": claimed["status"],
                    "poll": f"/api/jobs/{claimed['job_id']}",
                    "deduplicated": True,
                },
            ),
        )

    # Execution states a job can be in. ACTIVE = still doing (or about to do) work; the
queue view defaults
    # to these. done/failed are terminal and only surface under ?status=all (or an explicit
single status).
    ACTIVE_JOB_STATES = ("queued", "running", "waiting")
    _LISTABLE_JOB_STATES = frozenset(ACTIVE_JOB_STATES) | {"done", "failed"}


    @router.get("/api/jobs")
    def list_jobs(request: Request, status: str = "active"):
        """The caller's jobs (tenant-scoped), newest first. ``status=active`` (default) ?
        queued|running|waiting; ``status=all`` ? recent jobs of any state; a single explicit
state is also
        accepted (anything unknown degrades to the active queue ? never a raw 500). Light
projection: the
        heavy per-job ``play_segments`` payload is dropped; ``result_status`` carries the
terminal verdict."""
        try:
            owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
        except edge_auth.EdgeAuthError as e:
            raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
        if status == "all":
            statuses: Optional[List[str]] = None
        elif status in _LISTABLE_JOB_STATES:
            statuses = [status]
        else:  # "active" and any unknown value ? the active queue
            statuses = list(ACTIVE_JOB_STATES)
        rows = request.app.state.db.list_jobs(owner_id, statuses=statuses, limit=50)
        return {
            "jobs": [
                {
                    "job_id": j["id"],
                    "kind": j.get("kind") or "process",
                    "status": j["status"],
                    "progress": j.get("progress"),
                    "video_id": j.get("video_id"),
                    "source": j.get("video_path"),
                    "compute": j.get("compute"),
                    "result_status": j.get("result_status"),
                    "error": j.get("error"),
                    "created_at": j.get("created_at"),
                    "started_at": j.get("started_at"),
                    "finished_at": j.get("finished_at"),
                }
                for j in rows
            ]
        }

```

210. user (2026-07-05T10:35:18.722Z)

```
    def _execute_processing(
        config=None, db=None, req=None, progress=None
    ) -> Dict[str, Any]:
        """The full hash ? validate ? segment ? report pipeline for one video.

        Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
```

```
        module-level ``global_config``/``db_instance`` (backward-compatible with existing
        tests that pass only ``req``). Route handlers always pass explicit params; older
        test code that calls ``_execute_processing(req)`` still works.

        ``progress`` is an optional callable(stage: str) used to surface coarse job
progress.
        Raises on unexpected internal errors ? the caller records those as a job failure
        """
        import backend.main as _main  # call-time seam: main's globals/model stay patchable

        # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
        # shift positional args ? req arrives in position 0 (config), progress in position 1
(db).
        if config is not None and not isinstance(config, AppConfig):
            # Old signature: _execute_processing(req, progress=None)
            if req is None and db is not None and isinstance(db, _main.ProcessRequest):
                req = db
                db = None
            elif req is None and isinstance(config, _main.ProcessRequest):
                req = config
                config = None
        # Resolve config/db from module globals when not passed explicitly
        if config is None:
            config = _main.global_config
        if db is None:
            db = _main.db_instance

        notify = progress or (lambda stage: None)

        notify("hashing")
        # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
        # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
        # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
        # consumers (hash / validators / segmenter) use the resolved local path.
        local_video = ""
        try:
            local_video = storage.acquire_local_input(
                req.video_path, getattr(config, "storage", None)
            )
            video_id = _main.compute_video_id(local_video)
        except Exception:
            storage.cleanup_acquired_local_input(
                req.video_path, local_video, getattr(config, "storage", None)
            )
            raise
        try:
            existing_video = db.get_video(video_id)
        except Exception:
            storage.cleanup_acquired_local_input(
                req.video_path, local_video, getattr(config, "storage", None)
            )
            raise
        was_reingest = existing_video is not None

        # typed AppConfig: attribute access for the default segmenter (with safe fallback)
        default_seg = getattr(
            getattr(config, "indexing", None), "default_segmenter", "motion"
        )
        seg_name = req.segmenter or default_seg
        segmenter_actual = None
        segmentation_ran = False
        val_results: List[Any] = []
```

```
480         val_context: Dict[str, Any] = {}
481         # Compute-path PROVENANCE (the validation method): which backend actually ran the
DETECT ?
482         # "in_process" once the local segmenter is reached, set on the response as
"cloud_run" by
483         # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
484         compute_actual: Optional[str] = None
485         response: Optional[Dict[str, Any]] = None
486         try:
487             if (
488                 existing_video
489                 and existing_video.get("status") == "processed"
490                 and not req.force
491             ):
492                 cached_segments = db.query_play_segments(video_id, {})
493                 if cached_segments:
494                     logger.info(
495                         "[API] Reusing cached segmentation for video_id=%s; force=true
bypasses cache.",
496                         video_id,
497                     )
498                     response = {
499                         "video_id": video_id,
500                         "status": "success",
501                         "message": (
502                             f"Reused {len(cached_segments)} cached play segments. "
503                             "Pass force=true to reprocess this video."
504                         ),
505                         "results": existing_video.get("validation_results", []),
506                         "play_segments": cached_segments,
507                         "cached": True,
508                         "compute": {
509                             "path": "cached",
510                             "requested": getattr(req, "compute", None),
511                         },
512                     }
513                     return response
514
515             # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for
the report)
516             notify("validating")
517             logger.info(f"Running validations for {req.video_path}...")
518             val_results, val_context = registry.run_all_with_context(local_video, config)
519             failures = [r for r in val_results if not r.passed]
520
521             if failures:
522                 # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
523                 # status; it no longer destroys the video's prior good segments.
524                 db.add_video(
525                     video_id,
526                     req.video_path,
527                     "failed",
528                     [r.model_dump() for r in val_results],
529                 )
530                 response = {
531                     "video_id": video_id,
532                     "status": "failed",
533                     "message": _validation_failure_message(failures),
534                     "results": [r.model_dump() for r in val_results],
535                 }
536                 return response
537
538             # 2. Run segmenter & indexer
539             logger.info("[API] Video passed checks. Segmenting and indexing...")
```

```
540             db.add_video(
541                 video_id, req.video_path, "processed", [r.model_dump() for r in val_results]
542             )
543 
544             segmenter_cls = segmenter_registry.get(seg_name)
545             if not segmenter_cls:
546                 # Submit-time validation already 400s unknown names; this is the defensive
547                 # in-worker path (e.g. config default pointing at an unregistered
segmenter).
548                 available = list(segmenter_registry.list_available().keys())
549                 response = {
550                     "video_id": video_id,
551                     "status": "failed",
552                     "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
553                     "results": [r.model_dump() for r in val_results],
554                     "play_segments": [],
555                 }
556                 return response
557 
558             logger.info(f"[API] Using segmenter: {seg_name}")
559             notify(f"segmenting ({seg_name})")
560             segmenter_actual = seg_name
561             # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
562             # if the run yields segments; otherwise drop the new partial rows and keep the
old.
563             play_wm, cand_wm = db.segment_watermarks(video_id)
564             # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
565             # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
566             # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
567             # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
568             # they're dropped on the cloud path in this slice ? a documented follow-up.)
```

211. user (2026-07-05T10:36:29.282Z)

```
107     def _ingest_remote_segments(
108         db, video_id: str, outputs_uri: str, storage_config: dict, *, max_rows: int = 20000
109     ) -> int:
110         """Read a cloud job's ``intervals.csv`` (start,end,shot_count,ending_reason) from
the bucket and
111         persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters
call ? so
112         ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they
were produced.
113         Returns the count ingested. Mirrors ``cloud_run._read_json``'s fetch-to-a-real-dst
pattern (cloud
114         backends REQUIRE a dst). Hardened against a hostile/corrupt worker output: a
malformed row is
115         skipped, non-finite / overflowing values are rejected, the loop is bounded, and the
temp dir is
116         cleaned. May raise (missing/unreadable CSV, fetch error) ? the caller treats that as
a job failure
117         and prunes any partial rows (destroy-AFTER-confirm)."""
118         uri = f"{outputs_uri.rstrip('/')}/intervals.csv"
119         ref = storage.parse_uri(uri)
120         n = 0
121         with tempfile.TemporaryDirectory(prefix="cr-ingest-") as td:
122             local = storage.fetch(
123                 uri, os.path.join(td, ref.name or "intervals.csv"), config=storage_config
124             )
```

```
125             with open(local, newline="", encoding="utf-8") as f:
126                 for row in csv.DictReader(f):
127                     if n >= max_rows:
128                         logger.warning(
129                             "[API] cloud ingest hit the %d-row cap (%s) ? ignoring the
rest.",
130                             max_rows,
131                             uri,
132                         )
133                         break
134                     try:
135                         start, end = float(row["start"]), float(row["end"])
136                     except (KeyError, TypeError, ValueError):
137                         continue
138                     if not (math.isfinite(start) and math.isfinite(end)):
139                         continue  # reject inf/nan times rather than persist a junk rally
140                     meta: Dict[str, Any] = {
141                         "source": "cloud_run",
142                         "ending_reason": (row.get("ending_reason") or "").strip(),
143                     }
144                     sc = (row.get("shot_count") or "").strip()
145                     if sc:
146                         try:
147                             meta["shot_count"] = int(
148                                 float(sc)
149                             )  # OverflowError on 'inf'/'1e400'
150                         except (ValueError, OverflowError):
151                             meta["shot_count"] = sc
152                     db.add_play_segment(video_id, start, end, metadata=meta)
153                     n += 1
154         return n
155
156
157     def _cloud_available(cfg) -> bool:
158         """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item
3).
```

212. user (2026-07-05T10:36:31.351Z)

```
106:    WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`",
149:    Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done' means
150:    the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
210:    Best-effort ? never fails a completed job because cost bookkeeping hiccuped. ``job`` is
the
232:    except Exception as e:  # never wedge a completed job on bookkeeping
247:    Best-effort ? never fails a completed job because the flywheel hiccuped. ``job`` is the
claimed
257:    except Exception as e:  # never wedge a completed job on the flywheel
===
    def claim_or_create_process_job(
        self,
        job_id: str,
        video_path: str,
        segmenter: Optional[str] = None,
        player_pool: Optional[List[str]] = None,
        max_frames: Optional[int] = None,
        policy: Optional[Dict[str, Any]] = None,
        owner_id: Optional[str] = None,
        locale: Optional[str] = None,
        compute: Optional[str] = None,
        force: bool = False,
    ) -> Optional[Dict[str, Any]]:
        """ATOMIC idempotent submit for a PROCESS job (fixes the read-before-insert race ? PR
#481
        review): in ONE write-locked statement, insert ``job_id`` as 'queued' ONLY IF no active
```

```
        (queued|running|waiting) process job already exists for (owner_id, video_path). Two
concurrent
        identical submits (a double-click / retry) therefore can't both insert ? SQLite
serializes
        writers, and the second's ``INSERT ? WHERE NOT EXISTS`` sees the first's row (same
        compare-and-set idiom as ``claim_due_job``, no read-then-write gap). ``force`` skips the
guard
        (deliberate reprocess ? always inserts). Returns ``{"job_id","status","created"}`` ?
``created``
        False means an active job already existed and the caller must NOT dispatch a second run
? or
        ``None`` on failure."""
        params = json.dumps({"force": True}) if force else None
        vals = (
            job_id,
            video_path,
            segmenter,
            json.dumps(player_pool) if player_pool is not None else None,
            max_frames,
            json.dumps(policy) if policy is not None else None,
            owner_id,
            locale,
            compute,
            params,
        )
        owner_clause = "owner_id IS NULL" if owner_id is None else "owner_id = ?"
        guard = () if owner_id is None else (owner_id,)
        cols = (
            "id, video_path, segmenter, player_pool, max_frames, status, policy, "
            "owner_id, locale, compute, params"
        )
        try:
            with self._connect() as conn:
                cur = conn.cursor()
                if force:
                    cur.execute(
                        f"INSERT INTO jobs ({cols}) VALUES (?, ?, ?, ?, ?, 'queued', ?, ?, ?, ?,
?)",
                        vals,
                    )
                    conn.commit()
                    return {"job_id": job_id, "status": "queued", "created": True}
                # Conditional insert ? one atomic statement: only if no active process job
exists for
                # this (owner, input). SQLite serializes writers, so a racing second submit sees
the row.
                cur.execute(
                    f"INSERT INTO jobs ({cols}) "
                    "SELECT ?, ?, ?, ?, ?, 'queued', ?, ?, ?, ?, ? WHERE NOT EXISTS ("
                    "SELECT 1 FROM jobs WHERE " + owner_clause + " AND video_path = ? "
                    "AND status IN ('queued', 'running', 'waiting') "
                    "AND (kind IS NULL OR kind != 'compile'))",
                    vals + guard + (video_path,),
                )
                conn.commit()
                if cur.rowcount == 1:
                    return {"job_id": job_id, "status": "queued", "created": True}
                # Lost the race (or a pre-existing active job): return the winner, no second
dispatch.
                row = (
                    cur.execute(
                        "SELECT id, status FROM jobs WHERE " + owner_clause + " AND video_path =
? "
                        "AND status IN ('queued', 'running', 'waiting') "
                        "AND (kind IS NULL OR kind != 'compile') ORDER BY created_at DESC LIMIT
```

```
1",
                        guard + (video_path,),
                    ).fetchone()
                )
```

213. user (2026-07-05T10:36:50.216Z)

```
11:    def _connect(self):  # pragma: no cover - implemented by Database facade
14:    def _execute_write(  # pragma: no cover - implemented by Database facade
23:    def create_job(
62:    def create_compile_job(
91:    def count_jobs_by_locale(self) -> Dict[str, int]:
105:    def mark_job_running(self, job_id: str) -> bool:
113:    def set_job_progress(self, job_id: str, progress: str) -> bool:
120:    def set_job_video_id(self, job_id: str, video_id: str) -> bool:
127:    def mark_job_done(self, job_id: str, result: Dict[str, Any]) -> bool:
137:    def mark_job_failed(self, job_id: str, error: str) -> bool:
146:    def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
164:    def list_jobs(
201:    def find_active_process_job(
228:    def claim_or_create_process_job(
309:    def get_pricebook_rates(
339:    def record_job_cost(
404:    def get_job_cost(self, job_id: str) -> Optional[Dict[str, Any]]:
425:    def get_video_cost(self, video_id: str) -> Dict[str, Any]:
457:    def set_job_policy(self, job_id: str, policy: Optional[Dict[str, Any]]) -> bool:
465:    def set_job_waiting(
479:    def seconds_until_next_due(self) -> Optional[float]:
501:    def claim_due_job(self) -> Optional[Dict[str, Any]]:
542:    def fail_stale_jobs(self, reason: str = "interrupted by server restart") -> int:
564:    def recover_stale_jobs(
207:                video_id TEXT NOT NULL,
238:                video_id TEXT NOT NULL,
272:                video_id TEXT,
149:    Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done' means
179:        db.mark_job_done(job_id, result)
```

214. user (2026-07-05T10:37:52.117Z)

```
26      @dataclass(frozen=True)
27      class StorageStat:
28          """Lightweight object metadata (uniform across backends)."""
29
30          size: int
31          modified: Optional[datetime] = None
32          etag: Optional[str] = None
33          content_type: Optional[str] = None
34
35
```

215. user (2026-07-05T10:37:52.700Z)

```
1       """Google Cloud Storage backend ? first-class object buckets.
2
3       Uses ``google-cloud-storage``, imported lazily so ``import backend.storage`` never pulls
it;
4       install with the ``storage-gcs`` extra. Credentials come from Application Default
Credentials
5       (``GOOGLE_APPLICATION_CREDENTIALS`` pointing at a service-account JSON, or workload
identity) ?
6       never from config. A ``client`` may be injected for tests.
7       """
8
9       from __future__ import annotations
```

```
10
11      import logging
12      from datetime import timedelta
13      from importlib import import_module
14      from typing import Any, Iterator, Optional
15
16      from backend.storage.base import StorageBackend
17      from backend.storage.ref import StorageRef, StorageStat
18
19      logger = logging.getLogger(__name__)
20
```

216. user (2026-07-05T10:38:09.932Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p4-md5cache\backend\storage\ref.py has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

217. user (2026-07-05T10:38:11.817Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p4-md5cache\backend\storage\gcs.py has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

218. user (2026-07-05T10:38:14.612Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p4-md5cache\backend\storage\gcs.py has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

219. user (2026-07-05T10:38:18.409Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p4-md5cache\backend\storage\gcs.py has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

220. assistant (2026-07-05T10:38:24.057Z)

```
Now the claim layer ? `video_id` at INSERT time:
```

221. user (2026-07-05T10:38:24.965Z)

```
228         def claim_or_create_process_job(
229             self,
230             job_id: str,
231             video_path: str,
232             segmenter: Optional[str] = None,
233             player_pool: Optional[List[str]] = None,
234             max_frames: Optional[int] = None,
235             policy: Optional[Dict[str, Any]] = None,
236             owner_id: Optional[str] = None,
237             locale: Optional[str] = None,
238             compute: Optional[str] = None,
239             force: bool = False,
240         ) -> Optional[Dict[str, Any]]:
241             """ATOMIC idempotent submit for a PROCESS job (fixes the read-before-insert race
? PR #481
242             review): in ONE write-locked statement, insert ``job_id`` as 'queued' ONLY IF no
active
243             (queued|running|waiting) process job already exists for (owner_id, video_path).
Two concurrent
244             identical submits (a double-click / retry) therefore can't both insert ? SQLite
serializes
245             writers, and the second's ``INSERT ? WHERE NOT EXISTS`` sees the first's row
(same
246             compare-and-set idiom as ``claim_due_job``, no read-then-write gap). ``force``
skips the guard
247             (deliberate reprocess ? always inserts). Returns
```

``{"job_id","status","created"}`` ? ``created``
248   False means an active job already existed and the caller must NOT dispatch a
second run ? or
249   ``None`` on failure."""
250   params = json.dumps({"force": True}) if force else None
251   vals = (
252   job_id,
253   video_path,
254   segmenter,
255   json.dumps(player_pool) if player_pool is not None else None,
256   max_frames,
257   json.dumps(policy) if policy is not None else None,
258   owner_id,
259   locale,

222. user (2026-07-05T10:38:39.656Z)

The file C:\Users\avidu\Projects\khelsutra-
guru\wt-p4-md5cache\backend\infrastructure\database_jobs.py has been updated successfully. (file
state is current in your context ? no need to Read it back)

223. user (2026-07-05T10:38:45.998Z)

268   params,
269   )
270   owner_clause = "owner_id IS NULL" if owner_id is None else "owner_id = ?"
271   guard = () if owner_id is None else (owner_id,)
272   cols = (
273   "id, video_path, segmenter, player_pool, max_frames, status, policy, "
274   "owner_id, locale, compute, params"
275   )
276   try:
277   with self._connect() as conn:
278   cur = conn.cursor()
279   if force:
280   cur.execute(
281   f"INSERT INTO jobs ({cols}) VALUES (?, ?, ?, ?, ?, 'queued', ?,
?, ?, ?, ?)",
282   vals,
283   )
284   conn.commit()
285   return {"job_id": job_id, "status": "queued", "created": True}
286   # Conditional insert ? one atomic statement: only if no active process
job exists for
287   # this (owner, input). SQLite serializes writers, so a racing second
submit sees the row.
288   cur.execute(
289   f"INSERT INTO jobs ({cols}) "
290   "SELECT ?, ?, ?, ?, ?, 'queued', ?, ?, ?, ?, ? WHERE NOT EXISTS ("
291   "SELECT 1 FROM jobs WHERE " + owner_clause + " AND video_path = ? "
292   "AND status IN ('queued', 'running', 'waiting') "
293   "AND (kind IS NULL OR kind != 'compile'))",
294   vals + guard + (video_path,),
295   )
296   conn.commit()
297   if cur.rowcount == 1:
298   return {"job_id": job_id, "status": "queued", "created": True}
299   # Lost the race (or a pre-existing active job): return the winner, no
second dispatch.
300   row = (
301   cur.execute(
302   "SELECT id, status FROM jobs WHERE " + owner_clause + " AND
video_path = ? "
303   "AND status IN ('queued', 'running', 'waiting') "
304   "AND (kind IS NULL OR kind != 'compile') ORDER BY created_at

```
DESC LIMIT 1",
305                             guard + (video_path,),
306                         ).fetchone()
307                     )
308                     if row:
309                         return {"job_id": str(row["id"]), "status": str(row["status"]),
"created": False}
310                     return None
311             except Exception as e:
312                 logger.error(f"Failed to claim-or-create process job {job_id}: {e}")
313                 return None
314
315         # --- Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8)
----------------------------
316         def get_pricebook_rates(
317             self, version: str, gpu_type: Optional[str] = None
```

224. user (2026-07-05T10:39:00.988Z)

```
The file C:\Users\avidu\Projects\khelsutra-
guru\wt-p4-md5cache\backend\infrastructure\database_jobs.py has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

225. user (2026-07-05T10:39:53.469Z)

```
330         # Forward the engine's AI-handoff stance so the cloud job runs the SAME detection:
with
331         # skip_ai_handoff=True the WASB windows become rallies directly (no Gemini key
needed); without
332         # forwarding the cloud job would default to the handoff and yield 0 rallies when it
has no key.
333         # (player_pool / max_frames are still dropped on the cloud path ? a separate
documented follow-up.)
334         overrides = []
335         sk = getattr(getattr(cfg, "indexing", None), "skip_ai_handoff", None)
336         if sk is not None:
337             overrides.append(f"indexing.skip_ai_handoff={'true' if sk else 'false'}")
338         spec = ComputeJobSpec(
339             video_uri=req.video_path,
340             out_uri=out_root,
341             segmenter=seg_name,
342             config_overrides=tuple(overrides),
343         )
344         try:
345             result = compute.run_infer(
346                 spec
347             )  # dispatch the Cloud Run Job + bucket-poll to completion
348         except Exception as e:  # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
349             logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
350             return _failed(f"cloud dispatch error: {e}", True)
351         if result.returncode != 0:
352             return _failed(f"cloud job failed (returncode {result.returncode}).", True)
353         if result.video_id and result.video_id != video_id:
354             # The worker re-hashes the bucket bytes; a divergence means the input changed
between this
355             # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
diagnosable.
356             logger.warning(
357                 "[API] cloud video_id %s != local %s ? input bytes differ between the API
hash "
358                 "and the worker hash; ingesting under the local id.",
359                 result.video_id,
360                 video_id,
361             )
```

```
362
363         notify("ingesting (cloud_run)")
364         storage_dict = (
365             storage_cfg.model_dump()  # type: ignore[union-attr]  # hasattr-guarded; mypy
can't narrow
366             if hasattr(storage_cfg, "model_dump")
367             else {}
368         )
369         try:
370             _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
371         except Exception as e:  # noqa: BLE001 ? a missing/partial/corrupt result must not
500 or leak rows
372             logger.error("[API] cloud ingest failed: %s", e, exc_info=True)
373             return _failed(
374                 f"cloud-serving: could not read the result from {result.outputs_uri}: {e}",
375                 True,
376             )
377
378         # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
drops the old ones.
379         segments = [
380             s for s in db.query_play_segments(video_id, {}) if int(s.get("id", 0)) > play_wm
381         ]
382         _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
383         # Compute-path PROVENANCE: the GCP-verifiable proof the cloud path ran. `execution`
is the real
384         # Cloud Run execution name (cross-check: gcloud run jobs executions describe
<execution>).
385         provenance = {
386             "path": "cloud_run",
387             "requested": req.compute,
388             "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
389             "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
390             "execution": result.execution,
391             "outputs_uri": result.outputs_uri,
392         }
393         logger.info(
394             "[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s
video_id=%s",
395             result.execution,
396             provenance["job"],
397             provenance["region"],
398             result.outputs_uri,
399             video_id,
400         )
401         return (
402             {
403                 "video_id": video_id,
404                 "status": "success",
405                 "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
406                 "results": [r.model_dump() for r in val_results],
407                 "play_segments": segments,
408                 "compute": provenance,
409             },
410             True,
411         )
412
413
```

226. assistant (2026-07-05T10:40:05.001Z)

The finalize pattern is clear ? mirroring it exactly. Writing the orchestration helpers now.

227. user (2026-07-05T10:40:06.804Z)

```
157     def _cloud_available(cfg) -> bool:
158         """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item
3).
159
160         True when the control plane is wired for Cloud Run: either the configured default
target is
161         already ``cloud_run`` (the operator chose cloud serving), or the Cloud Run env
coordinates
162         (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT``) and an output bucket
(``storage.output_root``)
163         are present so a forced per-request override can actually dispatch + collect a
result.
164
165         Surfaced in ``/api/capabilities`` so the SPA only offers the cloud toggle when it
would work."""
166         deployment = getattr(cfg, "deployment", None)
167         if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
168             return True
169         has_job = bool(
170             os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT")
171         )
172         out_root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
173         return bool(has_job and out_root)
174
175
176     def _maybe_dispatch_remote(
177         cfg,
178         db,
```

228. user (2026-07-05T10:40:33.320Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p4-md5cache\backend\orchestration.py has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

229. user (2026-07-05T10:40:46.510Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p4-md5cache\backend\orchestration.py has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

230. user (2026-07-05T10:41:04.539Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p4-md5cache\backend\api\jobs.py has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

231. assistant (2026-07-05T10:41:09.520Z)

Now the main.py route swap ? replacing the claim block with the composed helper:

232. user (2026-07-05T10:41:18.561Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p4-md5cache\backend\main.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

233. user (2026-07-05T10:41:37.268Z)

```
895:    Counterweight = test_process_hosted_mode_rejects_nonlocal_uri (gs:// -> 400 when
allowed_video_root
993:    """Hosted mode (allowed_video_root set) must reject a bucket/Drive URI ? a tenant can't
point
998:    mp.setattr(m.global_config.security, "allowed_video_root", str(root))
===
    def mark_job_done(self, job_id: str, result: Dict[str, Any]) -> bool:
        """Job EXECUTED to completion. The pipeline's own verdict (success/failed
        validation) is inside `result` ? job status 'done' means 'the worker finished'."""
```

```
        return self._execute_write(
            "UPDATE jobs SET status='done', progress='finished', result=?, "
            "finished_at=CURRENT_TIMESTAMP WHERE id = ?",
            (json.dumps(result), job_id),
            f"Failed to mark job {job_id} done",
        )

    def mark_job_failed(self, job_id: str, error: str) -> bool:
        """Worker crashed / raised ? infrastructure failure, not a pipeline verdict."""
        return self._execute_write(
            "UPDATE jobs SET status='failed', error=?, "
            "finished_at=CURRENT_TIMESTAMP WHERE id = ?",
            (error, job_id),
            f"Failed to mark job {job_id} failed",
        )

    def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
===


class _CompileError(Exception):
    """A submit-time-validatable compile problem ? carries the HTTP status + detail so the API
path
    maps it to an HTTPException (fast 4xx) and the worker path maps it to a failed-job
result."""

    def __init__(self, status_code: int, detail: str):
        super().__init__(detail)
        self.status_code = status_code
        self.detail = detail


def _resolve_compile(
    db, cfg, video_id: str, segment_ids: List[int], output_filename: str
):
    """Shared compile resolution ? run at SUBMIT (fast-fail 4xx) AND in the WORKER (re-resolve,
since a
    row could change between submit and run). Verifies the video exists, the requested segments
match
    and survive the ``stitching.min_shot_count`` floor (segments WITHOUT a known shot_count are
exempt
    so manual/legacy picks can't silently vanish), and the output name is traversal-safe. Raises
    ``_CompileError(404/400)``. Returns ``(video_row, kept_segments, safe_out_name)``."""
    video = db.get_video(video_id)
    if not video:
        raise _CompileError(404, "Indexed video record not found.")
    # Reject path traversal / absolute names (os.path.join would otherwise write anywhere on
disk).
    try:
        out_name = safe_output_filename(output_filename)
    except ValueError as e:
        raise _CompileError(400, str(e)) from e

    all_segments = db.query_play_segments(video_id, {})
```

234. assistant (2026-07-05T10:42:18.991Z)

`mark_job_done` is an unconditional UPDATE ? instant-complete works from `queued`. Writing the P4 test suite.

235. assistant (2026-07-05T10:42:50.265Z)

```
"""Submit-time MD5 + completed-work cache-reuse (#484 ? CONTROL_PLANE_DURABLE_REBUILD P4).

Contract under test:
```

```
- a gs:// submit resolves the video's md5 from OBJECT METADATA (one stat, no bytes) and
  persists it on the job row at CREATE (the restart-recovery keystone, spec §4.2)
- when detection already exists ? DB first, bucket `outputs/<md5>/report.json` second ?
  /api/process completes the job INSTANTLY: no drain kick, no GPU dispatch ("MD5 ? instant")
- the bucket branch re-hydrates via the SAME ingest the real cloud path uses (watermarks +
  finalize), so a retry replaces rows rather than double-counting them (#485 mechanism)
- `force=true` bypasses the cache but still records the md5; no-md5 / failed-report inputs
  degrade to the normal queued dispatch
- GCS md5Hash (base64) ? compute_video_id (hex) conversion is exact
"""

import base64
import hashlib
import json
import os

import pytest

pytest.importorskip("httpx")  # TestClient needs httpx; skip cleanly where absent
from fastapi.testclient import TestClient

import backend.main as m
import backend.storage as storage_pkg
from backend.config.models import AppConfig
from backend.infrastructure.database import Database
from backend.orchestration import cached_process_result, submit_time_video_id
from backend.storage.gcs import _md5_hex
from backend.storage.ref import StorageStat, parse_uri

MD5 = hashlib.md5(b"video-bytes").hexdigest()
_INTERVALS = (
    "start,end,shot_count,ending_reason\n"
    "1.0,9.5,4,rally_end\n"
    "12.0,30.25,7,rally_end\n"
)


class _FakeBucketStorage:
    """The storage surface the submit path touches: stat (md5 metadata), exists, fetch."""

    def __init__(self, md5=MD5, report="success"):
        self.md5 = md5
        self.report = (
            None
            if report is None
            else {"video_id": md5, "segment_count": 2, "status": report}
        )
        self.stat_uris = []
        self.fetched = []

    def stat(self, ref):
        self.stat_uris.append(ref.uri)
        return StorageStat(size=123_456, md5=self.md5)

    def exists(self, ref):
        return self.report is not None and ref.uri.endswith("/report.json")

    def fetch_to_local(self, ref, dst=None, *, overwrite=False):
        self.fetched.append(ref.uri)
        os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
        if ref.uri.endswith("report.json") and self.report is not None:
            with open(dst, "w", encoding="utf-8") as f:
                json.dump(self.report, f)
        elif ref.uri.endswith("intervals.csv"):
            with open(dst, "w", encoding="utf-8", newline="") as f:
```

```
                f.write(_INTERVALS)
        else:
            raise FileNotFoundError(ref.uri)
        return dst


class _RecordingExecutor:
    def __init__(self, calls):
        self._calls = calls

    def submit(self, fn, *args, **kwargs):
        self._calls.append(fn)


@pytest.fixture
def env(tmp_path, monkeypatch):
    db = Database(str(tmp_path / "t.db"))
    cfg = AppConfig()
    cfg.output.output_dir = str(tmp_path / "out")
    # The hosted-alpha shape: gcs input/output roots + the #465 jail enabled.
    cfg.storage.backend = "gcs"
    cfg.storage.output_root = "gs://bkt"
    cfg.storage.input_backend = "gcs"
    cfg.storage.input_root = "gs://bkt/videos"
    cfg.security.allowed_video_root = str(tmp_path / "jail")
    os.makedirs(cfg.output.output_dir, exist_ok=True)
    fake = _FakeBucketStorage()
    monkeypatch.setattr(storage_pkg, "get_storage", lambda *a, **k: fake)
    drains = []
    monkeypatch.setattr(m, "_get_executor", lambda: _RecordingExecutor(drains))
    client = TestClient(m.create_app(cfg, db))
    return client, db, cfg, fake, drains


def _submit(client, force=False):
    body = {"video_path": "gs://bkt/videos/m.mp4"}
    if force:
        body["force"] = True
    return client.post("/api/process", json=body)
```

--- md5Hash (base64) ? hex identity ----------

```
def test_gcs_md5hash_base64_converts_to_video_id_hex():
    assert _md5_hex(base64.b64encode(bytes.fromhex(MD5)).decode()) == MD5
    assert _md5_hex(None) is None
    assert _md5_hex("") is None
    assert _md5_hex("not-base64!!") is None
    assert _md5_hex(base64.b64encode(b"short").decode()) is None  # not a 16-byte digest
```

--- the instant path ----------

```
def test_bucket_cache_hit_completes_job_instantly_without_dispatch(env):
    client, db, cfg, fake, drains = env
    r = _submit(client)
    assert r.status_code == 202, r.text
    body = r.json()
    assert body["status"] == "done"
    assert body["cached"] is True
    job = db.get_job(body["job_id"])
```

```
    assert job["status"] == "done"
    assert job["video_id"] == MD5  # persisted at CREATE ? the restart-recovery keystone
    result = json.loads(job["result"])
    assert result["cached"] is True and result["status"] == "success"
    assert len(result["play_segments"]) == 2
    # The rehydrate landed in the DB (bucket = truth, DB = cache)?
    assert db.get_video(MD5)["status"] == "processed"
    assert len(db.query_play_segments(MD5, {})) == 2
    # ?and NOTHING was dispatched: no drain kick means no GPU run.
    assert drains == []


def test_db_cache_hit_skips_the_bucket_entirely(env):
    client, db, cfg, fake, drains = env
    fake.report = None  # a bucket consult would find nothing ? the DB must satisfy it
    db.add_video(MD5, "gs://bkt/videos/m.mp4", "processed", [])
    db.add_play_segment(MD5, 1.0, 9.5, metadata={"source": "test"})
    r = _submit(client)
    assert r.json()["status"] == "done"
    assert drains == []
    assert fake.fetched == []  # no bucket reads at all


def test_force_bypasses_cache_but_still_records_md5(env):
    client, db, cfg, fake, drains = env
    r = _submit(client, force=True)
    body = r.json()
    assert body["status"] == "queued"
    assert len(drains) == 1  # dispatched normally
    assert db.get_job(body["job_id"])["video_id"] == MD5


def test_no_metadata_md5_degrades_to_normal_dispatch(env):
    client, db, cfg, fake, drains = env
    fake.md5 = None  # e.g. a composite object
    body = _submit(client).json()
    assert body["status"] == "queued"
    assert len(drains) == 1
    assert (db.get_job(body["job_id"])["video_id"] or "") == ""


def test_failed_bucket_report_dispatches_normally(env):
    client, db, cfg, fake, drains = env
    fake.report = {"video_id": MD5, "segment_count": 0, "status": "failed"}
    body = _submit(client).json()
    assert body["status"] == "queued"
    assert len(drains) == 1
```

--- helper-level contracts ----------

```
def test_submit_time_video_id_is_gcs_only():
    assert submit_time_video_id(parse_uri("C:/videos/m.mp4"), AppConfig()) is None


def test_rehydrate_is_idempotent_across_retries(tmp_path, monkeypatch):
    """A second re-hydrate must REPLACE the first's rows (the real cloud-ingest watermark +
    finalize semantics), never stack duplicates ? a partial earlier ingest self-heals."""
    db = Database(str(tmp_path / "t.db"))
    cfg = AppConfig()
    cfg.storage.backend = "gcs"
    cfg.storage.output_root = "gs://bkt"
    fake = _FakeBucketStorage()
```

```
    monkeypatch.setattr(storage_pkg, "get_storage", lambda *a, **k: fake)
    first = cached_process_result(db, cfg, MD5, "gs://bkt/videos/m.mp4")
    assert first is not None and len(first["play_segments"]) == 2
    # Wipe the video row's processed status? No ? retry the SAME call (e.g. after a crash
    # between ingest and respond): rows must not double.
    monkeypatch.setattr(db, "get_video", lambda vid: None)  # force the bucket branch again
    second = cached_process_result(db, cfg, MD5, "gs://bkt/videos/m.mp4")
    assert second is not None and len(second["play_segments"]) == 2
    monkeypatch.undo()
    assert len(db.query_play_segments(MD5, {})) == 2
```

236. user (2026-07-05T10:42:50.312Z)

```
File created successfully at: C:\Users\avidu\Projects\khelsutra-
guru\wt-p4-md5cache\tests\test_submit_md5_cache.py (file state is current in your context ? no
need to Read it back)
```

237. user (2026-07-05T10:43:06.395Z)

```
>               raise TypeError(f'the JSON object must be str, bytes or bytearray, '
                                f'not {s.__class__.__name__}')
E               TypeError: the JSON object must be str, bytes or bytearray, not dict

..\..\..\AppData\Local\Programs\Python\Python313\Lib\json\__init__.py:345: TypeError
=========================== short test summary info ===========================
FAILED
tests/test_submit_md5_cache.py::test_bucket_cache_hit_completes_job_instantly_without_dispatch
1 failed, 7 passed in 3.96s
```

238. user (2026-07-05T10:43:15.946Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p4-md5cache\tests\test_submit_md5_cache.py
has been updated successfully. (file state is current in your context ? no need to Read it back)
```

239. user (2026-07-05T10:43:24.138Z)

```
no tests ran in 0.00s
```

240. user (2026-07-05T10:44:24.698Z)

```
test_async_jobs.py
test_job_self_scheduling.py
test_job_worker_loop.py
test_job_worker_module_seam.py

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
97 passed, 1 skipped, 10 warnings in 48.71s
```

241. user (2026-07-05T10:48:05.936Z)

```
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
All checks passed!
Success: no issues found in 148 source files
rcs: suite=0 ruff=0 mypy=0 diff=0
```

242. user (2026-07-05T10:48:26.380Z)

```
1673 passed, 10 skipped, 18 warnings in 167.94s (0:02:47)
feat/submit-md5-cache-reuse
warning: in the working copy of 'tests/test_submit_md5_cache.py', LF will be replaced by CRLF
the next time Git touches it
[feat/submit-md5-cache-reuse 0aca0ff] feat(process): submit-time MD5 + completed-work cache-
reuse ? same video never re-runs the GPU (CP-rebuild P4, #484)
 7 files changed, 431 insertions(+), 35 deletions(-)
```

```
 create mode 100644 tests/test_submit_md5_cache.py
0aca0ff feat(process): submit-time MD5 + completed-work cache-reuse ? same video never re-runs
the GPU (CP-rebuild P4, #484)
 * [new branch]      feat/submit-md5-cache-reuse -> feat/submit-md5-cache-reuse
```

243. user (2026-07-05T10:48:54.030Z)

```
https://github.com/Khelsutra/badminton-highlight-indexer/pull/492
```

244. user (2026-07-05T10:49:27.593Z)

```
Updated task #8 status
```

245. user (2026-07-05T10:49:29.193Z)

```
backend/billing.py
backend/flywheel.py
backend/main.py
backend/orchestration.py
backend/results.py
backend/server_runtime.py
backend/pipeline/detectors/native_wasb_runner.py
backend/pipeline/detectors/__pycache__/native_wasb_runner.cpython-313.pyc
backend/worker/__main__.py
```

246. user (2026-07-05T10:49:36.808Z)

```
31:vs the cached WSL-disk CSV matched **1194/1195** frames (the lone diff a ~5e-5 px edge
37:import logging
51:logger = logging.getLogger(__name__)
60:    "import torch; print('torch', torch.__version__, 'cuda', torch.cuda.is_available())"
80:    scratch_dir: str = ""  # frame-extraction scratch (env); "" = next to the output dir
82:    keep_frames: bool = (
83:        False  # retain the decoded frame cache after success (debug only)
95:    def from_indexing_cfg(cls, idx_cfg: "IndexingConfig") -> "NativeWasbConfig":
114:            keep_frames=w.keep_frames,
124:    def __init__(self, cfg: NativeWasbConfig):
131:    def _run(
134:        logger.debug("native exec: %s (cwd=%s)", " ".join(argv), cwd)
144:    def _probe_cuda(self) -> bool:
153:    def _cuda_is_available(self) -> bool:
159:    def _effective_device(self) -> str:
166:    def _repo_src(self) -> str:
169:    def _copy_infer_script(self) -> bool:
177:            logger.error("Failed to copy wasb_infer.py into %s: %s", repo_src, e)
180:    def _rm_rf(self, path: Optional[str]) -> None:
185:    def healthcheck(self) -> Tuple[bool, str]:
229:            logger.warning(
236:    def run_predict(
255:            logger.error("Could not stage wasb_infer.py into the WASB repo src/.")
265:        frames_dir: Optional[str] = None
271:            frames_dir = os.path.join(scratch, f"{key}_frames")
272:            argv += ["--frames_out_dir", frames_dir]
293:        logger.info(
302:            logger.error(
307:            logger.error(
312:            logger.error(
===
  340 backend/pipeline/detectors/native_wasb_runner.py
  751 backend/worker/__main__.py
 1091 total
91:def _write_report_json(
167:    done_uri: str,
175:    """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer) for a
```

```
remote
187:        storage.put(local, done_uri, config=storage_cfg)
188:        logger.info("[worker] wrote completion marker -> %s", done_uri)
190:        logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
238:        # budget expires. DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
239:        if getattr(args, "done_uri", None):
241:                args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
315:    _write_report_json(
316:        video_id, segments, status, os.path.join(out_local, "report.json")
332:    # M6: a remote dispatcher passes --done-uri; write the SUCCESS completion marker so its
bucket-poll
```

247. user (2026-07-05T10:50:23.972Z)

```
HEAD is now at e80bca8 deploy(cloudrun): bake rev-00011 pins into the manifest generator + 8Gi
default (CP-rebuild P1) (#489)


def poll_until(
    check: Callable[[], Optional[T]],
    policy: ExecutionPolicy = DEFAULT_POLICY,
    *,
    label: str = "op",
    logger: Optional[logging.Logger] = None,
    sleep: Callable[[float], None] = _time.sleep,
    monotonic: Callable[[], float] = _time.monotonic,
) -> T:
    """Poll ``check()`` every ``poll_every_n_sec`` until it returns a non-``None`` value, or
raise
    ``PolicyTimeout`` after ``max_wait_sec``. ``check`` returns ``None`` while the op is still
in progress.

    Polls log at DEBUG (no INFO spam); the wait start + resolution + give-up log at
INFO/WARNING.
    """
    log = logger or LOG
    start = monotonic()
    polls = 0
    log.info(
        "%s: waiting (poll_every=%.0fs, max_wait=%.0fs)",
        label,
        policy.poll_every_n_sec,
        policy.max_wait_sec,
    )
    while True:
        result = check()
        if result is not None:
            log.info(
                "%s: ready after %d poll(s) / %.0fs", label, polls, monotonic() - start
            )
            return result
        elapsed = monotonic() - start
        if elapsed >= policy.max_wait_sec:
            log.warning(
                "%s: still not ready after %.0fs (max_wait) ? giving up", label, elapsed
            )
            raise PolicyTimeout(
                f"{label} not ready within max_wait_sec={policy.max_wait_sec}s"
            )
        polls += 1
        log.debug("%s: poll %d, %.0fs elapsed, not ready", label, polls, elapsed)
        sleep(policy.poll_every_n_sec)
backend/compute/base.py:56:    def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
backend/compute/cloud_run.py:179:    def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
```

248. user (2026-07-05T10:50:30.672Z)

```
        def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
            out_root = spec.out_uri.rstrip("/")
            token = self._token or uuid.uuid4().hex
            done_uri = f"{out_root}/_dispatch/{token}.done"
            argv: List[str] = [
                "infer",
                "--video-uri",
                spec.video_uri,
                "--out-uri",
                spec.out_uri,
                "--done-uri",
                done_uri,
            ]
            if spec.segmenter:
                argv += ["--segmenter", spec.segmenter]
            for kv in (*spec.config_overrides, *self._container_overrides):
                argv += ["--set", kv]

            execution = self._client.run_job(
                self._job_name, argv, region=self._region, project=self._project
            )
            LOG.info(
                "cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
                self._job_name,
                execution,
                done_uri,
            )

            try:
                marker = poll_until(
                    lambda: self._read_marker(done_uri),
                    self._policy,
                    label="cloud-run-infer",
                    logger=LOG,
                )
            except PolicyTimeout as timeout_exc:
                marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
            video_id = str(marker.get("video_id", ""))
            # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
            # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
            rc = int(marker.get("returncode", 1))
            if not video_id:
                LOG.warning(
                    "cloud-run: job=%s completion marker present but empty/unreadable (no "
                    "video_id) ? treating as failure",
                    self._job_name,
                )
                rc = rc or 1
            outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
            report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
            LOG.info(
                "cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc
            )
            return ComputeResult(
                returncode=rc,
                outputs_uri=outputs_uri,
                video_id=video_id,
                report=report,
                execution=str(execution or ""),
            )
```

249. user (2026-07-05T10:50:31.448Z)

```
44          # The backend's execution identity for PROVENANCE (e.g. the Cloud Run execution
name) ? surfaced
45          # on the job result so "the cloud path actually ran" is independently verifiable
(gcloud run jobs
46          # executions describe <execution>). Empty for the in-process path / backends without
one.
47          execution: str = ""
48
49
50      class ComputeBackend(ABC):
51          """Backend-agnostic compute. ``run_infer`` dispatches ONE inference job and returns
only once
52          its outputs are durably available (bucket-poll for cloud), so a caller treats local
and cloud
53          execution identically."""
54
55          @abstractmethod
56          def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
57              """Run ``spec`` to completion and return its ``ComputeResult``."""
58
```

250. user (2026-07-05T10:50:46.488Z)

```
tests/test_compute_backend.py:413:        def run_infer(self, spec):
backend/compute/base.py:56:    def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
```

251. user (2026-07-05T10:51:15.124Z)

```
395         )
396         assert result.returncode == 0
397         assert result.video_id == "vlate"  # the late result is ingested, not wasted
398         assert client.cancelled == []  # a finished execution is never cancelled
399         assert exists_calls["n"] == 2  # exactly ONE extra post-timeout check
400
401
402     # ---------------------------------------------------- worker wiring: dispatch branch +
done-marker
403
404
405     def test_cmd_infer_dispatches_when_cloud_target(tmp_path, monkeypatch):
406         """cmd_infer with compute_target=cloud_run hands off to the ComputeBackend (no in-
process run)."""
407         from backend.compute.base import ComputeResult
408         from backend.worker import __main__ as wmain
409
410         seen = {}
411
412         class _FakeBackend:
413             def run_infer(self, spec):
414                 seen["spec"] = spec
415                 return ComputeResult(
416                     returncode=0, video_id="vidX", outputs_uri="gs://b/outputs/vidX/"
417                 )
418
419         monkeypatch.setattr(wmain, "get_compute_backend", lambda config: _FakeBackend())
420         # cloud-serving startup checks require the AI-handoff key (AI_HANDOFF_KEY_MISSING is
fatal).
421         monkeypatch.setenv("GEMINI_API_KEY", "test-key")
422         cfg = tmp_path / "c.json"
423         cfg.write_text(
424             '{"deployment": {"profile": "cloud-serving"}, "storage": {"backend": "gcs"}}'
425         )
426         rc = wmain.cmd_infer(
```

```
427             wmain.build_parser().parse_args(
428                 [
429                     "infer",
430                     "--video-uri",
431                     "gs://b/videos/clip.mp4",
432                     "--out-uri",
433                     "gs://b",
434                     "--config",
435                     str(cfg),
436                     "--segmenter",
437                     "wasb_hybrid",
438                 ]
439             )
440         )
441         assert rc == 0
442         assert seen["spec"].video_uri == "gs://b/videos/clip.mp4"
443         assert seen["spec"].segmenter == "wasb_hybrid"
444
445
446     def test_worker_writes_done_marker_on_success(tmp_path, monkeypatch):
447         """The container side: cmd_infer with --done-uri writes a completion marker
(video_id + rc) so a
448         remote dispatcher's bucket-poll terminates. Driven by the deterministic stub + skip-
AI."""
449         from backend.worker import __main__ as wmain
```

252. user (2026-07-05T10:51:28.064Z)

```
tests/test_api_cloud_dispatch.py:128:    """Make backend.compute.get_compute_backend (re-
imported inside _maybe_dispatch_remote) build a
tests/test_api_cloud_dispatch.py:131:    real = compute_pkg.get_compute_backend
tests/test_api_cloud_dispatch.py:134:        "get_compute_backend",
tests/test_api_cloud_dispatch.py:332:    """compute_target='' ? get_compute_backend returns None
? the in-process segmenter runs
```

253. user (2026-07-05T10:51:35.417Z)

```
    )
    monkeypatch.setattr(
        m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
    )
    return db, bucket, monkeypatch


def _inject_fake(monkeypatch, fake):
    """Make backend.compute.get_compute_backend (re-imported inside _maybe_dispatch_remote)
build a
    real CloudRunCompute around the FAKE client + the fast no-wait policy + a fixed token.
``**kw``
    forwards the per-request ``target_override`` (item 3) so the cloud-picker path resolves
too."""
    real = compute_pkg.get_compute_backend
    monkeypatch.setattr(
        compute_pkg,
        "get_compute_backend",
        lambda config, **kw: real(
            config, run_client=fake, policy=_FAST, token="tok", **kw
        ),
    )


def _meta(row):
    md = row.get("metadata")
    return json.loads(md) if isinstance(md, str) else (md or {})
```

```
def test_cloud_dispatch_ingests_segments(cloud_env):
    db, _bucket, monkeypatch = cloud_env
    fake = _FakeCloudClient()
    _inject_fake(monkeypatch, fake)
```

254. assistant (2026-07-05T10:52:05.154Z)

Harness understood. Implementing the four-point change now.

255. user (2026-07-05T10:52:17.967Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

256. user (2026-07-05T10:52:26.842Z)

```
150
151
152     def poll_until(
153         check: Callable[[], Optional[T]],
154         policy: ExecutionPolicy = DEFAULT_POLICY,
155         *,
156         label: str = "op",
157         logger: Optional[logging.Logger] = None,
158         sleep: Callable[[float], None] = _time.sleep,
159         monotonic: Callable[[], float] = _time.monotonic,
160     ) -> T:
161         """Poll ``check()`` every ``poll_every_n_sec`` until it returns a non-``None``
value, or raise
162         ``PolicyTimeout`` after ``max_wait_sec``. ``check`` returns ``None`` while the op is
still in progress.
163
164         Polls log at DEBUG (no INFO spam); the wait start + resolution + give-up log at
INFO/WARNING.
165         """
166         log = logger or LOG
167         start = monotonic()
168         polls = 0
169         log.info(
170             "%s: waiting (poll_every=%.0fs, max_wait=%.0fs)",
171             label,
172             policy.poll_every_n_sec,
173             policy.max_wait_sec,
174         )
175         while True:
176             result = check()
177             if result is not None:
178                 log.info(
179                     "%s: ready after %d poll(s) / %.0fs", label, polls, monotonic() - start
180                 )
181                 return result
182             elapsed = monotonic() - start
183             if elapsed >= policy.max_wait_sec:
184                 log.warning(
185                     "%s: still not ready after %.0fs (max_wait) ? giving up", label, elapsed
186                 )
187                 raise PolicyTimeout(
188                     f"{label} not ready within max_wait_sec={policy.max_wait_sec}s"
189                 )
190             polls += 1
191             log.debug("%s: poll %d, %.0fs elapsed, not ready", label, polls, elapsed)
192             sleep(policy.poll_every_n_sec)
193
```

257. user (2026-07-05T10:52:44.150Z)

```
The file C:\Users\avidu\Projects\khelsutra-
guru\wt-p6-heartbeat\backend\infrastructure\execution_policy.py has been updated successfully.
(file state is current in your context ? no need to Read it back)
```

258. user (2026-07-05T10:52:53.153Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p6-heartbeat\backend\compute\base.py has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

259. user (2026-07-05T10:53:01.769Z)

```
13:from __future__ import annotations
15:from abc import ABC, abstractmethod
16:from dataclasses import dataclass, field
17:from typing import Optional
```

260. user (2026-07-05T10:53:09.196Z)

```
13      from __future__ import annotations
14
15      from abc import ABC, abstractmethod
16      from dataclasses import dataclass, field
17      from typing import Optional
18
```

261. user (2026-07-05T10:53:16.624Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p6-heartbeat\backend\compute\base.py has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

262. user (2026-07-05T10:53:26.567Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p6-heartbeat\backend\compute\cloud_run.py has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

263. user (2026-07-05T10:53:29.855Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p6-heartbeat\backend\compute\cloud_run.py has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

264. user (2026-07-05T10:53:38.330Z)

```
25:from __future__ import annotations
27:import json
28:import logging
29:import os
30:import tempfile
31:import uuid
32:from abc import ABC, abstractmethod
33:from typing import Any, List, Optional
35:from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
36:from backend.infrastructure.execution_policy import (
342-        config_overrides=tuple(overrides),
343-    )
344-    try:
345:        result = compute.run_infer(
346-            spec
347-        )  # dispatch the Cloud Run Job + bucket-poll to completion
348-    except Exception as e:  # noqa: BLE001 ? any dispatch/poll error is a clean job failure,
not a 500
349-        logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
350-        return _failed(f"cloud dispatch error: {e}", True)
351-    if result.returncode != 0:
```

265. user (2026-07-05T10:53:45.478Z)

```
cloud_run imports ok
```

266. assistant (2026-07-05T10:53:53.993Z)

Now the orchestration wiring ? the heartbeat message + threading `on_wait` through the dispatch call.

267. user (2026-07-05T10:53:55.123Z)

```
336         if sk is not None:
337             overrides.append(f"indexing.skip_ai_handoff={'true' if sk else 'false'}")
338         spec = ComputeJobSpec(
339             video_uri=req.video_path,
340             out_uri=out_root,
341             segmenter=seg_name,
342             config_overrides=tuple(overrides),
343         )
344         try:
345             result = compute.run_infer(
346                 spec
347             )  # dispatch the Cloud Run Job + bucket-poll to completion
348         except Exception as e:  # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
349             logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
350             return _failed(f"cloud dispatch error: {e}", True)
351         if result.returncode != 0:
352             return _failed(f"cloud job failed (returncode {result.returncode}).", True)
353         if result.video_id and result.video_id != video_id:
```

268. user (2026-07-05T10:54:06.947Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p6-heartbeat\backend\orchestration.py has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

269. user (2026-07-05T10:54:16.548Z)

```
157     def _cloud_available(cfg) -> bool:
158         """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item
3).
159
160         True when the control plane is wired for Cloud Run: either the configured default
target is
161         already ``cloud_run`` (the operator chose cloud serving), or the Cloud Run env
coordinates
162         (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT``) and an output bucket
(``storage.output_root``)
163         are present so a forced per-request override can actually dispatch + collect a
result.
164
165         Surfaced in ``/api/capabilities`` so the SPA only offers the cloud toggle when it
would work."""
166         deployment = getattr(cfg, "deployment", None)
167         if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
168             return True
169         has_job = bool(
170             os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT")
171         )
172         out_root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
173         return bool(has_job and out_root)
174
175
176     def _maybe_dispatch_remote(
```

270. user (2026-07-05T10:54:27.848Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p6-heartbeat\backend\orchestration.py has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

271. assistant (2026-07-05T10:54:51.182Z)

Code complete. Locating the existing test patterns to extend.

272. user (2026-07-05T10:54:52.853Z)

```
tests/test_compute_backend.py
tests/test_execution_policy.py
184:        ComputeJobSpec(
209:        ComputeJobSpec(
237:        ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bk"))
```

273. user (2026-07-05T10:55:19.241Z)

three test files extended

274. user (2026-07-05T10:55:54.379Z)

```
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
62 passed, 4 warnings in 24.94s
```

275. user (2026-07-05T10:59:05.569Z)

```
1669 passed, 10 skipped, 8 warnings in 137.67s (0:02:17)
Success: no issues found in 148 source files
warning: in the working copy of 'backend/compute/cloud_run.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'tests/test_api_cloud_dispatch.py', LF will be replaced by CRLF
the next time Git touches it
warning: in the working copy of 'tests/test_compute_backend.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'tests/test_execution_policy.py', LF will be replaced by CRLF
the next time Git touches it
rcs: suite=0 ruff=0 mypy=0 diff=0
```

276. assistant (2026-07-05T10:59:10.872Z)

All P6 gates green (1669 passed). Committing and opening the PR.

277. user (2026-07-05T10:59:23.877Z)

```
feat/cloud-poll-heartbeat
warning: in the working copy of 'backend/compute/cloud_run.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'tests/test_api_cloud_dispatch.py', LF will be replaced by CRLF
the next time Git touches it
warning: in the working copy of 'tests/test_compute_backend.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'tests/test_execution_policy.py', LF will be replaced by CRLF
the next time Git touches it
[feat/cloud-poll-heartbeat b9e2806] feat(progress): live cloud-poll heartbeat ? job.progress
moves every tick during a GPU run (CP-rebuild P6 slice, #482)
 7 files changed, 135 insertions(+), 7 deletions(-)
b9e2806 feat(progress): live cloud-poll heartbeat ? job.progress moves every tick during a GPU
run (CP-rebuild P6 slice, #482)
 * [new branch]      feat/cloud-poll-heartbeat -> feat/cloud-poll-heartbeat
```

278. user (2026-07-05T10:59:46.728Z)

```
https://github.com/Khelsutra/badminton-highlight-indexer/pull/493
https://github.com/Khelsutra/badminton-highlight-indexer/issues/482#issuecomment-4885756433
```

279. user (2026-07-05T11:00:11.758Z)

```
engine #490:       1 (lenient       3 fixtures       1 license       1 link       1 lint       1
scan       1 sweep       1 test       1 wheel
engine #491:       1 (lenient       3 fixtures       1 license       1 link       1 lint       1
scan       1 sweep       1 test       1 wheel
engine #492:       1 (lenient       3 fixtures       1 license       1 link       1 lint       1
scan       1 sweep       1 test       1 wheel
engine #493:       1 (lenient       3 fixtures       1 license       1 link       1 lint       1
scan       1 sweep       1 test       1 wheel
khelsutra #162:       1 Builds:       3 pass
```

280. user (2026-07-05T11:00:12.974Z)

```
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/wt-p1-genyaml': Permission
denied
error: failed to delete '.git/worktrees/wt-p1-genyaml': Permission denied
fatal: 'C:/Users/avidu/Projects/khelsutra-guru/wt-cp-rebuild-spec' is not a working tree
Deleted branch feat/genyaml-pins-8gi (was a74d947).
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer
e80bca8 [master]
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/cfg-wt        20b2d5a [claude/wasb-
config-validation]
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/compile-wt    b09a1d4 [claude/compile-
reliability]
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/pr458-pre-wt  355dad6 (detached HEAD)
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/pr458-wt      5de1ef2 (detached HEAD)
C:/Users/avidu/Projects/khelsutra-guru/wt-p2-upload-stream
ba08c9d [feat/upload-gcs-streaming]
C:/Users/avidu/Projects/khelsutra-guru/wt-p4-md5cache
0aca0ff [feat/submit-md5-cache-reuse]
C:/Users/avidu/Projects/khelsutra-guru/wt-p5a-pollknob
7cabf47 [feat/poll-knob-caps-emit]
C:/Users/avidu/Projects/khelsutra-guru/wt-p6-heartbeat
b9e2806 [feat/cloud-poll-heartbeat]
C:/Users/avidu/Projects/khelsutra-guru/wt-pr481-review
84403e7 (detached HEAD)
C:/Users/avidu/Projects/khelsutra-guru/wt-review-492-mergecheck
088d848 [review/pr492-mergecheck]
```

281. user (2026-07-05T11:00:42.732Z)

```
error: failed to delete '.git/worktrees/a1-master-worktree': Permission denied
error: failed to delete '.git/worktrees/fix-worktree': Permission denied
error: failed to delete '.git/worktrees/master-compare': Permission denied
error: failed to delete '.git/worktrees/pr465': Permission denied
error: failed to delete '.git/worktrees/pr465-current-merge': Permission denied
error: failed to delete '.git/worktrees/pr465-merge': Permission denied
error: failed to delete '.git/worktrees/pr467': Permission denied
error: failed to delete '.git/worktrees/pr469': Permission denied
error: failed to delete '.git/worktrees/wt-465fix': Permission denied
error: failed to delete '.git/worktrees/wt-a1-runlog': Permission denied
error: failed to delete '.git/worktrees/wt-claudemd-r25': Permission denied
error: failed to delete '.git/worktrees/wt-cloudrun-cancel': Permission denied
error: failed to delete '.git/worktrees/wt-cloudrun-dispatch': Permission denied
error: failed to delete '.git/worktrees/wt-cp-rebuild-spec': Permission denied
error: failed to delete '.git/worktrees/wt-dockerfile-auth': Permission denied
```

```
error: failed to delete '.git/worktrees/wt-docs': Permission denied
error: failed to delete '.git/worktrees/wt-docs-reconcile': Permission denied
error: failed to delete '.git/worktrees/wt-drain-concurrency': Permission denied
error: failed to delete '.git/worktrees/wt-ffmpeg-timeouts': Permission denied
error: failed to delete '.git/worktrees/wt-gdrive-api-safety': Permission denied
error: failed to delete '.git/worktrees/wt-idempotent-process': Permission denied
error: failed to delete '.git/worktrees/wt-input-jail': Permission denied
error: failed to delete '.git/worktrees/wt-output-dir-config': Permission denied
error: failed to delete '.git/worktrees/wt-p1-genyaml': Permission denied
error: failed to delete '.git/worktrees/wt-retention-sweep': Permission denied
error: failed to delete '.git/worktrees/wt-scratch-reclaim': Permission denied
error: failed to delete '.git/worktrees/wt-signed-download': Permission denied
error: failed to delete '.git/worktrees/wt-source-cache': Permission denied
error: failed to delete '.git/worktrees/wt-upload-dir': Permission denied
error: failed to delete '.git/worktrees/wt-upload-staging': Permission denied
wt-157-before
wt-157-ingest-ux
wt-465fix
wt-a1-runlog
wt-cloudrun-dispatch
wt-dockerfile-auth
wt-drain-concurrency
wt-input-jail
wt-p2-upload-stream
wt-p4-md5cache
wt-p5a-pollknob
wt-p5b-ceilings
wt-p6-heartbeat
wt-pr158-screenshot-assets
wt-pr481-review
wt-review-492-mergecheck
wt-signed-download
wt-upload-staging
wt-yaml-gen
```

282. user (2026-07-05T11:01:10.090Z)

```
Updated task #10 status
```

283. user (2026-07-05T11:01:26.234Z)

```
<tool_use_error>File has been modified since read, either by the user or by a linter. Read it
again before attempting to write it.</tool_use_error>
```

284. user (2026-07-05T11:01:36.527Z)

```
1       ---
2       name: session-handoff
3       description: "Living handoff: current state, next steps, ramp-up kit, and key
decisions."
4       metadata:
5         node_type: memory
6         type: project
7       ---
8
9       # Session handoff ? khelsutra-guru
10
11      > Last refreshed: 2026-07-05 evening (**standing CP-rebuild reviewer monitor ACTIVE**:
automation `khelsutra-cp-rebuild-pr-reviewer`, wakes this thread every ~10 min to inspect
incoming P-row PRs and apply the full verified-LGTM gate). **CP-rebuild SPEC PR
[#487](https://github.com/Khelsutra/badminton-highlight-indexer/pull/487) MERGED -> master
`cc5e83a`**; **A10/A11 guardrails PR [#488](https://github.com/Khelsutra/badminton-highlight-
indexer/pull/488) MERGED -> master `f1b60d0`**; **CP-rebuild P1 PR
[#489](https://github.com/Khelsutra/badminton-highlight-indexer/pull/489) MERGED -> master
```

`e80bca8`** and post-merge LGTM/verification posted
[here](https://github.com/Khelsutra/badminton-highlight-
indexer/pull/489#issuecomment-4885664770). **CP-rebuild P2 PR
[#490](https://github.com/Khelsutra/badminton-highlight-indexer/pull/490) OPEN + LGTM posted**
([comment](https://github.com/Khelsutra/badminton-highlight-
indexer/pull/490#issuecomment-4885682966)); exact merge result verified with full gate: `1675
passed, 10 skipped`, coverage `85.50%` vs base `85.49%`. **CP-rebuild P5a PR
[#491](https://github.com/Khelsutra/badminton-highlight-indexer/pull/491) OPEN + LGTM posted**
([comment](https://github.com/Khelsutra/badminton-highlight-
indexer/pull/491#issuecomment-4885706891)); exact merge result verified: `1668 passed, 10
skipped`, coverage `85.50%` vs base `85.49%`. #490/#491 both touch `backend/api/uploads.py`;
whichever merges second may need a tiny rebase and must be rechecked. Earlier same day:
khelsutra#157 UX polish [#158](https://github.com/avidullu/khelsutra/pull/158) MERGED -> master
`0e45e8f`; engine **#481 dedup+queue MERGED -> master `0a7b3ff`**; khelsutra#154 fix
[#155](https://github.com/avidullu/khelsutra/pull/155) MERGED -> `5ddb81b`). Prior: 2026-07-04 ?
A1 DONE (7 serving PRs merged, e2e PASSED, edge-auth restored, #477 master `122f174`).
12   > See [[current-state]] for the blow-by-blow.
13
14   ## ? YOU ARE HERE ? REVIEWER MONITOR ACTIVE; #490 P2 + #491 P5A LGTM, AWAITING MERGE
15
16   **A10/A11 limited-alpha decision is now merged via #488.** The recorded posture is:
fewer than 25
17   individually allowlisted testers; invite-only product/CUJ alpha; workflow-only claims;
no public signup,
18   no model/weight sharing, no quality benchmarks, and no owned/commercial model claim.
#172 stays open for
19   the real model/public ship-gate calibration, and newly filed #486 tracks WASB-C3
provenance/counsel or a
20   cleared-detector path before public/model/commercial posture.
21
22   **The rebuild spec is now merged.** `docs/CONTROL_PLANE_DURABLE_REBUILD.md` (PR #487 ->
master `cc5e83a`)
23   consolidates
24   #471/#480/#482/#483/#484/#485 + the SPA re-vendor into ONE image rebuild + redeploy: 8
GiB CP ·
25   upload staging STREAMED to the bucket (BlobWriter, never RAM-`/tmp`) · bucket-grounded
state (DB =
26   cache; re-hydrate reuses `_ingest_remote_segments`, orchestration.py:107) · **submit-
time MD5 as the
27   keystone** (instant re-runs #484 AND the restart re-association the 2026-07-05 loss
lacked ? job row
28   `video_id` at CREATE) · poll ceilings CP 1800?3900 s + SPA 10?60 min · **D5: rev-00011
pins baked
29   into `gen_service_yaml.py`** (today a regenerate silently DROPS them) · **D6: `use-
http2` REJECTED**
30   (uvicorn has no h2c ? the annotation would break the service). §7 is the execution plan:
P1?P8 one small
31   PR each, then P9 rebuild/deploy. **P1 is merged in #489**: `gen_service_yaml.py` bakes
the rev-00011
32   pins (`indexing.default_segmenter=wasb_hybrid`, `security.max_part_mb=24`) and defaults
CP memory to 8Gi
33   while keeping all three overridable. **P2 is open as #490 and has a verified LGTM**:
browser uploads stream
34   to GCS under `<input_root>/uploads/<upload_id>/<name>`, return the staged `gs://` path +
streamed md5, and
35   preserve local appliance behavior. **P5a is open as #491 and has a verified LGTM**:
`remote_poll_max_wait_sec`
36   defaults to 3900s and `/api/capabilities` emits upload caps. Await owner merges; after
either #490 or #491
37   lands, pull master and recheck/rebase the other if needed (`backend/api/uploads.py`
overlap). Next build P4?P3
38   and P5b?P8, bundle P6/#482 + P7/#483
39   (golden-gated, Launch Report) if ready, execute P9 per §4.8, verify §9 DoD (incl.
replaying the
40   restart-after-done-before-ingest failure shape), and write the runlog.

```
41
42      ## Standing state (pre-spec) ? A1 DONE + A13 DELIVERED; ball was with the OWNER
43
44      **A1 is DONE and e2e-verified** (runlog
`docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_a1-e2e_2026-07-04.md`)
45      and **A13 is DELIVERED** (`docs/ALPHA_LAUNCH_REVIEW_2026-07-04.md`, merged #478 ? master
`64fa4a2`,
```

285. user (2026-07-05T11:02:19.511Z)

```
The file c:\Users\avidu\.claude\projects\c--Users-avidu-Projects-khelsutra-guru\memory\session-
handoff.md has been updated successfully. (file state is current in your context ? no need to
Read it back)
```

286. user (2026-07-05T11:02:41.650Z)

```
The file c:\Users\avidu\.claude\projects\c--Users-avidu-Projects-khelsutra-guru\memory\session-
handoff.md has been updated successfully. (file state is current in your context ? no need to
Read it back)
```

287. user (2026-07-05T11:02:52.995Z)

```
75:**Next session grabs (priority order ? reviewer role):** (1) Watch/handle open
[#490](https://github.com/Khelsutra/badminton-highlight-indexer/pull/490) and
[#491](https://github.com/Khelsutra/badminton-highlight-indexer/pull/491): both already have
verified LGTM; do not merge unless owner explicitly asks; if either merges, pull master and re-
check/rebase the other as needed. (2) Review incoming
[#487](https://github.com/Khelsutra/badminton-highlight-indexer/pull/487) §7 rows with the full
verified-LGTM gate: P4 (submit-time MD5 + cache-reuse, needs P2) · P3 (restart-safe recovery,
needs P4) · P5b (khelsutra ceilings+pre-flight) · P8 (re-vendor via `scripts/bundle_ui.sh` after
P5b) ? one small PR each, reproduce locally on exact merge result, full gates before LGTM; then
**P9** rebuild+redeploy+CUJ per §4.8/§9 + runlog. (3) **Savers** (bundle if ready, never hold
P9): **#483 downsample** (P7, golden-gate + Launch Report), **#482 progress/heartbeat** (P6).
(4) **SPA processing-queue UI** on `feat/processing-queue-and-dedup` @ `23f3d15` ? build AFTER
the rebuild (nothing live until then). (5) owner 5-min CUJ + GO/NO-GO on A13. (6) least-
privilege SA (review §5.5). (7) retention (#301/#447) + processor-list / acceptable-use (#332).
(8) kn/hi native review (#156). (9) khelsutra UX backlog (#329) · rally finding-B (P2b).
77:**? NEXT-SESSION ICEBREAKER (paste this):** _Ramp up: `git pull --ff-only` both repos, `gh
auth status`, read this handoff + `docs/CONTROL_PLANE_DURABLE_REBUILD.md` (the merged spec) +
[[alpha-cuj-testing-2026-07-05]]. You are the standing code reviewer for CP-rebuild P1-P9.
Monitor incoming PRs; LGTM only after the full verified gate, otherwise comment findings or
pause for Avi. #490/P2 and #491/P5a are open with verified LGTMs already; do not merge unless
owner asks, and recheck/rebase whichever lands second because both touch
`backend/api/uploads.py`. Next review P4?P3 and P5b?P8. P9 must be holistic: rebuild/redeploy,
browser CUJ, restart/durable-state replay, upload OOM proof, cache/dedup proof, edge-auth
posture, generated manifest pins, SPA vendored-version proof, runlog/docs/memory. Honest
measured results only; Launch Report on every default flip._
```

288. assistant (2026-07-05T11:03:11.327Z)

```
The sibling reviewer-monitor owns the grabs/icebreaker framing ? updating the facts inside them
(it doesn't yet know #492/#493/#162 exist) while keeping its reviewer-role structure.
```

289. user (2026-07-05T11:03:25.619Z)

```
The file c:\Users\avidu\.claude\projects\c--Users-avidu-Projects-khelsutra-guru\memory\session-
handoff.md has been updated successfully. (file state is current in your context ? no need to
Read it back)
```

290. user (2026-07-05T11:03:34.440Z)

```
The file c:\Users\avidu\.claude\projects\c--Users-avidu-Projects-khelsutra-guru\memory\session-
handoff.md has been updated successfully. (file state is current in your context ? no need to
Read it back)
```

291. user (2026-07-05T11:03:43.599Z)

```
10      # Current state - khelsutra-guru
11
12      **Updated:** 2026-07-05
13
14      ## Live Checkpoint
15
16      **2026-07-05 evening heartbeat (standing reviewer caught #491; #490 + #491 both
LGTM/open).** Monitor found new [#491](https://github.com/Khelsutra/badminton-highlight-
indexer/pull/491) (CP-rebuild P5a: `deployment.remote_poll_max_wait_sec=3900` + top-level
`/api/capabilities` upload caps). Reviewed and posted verified LGTM
([comment](https://github.com/Khelsutra/badminton-highlight-
indexer/pull/491#issuecomment-4885706891)). No blocking issues: the config knob reaches the
actual `CloudRunCompute` `ExecutionPolicy` path while preserving explicitly injected policies,
and `max_upload_mb`/`max_part_mb` are emitted where the SPA pre-flight contract expects them.
Verified exact merge result (`0575334` over current master `e80bca8`): `ruff check .`, `mypy
backend training`, `python -m tools.check_md_links`, `python -m tools.leak_scan`, `git diff
--check origin/master...HEAD`, full coverage suite `1668 passed, 10 skipped`, coverage `85.50%`;
base `e80bca8` was verified earlier at `85.49%`, so no regression. **Open PRs now:** #490 P2
(LGTM posted) and #491 P5a (LGTM posted). Merge-order note: both touch `backend/api/uploads.py`;
whichever merges second may need a tiny rebase/conflict resolution and must be re-checked before
landing. Main checkout clean on `master`; temp review worktree removed. **? NEXT:** keep
monitoring; do not merge unless owner asks.
17
18      ---
19
```

292. user (2026-07-05T11:04:09.924Z)

```
The file c:\Users\avidu\.claude\projects\c--Users-avidu-Projects-khelsutra-guru\memory\current-
state.md has been updated successfully. (file state is current in your context ? no need to Read
it back)
```

293. user (2026-07-05T11:04:23.411Z)

```
Updated task #2 status
```

294. user (2026-07-05T11:04:23.882Z)

```
Updated task #3 status
```

295. user (2026-07-05T11:04:24.356Z)

```
Updated task #5 status
```

296. user (2026-07-05T11:04:24.662Z)

```
Updated task #6 status
```

297. user (2026-07-05T11:04:31.035Z)

```
Updated task #8 status
```

298. user (2026-07-05T11:04:31.500Z)

```
Updated task #10 status
```

299. user (2026-07-05T11:04:39.363Z)

```
#492: passing=11 non-passing=0
#493: passing=11 non-passing=0
```

300. assistant (2026-07-05T11:05:06.957Z)

Everything buildable is built and green ? the rebuild is now waiting only on your reviews.

Where the CP-rebuild stands

Merged earlier (pre-review-gate): the spec [#487](https://github.com/Khelsutra/badminton-highlight-indexer/pull/487) (now `IN PROGRESS`, your Q2?Q5 answers recorded) and P1 [#489](https://github.com/Khelsutra/badminton-highlight-indexer/pull/489) (rev-00011 pins baked into the generator + 8 GiB default) ? master is at `e80bca8`, plus the sibling session's A10/A11 [#488](https://github.com/Khelsutra/badminton-highlight-indexer/pull/488) (`f1b60d0`).

Five PRs open for your review, all 11/11 CI green, none will be self-merged:

PR	Row	What it does
[#490](https://github.com/Khelsutra/badminton-highlight-indexer/pull/490)	P2	Browser uploads stream into the bucket via a new `storage.open_writer` seam ? the video never assembles in tmpfs; streamed MD5 comes free. *Reviewer-monitor already posted a verified LGTM.*
[#491](https://github.com/Khelsutra/badminton-highlight-indexer/pull/491)	P5a	Poll budget knob (3900 s > worker task-timeout) + capabilities now emit `max_upload_mb`. *Verified LGTM posted.*
[#492](https://github.com/Khelsutra/badminton-highlight-indexer/pull/492)	P4	Submit-time MD5 from gs:// metadata, persisted at job CREATE, plus DB-first/bucket-second cache-reuse ? a re-submitted video completes instantly with no GPU dispatch (proven by test).
[#493](https://github.com/Khelsutra/badminton-highlight-indexer/pull/493)	P6 slice	`job.progress` now ticks "segmenting on cloud GPU ? N min elapsed" during the whole run instead of freezing for 22 minutes. Worker-side frame-% stays on #482 (needs a GPU pass).
[khelsutra #162](https://github.com/avidullu/khelsutra/pull/162)	P5b	Both SPA poll ceilings 10 ? 60 min. Shrank to ceilings-only because the SPA already consumes `capabilities.max_upload_mb` ? #491 revives the pre-flight with zero SPA change.

Merge-order notes for your reviews: #490/#491 share `backend/api/uploads.py` and #492/#493 share `backend/orchestration.py` ? disjoint hunks in both pairs, but whichever of each pair lands second deserves a recheck; I'll rebase if anything conflicts.

What's left after your merges: P3 (restart-safe recovery ? branches only after #492, since it keys on md5-at-create), P8 (SPA re-vendor after #162, which also batches the spec §7 row updates), then **P9** ? the actual rebuild + redeploy + browser CUJ with the full §9 DoD including a replay of the exact restart-after-done-before-ingest failure that lost your 29 rallies. #483 downsample stays golden-gated and won't hold P9, per your confirmed Q3.

The review-gate is recorded durably in the working-agreement memory (no session will merge without your explicit word), and the handoff/current-state are checkpointed with the full picture, unified with the sibling reviewer-monitor's updates. I'm blocked on your reviews now ? say the word on any of the five and the next row starts.

301. user (2026-07-05T11:05:08.871Z)

once you have sent out the PRs do keep listening for comments and then addressing them and merging the PRs when you get an LGTM

302. assistant (2026-07-05T11:05:39.173Z)

Got it ? the protocol is now: open PRs ? watch for comments ? address them ? **merge on LGTM**. Recording that in the working-agreement memory, then acting on it immediately: #490 and #491 already carry verified LGTMs, so they merge now (in overlap-safe order), and I'll sweep #492/#493/kh#162 for any new review activity.

303. user (2026-07-05T11:05:53.205Z)

The file c:\Users\avidu\.claude\projects\c--Users-avidu-Projects-khelsutra-guru\memory\working-agreement-merging.md has been updated successfully. (file state is current in your context ? no need to Read it back)

304. user (2026-07-05T11:07:04.244Z)

I can?t LGTM this yet. I reproduced two blocking issues on a fresh merge-check branch (`origin/master` at `e80bca8` + this PR cherry-picked as `088d848`). I stopped before the full gate because these are behavioral blockers.

1. **Cache rehydrate happens before the atomic submit claim, so concurrent cache hits duplicate rows.**

`submit_process_job` calls `cached_process_result(...)` before `claim_process_job(...)` (`backend/api/jobs.py:79-85`). If two identical submits arrive together after a CP restart/empty DB, both can rehydrate `outputs/<md5>/intervals.csv` before either job row exists. Repro with a fake bucket and two concurrent calls to `submit_process_job`: both returned `202 done cached=true`, two jobs were marked done, and a 2-row `intervals.csv` left **4** `play_segments` in SQLite:

```text
results: [(False, '{"job_id":"job1","status":"done",...}'), (False,
'{"job_id":"job2","status":"done",...}')]
jobs: ['done', 'done']
segments: 4
[(1.0, 2.0), (3.0, 4.0), (1.0, 2.0), (3.0, 4.0)]
```

I think the claim needs to happen before any bucket rehydrate side effect (or the rehydrate needs a per-`video_id` lock/transaction) so the first request owns the cache completion and the second request dedups to the active job. Please add a regression that forces two concurrent cache hits from an empty DB and proves one segment set remains.

2. **A valid cache hit can still be rejected by the pre-cache 507 guard.**

`/api/process` runs the remote-input free-space guard before the new submit/cache hook (`backend/main.py:951-967`). For a `gs://` source with a successful cached `outputs/<md5>/report.json`, the request should not need local scratch at all, but the route can return 507 before it checks the cache. Repro with a fake cached bucket result, a 10 GiB object size, and 1 MiB temp free:

```text
507
{"detail":"Not enough free disk space for fetch: need ~10240 MB, only ~1 MB free. Free some
space and retry."}
```

That defeats the ?same MD5 => instant/no GPU? path under low scratch, and it also matters once P2 streamed browser uploads hand `/api/process` a `gs://` path. The guard should run only on paths that will actually fetch to local scratch, or the cache/cloud-dispatch decision needs to happen before this guard.

305. user (2026-07-05T11:07:06.710Z)

Follow-up from the #492 review: please treat my earlier LGTM here as paused until this is addressed here or in a prerequisite before merge/deploy.

I reproduced that after this PR?s streamed upload path returns a `gs://...` source, `/api/process` still runs the remote-source scratch free-space guard before it decides whether the work will be cloud-dispatched or cache-reused. With a fake cached/remote object sized at 10 GiB and only 1 MiB temp free, the process request returns 507 before any no-local-fetch path can run:

```text
507
{"detail":"Not enough free disk space for fetch: need ~10240 MB, only ~1 MB free. Free some
space and retry."}
```

That conflicts with the P2/P4 rebuild intent: GCS-staged browser uploads and cache hits should not require CP scratch space for the whole video. The guard should be backend/compute aware, or moved after the path decision so it only applies when the CP will actually fetch locally.
=== 491 state ===
mergeable=MERGEABLE checks=SUCCESS

306. user (2026-07-05T11:08:20.547Z)

error: failed to delete '.git/worktrees/wt-p5a-pollknob': Permission denied
From https://github.com/Khelsutra/badminton-highlight-indexer
   e80bca8..97feff4 master   -> origin/master
97feff4 feat(config): remote-poll budget knob (3900s) + advertise upload caps in /api/capabilities (CP-rebuild P5a) (#491)
e80bca8 deploy(cloudrun): bake rev-00011 pins into the manifest generator + 8Gi default (CP-rebuild P1) (#489)

307. assistant (2026-07-05T11:08:28.967Z)

#491 merged ? master `97feff4`. Now updating #490's branch across the expected `uploads.py` conflict.

308. user (2026-07-05T11:08:32.260Z)

Auto-merging backend/api/uploads.py
CONFLICT (content): Merge conflict in backend/api/uploads.py
Automatic merge failed; fix conflicts and then commit the result.

309. user (2026-07-05T11:08:50.238Z)

The file C:\Users\avidu\Projects\khelsutra-guru\wt-p2-upload-stream\backend\api\uploads.py has been updated successfully. (file state is current in your context ? no need to Read it back)

310. user (2026-07-05T11:09:31.383Z)

M backend/api/uploads.py
M backend/compute/__init__.py
M backend/config/models.py
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
87 passed, 1 skipped, 4 warnings in 27.69s
[feat/upload-gcs-streaming dda8a68] Merge origin/master (P5a #491) ? keep both upload_caps and _staging_target at the uploads.py seam
dda8a68 Merge origin/master (P5a #491) ? keep both upload_caps and _staging_target at the uploads.py seam
97feff4 feat(config): remote-poll budget knob (3900s) + advertise upload caps in /api/capabilities (CP-rebuild P5a) (#491)
ba08c9d feat(uploads): stream browser-upload staging to the bucket ? never assemble in tmpfs (CP-rebuild P2)

311. user (2026-07-05T11:12:11.128Z)

1678 passed, 10 skipped, 8 warnings in 138.90s (0:02:18)
rcs: 0 0 0 0
   ba08c9d..dda8a68 feat/upload-gcs-streaming -> feat/upload-gcs-streaming

312. assistant (2026-07-05T11:12:54.379Z)

#490 is updated and green (1678 passed). Now the #492 fixes. Merging master into the P4 branch first.

313. user (2026-07-05T11:12:58.763Z)

 tests/test_compute_backend.py | 24 ++++++++++++++++++++++
 tests/test_config_deployment.py | 16 ++++++++++++++++

```
 7 files changed, 84 insertions(+)
7:import tempfile
93:from backend.utils.freespace import require_free_bytes  # HTTP-507 disk guard (P2, D11)
241:    _remote_input_size_bytes,
572:        need = _remote_input_size_bytes(input_ref, getattr(cfg, "storage", None))
573:        require_free_bytes(cache_dir, need, stage="source-cache")
801:            need = _remote_input_size_bytes(src_ref, getattr(cfg, "storage", None))
802:            require_free_bytes(tempfile.gettempdir(), need, stage="fetch")
846:                    require_free_bytes(
```

314. assistant (2026-07-05T11:13:09.141Z)

```
Clean merge. Implementing the race fix + guard reorder.
```

315. user (2026-07-05T11:13:10.194Z)

```
21:import csv
22:import json
23:import logging
24:import math
25:import os
26:import tempfile
27:import threading
28:from typing import TYPE_CHECKING, Any, Dict, List, Optional, Tuple
29:from urllib.parse import quote
31:from backend import storage
32:from backend.config import AppConfig, StitchingConfig
33:from backend.pipeline.segmenters.base import segmenter_registry
34:from backend.pipeline.stitcher.watermark_i18n import localize_watermark
35:from backend.pipeline.validators.base import registry
36:from backend.results import Failure
37:from backend.utils.paths import output_base, safe_output_filename
```

316. user (2026-07-05T11:13:46.179Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p4-md5cache\backend\orchestration.py has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

317. user (2026-07-05T11:14:13.675Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p4-md5cache\backend\api\jobs.py has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

318. user (2026-07-05T11:14:21.575Z)

```
6:can't see their running job from re-submitting the same video into a second, PAID GPU run
(#448). The
13:from typing import Any, List, Optional, Tuple
15:from fastapi import APIRouter, HTTPException, Request
16:from fastapi.responses import JSONResponse
18:from backend.api import (
```

319. user (2026-07-05T11:14:29.375Z)

```
11      """
12
13      from typing import Any, List, Optional, Tuple
14
15      from fastapi import APIRouter, HTTPException, Request
16      from fastapi.responses import JSONResponse
17
18      from backend.api import (
19          auth as edge_auth,  # M9: Cloudflare Access edge-auth (default-OFF)
20      )
```

```
21
22      router = APIRouter()
```

320. user (2026-07-05T11:14:37.956Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p4-md5cache\backend\api\jobs.py has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

321. assistant (2026-07-05T11:14:43.711Z)

Now removing the route-level guard from main.py (it moved into the hook):

322. user (2026-07-05T11:14:44.835Z)

```
944         if input_ref.backend == "local" and not os.path.exists(input_ref.key):
945             raise HTTPException(
946                 status_code=404, detail=f"Video file not found at '{req.video_path}'"
947             )
948         # P2 (D11): a bucket/Drive source is fetched to a scratch copy on the worker; fail
fast with 507
949         # if this box can't hold that copy. Best-effort ? an unstattable remote
(network/SDK) or an
950         # unreadable temp fs skips the guard (measuring must not deny a legit job); the
fetch itself
951         # still fails loudly downstream if space genuinely runs out. Local inputs need no
copy ? skipped.
952         if input_ref.backend != "local":
953             need = _remote_input_size_bytes(
954                 input_ref, getattr(request.app.state.config, "storage", None)
955             )
956             require_free_bytes(tempfile.gettempdir(), need, stage="fetch")
957         if req.segmenter and not segmenter_registry.get(req.segmenter):
958             available = list(segmenter_registry.list_available().keys())
959             raise HTTPException(
960                 status_code=400,
961                 detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}",
962             )
963
964         # ATOMIC idempotent submit (#448) + submit-time md5 cache (#484): one hook claims-
or-dedups,
965         # and completes the job INSTANTLY when this video's detection already exists in the
DB or the
```

323. user (2026-07-05T11:14:59.281Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p4-md5cache\backend\main.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

324. assistant (2026-07-05T11:15:41.183Z)

Now the regression tests ? extending the fake and adding the five review-driven cases.

325. user (2026-07-05T11:15:50.821Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p4-md5cache\tests\test_submit_md5_cache.py
has been updated successfully. (file state is current in your context ? no need to Read it back)
```

326. user (2026-07-05T11:15:59.786Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p4-md5cache\tests\test_submit_md5_cache.py
has been updated successfully. (file state is current in your context ? no need to Read it back)
```

327. user (2026-07-05T11:16:34.458Z)

```
6 review-driven tests appended
```

328. user (2026-07-05T11:16:47.468Z)

```
..............                                                           [100%]
14 passed in 3.86s
```

329. user (2026-07-05T11:19:49.530Z)

```
1 failed, 1681 passed, 10 skipped, 20 warnings in 164.08s (0:02:44)
Success: no issues found in 148 source files
warning: in the working copy of 'tests/test_submit_md5_cache.py', LF will be replaced by CRLF
the next time Git touches it
rcs: 1 1 0 0
```

330. user (2026-07-05T11:19:56.358Z)

```
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
========================== short test summary info ==========================
FAILED tests/test_api_endpoints.py::test_process_remote_input_507_when_scratch_too_small
1 failed, 1681 passed, 10 skipped, 20 warnings in 164.08s (0:02:44)
I001 [*] Import block is un-sorted or un-formatted
  --> backend\api\jobs.py:13:1
   |
11 |   """
12 |
```

331. user (2026-07-05T11:20:11.446Z)

```
Found 1 error (1 fixed, 0 remaining).
        )
        r = client.post(
            "/api/process",
            json={"video_path": "gs://bucket/videos/big.mp4", "segmenter": "x"},
        )
>       assert r.status_code == 507
E       assert 400 == 507
E        +  where 400 = <Response [400 Bad Request]>.status_code

tests\test_api_endpoints.py:862: AssertionError
[This command modified 1 file you've previously read: backend\api\jobs.py. Call Read before
editing.]
```

332. user (2026-07-05T11:20:28.918Z)

```
def test_process_remote_input_507_when_scratch_too_small(env):
    """P2 (D11): a bucket source whose fetched copy won't fit in scratch fails fast with 507 at
    submit ? before a job is queued ? rather than crashing the worker mid-fetch."""
    import types

    from backend.utils import freespace

    client, db, tmp_path, mp = env
    # Stat says the object is 10 GB; the scratch fs reports only 1 MB free ? shortfall.
    mp.setattr(m, "_remote_input_size_bytes", lambda *a, **k: 10 * 1024**3)
    mp.setattr(
        freespace.shutil,
        "disk_usage",
        lambda p: types.SimpleNamespace(total=0, used=0, free=1024 * 1024),
    )
    r = client.post(
        "/api/process",
```

```
        json={"video_path": "gs://bucket/videos/big.mp4", "segmenter": "x"},
    )
    assert r.status_code == 507
    assert "space" in r.json()["detail"].lower()


def test_process_remote_input_unstattable_is_not_blocked(env):
```

333. user (2026-07-05T11:20:52.251Z)

```
def test_process_remote_input_unstattable_is_not_blocked(env):
    """Best-effort: a remote we can't size (stat returns None) must NOT be blocked by the
guard."""
    client, db, tmp_path, mp = env
    mp.setattr(m, "_remote_input_size_bytes", lambda *a, **k: None)

    def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
        with open(dst, "wb") as fh:
            fh.write(b"bytes")
        return dst

    mp.setattr(m.storage, "fetch", fake_fetch)
    mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
    mp.setattr(m.segmenter_registry, "get", lambda name: _CaptureSeg)
    r = client.post(
        "/api/process",
        json={"video_path": "gs://bucket/videos/x.mp4", "segmenter": "x"},
    )
    assert r.status_code == 202  # unmeasurable ? guard degrades to a no-op


@pytest.mark.parametrize(
    "uri", ["gs://bucket/videos/x.mp4", "gcs://bucket/videos/x.mp4"]
)
def test_process_bucket_uri_yields_queryable_video_id(env, uri):
tests/test_api_annotations.py:91:    monkeypatch.setattr(m, "_remote_input_size_bytes", lambda
*a, **k: len(payload))
tests/test_api_assets.py:129:    monkeypatch.setattr(m, "_remote_input_size_bytes", lambda *a,
**k: len(payload))
tests/test_api_endpoints.py:816:    mp.setattr(m, "_remote_input_size_bytes", lambda *a, **k:
len(payload))
tests/test_api_endpoints.py:852:    mp.setattr(m, "_remote_input_size_bytes", lambda *a, **k: 10
* 1024**3)
tests/test_api_endpoints.py:869:    mp.setattr(m, "_remote_input_size_bytes", lambda *a, **k:
None)
tests/test_main_module_budget.py:49:    "_remote_input_size_bytes",
```

334. user (2026-07-05T11:22:10.537Z)

```
both route-guard tests updated
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
66 passed, 1 skipped, 10 warnings in 43.70s
```

335. user (2026-07-05T11:25:05.342Z)

```
1682 passed, 10 skipped, 18 warnings in 159.81s (0:02:39)
warning: in the working copy of 'tests/test_api_endpoints.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'tests/test_submit_md5_cache.py', LF will be replaced by CRLF
the next time Git touches it
rcs: 0 0 0 0
```

336. assistant (2026-07-05T11:25:12.891Z)

All gates green (1682 passed). Committing the review fixes and replying to both PRs.

337. user (2026-07-05T11:25:25.397Z)

```
feat/submit-md5-cache-reuse
warning: in the working copy of 'tests/test_api_endpoints.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'tests/test_submit_md5_cache.py', LF will be replaced by CRLF
the next time Git touches it
[feat/submit-md5-cache-reuse e4cf261] fix(process): address #492 review ? claim before re-
hydrate + per-video lock; scratch guard after the cache probe
 5 files changed, 353 insertions(+), 83 deletions(-)
   0aca0ff..e4cf261  feat/submit-md5-cache-reuse -> feat/submit-md5-cache-reuse
```

338. user (2026-07-05T11:25:43.744Z)

```
https://github.com/Khelsutra/badminton-highlight-indexer/pull/492#issuecomment-4885843089
```

339. user (2026-07-05T11:26:01.118Z)

```
https://github.com/Khelsutra/badminton-highlight-indexer/pull/490#issuecomment-4885844036
5
1
```

340. assistant (2026-07-05T11:26:07.986Z)

```
Replies posted. khelsutra #162 has a comment ? checking it, and #493 for any review activity
too.
```

341. user (2026-07-05T11:26:11.952Z)

Deploying with <a href="https://workers.dev"><img alt="Cloudflare Workers" src="https://workers.clo

```
The latest updates on your project. Learn more about [integrating Git with
Workers](https://developers.cloudflare.com/workers/ci-cd/builds/git-integration/).

| Status | Name | Latest Commit | Preview URL | Updated (UTC) |
| -|-|-|-|-|
| ? Deployment successful! <br>[View logs](https://dash.cloudflare.com/?to=/213c12cf60de9041dd8a
3521aa206472/workers/services/view/hidden-king-
khelsutra/production/builds/4218aada-939c-4fff-b7fa-ddd3f339df5a) | hidden-king-khelsutra |
0a3bb525 | <a href='https://f7f87538-hidden-king-khelsutra.avi-dullu.workers.dev'>Commit Preview
URL</a><br><br><a href='https://feat-poll-ceilings-60min-hidden-king-khelsutra.avi-
dullu.workers.dev'>Branch Preview URL</a> | Jul 05 2026, 10:33 AM |
=== 493 comments ===
2
```

342. user (2026-07-05T11:26:20.431Z)

LGTM for this CP-side P6 slice.

Verified on a fresh merge-check branch (`origin/master` `e80bca8` + this PR cherry-picked as `85c1e14`). I inspected the PR metadata/diff/CI/review threads and the changed paths in `execution_policy.poll_until`, `CloudRunCompute.run_infer`, the `ComputeBackend` interface, and `_maybe_dispatch_remote` progress plumbing.

Local gates run:

```text
ruff check .                                    PASS
mypy backend training                          PASS
python -m tools.check_md_links                 PASS
python -m tools.leak_scan                      PASS
git diff --check origin/master...HEAD          PASS
```

```
python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70
  -> 1669 passed, 10 skipped, coverage 85.51%
```

Coverage is non-regressing versus the exact base already verified in this monitor run (`e80bca8`: 85.49%). I did not exercise a real Cloud Run/GPU execution locally; this PR is the control-plane polling/progress slice and the Cloud Run path is covered with the repo?s fake client. The remaining #482 worker-side frame-%/ETA and per-job runlog work correctly stays out of this PR.

Merge-order note: this shares `backend/orchestration.py` with #492. If #492 lands first, please rebase/re-run the gate before merging this one.
Update after my LGTM: `master` advanced with #491 (`97feff4`) while I was running the local gate, and GitHub now reports this PR as `CONFLICTING`.

My verification above applies to the previous merge result (`origin/master` `e80bca8` + this PR as `85c1e14`) only. Please rebase/resolve onto current `origin/master` and I?ll re-run the verified gate before this merges.

343. user (2026-07-05T11:26:37.543Z)

```
Auto-merging tests/test_compute_backend.py
CONFLICT (content): Merge conflict in tests/test_compute_backend.py
Automatic merge failed; fix conflicts and then commit the result.
```

344. user (2026-07-05T11:26:46.466Z)

```
717:<<<<<<< HEAD
749:=======
772:>>>>>>> origin/master
```

345. user (2026-07-05T11:26:53.082Z)

```
710         assert (
711             "MAMBA_ROOT_PREFIX=/opt/conda" in text
712         )  # keeps the env at /opt/conda/envs/wasb
713         assert "Miniconda3" not in text  # the Anaconda installer is gone
714         assert "conda tos accept" not in text  # no ToS acceptance needed (conda-forge)
715
716
717     <<<<<<< HEAD
718     def test_run_infer_forwards_on_wait_into_the_bucket_poll(tmp_path, monkeypatch):
719         """The #482 heartbeat plumbing: the on_wait callback handed to run_infer must be the
720         exact on_tick the bucket-poll drives (cloud_run threads it through poll_until)."""
721         from backend.compute import cloud_run as cr
722
723         seen = {}
724
725         def fake_poll(check, policy, *, label="op", logger=None, on_tick=None, **kw):
726             seen["on_tick"] = on_tick
727             return {"video_id": "vidCloud", "returncode": 0}
728
729         monkeypatch.setattr(cr, "poll_until", fake_poll)
730         client = _FakeRunClient()  # still writes report.json so the post-marker read
succeeds
731         backend = CloudRunCompute(
732             run_client=client,
733             job_name="rally-worker",
734             region="asia-southeast1",
735             storage_config={},
736             policy=_FAST,
737             token="tok",
738         )
739
```

```
740         def heartbeat(_elapsed):
741             pass
742 
743         result = backend.run_infer(
744             ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bk")),
745             on_wait=heartbeat,
746         )
747         assert seen["on_tick"] is heartbeat
748         assert result.video_id == "vidCloud"
749     =======
750     def test_remote_poll_budget_comes_from_config_unless_policy_injected():
751         """CP-rebuild §4.3: deployment.remote_poll_max_wait_sec drives the dispatch poll
budget
752         (default 3900s > the Job's --task-timeout 3600s ? the old 1800s default could
abandon a
753         slow-but-healthy paid GPU run), and an explicitly injected policy always wins."""
754         from backend.infrastructure.execution_policy import DEFAULT_POLICY
755 
756         cfg = AppConfig()
757         cfg.deployment.compute_target = "cloud_run"
758 
759         be = get_compute_backend(cfg, run_client=_FakeRunClient())
760         assert be is not None
761         assert be._policy.max_wait_sec == 3900.0
762         # Only the budget is derived ? the other resilience knobs stay at their defaults.
763         assert be._policy.poll_every_n_sec == DEFAULT_POLICY.poll_every_n_sec
764         assert be._policy.op_timeout_sec == DEFAULT_POLICY.op_timeout_sec
765 
766         cfg.deployment.remote_poll_max_wait_sec = 42.0
767         assert get_compute_backend(cfg, run_client=_FakeRunClient())._policy.max_wait_sec ==
42.0
768 
769         # An injected policy is never overridden by config.
770         be = get_compute_backend(cfg, run_client=_FakeRunClient(), policy=_FAST)
771         assert be._policy.max_wait_sec == _FAST.max_wait_sec
772     >>>>>>> origin/master
773 
```

346. user (2026-07-05T11:27:16.919Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p6-heartbeat\tests\test_compute_backend.py
has been updated successfully. (file state is current in your context ? no need to Read it back)
```

347. user (2026-07-05T11:29:56.326Z)

```
70 passed in 6.39s
[feat/cloud-poll-heartbeat 0f3382b] Merge origin/master (P5a #491) ? keep both appended test
blocks in test_compute_backend.py
1672 passed, 10 skipped, 8 warnings in 132.22s (0:02:12)
rcs: 0 0 0 0
   b9e2806..0f3382b  feat/cloud-poll-heartbeat -> feat/cloud-poll-heartbeat
```

348. user (2026-07-05T11:30:11.931Z)

```
https://github.com/Khelsutra/badminton-highlight-indexer/pull/493#issuecomment-4885860105
MERGEABLE
```

349. assistant (2026-07-05T11:30:51.960Z)

```
#!/usr/bin/env bash
```

Watch the open CP-rebuild PRs for review activity (new comments / reviews / merge-s changes). Exits (triggering re-invocation) as soon as anything changes, or after ~50 m

```
set -u
```

```
cd /c/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer || exit 1

snapshot() {
  local out=""
  for pr in 490 492 493; do
    out+="engine#$pr:$(gh pr view "$pr" --json state,mergeable,comments,reviews \
      -q '"\(.state)/\(.mergeable)/c\(.comments|length)/r\(.reviews|length)"' 2>/dev/null) "
  done
  out+="kh#162:$(gh -R avidullu/khelsutra pr view 162 --json state,mergeable,comments,reviews \
    -q '"\(.state)/\(.mergeable)/c\(.comments|length)/r\(.reviews|length)"' 2>/dev/null)"
  printf '%s' "$out"
}

base="$(snapshot)"
echo "baseline: $base"
for i in $(seq 1 16); do
  sleep 180
  cur="$(snapshot)"
  if [ "$cur" != "$base" ]; then
    echo "CHANGE DETECTED (poll $i):"
    echo "  was: $base"
    echo "  now: $cur"
    exit 0
  fi
done
echo "no change in ~48 min; exiting for a fresh look"
exit 0
```

350. user (2026-07-05T11:30:52.021Z)

```
File created successfully at: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-
badminton-highlight-indexer\693b418f-8bf1-49e5-b443-b86a83235a63\scratchpad\pr_watch.sh (file
state is current in your context ? no need to Read it back)
```

351. user (2026-07-05T11:30:59.416Z)

```
Command running in background with ID: bw9sge92s. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\693b418f-8bf1-49e5-b443-b86a83235a63\tasks\bw9sge92s.output. You will be notified when
it completes. To check interim output, use Read on that file path.
```

352. assistant (2026-07-05T11:31:08.319Z)

```
Watcher running. One last check that #492's CI came back green after the fix push, then I'll
report status.
```

353. user (2026-07-05T11:31:12.579Z)

```
0
MERGEABLE
```

354. assistant (2026-07-05T11:31:40.694Z)

```
All review traffic is addressed and the listening loop is live. Status:

**Merged this round:** [#491](https://github.com/Khelsutra/badminton-highlight-indexer/pull/491)
(P5a) ? it carried an active verified LGTM ? master is now `97feff4`.

**Review feedback addressed:**
- **[#492](https://github.com/Khelsutra/badminton-highlight-indexer/pull/492) (P4)** ? both
reviewer blockers were real and are fixed in `e4cf261`: the submit order is now *probe (read-
only) ? atomic claim ? realize*, so a concurrent duplicate dedups **before** any re-hydrate side
effect, and the re-hydrate itself is serialized by a per-`video_id` lock with an in-lock DB re-
check. The 507 scratch guard moved after the cache probe ? the reviewer's 10 GiB/1 MiB-free
```

repro now returns instant `done` on a hit, still 507s pre-claim on a miss. Their exact two-thread repro is a regression test (proves one 2-row segment set survives), plus five more covering the lock path, both guard directions, and the probe-hit-realize-fail fallbacks. Full suite 1682 passed; CI 11/11 green again; replied and awaiting re-review.
- **[#490](https://github.com/Khelsutra/badminton-highlight-indexer/pull/490) (P2)** ? updated across the #491 conflict (both `upload_caps` and `_staging_target` kept, 1678 passed) and replied to the paused-LGTM note: the guard concern is fixed via #492, with the honest boundary stated ? a *fresh* uncached gs:// submit still fetches to drain scratch (pre-existing behavior); skipping that acquire entirely for cloud dispatch folds into P3.
- **[#493](https://github.com/Khelsutra/badminton-highlight-indexer/pull/493) (P6)** ? it received an LGTM conditioned on rebasing past #491; rebased (test-append collision resolved, 1672 passed), MERGEABLE again, replied for the re-gate.
- **[kh#162](https://github.com/avidullu/khelsutra/pull/162)** ? no review activity yet, only the Cloudflare preview bot.

Now listening: a background watcher polls all four open PRs every 3 minutes and wakes me on any comment, review, or merge-state change. On an LGTM I merge; the moment #492 lands I start P3 (restart-safe recovery ? the last engine row before the rebuild), and kh#162's merge unlocks P8. The merge protocol is recorded in the working-agreement memory so every session follows it.

355. user (2026-07-05T11:40:12.986Z)

<task-notification>
<task-id>bw9sge92s</task-id>
<tool-use-id>toolu_013sWHWuAZNjnRzsLx46uAuD</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\693b418f-8bf1-49e5-b443-b86a83235a63\tasks\bw9sge92s.output</output-file>
<status>completed</status>
<summary>Background command "Watch open PRs for review activity" completed (exit code 0)</summary>
</task-notification>

356. user (2026-07-05T11:40:26.887Z)

baseline: engine#490:OPEN/MERGEABLE/c4/r0 engine#492:OPEN/MERGEABLE/c2/r0 engine#493:OPEN/MERGEABLE/c3/r0 kh#162:OPEN/MERGEABLE/c1/r0
CHANGE DETECTED (poll 3):
  was: engine#490:OPEN/MERGEABLE/c4/r0 engine#492:OPEN/MERGEABLE/c2/r0 engine#493:OPEN/MERGEABLE/c3/r0 kh#162:OPEN/MERGEABLE/c1/r0
  now: engine#490:OPEN/MERGEABLE/c4/r0 engine#492:OPEN/MERGEABLE/c3/r0 engine#493:OPEN/MERGEABLE/c3/r0 kh#162:OPEN/MERGEABLE/c1/r0

357. assistant (2026-07-05T11:40:33.861Z)

New comment on #492 ? reading it.

358. user (2026-07-05T11:40:36.027Z)

Re-reviewed the new head `e4cf261` on the exact GitHub merge ref `7048191` (`97feff4` base + this PR). The two original blockers look addressed now:

- The cold-cache race is now probe -> claim -> realize, and the rehydrate path is serialized per `video_id` with an in-lock DB re-check. The new concurrent same-path and same-md5/different-path regressions cover the shapes I was worried about.
- The remote scratch 507 guard now runs after the read-only cache probe, so a real cache hit skips the full-source scratch reservation; cache misses still 507 pre-claim, and the probe-hit/realize-fail/no-scratch case marks the claimed row failed instead of leaving a phantom queued job.

Local gate on the merge ref:

```text
ruff check .                                          PASS
mypy backend training                                 PASS
```

```
python -m tools.check_md_links                          PASS
python -m tools.leak_scan                               PASS
git diff --check HEAD^1..HEAD                           PASS
python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70
  1682 passed, 10 skipped, coverage 85.54%              PASS floor
```

CI is also green.

I still cannot LGTM yet because the repo gate says coverage must not regress versus the exact base. I measured current base `97feff4` with the same command:

```text
base 97feff4: 1675 passed, 3 skipped, coverage 85.61%
PR merge ref: 1682 passed, 10 skipped, coverage 85.54%
```

So this is a small but real total coverage regression (`85.61% -> 85.54%`). Please add enough coverage to bring the merge ref back to at least the base percentage. The remaining uncovered new cache branches in `backend/orchestration.py` / `backend/api/jobs.py` look like good targets, but any focused coverage that restores the total is fine. Once that lands, I?ll rerun the full gate and LGTM if it stays clean.

359. user (2026-07-05T11:44:36.779Z)

```
backend\api\jobs.py                                    58      5    91%   54, 106, 169-170, 174
backend\orchestration.py                              425     57    87%   42, 75, 89-90, 97-98,
128-133, 136-137, 139, 150-151, 185, 260, 263, 269-273, 305, 308, 312, 315, 320, 328, 424-425,
443-444, 465-473, 487, 517-519, 603-604, 634-638, 708-716, 756-765, 889-890, 958-959, 1010-1016
backend\storage\gcs.py                                 75      3    96%   58, 63, 120
backend\storage\ref.py                                 49      0   100%
TOTAL                                                11825   1714    86%
```

360. user (2026-07-05T11:44:44.461Z)

```
        return None
        return None
    except edge_auth.EdgeAuthError as e:
        raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
        statuses = [status]
=== orch new-code misses ===
            return False
            return False
    except Exception:  # noqa: BLE001 ? a failed probe degrades to the guarded dispatch path
        logger.warning(
            "[API] cache probe failed for %s ? treating as a miss", video_id, exc_info=True
        )
        return False
                return None
                return None
                return None
                return None
                return None
                return None
```

361. user (2026-07-05T11:45:28.101Z)

```
    def _blob(self, ref: StorageRef):
        return self._client.bucket(ref.bucket).blob(ref.key)

    def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
        if dst is None:
            raise ValueError("GCSStorage.fetch_to_local requires a dst path")
        import os
```

```
        dst = str(dst)
        if os.path.exists(dst) and not overwrite:
            return dst
        os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
        self._blob(ref).download_to_filename(dst)
        return dst
            logger.error(f"[API] Segmenter failed: {result.message}")
            db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
            response = {
                "video_id": video_id,
                "status": "failed",
                "message": f"Segmenter '{seg_name}' failed: {result.message}",
                "results": [r.model_dump() for r in val_results],
                "play_segments": [],
            }
            return response

        segments = result.value if hasattr(result, "value") else result
        notify("finalizing")
```

362. assistant (2026-07-05T11:45:37.680Z)

Pre-existing misses aside, targeting the new-code gaps. Adding the coverage tests.

363. user (2026-07-05T11:45:59.080Z)

```
coverage tests appended
!!!!!!!!!!!!!!!!!!! Interrupted: 1 error during collection !!!!!!!!!!!!!!!!!!!
1 error in 0.49s
```

364. user (2026-07-05T11:46:07.813Z)

```
E     File "C:\Users\avidu\Projects\khelsutra-
guru\wt-p4-md5cache\tests\test_submit_md5_cache.py", line 464
E       empty.intervals_text = "start,end,shot_count,ending_reason
E                              ^
E   SyntaxError: unterminated string literal (detected at line 464)
=========================== short test summary info ============================
ERROR tests/test_submit_md5_cache.py
!!!!!!!!!!!!!!!!!!! Interrupted: 1 error during collection !!!!!!!!!!!!!!!!!!!
1 error in 0.45s
```

365. assistant (2026-07-05T11:46:16.171Z)

Heredoc escaping bit me ? fixing the broken string literal directly.

366. user (2026-07-05T11:46:17.051Z)

```
461         assert cached_process_result(db, cfg, MD5, "p") is None
462         # Success report but an EMPTY intervals.csv: n == 0 ? no payload, dispatch normally.
463         empty = _FakeBucketStorage()
464         empty.intervals_text = "start,end,shot_count,ending_reason
465     "
466         monkeypatch.setattr(storage_pkg, "get_storage", lambda *a, **k: empty)
467         assert cached_process_result(db, cfg, MD5, "p") is None
```

367. user (2026-07-05T11:46:24.951Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p4-md5cache\tests\test_submit_md5_cache.py
has been updated successfully. (file state is current in your context ? no need to Read it back)
```

368. user (2026-07-05T11:46:38.322Z)

```
..................                                                       [100%]
```

```
19 passed in 3.78s
```

369. assistant (2026-07-05T11:46:45.627Z)

19 pass. Now the coverage comparison ? branch vs. base in the same environment.

370. user (2026-07-05T11:50:03.562Z)

```
TOTAL                                              11825   1701    86%
1687 passed, 10 skipped, 25 warnings in 192.46s (0:03:12)
```

371. user (2026-07-05T11:50:13.902Z)

```
started
HEAD is now at 97feff4 feat(config): remote-poll budget knob (3900s) + advertise upload caps in
/api/capabilities (CP-rebuild P5a) (#491)
```

372. assistant (2026-07-05T11:50:24.128Z)

Running both coverage measurements precisely (JSON totals) ? branch first, then base.

373. user (2026-07-05T11:53:48.006Z)

```
branch: 85.6152
```

374. user (2026-07-05T11:56:33.447Z)

```
base 97feff4: 85.5044
```

375. user (2026-07-05T11:56:58.015Z)

```
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/wt-cov-base': Permission denied
ruff=0
mypy=0
warning: in the working copy of 'tests/test_submit_md5_cache.py', LF will be replaced by CRLF
the next time Git touches it
warning: in the working copy of 'tests/test_submit_md5_cache.py', LF will be replaced by CRLF
the next time Git touches it
[feat/submit-md5-cache-reuse dfe8216] test(process): cover the cache degradation branches ?
coverage back above base (#492 re-review)
 1 file changed, 83 insertions(+), 1 deletion(-)
   e4cf261..dfe8216  feat/submit-md5-cache-reuse -> feat/submit-md5-cache-reuse
```

376. user (2026-07-05T11:57:11.100Z)

```
https://github.com/Khelsutra/badminton-highlight-indexer/pull/492#issuecomment-4885950508
```

377. user (2026-07-05T11:57:21.723Z)

```
Command running in background with ID: boa011vqi. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\693b418f-8bf1-49e5-b443-b86a83235a63\tasks\boa011vqi.output. You will be notified when
it completes. To check interim output, use Read on that file path.
```

378. assistant (2026-07-05T11:57:31.575Z)

The reviewer's coverage finding on #492 is resolved: five new tests cover the cache code's degradation branches (probe/realize edges, DB-failure ? 500, raising stat, empty `intervals.csv`), and the same-environment comparison now shows the branch **+0.11pp above base** (85.5044% ? 85.6152%) with 1687 passed. Replied on the PR for the reviewer's gate re-run.

Standing state: #492 awaits re-gate + LGTM, #493 awaits re-verification of the rebase (its LGTM is conditional), #490 awaits LGTM re-activation, kh#162 awaits first review. The watcher is

polling every 3 minutes ? on each LGTM I merge, and #492's merge immediately unblocks P3.

379. user (2026-07-05T12:03:32.153Z)

<task-notification>
<task-id>boa011vqi</task-id>
<tool-use-id>toolu_01WhDSkrQmqcj11NZ2tCsuca</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\693b418f-8bf1-49e5-b443-b86a83235a63\tasks\boa011vqi.output</output-file>
<status>completed</status>
<summary>Background command "Watch open PRs for review activity" completed (exit code 0)</summary>
</task-notification>

380. user (2026-07-05T12:03:47.269Z)

baseline: engine#490:OPEN/MERGEABLE/c4/r0 engine#492:OPEN/MERGEABLE/c4/r0 engine#493:OPEN/MERGEABLE/c4/r0 kh#162:OPEN/MERGEABLE/c1/r0
CHANGE DETECTED (poll 2):
  was: engine#490:OPEN/MERGEABLE/c4/r0 engine#492:OPEN/MERGEABLE/c4/r0 engine#493:OPEN/MERGEABLE/c4/r0 kh#162:OPEN/MERGEABLE/c1/r0
  now: engine#490:OPEN/MERGEABLE/c4/r0 engine#492:OPEN/MERGEABLE/c5/r0 engine#493:OPEN/MERGEABLE/c4/r0 kh#162:OPEN/MERGEABLE/c1/r0

381. user (2026-07-05T12:04:03.481Z)

LGTM for #492 / CP-rebuild P4.

Re-reviewed head `dfe8216` on the exact GitHub merge ref `7b6e483` (`97feff4` base + this PR). The earlier behavioral blockers remain fixed, and the new coverage-only update is appropriately scoped to degradation branches: DB claim failure / route 500 mapping, metadata stat failure, probe miss/error edges, and cache-realize miss/non-success/empty-intervals edges.

Local verified gate:

```text
ruff check .                                          PASS
mypy backend training                                 PASS
python -m tools.check_md_links                        PASS
python -m tools.leak_scan                             PASS
git diff --check HEAD^1..HEAD                         PASS
python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70
  1687 passed, 10 skipped, coverage 85.65%            PASS
```

Coverage is now non-regressing versus the exact base measurement I took for `97feff4` (`85.61%`), and GitHub checks are all green, including the full offline suite. I did not exercise a real Cloud Run/GPU run locally; this PR's behavior is covered with the repo's fake storage/dispatch seams, which is appropriate for the submit-time MD5/cache path.

Merge-order note: #493 also touches `backend/orchestration.py`, so whichever lands second should be refreshed/re-gated on the new master before merge.

382. assistant (2026-07-05T12:04:15.657Z)

LGTM on #492 with a full verified gate ? merging now per protocol.

383. user (2026-07-05T12:04:23.776Z)

MERGEABLE SUCCESS

From https://github.com/Khelsutra/badminton-highlight-indexer
   97feff4..851f21d master   -> origin/master
851f21d feat(process): submit-time MD5 + completed-work cache-reuse ? same video never re-runs

```
the GPU (CP-rebuild P4, #484) (#492)
```

384. user (2026-07-05T12:04:41.755Z)

```
error: failed to delete '.git/worktrees/wt-p4-md5cache': Permission denied
Deleted branch feat/submit-md5-cache-reuse (was dfe8216).
Auto-merging backend/orchestration.py
CONFLICT (content): Merge conflict in backend/orchestration.py
Automatic merge failed; fix conflicts and then commit the result.
```

385. user (2026-07-05T12:04:49.846Z)

```
176:<<<<<<< HEAD
186:=======
354:>>>>>>> origin/master
```

386. user (2026-07-05T12:04:57.048Z)

```
170             os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT")
171         )
172         out_root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
173         return bool(has_job and out_root)
174
175
176     <<<<<<< HEAD
177     def _wait_progress_message(seg_name: str, elapsed_sec: float) -> str:
178         """The live job.progress line while a cloud GPU run is in flight (#482). Honest and
179         minute-granular: the control plane knows ELAPSED time, not frames ? frame-level %
comes
180         with the worker-side heartbeat (the #482 follow-up). Under a minute reads as
"starting"
181         (queue/cold-start), so the very first tick already shows movement."""
182         minutes = int(elapsed_sec) // 60
183         if minutes < 1:
184             return f"segmenting ({seg_name}) on cloud GPU ? starting up"
185         return f"segmenting ({seg_name}) on cloud GPU ? {minutes} min elapsed"
186     =======
187     def submit_time_video_id(input_ref, cfg) -> Optional[str]:
188         """The video's md5 (the canonical ``video_id``) resolvable CHEAPLY at submit time ?
gs://
189         object metadata only, one stat, no bytes (#484). ``None`` ? unknown: local files
keep
190         deferred hashing on the drain (DEFERRED_HARDENING #16), and composite objects carry
no
191         ``md5Hash``. Never raises ? a metadata miss must not block a submit."""
192         if getattr(input_ref, "backend", "") != "gcs":
193             return None
194         sc = _storage_settings(cfg)
195         if not sc:
196             return None
197         try:
198             st = storage.get_storage("gcs", config=sc).stat(input_ref)
199             return getattr(st, "md5", None) or None
```

387. user (2026-07-05T12:05:12.723Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p6-heartbeat\backend\orchestration.py has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

388. user (2026-07-05T12:05:20.308Z)

```
346                 return _cached_process_payload(video_id, [], segments, requested_compute)
347         except Exception:  # noqa: BLE001 ? cache misses must degrade to a normal dispatch
348             logger.warning(
```

```
349                 "[API] submit-time cache check failed for %s ? dispatching normally",
350                 video_id,
351                 exc_info=True,
352             )
353             return None
354     >>>>>>> origin/master
355
356
357     def _maybe_dispatch_remote(
358         cfg,
359         db,
```

389. user (2026-07-05T12:05:28.664Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p6-heartbeat\backend\orchestration.py has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

390. user (2026-07-05T12:09:07.218Z)

```
68 passed, 8 warnings in 39.51s
[feat/cloud-poll-heartbeat c8f3031] Merge origin/master (P4 #492) ? keep both the heartbeat
helper and the cache helpers at the orchestration seam
1691 passed, 10 skipped, 18 warnings in 156.11s (0:02:36)
rcs: 0 0 0 0
   0f3382b..c8f3031  feat/cloud-poll-heartbeat -> feat/cloud-poll-heartbeat
```

391. user (2026-07-05T12:09:25.476Z)

```
https://github.com/Khelsutra/badminton-highlight-indexer/pull/493#issuecomment-4885981631
MERGEABLE
```

392. user (2026-07-05T12:10:32.566Z)

```
Updated task #4 status
```

393. user (2026-07-05T12:10:35.822Z)

```
HEAD is now at 851f21d feat(process): submit-time MD5 + completed-work cache-reuse ? same video
never re-runs the GPU (CP-rebuild P4, #484) (#492)
                )
                conn.commit()
                return cur.rowcount
        except Exception as e:
            logger.error(f"Failed to sweep stale jobs: {e}")
            return 0

    def recover_stale_jobs(
        self, reason: str = "interrupted by server restart"
    ) -> Tuple[int, int]:
        """Crash recovery, kind-aware (#329 serving reliability). Called once at startup.

        A ``compile`` job is a **CPU-only, idempotent** ffmpeg re-stitch (re-resolving the same
        segments yields the same reel), so a compile left queued/running by a killed instance is
        **re-queued** ? the startup drain re-runs it and the client keeps polling until it gets
        the reel, instead of seeing a spurious "failed". A ``process`` job is an expensive,
        non-idempotent GPU detection pass; auto-retrying could double-spend, so it is **failed**
        (terminal, as before) and the client re-submits deliberately.

        Returns ``(requeued_compile, failed_process)``."""
        try:
            with self._connect() as conn:
                cur = conn.cursor()
                # Re-queue interrupted COMPILE jobs (idempotent ? safe to retry automatically).
                cur.execute(
```

```
                    "UPDATE jobs SET status='queued', started_at=NULL, next_poll_at=NULL, "
                    "error=NULL WHERE status IN ('queued', 'running') AND kind='compile'"
                )
                requeued = cur.rowcount
                # Fail interrupted PROCESS jobs (GPU, not safe to auto-retry).
                cur.execute(
                    "UPDATE jobs SET status='failed', error=?, finished_at=CURRENT_TIMESTAMP "
                    "WHERE status IN ('queued', 'running') AND (kind IS NULL OR kind !=
'compile')",
                    (reason,),
                )
                failed = cur.rowcount
                conn.commit()
                return requeued, failed
        except Exception as e:
            logger.error(f"Failed to recover stale jobs: {e}")
            return 0, 0
```

394. user (2026-07-05T12:11:30.293Z)

```
def _write_report_json(
    video_id: str, segments: List[dict], status: str, path: str
) -> None:
    with open(path, "w") as f:
        json.dump(
            {
                "video_id": video_id,
                "status": status,
                "segment_count": len(segments),
                "segments": [
                    {
                        "start": float(s.get("start_time", 0.0)),
                        "end": float(s.get("end_time", 0.0)),
                    }
                    for s in segments
                ],
            },
            f,
            indent=2,
        )


def _run_startup_checks(config: Any) -> bool:
    """M5 per-profile startup checks for the worker (the compute process, so it probes CUDA
    best-effort). Logs warnings; returns False on a FATAL coherence error so the caller aborts
    cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true no-op
    (returns True, ZERO work) when no deployment.profile is selected.

    The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily imports
    torch, and pre-M5 the worker's infer/train path was torch-free (detection is delegated to a
    WSL/native subprocess). Probing only when a profile is set keeps the default-OFF path a true
    no-op of work, not just of output."""
    profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
    cuda = (
        probe_cuda_available() if profile else None
    )  # torch-free on the default-OFF path
    try:
        enforce_startup_checks(config, cuda_available=cuda, logger=logger)
        return True
    except StartupCheckError:
=== recover call site ===
backend/main.py:1471:    requeued, failed = db_instance.recover_stale_jobs()
=== read routes ===
990:@app.get("/api/videos/{video_id}/cost")
1080:@app.post("/api/query")
```

```
=== SPA load ===
167:/** List a video's detected rallies. POST /api/query {video_id} ? {play_segments} [engine
query route]. */
```

395. user (2026-07-05T12:12:01.826Z)

```
@app.post("/api/query")
def query_play_segments(req: QueryRequest, request: Request):
    filters = {
        "server": req.server,
        "receiver": req.receiver,
        "duration_min": req.duration_min,
        "event_type": req.event_type,
    }
    segments = request.app.state.db.query_play_segments(req.video_id, filters)
    return {"play_segments": segments}


@app.post("/api/compile", status_code=202)
def compile_highlights(req: CompileRequest, request: Request):
    """Submit a reel-COMPILE job (202 + job_id); poll GET /api/jobs/{job_id} for the reel
(#329).

    Async, mirroring /api/process: the ffmpeg stitch (trim + re-encode each clip + concat +
watermark)
    can run minutes on CPU ? past the ~100 s edge/proxy timeout ? so doing it synchronously
returned a
    Cloudflare 524 while the engine kept going (the reel built, but the client saw "failed" and
never
    got the URL). The cheap checks (video exists, segments match + survive the floor, safe
filename)
    fast-fail HERE with a 4xx; the stitch runs on the worker. On `done`, the job's `result`
carries
    {filepath, download_url} ? the same shape the sync endpoint returned, now under
/api/jobs/{id}."""
    # M9 edge-auth tenant (DEFAULT-OFF ? None); a missing/spoofed identity fails 401 before
queuing.
    try:
        owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
    except edge_auth.EdgeAuthError as e:
        raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
    try:
        video, _kept, out_name = _resolve_compile(
            request.app.state.db,
            request.app.state.config,
            req.video_id,
            req.segment_ids,
            req.output_filename,
        )
    except _CompileError as e:
        raise HTTPException(status_code=e.status_code, detail=e.detail) from e

    job_id = uuid.uuid4().hex
    # Persist the video's SOURCE ref (the NOT NULL video_path); the worker re-fetches the row by
id so
    # the freshest filepath wins. owner_id/locale ride along for M9 multi-tenant + the localized
reel.
    if not request.app.state.db.create_compile_job(
        job_id,
        req.video_id,
        video["filepath"],
        req.segment_ids,
        out_name,
        locale=req.locale,
        owner_id=owner_id,
```

```
    ):
        raise HTTPException(
            status_code=500, detail="Failed to persist compile job record."
        )
    _get_executor().submit(
        _drain_due_jobs, request.app.state.config, request.app.state.db
    )
===startup===
            print(
                f"[FATAL] Another instance is already using this database ({db_path}). Refusing
to "
                f"start ? a second instance would corrupt in-flight job state. Stop the other
one, or "
                f"use a different --output-dir (RALLY_APP_HOME) to run an independent instance."
            )
            sys.exit(1)

    db_instance = Database(db_path)

    # Crash recovery (#16, #329): jobs left queued/running by a previous process are dead ?
    # the executor is in-process. Compile jobs (idempotent CPU stitch) are RE-QUEUED so a reel
    # interrupted by a Cloud Run scale-down finishes on the next drain instead of failing the
    # client; process jobs (GPU, non-idempotent) are failed so pollers see a terminal state.
    requeued, failed = db_instance.recover_stale_jobs()
    if requeued or failed:
        logger.warning(
            f"[JOBS] Recovered stale jobs from a previous run ? {requeued} compile re-queued, "
            f"{failed} process failed."
        )

    # O1/P11: sweep Gemini File-API uploads orphaned by a hard process death (the per-call
    # _safe_delete covers normal/except paths, but a SIGKILL/OOM mid-upload leaves a clip on
    # Google's servers forever). Best-effort, video-only, grace-gated; never blocks startup.
    _maybe_sweep_gemini_orphans(global_config)

    # CLI processing mode (no web UI needed). --app/--server always run the server, never CLI
single-shot.
    if args.video_path and not args.server and not args.app:
        print(f"[CLI] Processing video file: {args.video_path}")
        # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is fetched to
scratch.
        # The local-existence fast-fail only applies to a local input (stat the RESOLVED key,
not the
        # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error, not a
crash.
        storage_cfg = getattr(global_config, "storage", None)
        try:
            input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
        except ValueError as e:
            print(f"[FAIL] {e}")
            return
        if input_ref.backend == "local" and not os.path.exists(input_ref.key):
```

396. user (2026-07-05T12:13:24.416Z)

```
19:import json
20:import logging
21:import os
22:import threading
23:import traceback
24:from concurrent.futures import ThreadPoolExecutor
25:from typing import Any, Dict, Optional
33:# EXCEPTION (#466): when the server CAN dispatch to Cloud Run (see _cloud_dispatch_possible),
a REMOTE
56:def _cloud_dispatch_possible(config: Any = None) -> bool:
```

```
72:def _resolve_drain_workers(config: Any = None) -> int:
73:    """Drain concurrency (#466): a server that CAN dispatch to Cloud Run (see
_cloud_dispatch_possible)
80:    if _cloud_dispatch_possible(config):
86:def configure_drain(config: Any = None) -> None:
93:def _get_executor() -> ThreadPoolExecutor:
122:def _job_to_request(job: Dict[str, Any]):
145:def _run_claimed_job(job: Dict[str, Any], config: Any, db: Any) -> None:
196:def _maybe_record_job_cost(
238:def _maybe_run_flywheel(
261:def _drain_due_jobs(config: Any, db: Any) -> int:
286:def _schedule_next_drain_if_waiting(config: Any, db: Any) -> None:
303:def _arm_wake_timer(delay_sec: float, config: Any, db: Any) -> None:
===
            segmenter_requested=args.segmenter,
            segmenter_actual=segmenter_actual,
            was_reingest=False,
            segmentation_ran=segmentation_ran,
        )

    # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
    out_local = os.path.join(scratch, "outputs", video_id)
    os.makedirs(out_local, exist_ok=True)
    _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
    _write_report_json(
        video_id, segments, status, os.path.join(out_local, "report.json")
    )

    out_prefix = args.out_uri.rstrip("/")
    pushed = []
    for fn in sorted(os.listdir(out_local)):
        uri = f"{out_prefix}/outputs/{video_id}/{fn}"
        storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
        pushed.append(uri)

    print(
        f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)} artifact(s):"
    )
    for u in pushed:
        print("   ", u)

    # M6: a remote dispatcher passes --done-uri; write the SUCCESS completion marker so its
bucket-poll
    # terminates (carries video_id ? the dispatcher's output path + rc=0). Worker FAILURES (rc
2/3)
    # write their own marker via _fail() above, so the dispatcher fails FAST either way (no
30-min hang
    # on a bad video). DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
    if getattr(args, "done_uri", None):
        _write_done_marker(
            args.done_uri, video_id, 0, len(segments), status, storage_cfg, scratch
        )
    return 0
===
tests/test_async_jobs.py
```

397. user (2026-07-05T12:13:41.958Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

398. user (2026-07-05T12:13:44.433Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

399. user (2026-07-05T12:13:52.810Z)

```
91      def _write_report_json(
92          video_id: str, segments: List[dict], status: str, path: str
93      ) -> None:
94          with open(path, "w") as f:
95              json.dump(
96                  {
97                      "video_id": video_id,
98                      "status": status,
99                      "segment_count": len(segments),
100                     "segments": [
101                         {
102                             "start": float(s.get("start_time", 0.0)),
103                             "end": float(s.get("end_time", 0.0)),
104                         }
105                         for s in segments
106                     ],
107                 },
108                 f,
109                 indent=2,
110             )
111
112
```

400. user (2026-07-05T12:14:04.115Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p3-recovery\backend\worker\__main__.py has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

401. user (2026-07-05T12:14:12.317Z)

```
314
315         # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
316         out_local = os.path.join(scratch, "outputs", video_id)
317         os.makedirs(out_local, exist_ok=True)
318         _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
319         _write_report_json(
320             video_id, segments, status, os.path.join(out_local, "report.json")
321         )
322
323         out_prefix = args.out_uri.rstrip("/")
```

402. user (2026-07-05T12:14:21.711Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p3-recovery\backend\worker\__main__.py has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

403. assistant (2026-07-05T12:14:29.849Z)

Now the recovery carve-out in `database_jobs.py`:

404. user (2026-07-05T12:14:30.783Z)

```
576             A ``compile`` job is a **CPU-only, idempotent** ffmpeg re-stitch (re-resolving
the same
577             segments yields the same reel), so a compile left queued/running by a killed
instance is
578             **re-queued** ? the startup drain re-runs it and the client keeps polling until
it gets
579             the reel, instead of seeing a spurious "failed". A ``process`` job is an
expensive,
580             non-idempotent GPU detection pass; auto-retrying could double-spend, so it is
**failed**
```

```
581             (terminal, as before) and the client re-submits deliberately.
582
583             Returns ``(requeued_compile, failed_process)``."""
584             try:
585                 with self._connect() as conn:
586                     cur = conn.cursor()
587                     # Re-queue interrupted COMPILE jobs (idempotent ? safe to retry
automatically).
588                     cur.execute(
589                         "UPDATE jobs SET status='queued', started_at=NULL,
next_poll_at=NULL, "
590                         "error=NULL WHERE status IN ('queued', 'running') AND
kind='compile'"
591                     )
592                     requeued = cur.rowcount
593                     # Fail interrupted PROCESS jobs (GPU, not safe to auto-retry).
594                     cur.execute(
595                         "UPDATE jobs SET status='failed', error=?,
finished_at=CURRENT_TIMESTAMP "
596                         "WHERE status IN ('queued', 'running') AND (kind IS NULL OR kind !=
'compile')",
597                         (reason,),
598                     )
599                     failed = cur.rowcount
600                     conn.commit()
601                     return requeued, failed
602             except Exception as e:
603                 logger.error(f"Failed to recover stale jobs: {e}")
604                 return 0, 0
605
606
```

405. user (2026-07-05T12:14:54.308Z)

```
The file C:\Users\avidu\Projects\khelsutra-
guru\wt-p3-recovery\backend\infrastructure\database_jobs.py has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

406. assistant (2026-07-05T12:15:00.744Z)

Now the resume pass in `job_worker.py`:

407. user (2026-07-05T12:15:02.057Z)

```
19      import json
20      import logging
21      import os
22      import threading
23      import traceback
24      from concurrent.futures import ThreadPoolExecutor
25      from typing import Any, Dict, Optional
26
27      logger = logging.getLogger(__name__)
28
29      # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 by default: a single
30      # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
31      # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
32      # already parallelize their AI handoff internally via their own pools.
33      # EXCEPTION (#466): when the server CAN dispatch to Cloud Run (see
_cloud_dispatch_possible), a REMOTE
34      # job is only a bucket-poll (I/O-bound; GPU is on ephemeral L4s, WASB-only has no
Gemini), so the drain
35      # scales to deployment.max_concurrent_remote_jobs to overlap those. This does NOT
parallelize local
```

```
36      # work: CPU/GPU-heavy in-process detection + the compile stitch serialize via
37      # orchestration._LOCAL_COMPUTE_LANE regardless of the worker count, so the single-box
invariant holds
38      # and compile/local-override jobs stay serial. The count is fixed at first
_get_executor() creation.
39      #
40      # ? O2 RISK (CODE_CLEANUP_AUDIT §3c): a single worker means ONE stuck or hung job
41      # (e.g. hung Gemini generate, hung WSL GPU call, hung Cloud Run poll) blocks ALL
42      # subsequent jobs ? they stay 'queued' forever. Mitigations already in place:
43      #   - ExecutionPolicy op_timeout_sec kills hung generate/processing calls
44      #   - detector timeout_sec kills hung WSL/GPU calls
45      #   - Cloud Run poll has a max_wait_sec bound
46      #   - ffmpeg/ffprobe subprocesses use bounded media timeouts
47      # A future slice should add a per-job wall-clock timeout (the executor itself
48      # cannot enforce timeouts on the Thread).
49      _job_executor: Optional[ThreadPoolExecutor] = None
50      # Drain worker count, fixed at first _get_executor() creation. 1 (serial) by default;
configure_drain()
51      # raises it for cloud_run (#466). A module global (not a _get_executor arg) so the many
call sites +
52      # the tests that monkeypatch `_get_executor` as a no-arg lambda are unchanged.
53      _drain_max_workers: int = 1
54
55
56      def _cloud_dispatch_possible(config: Any = None) -> bool:
57          """True if this server CAN dispatch a job to Cloud Run ? either the profile sets
58          ``deployment.compute_target=cloud_run``, OR the per-request ``compute='cloud'``
override path is
59          wired (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT`` env + an output bucket). Mirrors
60          orchestration._cloud_available so drain concurrency tracks the ACTUAL remote-
dispatch capability,
61          not just the server default (so compute='cloud' overrides on a local-default server
parallelize too)."""
62          dep = getattr(config, "deployment", None)
63          if getattr(dep, "compute_target", "") == "cloud_run":
64              return True
65          if os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT"):
66              stor = getattr(config, "storage", None)
67              if (getattr(stor, "output_root", "") or "").strip():
68                  return True
69          return False
70
71
72      def _resolve_drain_workers(config: Any = None) -> int:
73          """Drain concurrency (#466): a server that CAN dispatch to Cloud Run (see
_cloud_dispatch_possible)
```

408. user (2026-07-05T12:15:10.983Z)

```
259-
260-
261:def _drain_due_jobs(config: Any, db: Any) -> int:
    _main._arm_wake_timer(max(0.0, min(delay, _main._MAX_DRAIN_WAKE_SEC)), config, db)


def _arm_wake_timer(delay_sec: float, config: Any, db: Any) -> None:
    """Arm a single daemon timer that re-submits a drain after ``delay_sec`` (no central
    scheduler ? the DB is the clock). Isolated as the sole timer-creating seam so tests can
    monkeypatch it to a no-op and assert the *scheduling decision* without real waiting."""
    import backend.main as _main

    timer = threading.Timer(
        delay_sec,
        lambda: _main._get_executor().submit(_main._drain_due_jobs, config, db),
    )
```

```
    timer.daemon = True
    timer.start()


#: A parked job further out than this is re-checked by a capped wake rather than one long
#: sleeping timer ? keeps the wake cadence bounded without a central scheduler.
_MAX_DRAIN_WAKE_SEC = 60.0
```

409. user (2026-07-05T12:15:42.997Z)

```
resume pass added
```

410. assistant (2026-07-05T12:16:11.598Z)

```
Now the orchestration read-path helpers and source-URI recovery:
```

411. user (2026-07-05T12:16:25.243Z)

```
orchestration helpers added
```

412. assistant (2026-07-05T12:16:32.455Z)

```
Now wiring main.py ? the query-route hook and the startup resume call:
```

413. user (2026-07-05T12:16:34.035Z)

dispatch+ingest, compile resolution + worker handler) now lives in backend.orchestra (P23 god-module split; re-cut of #390). Re-exported here so the names stay addressab backend.main ? `import backend.main as m; m._execute_processing(...)`, the job work call-time `_main._execute_compile(...)` seam, and monkeypatch.setattr(backend.main, all keep working unchanged.

```
from backend.orchestration import (  # noqa: F401  (re-exports: names stay addressable on
backend.main)
    _cloud_available,
    _CompileError,
    _execute_compile,
    _execute_processing,
    _finalize_reingest,
    _ingest_remote_segments,
    _maybe_dispatch_remote,
    _remote_input_size_bytes,
    _report_config_for_input,
    _resolve_compile,
    signed_reel_url,
)

from backend.api.job_worker import (  # noqa: F401  (re-exports: names stay addressable on
backend.main)
    _MAX_DRAIN_WAKE_SEC,
    JobWaiting,
    _arm_wake_timer,
    _drain_due_jobs,
    _get_executor,
    _job_to_request,
    _maybe_record_job_cost,
    _run_claimed_job,
```

414. user (2026-07-05T12:16:55.734Z)

```
main.py wired
```

```
1 failed, 2 passed in 2.49s
```

415. user (2026-07-05T12:17:07.058Z)

```
>       assert n <= MAIN_LINE_BUDGET, (
E       assert 1654 <= 1653
```

416. assistant (2026-07-05T12:17:18.331Z)

One line over budget ? trimming the startup comment to a single line.

417. user (2026-07-05T12:17:19.618Z)

```
1479            logger.warning(
1480                f"[JOBS] Recovered stale jobs from a previous run ? {requeued} compile re-
queued, "
1481                f"{failed} process failed."
1482            )
1483        # CP-rebuild P3 (#471): md5-keyed RUNNING remote jobs are resumed on the durable
1484        # done-signal (outputs/<md5>/report.json) instead of failed ? never re-dispatched.
1485        resume_interrupted_remote_jobs(global_config, db_instance)
1486
1487        # O1/P11: sweep Gemini File-API uploads orphaned by a hard process death (the per-
call
1488        # _safe_delete covers normal/except paths, but a SIGKILL/OOM mid-upload leaves a
clip on
```

418. user (2026-07-05T12:17:29.669Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p3-recovery\backend\main.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

419. assistant (2026-07-05T12:18:03.612Z)

Now the P3 test suite:

420. user (2026-07-05T12:18:05.788Z)

```
    def create_job(
        self,
        job_id: str,
        video_path: str,
        segmenter: Optional[str] = None,
        player_pool: Optional[List[str]] = None,
        max_frames: Optional[int] = None,
        policy: Optional[Dict[str, Any]] = None,
        owner_id: Optional[str] = None,
        locale: Optional[str] = None,
        compute: Optional[str] = None,
        force: bool = False,
    ) -> bool:
        """Insert a new job in state 'queued'. ``policy`` is an optional JSON ExecutionPolicy
        (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy.
``owner_id``
        (M9, D9) is the verified tenant from edge auth ? NULL for the local single-user
(default).
        ``locale`` (v7) is the UI language the submission came from ? NULL when unrecorded.
        ``compute`` (v8) is the SPA compute-picker choice ("local"/"cloud") the worker
reconstructs +
        honours; NULL ? the server default (byte-identical to pre-picker behaviour). ``force``
opts
        into deliberate reprocessing; false leaves params NULL for old-row compatibility."""
        params = json.dumps({"force": True}) if force else None
        return self._execute_write(
```

```
            "INSERT INTO jobs (id, video_path, segmenter, player_pool, max_frames, status,
policy, "
            "owner_id, locale, compute, params) VALUES (?, ?, ?, ?, ?, 'queued', ?, ?, ?, ?,
?)",
            (
                job_id,
                video_path,
                segmenter,
                json.dumps(player_pool) if player_pool is not None else None,
                max_frames,
                json.dumps(policy) if policy is not None else None,
                owner_id,
                locale,
                compute,
                params,
            ),
            f"Failed to create job {job_id}",
        )

    def create_compile_job(
296:class QueryRequest(BaseModel):
297-    video_id: str
298-    server: Optional[str] = None
299-    receiver: Optional[str] = None
300-    duration_min: Optional[float] = None
301-    event_type: Optional[str] = None
302-
303-
304-class CompileRequest(BaseModel):
def test_worker_writes_done_marker_on_success(tmp_path, monkeypatch):
    """The container side: cmd_infer with --done-uri writes a completion marker (video_id + rc)
so a
    remote dispatcher's bucket-poll terminates. Driven by the deterministic stub + skip-AI."""
    from backend.worker import __main__ as wmain

    class _OK:
        passed = True
        validator_name = "fake"
        message = "ok"

        def model_dump(self):
            return {"validator_name": "fake", "passed": True, "message": "ok"}

    monkeypatch.setattr(
        wmain.validator_registry,
        "run_all_with_context",
        lambda path, cfg: ([_OK()], {}),
    )
    monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
    monkeypatch.setattr(
        "backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
        lambda p: 30.0,
    )

    cfg = tmp_path / "c.json"
    cfg.write_text(
        json.dumps(
            {
                "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
                "output": {"output_dir": str(tmp_path / "o")},
            }
        )
    )
    video = tmp_path / "in.mp4"
    video.write_bytes(b"FAKEVIDEO")
```

```
    out = tmp_path / "bucket"
    done = str(out / "_dispatch" / "tok.done")

    rc = wmain.cmd_infer(
        wmain.build_parser().parse_args(
```

421. user (2026-07-05T12:18:17.916Z)

```
        wmain.build_parser().parse_args(
            [
                "infer",
                "--video-uri",
                str(video),
                "--out-uri",
                str(out),
                "--config",
                str(cfg),
                "--scratch",
                str(tmp_path / "s"),
                "--segmenter",
                "wasb_hybrid",
                "--done-uri",
                done,
            ]
        )
    )
    assert rc == 0
    marker = json.loads(open(done, encoding="utf-8").read())
    assert marker["returncode"] == 0 and marker["status"] == "success"
    assert marker["video_id"] and marker["segment_count"] >= 1


def test_worker_no_done_uri_is_unchanged(tmp_path, monkeypatch):
    """DEFAULT-OFF: without --done-uri the inline run writes NO marker (today's behaviour)."""
    from backend.worker import __main__ as wmain

    class _OK:
        passed = True
        validator_name = "fake"
        message = "ok"

        def model_dump(self):
            return {"validator_name": "fake", "passed": True, "message": "ok"}

    monkeypatch.setattr(
        wmain.validator_registry,
        "run_all_with_context",
        lambda path, cfg: ([_OK()], {}),
    )
```

422. user (2026-07-05T12:18:36.791Z)

```
worker report assertion added
1 passed in 2.33s
```

423. assistant (2026-07-05T12:19:27.355Z)

```
"""Restart-safe remote jobs + read-path re-hydrate (#471 / #485 ? CP-rebuild P3).

The 2026-07-05 loss shape under test: a running cloud detection's done-marker poll lived only
in process memory; a restart orphaned the job while the L4 worker kept computing, and its 29
rallies landed in the bucket with no listener. Contract now:

- ``recover_stale_jobs`` leaves RUNNING md5-keyed process jobs alone (everything else keeps
```

```
  today's semantics: compiles re-queue, other process rows fail terminally)
- ``resume_interrupted_remote_jobs`` re-arms the durable done-signal poll
  (``outputs/<md5>/report.json``) for those rows ? completing or failing them honestly,
  NEVER re-dispatching (no double GPU spend); on a box that can't dispatch to Cloud Run the
  rows get today's terminal failure
- ``POST /api/query`` re-hydrates an md5-shaped id from the bucket before reporting nothing
  (#485: DB = cache, bucket = truth), restoring the video row's truthful source path from the
  report's ``video_uri`` (or the ``videos/<md5>/`` listing for older reports)
"""

import hashlib
import json
import os

import pytest

pytest.importorskip("httpx")  # TestClient needs httpx; skip cleanly where absent
from fastapi.testclient import TestClient

import backend.main as m
import backend.storage as storage_pkg
from backend.api import job_worker
from backend.config.models import AppConfig
from backend.infrastructure.database import Database
from backend.storage.ref import StorageRef, StorageStat

MD5 = hashlib.md5(b"recovery-bytes").hexdigest()
SOURCE_URI = "gs://bkt/videos/original-match.mp4"
_INTERVALS = (
    "start,end,shot_count,ending_reason\n"
    "2.0,11.0,5,rally_end\n"
    "14.5,29.0,8,rally_end\n"
)


class _FakeBucket:
    """stat/exists/fetch/list surface for outputs/<md5>/ recovery reads."""

    def __init__(self, report="success", video_uri=SOURCE_URI):
        self.report = (
            None
            if report is None
            else {
                "video_id": MD5,
                "segment_count": 2,
                "status": report,
                "video_uri": video_uri,
            }
        )
        self.listed = []
        self.staged_under_videos_md5 = []  # optional videos/<md5>/ fallback objects
        self.exists_calls = []

    def stat(self, ref):
        return StorageStat(size=1234, md5=MD5)

    def exists(self, ref):
        self.exists_calls.append(ref.uri)
        return self.report is not None and ref.uri.endswith("/report.json")

    def list(self, prefix):
        self.listed.append(prefix.uri)
        for uri in self.staged_under_videos_md5:
            yield StorageRef("gcs", "bkt", uri.split("/", 3)[-1], uri)
```

```
    def fetch_to_local(self, ref, dst=None, *, overwrite=False):
        os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
        if ref.uri.endswith("report.json") and self.report is not None:
            with open(dst, "w", encoding="utf-8") as f:
                json.dump(self.report, f)
        elif ref.uri.endswith("intervals.csv"):
            with open(dst, "w", encoding="utf-8", newline="") as f:
                f.write(_INTERVALS)
        else:
            raise FileNotFoundError(ref.uri)
        return dst


def _gcs_cfg(tmp_path):
    cfg = AppConfig()
    cfg.output.output_dir = str(tmp_path / "out")
    cfg.storage.backend = "gcs"
    cfg.storage.output_root = "gs://bkt"
    cfg.storage.input_backend = "gcs"
    cfg.storage.input_root = "gs://bkt/videos"
    return cfg


def _running_md5_job(db, job_id="jrun", path="gs://bkt/videos/original-match.mp4"):
    claimed = db.claim_or_create_process_job(job_id, path, video_id=MD5)
    assert claimed and claimed["created"]
    assert db.mark_job_running(job_id)
    return job_id
```

--- recover_stale_jobs carve-out ----------

```
def test_recover_leaves_running_md5_process_jobs_alone(tmp_path):
    db = Database(str(tmp_path / "t.db"))
    _running_md5_job(db, "jresume")  # running + md5 ? the resume pass owns it
    db.claim_or_create_process_job("jqueued", "gs://bkt/videos/q.mp4", video_id=MD5)
    db.claim_or_create_process_job("jnomd5", "gs://bkt/videos/n.mp4", force=True)
    db.mark_job_running("jnomd5")  # running but md5-less ? nothing durable to re-associate

    requeued, failed = db.recover_stale_jobs()

    assert requeued == 0
    assert failed == 2  # the queued row + the md5-less running row
    assert db.get_job("jresume")["status"] == "running"  # left for the resume pass
    assert db.get_job("jqueued")["status"] == "failed"
    assert db.get_job("jnomd5")["status"] == "failed"
    rows = db.interrupted_remote_process_jobs()
    assert [r["id"] for r in rows] == ["jresume"]
    assert rows[0]["video_id"] == MD5


def test_recover_still_requeues_compiles(tmp_path):
    db = Database(str(tmp_path / "t.db"))
    db.create_compile_job("jc", "vid1", "/v/x.mp4", [1, 2], "reel.mp4")
    db.mark_job_running("jc")
    requeued, failed = db.recover_stale_jobs()
    assert requeued == 1 and failed == 0
    assert db.get_job("jc")["status"] == "queued"
```

--- resume_interrupted_remote_jobs ----------

```
def test_resume_fails_rows_when_cloud_dispatch_impossible(tmp_path):
    db = Database(str(tmp_path / "t.db"))
    job_id = _running_md5_job(db)
    cfg = AppConfig()  # local everything: no cloud target, no env coordinates
    assert job_worker.resume_interrupted_remote_jobs(cfg, db) == 0
    job = db.get_job(job_id)
    assert job["status"] == "failed"
    assert "interrupted by server restart" in (job["error"] or "")


def test_resume_spawns_one_thread_per_row(tmp_path, monkeypatch):
    db = Database(str(tmp_path / "t.db"))
    a = _running_md5_job(db, "ja", "gs://bkt/videos/a.mp4")
    b = _running_md5_job(db, "jb", "gs://bkt/videos/b.mp4")
    cfg = _gcs_cfg(tmp_path)
    cfg.deployment.compute_target = "cloud_run"
    resumed = []
    monkeypatch.setattr(
        job_worker,
        "_resume_one_remote_job",
        lambda config, database, job: resumed.append(job["id"]),
    )
    assert job_worker.resume_interrupted_remote_jobs(cfg, db) == 2
    # Daemon threads run the stub synchronously-fast; poll briefly for both.
    import time

    for _ in range(50):
        if sorted(resumed) == ["ja", "jb"]:
            break
        time.sleep(0.02)
    assert sorted(resumed) == ["ja", "jb"]


def test_resume_one_completes_from_the_bucket_report(tmp_path, monkeypatch):
    """The headline: the orphaned job is completed from the worker's durable output ?
    segments ingested, job done with the cached payload, no dispatch anywhere."""
    db = Database(str(tmp_path / "t.db"))
    job_id = _running_md5_job(db)
    cfg = _gcs_cfg(tmp_path)
    fake = _FakeBucket()
    monkeypatch.setattr(storage_pkg, "get_storage", lambda *a, **k: fake)
    monkeypatch.setattr(job_worker, "_RESUME_POLL_EVERY_SEC", 0.0)

    job_worker._resume_one_remote_job(cfg, db, db.get_job(job_id))

    job = db.get_job(job_id)
    assert job["status"] == "done"
    result = job["result"]
    if isinstance(result, str):
        result = json.loads(result)
    assert result["status"] == "success" and result["cached"] is True
    assert len(result["play_segments"]) == 2
    assert len(db.query_play_segments(MD5, {})) == 2


def test_resume_one_surfaces_a_failed_worker_verdict(tmp_path, monkeypatch):
    db = Database(str(tmp_path / "t.db"))
    job_id = _running_md5_job(db)
    cfg = _gcs_cfg(tmp_path)
    fake = _FakeBucket(report="failed")
    monkeypatch.setattr(storage_pkg, "get_storage", lambda *a, **k: fake)
    monkeypatch.setattr(job_worker, "_RESUME_POLL_EVERY_SEC", 0.0)

    job_worker._resume_one_remote_job(cfg, db, db.get_job(job_id))
```

```
    job = db.get_job(job_id)
    assert job["status"] == "failed"
    assert "report status: failed" in (job["error"] or "")


def test_resume_one_times_out_honestly_when_no_result_appears(tmp_path, monkeypatch):
    db = Database(str(tmp_path / "t.db"))
    job_id = _running_md5_job(db)
    cfg = _gcs_cfg(tmp_path)
    cfg.deployment.remote_poll_max_wait_sec = 0.05  # tiny budget for the test
    fake = _FakeBucket(report=None)  # the worker never wrote anything
    monkeypatch.setattr(storage_pkg, "get_storage", lambda *a, **k: fake)
    monkeypatch.setattr(job_worker, "_RESUME_POLL_EVERY_SEC", 0.0)

    job_worker._resume_one_remote_job(cfg, db, db.get_job(job_id))

    job = db.get_job(job_id)
    assert job["status"] == "failed"
    assert "never appeared" in (job["error"] or "")


def test_resume_one_fails_cleanly_without_an_output_bucket(tmp_path):
    db = Database(str(tmp_path / "t.db"))
    job_id = _running_md5_job(db)
    cfg = AppConfig()  # no output_root to recover from
    job_worker._resume_one_remote_job(cfg, db, db.get_job(job_id))
    job = db.get_job(job_id)
    assert job["status"] == "failed"
    assert "no output bucket" in (job["error"] or "")
```

--- /api/query read-path re-hydrate (#485) ----------

```
@pytest.fixture
def env(tmp_path, monkeypatch):
    db = Database(str(tmp_path / "t.db"))
    cfg = _gcs_cfg(tmp_path)
    os.makedirs(cfg.output.output_dir, exist_ok=True)
    fake = _FakeBucket()
    monkeypatch.setattr(storage_pkg, "get_storage", lambda *a, **k: fake)
    client = TestClient(m.create_app(cfg, db))
    return client, db, fake


def test_query_rehydrates_md5_id_from_the_bucket(env):
    """Reload ?video=<md5> after a restart wipe ? the rallies appear (the §9 DoD shape),
    and the video row carries the report's truthful source path."""
    client, db, fake = env
    r = client.post("/api/query", json={"video_id": MD5})
    assert r.status_code == 200
    assert len(r.json()["play_segments"]) == 2
    assert db.get_video(MD5)["filepath"] == SOURCE_URI


def test_query_non_md5_id_never_touches_the_bucket(env):
    client, db, fake = env
    r = client.post("/api/query", json={"video_id": "not-an-md5"})
    assert r.status_code == 200
    assert r.json()["play_segments"] == []
    assert fake.exists_calls == []


def test_query_rehydrate_falls_back_to_videos_prefix_for_old_reports(env):
```

```
    """Reports written before P3 lack video_uri ? the staged-canonical videos/<md5>/
    listing is the fallback source of the truthful path."""
    client, db, fake = env
    fake.report.pop("video_uri")
    staged = f"gs://bkt/videos/{MD5}/original-match.mp4"
    fake.staged_under_videos_md5 = [staged]
    r = client.post("/api/query", json={"video_id": MD5})
    assert len(r.json()["play_segments"]) == 2
    assert db.get_video(MD5)["filepath"] == staged


def test_query_rehydrate_tolerates_unknown_source(env):
    """No video_uri and nothing staged: segments still recover (rallies render); the path
    re-binds on the next same-md5 submit via add_video's upsert."""
    client, db, fake = env
    fake.report.pop("video_uri")
    r = client.post("/api/query", json={"video_id": MD5})
    assert len(r.json()["play_segments"]) == 2
    assert db.get_video(MD5)["filepath"] == ""
```

424. user (2026-07-05T12:19:27.391Z)

```
File created successfully at: C:\Users\avidu\Projects\khelsutra-
guru\wt-p3-recovery\tests\test_restart_recovery.py (file state is current in your context ? no
need to Read it back)
```

425. user (2026-07-05T12:19:41.342Z)

```
TypeError: poll_until() got an unexpected keyword argument 'on_tick'
========================== short test summary info ==========================
FAILED tests/test_restart_recovery.py::test_resume_one_completes_from_the_bucket_report
FAILED tests/test_restart_recovery.py::test_resume_one_surfaces_a_failed_worker_verdict
FAILED tests/test_restart_recovery.py::test_resume_one_times_out_honestly_when_no_result_appears
3 failed, 9 passed in 3.98s
```

426. user (2026-07-05T12:19:59.676Z)

```
<tool_use_error>File has been modified since read, either by the user or by a linter. Read it
again before attempting to write it.</tool_use_error>
```

427. user (2026-07-05T12:20:08.325Z)

```
16:module; this module must not import main at import time).
19:import json
20:import logging
21:import os
22:import threading
23:import traceback
409:                on_tick=lambda elapsed: db.set_job_progress(
```

428. user (2026-07-05T12:20:15.409Z)

```
396             )
397             policy = ExecutionPolicy(
398                 poll_every_n_sec=_RESUME_POLL_EVERY_SEC,
399                 op_timeout_sec=120.0,
400                 max_wait_sec=max_wait,
401             )
402             db.set_job_progress(job_id, "recovering: waiting for the in-flight cloud run")
403             try:
404                 poll_until(
405                     lambda: True if storage.exists(report_uri, config=sc) else None,
406                     policy,
407                     label=f"resume-{job_id[:8]}",
```

```
408                     logger=logger,
409                     on_tick=lambda elapsed: db.set_job_progress(
410                         job_id,
411                         "recovering: waiting for the in-flight cloud run ? "
412                         f"{int(elapsed) // 60} min",
413                     ),
414                 )
415             except PolicyTimeout:
416                 db.mark_job_failed(
417                     job_id,
418                     "control plane restarted mid-run and the cloud worker's result never "
419                     "appeared within the poll budget; re-submit the video (a finished result
"
420                     "re-hydrates instantly)",
421                 )
422                 return
423             payload = orchestration.cached_process_result(
```

429. user (2026-07-05T12:20:26.756Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p3-recovery\backend\api\job_worker.py has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

430. user (2026-07-05T12:20:29.341Z)

```
time import added
```

431. user (2026-07-05T12:20:43.692Z)

```
............                                                             [100%]
12 passed in 3.45s
```

432. user (2026-07-05T12:24:44.418Z)

```
1699 passed, 10 skipped, 18 warnings in 181.20s (0:03:01)
Found 3 errors.
[*] 1 fixable with the `--fix` option (2 hidden fixes can be enabled with the `--unsafe-fixes`
option).
Success: no issues found in 148 source files
warning: in the working copy of 'backend/api/job_worker.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'backend/main.py', LF will be replaced by CRLF the next time Git
touches it
warning: in the working copy of 'backend/orchestration.py', LF will be replaced by CRLF the next
time Git touches it
warning: in the working copy of 'tests/test_compute_backend.py', LF will be replaced by CRLF the
next time Git touches it
rcs: 0 1 0 0
```

433. user (2026-07-05T12:24:52.929Z)

```
I001 [*] Import block is un-sorted or un-formatted
   --> backend\main.py:214:1
    |
212 |   # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
213 |   # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
214 | / from backend.api.job_worker import (  # noqa: F401  (re-exports: names stay addressable
on backend.main)
215 | |     _MAX_DRAIN_WAKE_SEC,
216 | |     JobWaiting,
217 | |     _arm_wake_timer,
218 | |     _drain_due_jobs,
219 | |     _get_executor,
220 | |     _job_to_request,
```

```
221 | |     _maybe_record_job_cost,
222 | |     _run_claimed_job,
223 | |     resume_interrupted_remote_jobs,
224 | |     _schedule_next_drain_if_waiting,
225 | |     configure_drain,
226 | | )
227 | |
228 | | # The processing/compile ORCHESTRATION (hash?validate?segment?report pipeline, Cloud Run
```

434. user (2026-07-05T12:25:06.104Z)

```
Found 3 errors (1 fixed, 2 remaining).
No fixes available (2 hidden fixes can be enabled with the `--unsafe-fixes` option).
No fixes available (2 hidden fixes can be enabled with the `--unsafe-fixes` option).
3 passed in 2.47s
[This command modified 1 file you've previously read: backend\main.py. Call Read before
editing.]
```

435. user (2026-07-05T12:25:14.340Z)

```
F841 Local variable `a` is assigned to but never used
   --> tests\test_restart_recovery.py:149:5
    |
147 | def test_resume_spawns_one_thread_per_row(tmp_path, monkeypatch):
148 |     db = Database(str(tmp_path / "t.db"))
149 |     a = _running_md5_job(db, "ja", "gs://bkt/videos/a.mp4")
    |     ^
150 |     b = _running_md5_job(db, "jb", "gs://bkt/videos/b.mp4")
151 |     cfg = _gcs_cfg(tmp_path)
    |
help: Remove assignment to unused variable `a`

F841 Local variable `b` is assigned to but never used
   --> tests\test_restart_recovery.py:150:5
    |
148 |     db = Database(str(tmp_path / "t.db"))
149 |     a = _running_md5_job(db, "ja", "gs://bkt/videos/a.mp4")
150 |     b = _running_md5_job(db, "jb", "gs://bkt/videos/b.mp4")
    |     ^
151 |     cfg = _gcs_cfg(tmp_path)
```

436. user (2026-07-05T12:25:24.539Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p3-recovery\tests\test_restart_recovery.py
has been updated successfully. (file state is current in your context ? no need to Read it back)
```

437. user (2026-07-05T12:29:01.492Z)

```
ruff=0
1699 passed, 10 skipped, 25 warnings in 197.91s (0:03:17)
P3 branch: 85.6003
mypy=0
warning: in the working copy of 'backend/api/job_worker.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'backend/main.py', LF will be replaced by CRLF the next time Git
touches it
warning: in the working copy of 'backend/orchestration.py', LF will be replaced by CRLF the next
time Git touches it
warning: in the working copy of 'tests/test_compute_backend.py', LF will be replaced by CRLF the
next time Git touches it
diff=0
```

438. user (2026-07-05T12:30:11.672Z)

```
backend\api\job_worker.py     162     40    75%   67-69, 81-84, 91, 98-102, 118-120, 190-191,
217-234, 254-259, 301, 308-315, 447-448, 454-459
backend\orchestration.py      441    137    69%   43, 76, 90-91, 98-99, 129-134, 137-138, 140,
151-152, 169, 186, 189, 258, 261, 264, 267, 270-274, 296-298, 306, 309, 313, 330, 338, 346-352,
359-360, 376, 405-410, 432-436, 438, 440-446, 454-605, 637-642, 645, 647, 669-673, 743-751,
768-771, 791-800, 924-925, 950, 966-972, 993-994, 1045-1051
```

439. user (2026-07-05T12:30:19.979Z)

```
        except Exception:  # noqa: BLE001 ? the failure message is best-effort
            pass
    except Exception as e:  # noqa: BLE001 ? a resume crash must land as a job failure, not
vanish
        logger.error("[JOBS] resume of job %s crashed: %s", job_id, e, exc_info=True)
        try:
            db.mark_job_failed(job_id, f"restart recovery failed: {e}")
        except Exception:  # noqa: BLE001
            pass
===
        return None
        return getattr(st, "md5", None) or None
```

440. user (2026-07-05T12:30:40.505Z)

```
3 more recovery tests
15 passed in 3.15s
```

441. user (2026-07-05T12:33:57.493Z)

```
1702 passed, 10 skipped, 25 warnings in 179.10s (0:02:59)
P3 branch now: 85.6507 (base 851f21d == 85.6152)
ruff=0
mypy=0
```

442. user (2026-07-05T12:34:21.259Z)

```
feat/restart-safe-recovery
warning: in the working copy of 'backend/api/job_worker.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'backend/main.py', LF will be replaced by CRLF the next time Git
touches it
warning: in the working copy of 'backend/orchestration.py', LF will be replaced by CRLF the next
time Git touches it
warning: in the working copy of 'tests/test_compute_backend.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'tests/test_restart_recovery.py', LF will be replaced by CRLF
the next time Git touches it
[feat/restart-safe-recovery 627e8f4] feat(recovery): restart-safe remote jobs + read-path re-
hydrate (CP-rebuild P3, #471 #485)
 7 files changed, 555 insertions(+), 5 deletions(-)
 create mode 100644 tests/test_restart_recovery.py
627e8f4 feat(recovery): restart-safe remote jobs + read-path re-hydrate (CP-rebuild P3, #471
#485)
 * [new branch]      feat/restart-safe-recovery -> feat/restart-safe-recovery
```

443. user (2026-07-05T12:34:51.496Z)

```
https://github.com/Khelsutra/badminton-highlight-indexer/pull/494
```

444. user (2026-07-05T12:35:04.805Z)

```
The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\693b418f-8bf1-49e5-b443-b86a83235a63\scratchpad\pr_watch.sh has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

445. user (2026-07-05T12:35:09.957Z)

```
--- #490 latest ---
Re the paused LGTM: the guard concern is fixed on #492 (`e4cf261`) ? the remote-input scratch
guard now runs **after the cache probe** inside `jobs.submit_process_job`, so it fires only on
paths that will actually fetch to local scratch. Your 10 GiB / 1 MiB-free repro returns the
instant `done` on a cache hit, and a cache miss still 507s pre-claim (both regression-tested
there).

Honest boundary so the re-review is precise: after #492, a **fresh** (uncached) gs:// submit ?
including a freshly staged browser upload from this PR ? still fetches to CP scratch on the
drain for hash+validate before cloud dispatch; that's pre-existing #467-era behavior, the guard
stays correct for it, and P1's 8 GiB covers it at the 4096 MB cap. Skipping the CP-side acquire
entirely for cloud dispatch when the md5 is already known is the natural next slice (it belongs
with P3's drain/recovery work ? happy to fold it in there).

This branch itself is updated: origin/master (#491) merged in with the `uploads.py` seam
conflict resolved (both `upload_caps` and `_staging_target` kept), full suite **1678 passed / 10
skipped**, ruff/mypy/diff-check clean on the branch head `dda8a68`.
--- #493 latest ---
LGTM for the rebased P6 control-plane heartbeat slice.

Verified the current non-draft, mergeable head `c8f3031` on the exact GitHub merge ref `0df7310`
(`851f21d` base + this PR). I inspected the PR metadata, diff, CI, comments, and review-thread
state (no active review threads). The implementation remains appropriately scoped:
`poll_until(on_tick=...)` is best-effort and cannot kill the wait,
`CloudRunCompute.run_infer(on_wait=...)` forwards the callback into the bucket poll, and
`_maybe_dispatch_remote` updates `job.progress` with honest elapsed-time wording rather than
frame-level progress.

Local verified gate:

--- kh162 count ---
1
```

446. user (2026-07-05T12:35:31.840Z)

```
MERGEABLE SUCCESS

From https://github.com/Khelsutra/badminton-highlight-indexer
   851f21d..4cd36fd  master     -> origin/master
4cd36fd feat(progress): live cloud-poll heartbeat ? job.progress moves every tick during a GPU
run (CP-rebuild P6 slice, #482) (#493)
error: failed to delete '.git/worktrees/wt-p6-heartbeat': Permission denied
Deleted branch feat/cloud-poll-heartbeat (was c8f3031).
```

447. user (2026-07-05T12:35:44.046Z)

```
 tests/test_api_cloud_dispatch.py           | 11 ++++++++
 tests/test_compute_backend.py              | 33 ++++++++++++++++++++++++
 tests/test_execution_policy.py             | 42 ++++++++++++++++++++++++++++++++++++++
 7 files changed, 135 insertions(+), 7 deletions(-)
```

448. user (2026-07-05T12:35:59.005Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p3-recovery\backend\api\job_worker.py has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

449. user (2026-07-05T12:36:06.558Z)

```
23:import time
0
```

450. user (2026-07-05T12:39:05.241Z)

```
unused time import dropped
15 passed in 3.13s
warning: in the working copy of 'backend/api/job_worker.py', LF will be replaced by CRLF the
next time Git touches it
 1 file changed, 7 insertions(+), 14 deletions(-)
1706 passed, 10 skipped, 18 warnings in 153.53s (0:02:33)
rcs: 0 0 0 0
   627e8f4..368261b  feat/restart-safe-recovery -> feat/restart-safe-recovery
```

451. user (2026-07-05T12:39:20.282Z)

```
https://github.com/Khelsutra/badminton-highlight-indexer/pull/494#issuecomment-4886068008
```

452. user (2026-07-05T12:39:20.303Z)

```
Updated task #4 status
```

453. user (2026-07-05T12:39:29.048Z)

```
Command running in background with ID: btgrrfazk. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\693b418f-8bf1-49e5-b443-b86a83235a63\tasks\btgrrfazk.output. You will be notified when
it completes. To check interim output, use Read on that file path.
```

454. assistant (2026-07-05T12:39:43.713Z)

Big step forward this round ? two more merges and the final engine row is built:

Merged: [#492](https://github.com/Khelsutra/badminton-highlight-indexer/pull/492) (P4, after its re-review LGTM on the coverage fix) and [#493](https://github.com/Khelsutra/badminton-highlight-indexer/pull/493) (P6 heartbeat, after its rebase re-verification LGTM). Master is at `4cd36fd`.

Built and opened: [#494](https://github.com/Khelsutra/badminton-highlight-indexer/pull/494) ? **P3, the last engine row before the rebuild.** It closes the exact 2026-07-05 loss: `recover_stale_jobs` now spares running md5-keyed process jobs, a startup resume pass re-arms the durable done-signal poll on `outputs/<md5>/report.json` (completing or failing honestly, never re-dispatching ? zero double GPU spend), `POST /api/query` re-hydrates an md5-shaped id from the bucket before reporting nothing (so `?video=<md5>` recovers even after a full DB wipe), and the worker's `report.json` now records `video_uri` so recovered videos keep their truthful source path. 15 new tests including the full carve-out matrix and every honest-failure path; 1706 passed; coverage above base; already refreshed across #493's merge with the heartbeat folded onto `on_tick` as promised.

Remaining before P9: #494 review ? merge; #490 (P2) awaits the reviewer re-activating its paused LGTM (my reply addressed the guard concern via the now-merged #492); kh#162 (P5b) awaits its first review ? then P8 (SPA re-vendor + the batched spec §7 updates), and the rebuild itself. The watcher is back on all three.

455. user (2026-07-05T12:42:36.386Z)

```
<task-notification>
<task-id>btgrrfazk</task-id>
<tool-use-id>toolu_01Th6GV9KoGwZMQqA11ZsMcK</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-
highlight-indexer\693b418f-8bf1-49e5-b443-b86a83235a63\tasks\btgrrfazk.output</output-file>
<status>completed</status>
<summary>Background command "Watch PRs 490, 493 slot replaced by 494, kh162" completed (exit
code 0)</summary>
</task-notification>
```

456. user (2026-07-05T12:42:45.138Z)

```
baseline: engine#490:OPEN/MERGEABLE/c4/r0 engine#493:MERGED/UNKNOWN/c6/r0
```

```
engine#494:OPEN/MERGEABLE/c1/r0 kh#162:OPEN/MERGEABLE/c1/r0
CHANGE DETECTED (poll 1):
  was: engine#490:OPEN/MERGEABLE/c4/r0 engine#493:MERGED/UNKNOWN/c6/r0
engine#494:OPEN/MERGEABLE/c1/r0 kh#162:OPEN/MERGEABLE/c1/r0
  now: engine#490:OPEN/MERGEABLE/c4/r0 engine#493:MERGED/UNKNOWN/c6/r0
engine#494:OPEN/MERGEABLE/c2/r0 kh#162:OPEN/MERGEABLE/c1/r0
```

457. user (2026-07-05T12:42:55.997Z)

I found a blocking restart-recovery correctness issue, so I?m pausing LGTM on this head.

The new carve-out in `recover_stale_jobs()` leaves every `running` process row with a `video_id` for the resume pass, and `_resume_one_remote_job()` then polls only `outputs/<md5>/report.json`. That breaks `force=true` reprocess jobs: `force` deliberately bypasses the completed-work cache, but if an older successful `outputs/<md5>/report.json` already exists, a control-plane restart can mark the forced job `done` from that stale cached result immediately, while the newly forced Cloud Run execution is still running or may later overwrite the same output prefix.

I reproduced this locally on the exact merge ref `d3327c7`:

```text
before recover running {"force": true}
recover (0, 0)
after recover running
after resume done cached= True segments= 2
```

Repro shape:

1. Create a process job with `video_id=<md5>`, `compute="cloud"`, and `force=True`.
2. Mark it `running` to simulate a CP restart while the forced Cloud Run run is in flight.
3. Leave a pre-existing successful `outputs/<md5>/report.json` in the bucket from an older completed run.
4. `recover_stale_jobs()` leaves the row running, and `_resume_one_remote_job()` immediately calls `cached_process_result(...)`, marking the forced job done with `cached=true`.

That violates the explicit force contract and loses the result of the in-flight forced run. Since the current schema does not persist a dispatch token/execution/run generation, I think the safe fix is to exclude forced process rows from the resume carve-out and fail them honestly on restart, or otherwise persist enough per-dispatch identity to prove the bucket result belongs to this run before completing it.

I stopped before the long full-suite gate because this behavior blocker is reproduced. Once fixed, I?ll rerun the full verified gate and coverage comparison.

458. user (2026-07-05T12:43:47.739Z)

carve-out excludes forced rows

459. user (2026-07-05T12:44:17.909Z)

```
resume force-guard added
force regression tests added
..................                                                      [100%]
18 passed in 3.57s
```

460. user (2026-07-05T12:47:11.378Z)

```
1709 passed, 10 skipped, 18 warnings in 157.60s (0:02:37)
warning: in the working copy of 'backend/api/job_worker.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'backend/infrastructure/database_jobs.py', LF will be replaced
by CRLF the next time Git touches it
warning: in the working copy of 'tests/test_restart_recovery.py', LF will be replaced by CRLF
```

```
the next time Git touches it
rcs: 0 0 0 0
```

461. user (2026-07-05T12:47:35.574Z)

```
warning: in the working copy of 'backend/api/job_worker.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'backend/infrastructure/database_jobs.py', LF will be replaced
by CRLF the next time Git touches it
warning: in the working copy of 'tests/test_restart_recovery.py', LF will be replaced by CRLF
the next time Git touches it
[feat/restart-safe-recovery c824dfc] fix(recovery): forced reprocess jobs never resume from the
cache (#494 review)
 3 files changed, 79 insertions(+), 2 deletions(-)
   368261b..c824dfc  feat/restart-safe-recovery -> feat/restart-safe-recovery
https://github.com/Khelsutra/badminton-highlight-indexer/pull/494#issuecomment-4886091114
```

462. user (2026-07-05T12:47:46.121Z)

```
Command running in background with ID: bu3em18u4. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\693b418f-8bf1-49e5-b443-b86a83235a63\tasks\bu3em18u4.output. You will be notified when
it completes. To check interim output, use Read on that file path.
```

463. assistant (2026-07-05T12:47:57.675Z)

```
The reviewer caught a real force-contract violation on #494: the resume pass could complete a
**forced** reprocess job from the *older* cached `outputs/<md5>/report.json` ? exactly the stale
result `force=true` exists to bypass. Fixed in `c824dfc` with their suggested schema-light
approach:

- **Carve-out exclusion:** `recover_stale_jobs` and `interrupted_remote_process_jobs` now share
the resumable predicate with `COALESCE(json_extract(params,'$.force'),0) != 1` ? an interrupted
forced run gets today's honest terminal failure instead of a stale "done".
- **Defense-in-depth:** `_resume_one_remote_job` refuses forced rows outright even if one
somehow reached it, failing with an explicit message rather than ever trusting the cache. (The
deeper per-dispatch-identity fix that would make forced runs resumable needs schema ?
deliberately left out, noted on the PR.)
- **Three regressions** including the reviewer's exact repro shape: forced running row + pre-
existing successful report ? `failed`, never `done cached=true`.

Gates: 1709 passed, ruff/mypy/diff-check clean, pushed, replied for the re-gate. Watcher is back
on #490 / #494 / kh#162.
```

464. user (2026-07-05T12:53:56.315Z)

```
<task-notification>
<task-id>bu3em18u4</task-id>
<tool-use-id>toolu_01KhetYXqot3HNiLmfyPVXAu</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-
highlight-indexer\693b418f-8bf1-49e5-b443-b86a83235a63\tasks\bu3em18u4.output</output-file>
<status>completed</status>
<summary>Background command "Watch PRs for review activity" completed (exit code 0)</summary>
</task-notification>
```

465. user (2026-07-05T12:54:05.616Z)

```
baseline: engine#490:OPEN/MERGEABLE/c4/r0 engine#493:MERGED/UNKNOWN/c6/r0
engine#494:OPEN/MERGEABLE/c3/r0 kh#162:OPEN/MERGEABLE/c1/r0
CHANGE DETECTED (poll 2):
  was: engine#490:OPEN/MERGEABLE/c4/r0 engine#493:MERGED/UNKNOWN/c6/r0
engine#494:OPEN/MERGEABLE/c3/r0 kh#162:OPEN/MERGEABLE/c1/r0
  now: engine#490:OPEN/MERGEABLE/c4/r0 engine#493:MERGED/UNKNOWN/c6/r0
engine#494:OPEN/MERGEABLE/c4/r0 kh#162:OPEN/MERGEABLE/c1/r0
```

466. user (2026-07-05T12:54:17.805Z)

I found another restart-recovery blocker on the fixed head, so I?m still pausing LGTM.

The fix correctly excludes `force=true`, but the resumable-row predicate is still broader than ?remote job?: `recover_stale_jobs()` leaves any `running` process row with a `video_id` alive, and `interrupted_remote_process_jobs()` returns it even when the original request explicitly set `compute="local"`. Per `_maybe_dispatch_remote()`, `compute="local"` forces the in-process segmenter and never dispatches Cloud Run, so after a control-plane restart there is no remote worker to resume.

I reproduced this locally on the exact merge ref `f22a40d` (`c824dfc` into `4cd36fd`):

```text
before recover running local
recover (0, 0)
after recover running local
interrupted rows [{'id': 'jlocal', 'video_id': '3354743ad44e9fcd363ce6d597fb8481', 'video_path':
'gs://bkt/videos/original-match.mp4', 'owner_id': None, 'compute': 'local'}]
after resume done cached= True segments= 2 compute= local
```

Repro shape:

1. Create a process job with `video_id=<md5>` and `compute="local"`.
2. Mark it `running` to simulate a restart while the in-process/local detector was running.
3. Leave an older successful `outputs/<md5>/report.json` in the bucket.
4. `recover_stale_jobs()` leaves the row running, `interrupted_remote_process_jobs()` returns it, and `_resume_one_remote_job()` completes the local job from the bucket cache as `done cached=true`.

That misclassifies a local run as an in-flight cloud run, can ingest stale/unrelated bucket output, and otherwise can leave a local job waiting up to the remote poll budget instead of failing honestly at startup. I think the resumable predicate needs to prove the job was actually cloud-dispatched, not merely md5-keyed. At minimum, exclude `compute='local'`; also please consider the default-local + per-request-cloud-capable case, where `compute IS NULL` means ?server default?, not necessarily cloud, even though `_cloud_dispatch_possible(config)` may be true because `compute='cloud'` overrides are available.

I stopped before the long full-suite gate because this behavior blocker is reproduced. Once fixed, I?ll rerun the full verified gate and coverage comparison.

467. user (2026-07-05T12:55:18.665Z)

SQL predicates exclude compute='local'
resume pass proves cloud dispatch

468. user (2026-07-05T12:55:49.054Z)

```text
4 round-2 regressions added
......................                                                  [100%]
22 passed in 3.87s
```

469. user (2026-07-05T12:58:46.751Z)

```text
1713 passed, 10 skipped, 18 warnings in 158.06s (0:02:38)
warning: in the working copy of 'backend/api/job_worker.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'backend/infrastructure/database_jobs.py', LF will be replaced
by CRLF the next time Git touches it
warning: in the working copy of 'tests/test_restart_recovery.py', LF will be replaced by CRLF
the next time Git touches it
rcs: 0 0 0 0
```

470. user (2026-07-05T12:59:10.776Z)

```
warning: in the working copy of 'backend/api/job_worker.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'backend/infrastructure/database_jobs.py', LF will be replaced
by CRLF the next time Git touches it
warning: in the working copy of 'tests/test_restart_recovery.py', LF will be replaced by CRLF
the next time Git touches it
[feat/restart-safe-recovery 9c98b47] fix(recovery): resume only PROVEN cloud-dispatched rows
(#494 review round 2)
 3 files changed, 122 insertions(+), 7 deletions(-)
   c824dfc..9c98b47  feat/restart-safe-recovery -> feat/restart-safe-recovery
https://github.com/Khelsutra/badminton-highlight-indexer/pull/494#issuecomment-4886123791
```

471. user (2026-07-05T12:59:19.217Z)

```
Command running in background with ID: bwio65zux. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\693b418f-8bf1-49e5-b443-b86a83235a63\tasks\bwio65zux.output. You will be notified when
it completes. To check interim output, use Read on that file path.
```

472. assistant (2026-07-05T12:59:30.910Z)

Round 2 on #494 addressed ? the reviewer was right again: md5-keyed didn't prove cloud-dispatched. Fixed in `9c98b47` with the proof at the right layers:

- **SQL carve-out** excludes explicit `compute='local'` rows (in-process runs have no remote worker ? they fail terminally as before).
- **The resume pass**, which holds the config, gates `NULL`-compute rows on the server default actually being `cloud_run` ? covering the reviewer's subtle case where a default-local server merely *has* cloud-override capability: those NULL rows now fail honestly at startup rather than waiting out the remote poll budget, while explicit `compute='cloud'` rows still resume.
- **Defense-in-depth** in `_resume_one_remote_job` refuses explicit-local rows, parallel to the force guard.

Four regressions added including their exact repro and both halves of the NULL split. Gates: 1713 passed, ruff/mypy/diff clean, pushed, replied for the re-gate. Watcher back on.

473. user (2026-07-05T13:05:29.942Z)

```
<task-notification>
<task-id>bwio65zux</task-id>
<tool-use-id>toolu_01TXtt5pDwy1q3YErD4cRnwG</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-
highlight-indexer\693b418f-8bf1-49e5-b443-b86a83235a63\tasks\bwio65zux.output</output-file>
<status>completed</status>
<summary>Background command "Watch PRs for review activity" completed (exit code 0)</summary>
</task-notification>
```

474. user (2026-07-05T13:05:41.020Z)

```
baseline: engine#490:OPEN/MERGEABLE/c4/r0 engine#493:MERGED/UNKNOWN/c6/r0
engine#494:OPEN/MERGEABLE/c5/r0 kh#162:OPEN/MERGEABLE/c1/r0
CHANGE DETECTED (poll 2):
  was: engine#490:OPEN/MERGEABLE/c4/r0 engine#493:MERGED/UNKNOWN/c6/r0
engine#494:OPEN/MERGEABLE/c5/r0 kh#162:OPEN/MERGEABLE/c1/r0
  now: engine#490:OPEN/MERGEABLE/c5/r0 engine#493:MERGED/UNKNOWN/c6/r0
engine#494:OPEN/MERGEABLE/c5/r0 kh#162:OPEN/MERGEABLE/c1/r0
Round 2 fixed in `9c98b47` ? right again: md5-keyed ? cloud-dispatched.

The predicate now PROVES the dispatch, at the layer that has the knowledge to do so: the **SQL
carve-out** (recover + interrupted SELECT) excludes explicit `compute='local'` rows ? they get
today's terminal failure; the **resume pass**, which has the config, gates `NULL`-compute rows
on `deployment.compute_target == 'cloud_run'` (your default-local-with-override-capability case:
```

a NULL row there ran in-process and now **fails honestly at startup** instead of waiting out the remote poll budget, while an explicit `compute='cloud'` row still resumes); and **defense-in-depth** in `_resume_one_remote_job` refuses explicit-local rows outright, parallel to the force guard.

Four regressions added, including your exact repro (local running row + older successful `outputs/<md5>/report.json` ? `failed`, `interrupted_remote_process_jobs()` empty, stale result never ingested) and both halves of the NULL split (fails on a default-local override-capable server; resumes on a cloud-default server). Gates on the new head: suite **1713 passed / 10 skipped**, ruff / `mypy backend training` / `git diff --check` clean. Ready for your full gate re-run.

475. assistant (2026-07-05T13:05:49.055Z)

New activity is on #490 this time ? reading it.

476. user (2026-07-05T13:05:51.251Z)

Restored LGTM for CP-rebuild P2 on the current master base.

The earlier paused-LGTM blocker is resolved by the now-merged #492 path: `/api/process` runs the remote-source scratch guard after the submit-time cache probe/decision, so a cached `gs://` result no longer needs CP scratch. I re-ran the old shape on the current-base merge: the 10 GiB object / ~1 MiB free cache hit returns instant `done`, while a true cache miss still 507s pre-claim.

I reviewed the current merge result locally as `2440037` = current `origin/master` `4cd36fd` + PR head `dda8a68` (GitHub's `pull/490/merge` ref was still stale against `97feff4`, so I simulated the real current merge in a fresh worktree). No blocking issues found in the upload streaming diff: staged objects stay under `input_root/uploads/<upload_id>/`, local profile behavior stays untouched, overflow/abort delete the partial object, and the streamed md5 matches the canonical `compute_video_id` contract.

Verified locally:

```text
ruff check .                              PASS
mypy backend training                     PASS
python -m tools.check_md_links            PASS
python -m tools.leak_scan                 PASS
git diff --check HEAD^1..HEAD             PASS
focused upload + prior 507/cache tests    13 passed
full coverage suite on PR merge           1701 passed / 10 skipped, coverage 85.66%
full coverage suite on exact base 4cd36fd 1691 passed / 10 skipped, coverage 85.66%
```

GitHub checks are green and the PR is `CLEAN`. Since this is authored by the signed-in owner account, treating this as my verified LGTM/comment rather than a formal approval.

477. assistant (2026-07-05T13:06:11.081Z)

#490's LGTM is restored with a full verified gate on the current-master merge ? merging now.

478. user (2026-07-05T13:06:20.346Z)

MERGEABLE SUCCESS

From https://github.com/Khelsutra/badminton-highlight-indexer
   4cd36fd..6acddce master   -> origin/master
6acddce feat(uploads): stream browser-upload staging to the bucket ? never assemble in tmpfs (CP-rebuild P2) (#490)
error: failed to delete '.git/worktrees/wt-p2-upload-stream': Permission denied
Deleted branch feat/upload-gcs-streaming (was dda8a68).

479. user (2026-07-05T13:06:41.086Z)

```
MERGEABLE SUCCESS
```

480. user (2026-07-05T13:06:42.011Z)

```
Command running in background with ID: bv4wk6ss5. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\693b418f-8bf1-49e5-b443-b86a83235a63\tasks\bv4wk6ss5.output. You will be notified when
it completes. To check interim output, use Read on that file path.
```

481. assistant (2026-07-05T13:06:54.027Z)

#490 (P2 streamed staging) is merged ? its LGTM was restored after the reviewer re-verified the guard fix on the current-master merge with non-regressing coverage. Master is at `6acddce`.

The rebuild's engine work is now **5 of 6 rows merged**: P1 pins+8Gi, P2 streamed staging, P4 md5+cache, P5a poll knob+caps, P6 heartbeat. Outstanding:

- **[#494](https://github.com/Khelsutra/badminton-highlight-indexer/pull/494) (P3)** ? round-2 fix pushed, still MERGEABLE and green after #490's merge; awaiting the reviewer's full re-gate.
- **[kh#162](https://github.com/avidullu/khelsutra/pull/162) (P5b)** ? still awaiting its first review. Worth noting: the reviewer-monitor watches the engine repo only, so this khelsutra PR likely needs your eyes directly ? it's the small one (two poll ceilings 10?60 min plus a test).

Once those two land: P8 (re-vendor the SPA snapshot + batch the spec §7 row updates) and then P9 ? the rebuild, redeploy, and the browser CUJ with the restart-replay DoD. Watcher is running.

482. user (2026-07-05T13:12:52.607Z)

```
<task-notification>
<task-id>bv4wk6ss5</task-id>
<tool-use-id>toolu_01Bd2TeNns9vbcrDsmA1rovj</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-
highlight-indexer\693b418f-8bf1-49e5-b443-b86a83235a63\tasks\bv4wk6ss5.output</output-file>
<status>completed</status>
<summary>Background command "Watch remaining PRs" completed (exit code 0)</summary>
</task-notification>
```

483. user (2026-07-05T13:13:01.416Z)

```
baseline: engine#490:MERGED/UNKNOWN/c5/r0 engine#493:MERGED/UNKNOWN/c6/r0
engine#494:OPEN/MERGEABLE/c5/r0 kh#162:OPEN/MERGEABLE/c1/r0
CHANGE DETECTED (poll 2):
  was: engine#490:MERGED/UNKNOWN/c5/r0 engine#493:MERGED/UNKNOWN/c6/r0
engine#494:OPEN/MERGEABLE/c5/r0 kh#162:OPEN/MERGEABLE/c1/r0
  now: engine#490:MERGED/UNKNOWN/c5/r0 engine#493:MERGED/UNKNOWN/c6/r0
engine#494:OPEN/MERGEABLE/c6/r0 kh#162:OPEN/MERGEABLE/c1/r0
```

484. user (2026-07-05T13:13:12.189Z)

Thanks for the round-2 fix; the lowercase `compute="local"` repro is covered now. I found one remaining blocker in the same recovery predicate: the submit path normalizes `req.compute` with `.strip().lower()` when deciding whether to run local/cloud, but the recovery predicate and resume guard compare the stored `compute` value case-sensitively.

That means an API client that submits `compute="LOCAL"` gets the local/in-process path at execution time, but restart recovery later treats the row as remote-resumable on a cloud-default server and can complete it from an older bucket report.

I reproduced this locally on the current-base merge `0595a9d` (`6acddce` + `9c98b47`):

```text
before recover running LOCAL
```

```
recover (0, 0)
after recover running
interrupted rows [{'id': 'jupper', 'video_id': '3354743ad44e9fcd363ce6d597fb8481', 'video_path':
'gs://bkt/videos/original-match.mp4', 'owner_id': None, 'compute': 'LOCAL'}]
after resume done cached= True segments= 2 compute= LOCAL
```

The root is the mismatch between execution-time normalization and recovery-time checks:

- `_maybe_dispatch_remote()` uses `(req.compute or "").strip().lower()`, so `"LOCAL"` is explicit local.
- `interrupted_remote_process_jobs()` excludes only `compute != 'local'` case-sensitively.
- `resume_interrupted_remote_jobs()` uses `requested = str(...).strip()` and compares to `"cloud"` case-sensitively.
- `_resume_one_remote_job()` refuses only exactly `"local"`.

I think the fix is to normalize the persisted/requested compute value consistently in the recovery path too, e.g. use `LOWER(TRIM(compute))` in the SQL predicate and `.strip().lower()` in the resume/defense checks, with a regression for uppercase or padded `LOCAL`. I stopped before the long full-suite gate because this behavior blocker reproduced.

485. user (2026-07-05T13:13:50.212Z)

```
SQL normalized
resume checks normalized
case regressions added
24 passed in 3.90s
```

486. user (2026-07-05T13:17:32.246Z)

```
1715 passed, 10 skipped, 18 warnings in 188.14s (0:03:08)
warning: in the working copy of 'backend/api/job_worker.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'backend/infrastructure/database_jobs.py', LF will be replaced
by CRLF the next time Git touches it
warning: in the working copy of 'tests/test_restart_recovery.py', LF will be replaced by CRLF
the next time Git touches it
rcs: 0 0 0 0
warning: in the working copy of 'backend/api/job_worker.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'backend/infrastructure/database_jobs.py', LF will be replaced
by CRLF the next time Git touches it
warning: in the working copy of 'tests/test_restart_recovery.py', LF will be replaced by CRLF
the next time Git touches it
 3 files changed, 65 insertions(+), 4 deletions(-)
   9c98b47..d915444  feat/restart-safe-recovery -> feat/restart-safe-recovery
https://github.com/Khelsutra/badminton-highlight-indexer/pull/494#issuecomment-4886175563
```

487. user (2026-07-05T13:17:41.044Z)

```
Command running in background with ID: bfcmmizog. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\693b418f-8bf1-49e5-b443-b86a83235a63\tasks\bfcmmizog.output. You will be notified when
it completes. To check interim output, use Read on that file path.
```

488. assistant (2026-07-05T13:17:51.377Z)

Round 3 on #494 fixed in `d915444`: the recovery predicates now apply the exact normalization the execution path uses ? `LOWER(TRIM(compute))` in both SQL predicates and `.strip().lower()` in the resume pass and defense guard ? so `"LOCAL"` or `"  Local  "` means at recovery precisely what it meant at run. Regressions cover the reviewer's uppercase repro, the padded-`CLOUD` row still resuming, and the cased-local defense refusal. 1715 passed, all gates clean, replied for the re-gate. Watcher is back on #494 and kh#162 ? those two are the last gates before P8 and the rebuild.

489. user (2026-07-05T13:20:48.745Z)

```
<task-notification>
<task-id>bfcmmizog</task-id>
<tool-use-id>toolu_01ExyUtbbCcXUiNjnFZeD8Tu</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-
highlight-indexer\693b418f-8bf1-49e5-b443-b86a83235a63\tasks\bfcmmizog.output</output-file>
<status>completed</status>
<summary>Background command "Watch remaining PRs" completed (exit code 0)</summary>
</task-notification>
```

490. user (2026-07-05T13:21:00.189Z)

```
baseline: engine#490:MERGED/UNKNOWN/c5/r0 engine#493:MERGED/UNKNOWN/c6/r0
engine#494:OPEN/MERGEABLE/c7/r0 kh#162:OPEN/MERGEABLE/c1/r0
CHANGE DETECTED (poll 1):
  was: engine#490:MERGED/UNKNOWN/c5/r0 engine#493:MERGED/UNKNOWN/c6/r0
engine#494:OPEN/MERGEABLE/c7/r0 kh#162:OPEN/MERGEABLE/c1/r0
  now: engine#490:MERGED/UNKNOWN/c5/r0 engine#493:MERGED/UNKNOWN/c6/r0
engine#494:OPEN/MERGEABLE/c7/r0 kh#162:OPEN/MERGEABLE/c2/r0
Round 3 fixed in `d915444` ? the normalization now matches execution end-to-end: SQL predicates
use `LOWER(TRIM(compute)) != 'local'` (recover carve-out + interrupted SELECT), the resume pass
and the defense-in-depth guard apply `.strip().lower()` before comparing, so `\"LOCAL\"`/`\"
Local  \"` mean at recovery exactly what they meant at run. Regressions cover your repro
(uppercase local running row + older successful report ? failed at recover, interrupted list
empty) plus the padded/cased-`CLOUD` row still resuming on a default-local override-capable
server, and the cased-local defense-in-depth refusal. Gates on the new head: suite **1715 passed
/ 10 skipped**, ruff / `mypy backend training` / `git diff --check` clean. Ready for your full
gate re-run.
```

491. assistant (2026-07-05T13:21:07.855Z)

```
New comment on kh#162 ? reading it.
```

492. user (2026-07-05T13:21:10.154Z)

```
Verified LGTM for the P5b SPA ceilings change.

Reviewed the current merge result locally: `0d3a88e` = base `0e45e8f` + head `0a3bb52`. The
change is scoped to raising the ingest and compile poll ceilings from 10 min to 60 min, with the
ingest timeout regression updated. I do not see a correctness or UX-copy blocker: this preserves
the existing "server may still be working" behavior while letting the browser outlast honest
cloud detection/queued compile durations.

Local verification:

- GitHub checks are green (`site`, `web`, `layout`, Workers build).
- `git diff --check HEAD^1..HEAD` clean.
- `web`: `npm ci`; `npm run typecheck`; `npm run test:coverage` -> 301 passed, coverage above
floors.
- Exact-base coverage comparison with the same `json-summary` reporter: PR and base both report
statements/lines 94.79% (2421/2554), branches 88.23% (765/867), functions 89.40% (135/151), so
no measured regression.
- `web`: `npm run e2e:ci` -> build + mock contract smoke + 5 Playwright CUJs passed.
- `site`: `npm ci`; `npm test` -> 95 passed; shell script `bash -n` parse checks passed; `npm
run test:layout` -> 3 passed; `npm run test:render` -> 1 passed; `npm run test:a11y` -> 1
passed.

I used local Node v26.1.0 / npm 11.13.0 while CI uses Node 24; CI is green on the PR, so this
looks good to merge when Avi is ready.
```

493. user (2026-07-05T13:21:40.923Z)

```
MERGEABLE SUCCESS
```

```
From https://github.com/avidullu/khelsutra
   0e45e8f..90d1d90  master     -> origin/master
90d1d90 Raise SPA poll ceilings 10 -> 60 min for honest cloud job lengths (CP-rebuild P5b)
(#162)
```

494. user (2026-07-05T13:21:47.610Z)

```
Updated task #7 status
```

495. user (2026-07-05T13:21:52.244Z)

```
error: failed to delete '.git/worktrees/wt-p5b-ceilings': Permission denied
Deleted branch feat/poll-ceilings-60min (was 0a3bb52).
HEAD is now at 90d1d90 Raise SPA poll ceilings 10 -> 60 min for honest cloud job lengths (CP-
rebuild P5b) (#162)
HEAD is now at 6acddce feat(uploads): stream browser-upload staging to the bucket ? never
assemble in tmpfs (CP-rebuild P2) (#490)
```

496. user (2026-07-05T13:22:19.856Z)

```
[bundle_ui] files:
assets/index-5O3c-FCB.js
assets/index-dvIzm83F.css
assets/noto-sans-devanagari-devanagari-wght-normal-DWI_L5Cu.woff2
assets/noto-sans-devanagari-latin-ext-wght-normal-CLkupaxV.woff2
assets/noto-sans-devanagari-latin-wght-normal-Bxfr4UJH.woff2
assets/noto-sans-kannada-kannada-wght-normal-DuwB-P8d.woff2
assets/noto-sans-kannada-latin-ext-wght-normal-MdBBF41x.woff2
assets/noto-sans-kannada-latin-wght-normal-DeLZCgFT.woff2
index.html
THIRD_PARTY_NOTICES.md
ui_version.json
```

497. user (2026-07-05T13:22:28.044Z)

```
{
  "spa_repo": "https://github.com/avidullu/khelsutra.git",
  "spa_commit": "90d1d90cebee057b98ec4adf165b036257f14978",
  "engine_api_version": 1,
  "content_hash": "a6fd1182afef0f7d600800b793dc26f0549d521e56e1890430639dada1b448e1",
  "built_at": "2026-07-05T13:22:19Z"
}
 M backend/ui/THIRD_PARTY_NOTICES.md
 D backend/ui/assets/index-Cqr7zJeJ.js
 D backend/ui/assets/index-DYSqmz09.css
 M backend/ui/index.html
 M backend/ui/ui_version.json
?? backend/ui/assets/index-5O3c-FCB.js
?? backend/ui/assets/index-dvIzm83F.css
```

498. user (2026-07-05T13:22:42.144Z)

```
239:| Proxy detection regresses quality (#483) | worse rallies for testers | D10 golden gate +
Launch Report before default-ON; most-conservative-passing setting |
266:| P0 | This spec; owner confirmation opens P9 | engine | ? | **Owner** | ? |
[#487](https://github.com/Khelsutra/badminton-highlight-indexer/pull/487) (confirmed 2026-07-05)
|
267:| P1 | `gen_service_yaml.py` v2: rev-00011 pins in `build_config` + `--memory` 8Gi default |
engine | P0 | No | ? | (this PR) |
268:| P2 | GCS-streamed upload staging (`BlobWriter` under `<input_root>/uploads/`, streamed
md5, backend-aware 507 guard, abort cleanup) | engine | P0 | No | ? | ? |
269:| P3 | Restart-safe remote jobs: md5-keyed startup recovery (extend `recover_stale_jobs`) +
read-path re-hydrate via `_ingest_remote_segments` (#471 #485) | engine | P4 | No | ? | ? |
270:| P4 | Submit-time MD5 on the job row + completed-work cache-reuse with `force` bypass
```

(#484) | engine | P2 (upload hash) | No | ? | ? |
271:| P5a | `deployment.remote_poll_max_wait_sec` knob (default 3900) + emit
`max_upload_mb`/`max_part_mb` in `/api/capabilities` (#480 item 5 ? revives the SPA pre-flight)
| engine | P0 | No | ? | ? |
272:| P5b | SPA: both poll ceilings 10 ? 60 min + capabilities-driven upload pre-flight |
khelsutra | P5a | No | ? | ? |
273:| P6 | #482 progress heartbeat ? `job.progress` + per-job runlog (SPA stall-detector rides
the next SPA PR) | engine (+khelsutra) | P0 | No | ? | ? |
274:| P7 | #483 downsample-for-detection behind config; default set by the golden gate | engine
| P0 | **Golden gate + Launch Report (D10)** | ? | ? |
275:| P8 | Re-vendor the SPA snapshot (`scripts/bundle_ui.sh`) at khelsutra master ? P5b |
engine | P5b | No | ? | ? |
276:| P9 | **REBUILD + REDEPLOY + browser-CUJ verify + runlog** (§4.8, §9) | ops | P1?P5b, P8
(P6/P7 if ready) | **Owner: spec confirm + deploy/spend authorization** | ? | ? |

499. user (2026-07-05T13:22:50.881Z)

```
263
264     | ID | Deliverable | Repo | Depends on | Gated? | Status | PR |
265     |----|-------------|------|-----------|--------|--------|----|
266     | P0 | This spec; owner confirmation opens P9 | engine | ? | **Owner** | ? |
[#487](https://github.com/Khelsutra/badminton-highlight-indexer/pull/487) (confirmed 2026-07-05)
|
267     | P1 | `gen_service_yaml.py` v2: rev-00011 pins in `build_config` + `--memory` 8Gi
default | engine | P0 | No | ? | (this PR) |
268     | P2 | GCS-streamed upload staging (`BlobWriter` under `<input_root>/uploads/`, streamed
md5, backend-aware 507 guard, abort cleanup) | engine | P0 | No | ? | ? |
269     | P3 | Restart-safe remote jobs: md5-keyed startup recovery (extend
`recover_stale_jobs`) + read-path re-hydrate via `_ingest_remote_segments` (#471 #485) | engine
| P4 | No | ? | ? |
270     | P4 | Submit-time MD5 on the job row + completed-work cache-reuse with `force` bypass
(#484) | engine | P2 (upload hash) | No | ? | ? |
271     | P5a | `deployment.remote_poll_max_wait_sec` knob (default 3900) + emit
`max_upload_mb`/`max_part_mb` in `/api/capabilities` (#480 item 5 ? revives the SPA pre-flight)
| engine | P0 | No | ? | ? |
272     | P5b | SPA: both poll ceilings 10 ? 60 min + capabilities-driven upload pre-flight |
khelsutra | P5a | No | ? | ? |
273     | P6 | #482 progress heartbeat ? `job.progress` + per-job runlog (SPA stall-detector
rides the next SPA PR) | engine (+khelsutra) | P0 | No | ? | ? |
274     | P7 | #483 downsample-for-detection behind config; default set by the golden gate |
engine | P0 | **Golden gate + Launch Report (D10)** | ? | ? |
275     | P8 | Re-vendor the SPA snapshot (`scripts/bundle_ui.sh`) at khelsutra master ? P5b |
engine | P5b | No | ? | ? |
276     | P9 | **REBUILD + REDEPLOY + browser-CUJ verify + runlog** (§4.8, §9) | ops | P1?P5b,
P8 (P6/P7 if ready) | **Owner: spec confirm + deploy/spend authorization** | ? | ? |
277
278     **Bundling note:** P6/P7 ride the same image if ready before P9 executes; otherwise they
trigger a
```

500. user (2026-07-05T13:23:11.922Z)

```
The file C:\Users\avidu\Projects\khelsutra-
guru\wt-p8-vendor\docs\CONTROL_PLANE_DURABLE_REBUILD.md has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

501. user (2026-07-05T13:23:32.570Z)

```
changelog updated
[check-md-links] clean ? no broken internal links in live docs.
warning: in the working copy of 'docs/CONTROL_PLANE_DURABLE_REBUILD.md', LF will be replaced by
CRLF the next time Git touches it
DOCS_OK
```

502. user (2026-07-05T13:28:02.452Z)

```
1701 passed, 10 skipped, 18 warnings in 200.03s (0:03:20)
[leak-scan] clean ? no MD5 / personal-path / private-name leaks in the code surface.
warning: in the working copy of 'docs/CONTROL_PLANE_DURABLE_REBUILD.md', LF will be replaced by
CRLF the next time Git touches it
rcs: suite=0 ruff=0 mypy=0 diff=0 leak=0
```

503. user (2026-07-05T13:28:20.119Z)

```
warning: in the working copy of 'docs/CONTROL_PLANE_DURABLE_REBUILD.md', LF will be replaced by
CRLF the next time Git touches it
[feat/revendor-spa-p8 bc031e0] chore(ui): re-vendor the SPA snapshot at khelsutra 90d1d90 (CP-
rebuild P8) + batch the Sec.7 tracker updates
 8 files changed, 83 insertions(+), 77 deletions(-)
 create mode 100644 backend/ui/assets/index-5O3c-FCB.js
 delete mode 100644 backend/ui/assets/index-Cqr7zJeJ.js
 delete mode 100644 backend/ui/assets/index-DYSqmz09.css
 create mode 100644 backend/ui/assets/index-dvIzm83F.css
bc031e0 chore(ui): re-vendor the SPA snapshot at khelsutra 90d1d90 (CP-rebuild P8) + batch the
Sec.7 tracker updates
 * [new branch]      feat/revendor-spa-p8 -> feat/revendor-spa-p8
```

504. user (2026-07-05T13:28:42.386Z)

```
https://github.com/Khelsutra/badminton-highlight-indexer/pull/495
```

505. user (2026-07-05T13:28:55.813Z)

```
The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\693b418f-8bf1-49e5-b443-b86a83235a63\scratchpad\pr_watch.sh has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

506. user (2026-07-05T13:28:56.992Z)

```
Command running in background with ID: b3c1hpzgf. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\693b418f-8bf1-49e5-b443-b86a83235a63\tasks\b3c1hpzgf.output. You will be notified when
it completes. To check interim output, use Read on that file path.
```

507. user (2026-07-05T13:29:08.352Z)

```
11:> Last refreshed: 2026-07-05 night (**PRE-MERGE P-ROW REVIEW-GATE ACTIVE**; standing reviewer
monitor `khelsutra-cp-rebuild-pr-reviewer` wakes every ~10 min and applies the verified-LGTM
gate). **? REVIEW-GATE (owner, 2026-07-05): nobody merges without the owner's explicit say-so**
? [[working-agreement-merging]] overridden; open PRs, keep working, wait. **MERGED:** spec #487
(`cc5e83a`, doc `IN PROGRESS`) · P1 #489 (`e80bca8`, [post-merge
verification](https://github.com/Khelsutra/badminton-highlight-
indexer/pull/489#issuecomment-4885664770)) · sibling A10/A11 #488 (`f1b60d0`) · **P5a #491
(`97feff4`)** · **P4 #492 (`851f21d`, [verified LGTM](https://github.com/Khelsutra/badminton-
highlight-indexer/pull/492#issuecomment-4885966056))** · **P6 CP slice #493 (`4cd36fd`,
[verified LGTM](https://github.com/Khelsutra/badminton-highlight-
indexer/pull/493#issuecomment-4886024640))** · **P2 #490 (`6acddce`, restored LGTM
[comment](https://github.com/Khelsutra/badminton-highlight-
indexer/pull/490#issuecomment-4886139864))**. **OPEN:** P3
[#494](https://github.com/Khelsutra/badminton-highlight-indexer/pull/494) restart-safe recovery
(**BEHAVIOR-BLOCKED** on compute normalization: `compute="LOCAL"` runs local at execution time
but recovery treats it as remote-resumable and can complete from stale
`outputs/<md5>/report.json`; [comment](https://github.com/Khelsutra/badminton-highlight-
indexer/pull/494#issuecomment-4886155497); prior `force=true` and lowercase `compute="local"`
blockers were fixed) · khelsutra P5b [#162](https://github.com/avidullu/khelsutra/pull/162)
60-min ceilings (**VERIFIED-LGTM**
[comment](https://github.com/avidullu/khelsutra/pull/162#issuecomment-4886183418)). Earlier:
#481 merged (`0a7b3ff`); khelsutra #158/#155 merged. Prior: 2026-07-04 ? **A1 DONE** (#477,
`122f174`).
```

508. assistant (2026-07-05T13:29:21.126Z)

The handoff header is a step behind (round-3 fix, kh#162 merge, #495). Bringing it to current truth, including the refined merge-on-LGTM protocol.

509. user (2026-07-05T13:29:22.761Z)

11 > Last refreshed: 2026-07-05 night (**PRE-MERGE P-ROW REVIEW-GATE ACTIVE**; standing reviewer monitor `khelsutra-cp-rebuild-pr-reviewer` wakes every ~10 min and applies the verified-LGTM gate). **? REVIEW-GATE (owner, 2026-07-05): nobody merges without the owner's explicit say-so** ? [[working-agreement-merging]] overridden; open PRs, keep working, wait. **MERGED:** spec #487 (`cc5e83a`, doc `IN PROGRESS`) · P1 #489 (`e80bca8`, [post-merge verification](https://github.com/Khelsutra/badminton-highlight-indexer/pull/489#issuecomment-4885664770)) · sibling A10/A11 #488 (`f1b60d0`) · **P5a #491 (`97feff4`)** · **P4 #492 (`851f21d`, [verified LGTM](https://github.com/Khelsutra/badminton-highlight-indexer/pull/492#issuecomment-4885966056))** · **P6 CP slice #493 (`4cd36fd`, [verified LGTM](https://github.com/Khelsutra/badminton-highlight-indexer/pull/493#issuecomment-4886024640))** · **P2 #490 (`6acddce`, restored LGTM [comment](https://github.com/Khelsutra/badminton-highlight-indexer/pull/490#issuecomment-4886139864))**. **OPEN:** P3 [#494](https://github.com/Khelsutra/badminton-highlight-indexer/pull/494) restart-safe recovery (**BEHAVIOR-BLOCKED** on compute normalization: `compute="LOCAL"` runs local at execution time but recovery treats it as remote-resumable and can complete from stale `outputs/<md5>/report.json`; [comment](https://github.com/Khelsutra/badminton-highlight-indexer/pull/494#issuecomment-4886155497); prior `force=true` and lowercase `compute="local"` blockers were fixed) · khelsutra P5b [#162](https://github.com/avidullu/khelsutra/pull/162) 60-min ceilings (**VERIFIED-LGTM** [comment](https://github.com/avidullu/khelsutra/pull/162#issuecomment-4886183418)). Earlier: #481 merged (`0a7b3ff`); khelsutra #158/#155 merged. Prior: 2026-07-04 ? **A1 DONE** (#477, `122f174`).
12 > See [[current-state]] for the blow-by-blow.
13
14 ## ? YOU ARE HERE ? REVIEW-GATE MODE: #494 P3 BLOCKED; kh#162 VERIFIED-LGTM

510. user (2026-07-05T13:29:47.819Z)

The file c:\Users\avidu\.claude\projects\c--Users-avidu-Projects-khelsutra-guru\memory\session-handoff.md has been updated successfully. (file state is current in your context ? no need to Read it back)

511. user (2026-07-05T13:30:09.229Z)

<tool_use_error>File has been modified since read, either by the user or by a linter. Read it again before attempting to write it.</tool_use_error>

512. user (2026-07-05T13:30:21.905Z)

10 # Current state - khelsutra-guru
11
12 **Updated:** 2026-07-05
13
14 ## Live Checkpoint
15
16 **2026-07-05 evening heartbeat (khelsutra #162 P5b -> VERIFIED LGTM; #494 still BLOCKED).** Monitor found
17 the sibling SPA P5b PR [avidullu/khelsutra#162](https://github.com/avidullu/khelsutra/pull/162) still open,

513. user (2026-07-05T13:30:41.232Z)

The file c:\Users\avidu\.claude\projects\c--Users-avidu-Projects-khelsutra-guru\memory\current-state.md has been updated successfully. (file state is current in your context ? no need to Read it back)

514. assistant (2026-07-05T13:30:58.307Z)

The board is nearly clear. Tonight's sweep under your merge-on-LGTM protocol:

Merged (7 PRs): P5a #491, P4 #492 (two review blockers fixed + a coverage top-up), P6 #493 (after a rebase), P2 #490 (LGTM restored once the guard fix landed), and **kh#162** (P5b, verified LGTM with full e2e) ? plus the earlier spec #487 and P1 #489. Engine master `6acddce`, khelsutra master `90d1d90`.

Open ? the last two before P9:
- **[#494](https://github.com/Khelsutra/badminton-highlight-indexer/pull/494) (P3)** ? three genuinely good review rounds, all addressed: forced rows never resume from the cache, only *proven* cloud-dispatched rows resume (including the NULL-means-server-default subtlety), and compute case-normalization now matches execution exactly. 24 recovery tests; awaiting the reviewer's full re-gate.
- **[#495](https://github.com/Khelsutra/badminton-highlight-indexer/pull/495) (P8)** ? the SPA snapshot re-vendored at khelsutra `90d1d90`, so the image will finally carry #153/#155/#158/#162 (the medium-picker trap you hit live dies with the deploy), plus the batched spec §7 tracker updates. All gates including leak-scan clean.

Once those two land, **P9 is fully unblocked**: image rebuild, worker re-pin, `gen_service_yaml --edge-auth on` (pins + 8 GiB now baked in code), `services replace`, then the §9 browser CUJ including the restart-replay of your exact 2026-07-05 failure, and the deploy runlog. Handoff and current-state are checkpointed; the watcher is live on both PRs.

515. user (2026-07-05T13:35:05.463Z)

<task-notification>
<task-id>b3c1hpzgf</task-id>
<tool-use-id>toolu_01BN8CiqmR6tdEbK4HEJh2za</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\693b418f-8bf1-49e5-b443-b86a83235a63\tasks\b3c1hpzgf.output</output-file>
<status>completed</status>
<summary>Background command "Watch 494 and 495" completed (exit code 0)</summary>
</task-notification>

516. user (2026-07-05T13:35:23.642Z)

CHANGE DETECTED (poll 2):
  was: engine#494:OPEN/MERGEABLE/c7/r0 engine#495:OPEN/MERGEABLE/c0/r0 kh#162:MERGED/UNKNOWN/c2/r0
  now: engine#494:OPEN/MERGEABLE/c8/r0 engine#495:OPEN/MERGEABLE/c0/r0 kh#162:MERGED/UNKNOWN/c2/r0
Verified LGTM for the current P3 restart-recovery head.

Reviewed the current-base merge locally: `14acc53` = current `origin/master` `6acddce` + PR head `d915444` (GitHub's PR base metadata is still pre-#490, so I explicitly merged onto current master). The prior blocker is fixed: recovery now normalizes `compute` the same way execution does (`LOWER(TRIM(compute))` in the SQL predicates, `.strip().lower()` in the resume path), so `"LOCAL"` / padded local rows fail honestly on restart instead of resuming as remote; cased/padded `cloud` rows still resume when they were actually cloud-dispatched. The earlier `force=true`, explicit-local, and NULL-default split blockers are also covered by the regression suite.

Local verification:

- GitHub checks are green and PR is clean/mergeable.
- `git diff --check HEAD^1..HEAD` clean.

517. user (2026-07-05T13:35:41.388Z)

MERGEABLE SUCCESS

From https://github.com/Khelsutra/badminton-highlight-indexer

```
   6acddce..f6864f1  master     -> origin/master
f6864f1 feat(recovery): restart-safe remote jobs + read-path re-hydrate (CP-rebuild P3, #471
#485) (#494)
Deleted branch feat/restart-safe-recovery (was d915444).
```

518. user (2026-07-05T13:36:04.460Z)

```
 create mode 100644 tests/test_restart_recovery.py
P3 row flipped
[check-md-links] clean ? no broken internal links in live docs.
warning: in the working copy of 'docs/CONTROL_PLANE_DURABLE_REBUILD.md', LF will be replaced by
CRLF the next time Git touches it
 1 file changed, 1 insertion(+), 1 deletion(-)
   bc031e0..6644736  feat/revendor-spa-p8 -> feat/revendor-spa-p8
MERGEABLE
```

519. user (2026-07-05T13:36:14.707Z)

```
Command running in background with ID: bookini5i. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\693b418f-8bf1-49e5-b443-b86a83235a63\tasks\bookini5i.output. You will be notified when
it completes. To check interim output, use Read on that file path.
```

520. assistant (2026-07-05T13:36:31.799Z)

```
One correction ? that launch nested the watcher so its output would be lost. Relaunching it
cleanly.
```

521. user (2026-07-05T13:36:33.443Z)

```
Command running in background with ID: b30ygm2an. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\693b418f-8bf1-49e5-b443-b86a83235a63\tasks\b30ygm2an.output. You will be notified when
it completes. To check interim output, use Read on that file path.
```

522. assistant (2026-07-05T13:36:46.740Z)

The last engine row is in: **#494 (P3) merged** on its verified LGTM ? master is `f6864f1`, and every code row of the rebuild (P1?P6) is now on master.

[#495](https://github.com/Khelsutra/badminton-highlight-indexer/pull/495) (P8, the re-vendored SPA + tracker batch) is the **only PR left before P9**. Its branch is refreshed on the new master with the P3 tracker row flipped to done, still MERGEABLE, and the watcher is on it. The moment it gets an LGTM I merge it ? and then P9 (image rebuild ? redeploy with the baked pins + 8 GiB ? the §9 browser CUJ including the restart replay ? deploy runlog) is fully unblocked under the authorization you gave with the spec confirm.

523. user (2026-07-05T13:38:56.947Z)

with the P9 would the optimization of downsampling to 1080p and then running segmentation on it also be checked in?

524. assistant (2026-07-05T13:39:41.445Z)

Short answer: **no ? as confirmed in the spec (§8 Q3), P9 ships without it.** The rebuild image carries P1?P6 + P8 (streamed staging, durable state, MD5 reuse, heartbeat, timeouts, pins, new SPA), but detection still runs on the original file. #483 downsample-for-detection is row **P7, deliberately golden-gated** (decision D10): it changes the *input* to detection, so before it can be default-ON it needs a golden-set run (n=15) at the proxy settings with F1/tolF1 compared against baseline, plus a Launch Report ? honest measured results, not the one dev-box experiment.

Two clarifications on the shape of it:

- What was validated on 2026-07-05 was **1080p60 ? 1080p30** (fps halving, ~10× realtime on the dev 2070) ? that's the conservative candidate. The spec's golden gate would also test 720p30 and pick the **most aggressive setting that holds quality**, rather than assuming.
- The cost of it *not* being in P9 is bounded: detection stays ~22 min for a 6-min 1080p60 clip, but the #493 heartbeat now shows live progress, the 60-min ceilings outlast it, and #484 makes any re-run of the same video instant. And adding #483 later is cheap operationally ? one more PR + a ~10-minute image rebuild + `services update`.

My recommendation stands as confirmed: **don't hold P9 for it** ? P9 is what unblocks DoD #1 and kills the OOM/lost-results fragility testers actually hit; #483 follows in the next image once the gate passes.

That said, if you want the wall-clock win sooner, I can start P7 now while #495 sits in review: write the worker-side PR (GPU ffmpeg proxy ? detect on proxy ? map timestamps back ? reel from the original, behind a default-OFF config), and prep the golden-gate harness runs. The gate itself needs real GPU time ? roughly a full corpus re-detection at each candidate setting (hours on the WSL 2070, or faster on an L4 with your spend OK) ? so I'd hold the actual runs for your go. Want me to start the P7 code in parallel, or keep the sequence as planned (P9 first, P7 next)?

525. user (2026-07-05T13:42:38.963Z)

<task-notification>
<task-id>b30ygm2an</task-id>
<tool-use-id>toolu_01DUhqwACz9RwGSJ5CYSCmXC</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\693b418f-8bf1-49e5-b443-b86a83235a63\tasks\b30ygm2an.output</output-file>
<status>completed</status>
<summary>Background command "Watch PR 495 for review activity" completed (exit code 0)</summary>
</task-notification>

526. user (2026-07-05T13:42:49.651Z)

CHANGE DETECTED (poll 2):
  was: engine#495:OPEN/MERGEABLE/c0/r0 kh#162:MERGED/UNKNOWN/c2/r0
  now: engine#495:OPEN/MERGEABLE/c1/r0 kh#162:MERGED/UNKNOWN/c2/r0
Blocking provenance mismatch in the vendored UI manifest.

I verified the good part first: a fresh build of `avidullu/khelsutra` at `90d1d90cebee057b98ec4adf165b036257f14978` produces the same payload filenames as this PR (`assets/index-5O3c-FCB.js`, `assets/index-dvIzm83F.css`, fonts, and `index.html`), and a byte/hash comparison of `web/dist` versus `backend/ui` passes for every payload file.

But the committed `backend/ui/ui_version.json` has:

```json
"content_hash": "a6fd1182afef0f7d600800b793dc26f0549d521e56e1890430639dada1b448e1"
```

Running the current bundle script hash recipe over the committed `backend/ui` payload gives a different value:

```bash
cd backend/ui
(find . -type f ! -name ui_version.json ! -name THIRD_PARTY_NOTICES.md | LC_ALL=C sort | xargs sha256sum) | sha256sum
364da0696aebdc8eff57ebf51e306a33f450330f3c2386791f75324a80167abd  -
```

I also reran `scripts/bundle_ui.sh` in a detached throwaway worktree with `BUILD=0` against the freshly built `90d1d90` dist; it rewrote the manifest to the same `364da069...` content hash. That means the PR's actual SPA bytes look correct, but the generated provenance field is stale/incorrect, which matters for the P8/P9 manifest-pin verification.

Please rerun the bundling step or otherwise update `ui_version.json.content_hash` so it matches the committed payload. It would also be worth adding a small `tests/test_ui_bundle.py` assertion that recomputes this hash so CI catches this class of drift next time.

527. user (2026-07-05T13:43:42.191Z)

```
78:# content hash is over the SPA build output only (relative paths ? reproducible across
checkouts),
81:CONTENT_HASH="$( (cd "$UI_DEST" && find . -type f ! -name ui_version.json ! -name
THIRD_PARTY_NOTICES.md | LC_ALL=C sort | xargs sha256sum) | sha256sum | cut -d' ' -f1)"
89:  "content_hash": "$CONTENT_HASH",
  "content_hash": "$CONTENT_HASH",
  "built_at": "$BUILT_AT"
}
JSON

echo "[bundle_ui] vendored SPA -> $UI_DEST (spa_commit=$SPA_COMMIT,
engine_api_version=$ENGINE_API_VERSION)"
echo "[bundle_ui] files:"; find "$UI_DEST" -type f | sed "s|$UI_DEST/||" | sort
```

528. user (2026-07-05T13:44:29.650Z)

```
i/lf    w/lf    attr/-text              backend/ui/THIRD_PARTY_NOTICES.md
i/lf    w/lf    attr/-text              backend/ui/assets/index-5O3c-FCB.js
i/lf    w/lf    attr/-text              backend/ui/assets/index-dvIzm83F.css
i/-text w/-text attr/-text              backend/ui/assets/noto-sans-devanagari-devanagari-wght-
normal-DWI_L5Cu.woff2
i/-text w/-text attr/-text              backend/ui/assets/noto-sans-devanagari-latin-ext-wght-
normal-CLkupaxV.woff2
i/-text w/-text attr/-text              backend/ui/assets/noto-sans-devanagari-latin-wght-
normal-Bxfr4UJH.woff2
i/-text w/-text attr/-text              backend/ui/assets/noto-sans-kannada-kannada-wght-normal-
DuwB-P8d.woff2
i/-text w/-text attr/-text              backend/ui/assets/noto-sans-kannada-latin-ext-wght-
normal-MdBBF41x.woff2
i/-text w/-text attr/-text              backend/ui/assets/noto-sans-kannada-latin-wght-normal-
DeLZCgFT.woff2
i/lf    w/lf    attr/-text              backend/ui/index.html
i/lf    w/lf    attr/-text              backend/ui/ui_version.json
python recompute: 364da0696aebdc8eff57ebf51e306a33f450330f3c2386791f75324a80167abd
```

529. assistant (2026-07-05T13:44:43.235Z)

Python recompute matches the reviewer exactly (`364da069?`), and `backend/ui/**` is `-text` (LF everywhere) ? the mirror is stable cross-OS. Applying the three-layer fix.

530. user (2026-07-05T13:44:44.101Z)

```
1       {
2         "spa_repo": "https://github.com/avidullu/khelsutra.git",
3         "spa_commit": "90d1d90cebee057b98ec4adf165b036257f14978",
4         "engine_api_version": 1,
5         "content_hash": "a6fd1182afef0f7d600800b793dc26f0549d521e56e1890430639dada1b448e1",
6         "built_at": "2026-07-05T13:22:19Z"
7       }
8
```

531. user (2026-07-05T13:44:59.940Z)

The file C:\Users\avidu\Projects\khelsutra-guru\wt-p8-vendor\backend\ui\ui_version.json has been updated successfully. (file state is current in your context ? no need to Read it back)

532. user (2026-07-05T13:45:00.335Z)

```
76
77      # Provenance + the SPA<->engine contract version (the engine's API_VERSION at bundle
time). The
78      # content hash is over the SPA build output only (relative paths ? reproducible across
checkouts),
79      # excluding the two generated metadata files.
80      ENGINE_API_VERSION="$(grep -E '^API_VERSION\b[^=]*=' "$ENGINE_ROOT/backend/main.py" |
grep -oE '[0-9]+' | head -1)"
81      CONTENT_HASH="$( (cd "$UI_DEST" && find . -type f ! -name ui_version.json ! -name
THIRD_PARTY_NOTICES.md | LC_ALL=C sort | xargs sha256sum) | sha256sum | cut -d' ' -f1)"
82      BUILT_AT="$(date -u +%Y-%m-%dT%H:%M:%SZ 2>/dev/null || echo unknown)"
83
84      cat > "$UI_DEST/ui_version.json" <<JSON
85      {
```

533. user (2026-07-05T13:45:15.161Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p8-vendor\scripts\bundle_ui.sh has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

534. assistant (2026-07-05T13:45:33.445Z)

```
"""Vendored-SPA provenance guard (#495 review ? CP-rebuild P8).

``backend/ui/ui_version.json.content_hash`` must equal a recompute over the committed payload,
so a hand-edited/drifted snapshot (or a platform-skewed bundling run) can't ship silently: the
P9 deploy verification pins the served UI against exactly this manifest.

The recompute mirrors scripts/bundle_ui.sh's recipe deterministically in Python ? per-file
sha256 lines in the POSIX text form ("<hash>  ./<relpath>\\n", two spaces, LC_ALL=C path sort),
hashed again. The shell recipe normalizes MSYS sha256sum's "*" binary-mode marker to this same
form (the #495 finding: the marker silently changed the outer hash on Windows). Stable across
OSes because backend/ui/** is gitattributes ``-text`` (bytes identical in every checkout).
"""

import hashlib
import json
from pathlib import Path

_UI = Path(__file__).resolve().parents[1] / "backend" / "ui"
_EXCLUDED = {"ui_version.json", "THIRD_PARTY_NOTICES.md"}


def _recompute_content_hash() -> str:
    rels = sorted(
        "./" + p.relative_to(_UI).as_posix()
        for p in _UI.rglob("*")
        if p.is_file() and p.name not in _EXCLUDED
    )
    lines = []
    for rel in rels:
        digest = hashlib.sha256((_UI / rel[2:]).read_bytes()).hexdigest()
        lines.append(f"{digest}  {rel}\n")
    return hashlib.sha256("".join(lines).encode("ascii")).hexdigest()


def test_ui_manifest_content_hash_matches_the_committed_payload():
    manifest = json.loads((_UI / "ui_version.json").read_text(encoding="utf-8"))
    assert manifest["content_hash"] == _recompute_content_hash(), (
        "backend/ui/ui_version.json.content_hash does not match the committed SPA payload ? "
        "re-run scripts/bundle_ui.sh (it recomputes the manifest) instead of hand-editing "
        "either side."
    )
```

```
def test_ui_manifest_provenance_shape():
    manifest = json.loads((_UI / "ui_version.json").read_text(encoding="utf-8"))
    assert len(manifest["spa_commit"]) == 40 and int(manifest["spa_commit"], 16) >= 0
    assert manifest["spa_repo"].startswith("https://")
    assert isinstance(manifest["engine_api_version"], int)
    # The payload the manifest vouches for actually exists.
    assert (_UI / "index.html").is_file()
    assert any((_UI / "assets").glob("index-*.js"))
```

535. user (2026-07-05T13:45:33.449Z)

```
<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>
```

536. user (2026-07-05T13:45:43.321Z)

```
-rw-r--r-- 2 avidu 197609 2206 Jul  5 18:51 tests/test_ui_bundle.py
"""The bundled SPA is served single-origin + the SPA<->engine contract (PACKAGING_PLAN.md
P1b)."""

import json
import os

import pytest

pytest.importorskip("httpx")
from fastapi.testclient import TestClient

import backend.main as m

client = TestClient(m.app)
UI_DIR = os.path.join(os.path.dirname(os.path.abspath(m.__file__)), "ui")


def test_bundled_ui_snapshot_is_committed():
    assert os.path.isfile(os.path.join(UI_DIR, "index.html")), (
        "backend/ui/index.html missing ? the SPA snapshot must be committed (run
scripts/bundle_ui.sh)."
    )
```

537. assistant (2026-07-05T13:45:51.617Z)

```
The file already exists (P1b's single-origin tests) ? extending it rather than replacing.
```

538. user (2026-07-05T13:45:52.611Z)

```
1       """The bundled SPA is served single-origin + the SPA<->engine contract
(PACKAGING_PLAN.md P1b)."""
2
3       import json
4       import os
5
6       import pytest
7
8       pytest.importorskip("httpx")
9       from fastapi.testclient import TestClient
10
11      import backend.main as m
12
13      client = TestClient(m.app)
14      UI_DIR = os.path.join(os.path.dirname(os.path.abspath(m.__file__)), "ui")
15
16
17      def test_bundled_ui_snapshot_is_committed():
18          assert os.path.isfile(os.path.join(UI_DIR, "index.html")), (
```

```
19              "backend/ui/index.html missing ? the SPA snapshot must be committed (run
scripts/bundle_ui.sh)."
20          )
21          assert os.path.isfile(os.path.join(UI_DIR, "ui_version.json"))
22
23
24      def test_serves_spa_at_root():
25          r = client.get("/")
26          assert r.status_code == 200
27          # The bundled index.html references its single-origin /assets bundle (Vite
base='/').
28          assert "/assets/" in r.text
29
30
31      def test_serves_spa_assets_byte_for_byte():
32          assets = os.path.join(UI_DIR, "assets")
33          js = next(f for f in os.listdir(assets) if f.endswith(".js"))
34          r = client.get(f"/assets/{js}")
35          assert r.status_code == 200
36          with open(os.path.join(assets, js), "rb") as f:
37              assert (
38                  r.content == f.read()
39              )  # served verbatim (mimetype varies by OS, so compare bytes)
40
41
42      def test_api_version_reports_bundled_ui():
43          body = client.get("/api/version").json()
44          assert body["api_version"] == m.API_VERSION
45          assert body["ui"] is not None
46          assert body["ui"]["spa_commit"]  # provenance recorded
47
48
49      def test_spa_engine_contract_version_matches():
50          """Skew guard: the committed snapshot records the engine API_VERSION it was built
against. If
51          someone bumps API_VERSION without re-running scripts/bundle_ui.sh, this FAILS ?
forcing a
52          re-bundle so the shipped SPA can't silently target a stale API contract."""
53          with open(os.path.join(UI_DIR, "ui_version.json"), encoding="utf-8") as f:
54              uiv = json.load(f)
55          assert uiv["engine_api_version"] == m.API_VERSION, (
56              f"bundled SPA built against engine api_version {uiv['engine_api_version']} but
engine is now "
57              f"{m.API_VERSION} ? re-run scripts/bundle_ui.sh and commit backend/ui/."
58          )
59
```

539. user (2026-07-05T13:46:12.423Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-p8-vendor\tests\test_ui_bundle.py has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

540. user (2026-07-05T13:49:14.965Z)

```
7 passed in 2.45s
script parses
1727 passed, 10 skipped, 18 warnings in 160.59s (0:02:40)
rcs: 0 0 0 0
```

541. user (2026-07-05T13:49:38.969Z)

```
[feat/revendor-spa-p8 9af45b2] fix(ui): platform-stable content_hash + drift guard (#495 review)
 3 files changed, 51 insertions(+), 2 deletions(-)
   6644736..9af45b2  feat/revendor-spa-p8 -> feat/revendor-spa-p8
```

https://github.com/Khelsutra/badminton-highlight-indexer/pull/495#issuecomment-4886271892

542. user (2026-07-05T13:49:47.946Z)

Command running in background with ID: bpcz52rln. Output is being written to: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\693b418f-8bf1-49e5-b443-b86a83235a63\tasks\bpcz52rln.output. You will be notified when it completes. To check interim output, use Read on that file path.

543. assistant (2026-07-05T13:49:58.296Z)

The #495 provenance finding is fixed ? and it was a genuinely interesting root cause: **MSYS `sha256sum` emits a `*` binary-mode marker in its output lines while Linux emits two spaces**, so the manifest's outer hash was platform-dependent even though the payload bytes were verified identical to a fresh khelsutra build. Fixed at three layers in `9af45b2`: the manifest now carries the canonical POSIX-form value (my independent Python recompute matches the reviewer's `364da069?` exactly), `bundle_ui.sh` normalizes the marker so future Windows bundling runs agree with CI, and `test_ui_bundle.py` gains the reviewer-suggested drift guard ? a deterministic cross-OS Python mirror of the recipe that CI runs on every change. 1727 passed with the new guard; replied for the re-gate.

#495 is now the only thing between us and P9. Watcher is on it.

544. user (2026-07-05T14:03:04.414Z)

as for the P7, feel free to keep it as a "warm backlog"

545. assistant (2026-07-05T14:03:22.221Z)

Noted ? P7 stays a warm backlog item: nothing blocks on it, no code now, and it's the natural first pick-up after the rebuild lands. Recording that in the two places the next session will look.

546. user (2026-07-05T14:03:30.147Z)

https://github.com/Khelsutra/badminton-highlight-indexer/issues/483#issuecomment-4886310288

547. user (2026-07-05T14:03:43.856Z)

anchor not found - handoff may have been rewritten; skipping

548. user (2026-07-05T14:03:56.009Z)

24:#471/#480/#482/#483/#484/#485 + the SPA re-vendor into ONE image rebuild + redeploy: 8 GiB CP
.
78:replaying the restart-after-done-before-ingest failure shape) + runlog. P7/#483 downsample stays
98:2. **5-minute browser CUJ (DoD #1) ? ATTEMPTED 2026-07-05 by the owner -> it's a BLOCKER on the current build.** Full detail [[alpha-cuj-testing-2026-07-05]]. WASB detection works (29 rallies) but the deploy infra fails the round-trip: a 2.68 GB upload OOM'd the 4 GiB / RAM-`/tmp` CP -> restart -> ephemeral SQLite wiped (#471) -> the detected rallies (in the bucket) never ingested -> UI empty. Config rev 00011 already fixed the segmenter (#479) + upload part-size (#480); still broken in the live build: #471 (durable state), the 10-min SPA / 30-min CP timeouts, no MD5 dedup (#484), and the medium-picker trap until merged master `0e45e8f` is deployed. **The fix is the CP IMAGE REBUILD (grab #2 below).** Reel was recovered manually + delivered; new issues #482/#483/#484/#485 filed; engine #481 (dedup + `GET /api/jobs` queue) ? atomic concurrent-submit fix (`d27f2d7`, 12-thread regression), all CI/local gates green, **MERGED on owner LGTM ? master `0a7b3ff`** (2026-07-05 09:15Z; `backend/api/jobs.py` now on master, branch deleted). The CUJ DID succeed in the UI on a gs:// re-run (29 rallies pickable) once the CP survived ? the big-upload OOM is the only killer.
103:**Next session grabs (priority order ? role-dependent):** (1) **Watch/re-review [#495](https://github.com/Khelsutra/badminton-highlight-indexer/pull/495)** after the author fixes the manifest hash blocker: verify `ui_version.json.content_hash` matches the committed `backend/ui` payload, then run the full verified-LGTM gate (ruff, mypy, md-links, leak-scan,

diff-check, full suite with coverage vs exact base, and CI/review-thread check). (2) **Still-unbuilt §7 row:** **P9** after #495 lands ? rebuild+redeploy+CUJ per §4.8/§9 + runlog (holistic: OOM proof, restart replay, cache/dedup proof, pins, edge-auth, ui_version, runlog). (3) **Savers** (never hold P9): **#483 downsample** (P7, golden-gate + Launch Report) · **#482 worker-side follow-up** (frame-% + per-job runlog ? needs a GPU verification pass; CP-side slice #493 is merged). (4) **SPA processing-queue UI** on `feat/processing-queue-and-dedup` @ `23f3d15` ? AFTER the rebuild. (5) owner 5-min CUJ + GO/NO-GO on A13. (6) least-privilege SA (review §5.5). (7) retention (#301/#447) + processor-list / acceptable-use (#332). (8) kn/hi native review (#156). (9) khelsutra UX backlog (#329) · rally finding-B (P2b).
105:**? NEXT-SESSION ICEBREAKER (paste this):** _Ramp up: `git pull --ff-only` both repos, `gh auth status`, read this handoff + `docs/CONTROL_PLANE_DURABLE_REBUILD.md` (the merged spec, IN PROGRESS) + [[alpha-cuj-testing-2026-07-05]]. Active state: P1 #489, P5a #491, P4 #492, P6 #493, P2 #490, P3 #494, and kh#162/P5b are merged. OPEN = #495/P8 (re-vendored SPA @ kh `90d1d90` + §7 tracker updates) but it is blocked: payload bytes match the fresh SPA build, while `ui_version.json.content_hash` is wrong (`a6fd...` committed vs `364da069...` recomputed); see comment `4886248799`. Re-review after the author updates the head, then run the full verified-LGTM gate. After #495 lands ? P9 holistic per §4.8/§9: rebuild/redeploy, browser CUJ, big-upload OOM proof, restart/durable-state replay, cache/dedup proof, manifest pins, edge-auth posture, SPA ui_version proof, runlog/docs/memory. #483 (P7, golden-gated) + #482 worker-side never hold P9. Honest measured results only; Launch Report on every default flip._

549. user (2026-07-05T14:04:12.172Z)

handoff updated with warm-backlog status

550. assistant (2026-07-05T14:04:24.783Z)

Done ? P7's warm-backlog status is recorded durably in both places: a pick-up-shape comment on [#483](https://github.com/Khelsutra/badminton-highlight-indexer/issues/483#issuecomment-4886310288) (proxy candidates, golden-gate procedure, and the owner-go needed for the GPU spend) and the session handoff's grabs + icebreaker, so no future session starts it prematurely or relitigates the sequencing.

Current state stands: [#495](https://github.com/Khelsutra/badminton-highlight-indexer/pull/495) (P8) is the last PR before P9, with the platform-stable hash fix pushed and awaiting the reviewer's re-gate ? the watcher will wake me when it lands, and on its LGTM I merge and P9 becomes pure execution: rebuild, redeploy with the baked pins + 8 GiB, the §9 browser CUJ with the restart replay, and the deploy runlog.

551. user (2026-07-05T14:08:00.080Z)

<task-notification>
<task-id>bpcz52rln</task-id>
<tool-use-id>toolu_014yNhRvcvBW8DiMfg3UuQiV</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\693b418f-8bf1-49e5-b443-b86a83235a63\tasks\bpcz52rln.output</output-file>
<status>completed</status>
<summary>Background command "Watch PR 495" completed (exit code 0)</summary>
</task-notification>

552. user (2026-07-05T14:08:10.453Z)

CHANGE DETECTED (poll 6):
  was: engine#495:OPEN/MERGEABLE/c2/r0 kh#162:MERGED/UNKNOWN/c2/r0
  now: engine#495:OPEN/MERGEABLE/c3/r0 kh#162:MERGED/UNKNOWN/c2/r0
Verified LGTM for #495 at `9af45b23b314dae77db90f6786c28c108f8edaeb`.

What I verified locally on a fresh current-base merge (`origin/master` `f6864f1` + PR head):

- Prior blocker is fixed: `backend/ui/ui_version.json.content_hash` now matches the bundle recipe over the committed payload (`364da0696aebdc8eff57ebf51e306a33f450330f3c2386791f75324a80167abd`).
- Fresh build of `avidullu/khelsutra` at `90d1d90cebee057b98ec4adf165b036257f14978` byte-matches the committed `backend/ui` payload files (`index.html`, JS/CSS, fonts), excluding generated

metadata.
- `tests/test_ui_bundle.py` now includes the requested deterministic drift guard, and `python -m pytest tests/test_ui_bundle.py -q` passed (`7 passed`).
- `scripts/bundle_ui.sh` parses with `bash -n`; the MSYS `sha256sum` marker normalization is scoped to the hash-line format and does not affect payload bytes.
- §7 tracker/docs are consistent with current state: P3 #494 merged, P8 is this PR, P9 remains the deploy/CUJ/runlog step after P8 lands.

Full verified gate:

- GitHub checks: all green on `9af45b2`.
- `git diff --check HEAD^1..HEAD`: clean.
- `ruff check .`: clean.
- `mypy backend training`: clean (notes only).
- `python -m tools.check_md_links`: clean.
- `python -m tools.leak_scan`: clean.
- PR merge full suite: `1727 passed, 10 skipped`, coverage `85.71%` (floor 70%).
- Exact base `f6864f1` full suite: `1725 passed, 10 skipped`, coverage `85.71%`; no coverage regression.

No blocking findings remain from my review. Since this PR is authored by the signed-in GitHub user, treating this comment as the verified LGTM/signoff rather than a formal approval.

553. assistant (2026-07-05T14:08:50.169Z)

#495's verified LGTM is in ? merging the final PR. That makes P9 pure execution, and it's already authorized via the spec confirm.

554. user (2026-07-05T14:09:03.383Z)

MERGEABLE SUCCESS

From https://github.com/Khelsutra/badminton-highlight-indexer
   f6864f1..257f469 master   -> origin/master
257f469 chore(ui): re-vendor the SPA snapshot at khelsutra 90d1d90 (CP-rebuild P8) + batch the §7 tracker updates (#495)
Deleted branch feat/revendor-spa-p8 (was 9af45b2).
cleaned

555. user (2026-07-05T14:09:39.704Z)

Updated task #7 status

556. user (2026-07-05T14:09:39.940Z)

Updated task #9 status

557. user (2026-07-05T14:09:42.252Z)

HEAD is now at 257f469 chore(ui): re-vendor the SPA snapshot at khelsutra 90d1d90 (CP-rebuild P8) + batch the §7 tracker updates (#495)

558. user (2026-07-05T14:10:01.477Z)

avi.dullu@gmail.com
gen-lang-client-0306026826
rally-control-plane-00011-dx6   4Gi
asia-southeast1-docker.pkg.dev/gen-lang-client-0306026826/rally/rally-worker:latest

559. user (2026-07-05T14:10:13.710Z)

Command running in background with ID: bq5u3urv7. Output is being written to: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\693b418f-8bf1-49e5-b443-b86a83235a63\tasks\bq5u3urv7.output. You will be notified when

```
it completes. To check interim output, use Read on that file path.
```

560. user (2026-07-05T14:10:30.323Z)

```
{"deployment":{"profile":"cloud-serving","compute_target":"cloud_run"},"indexing":{"skip_ai_hand
off":true,"default_segmenter":"wasb_hybrid"},"storage":{"backend":"gcs","output_root":"gs://khel
sutra-rally-serving","input_backend":"gcs","input_root":"gs://khelsutra-rally-serving/videos"},"
security":{"edge_auth_enabled":true,"cf_team_domain":"https://khelsutra.cloudflareaccess.com","c
f_access_aud":"c980da67ed888fff935fcd913ae1eb4da63f9d65805d6e1c5ca76f92f517082b","allowed_video_
root":"/tmp/apphome/output","max_part_mb":24}}
apiVersion: serving.knative.dev/v1
kind: Service
metadata:
  name: rally-control-plane
  labels:
    cloud.googleapis.com/location: asia-southeast1
spec:
  template:
    metadata:
      annotations:
        autoscaling.knative.dev/minScale: "1"
        autoscaling.knative.dev/maxScale: "1"
        run.googleapis.com/cpu-throttling: "false"
    spec:
      timeoutSeconds: 300
      containers:
        - image: asia-southeast1-docker.pkg.dev/gen-lang-client-0306026826/rally/rally-
worker:latest
          command: ["/bin/bash"]
          args: ["-c", "mkdir -p /tmp/apphome && printf %s 'eyJkZXBsb3ltZW50Ijp7InByb2ZpbGUiOiJj
bG91ZC1zZXJ2aW5nIiwiY29tcHV0ZV90YXJnZXQiOiJjbG91ZF9ydW4ifSwiaW5kZXhpbmciOnsic2tpcF9haV9oYW5kb2Zm
Ijp0cnVlLCJkZWZhdWx0X3NlZ21lbnRlciI6Indhc2JfaHlicmlkIn0sInN0b3JhZ2UiOnsiYmFja2VuZCI6ImdjcyIsIm91
dHB1dF9yb290IjoiZ3M6Ly9raGVsc3V0cmEtcmFsbHktc2VydmluZyIsImlucHV0X2JhY2tlbmQiOiJnY3MiLCJpbnB1dF9y
b290IjoiZ3M6Ly9raGVsc3V0cmEtcmFsbHktc2VydmluZy92aWRlb3MifSwic2VjdXJpdHkiOnsiZWRnZV9hdXRoX2VuYWJs
ZWQiOnRydWUsImNmX3RlYW1fZG9tYWluIjoiaHR0cHM6Ly9raGVsc3V0cmEuY2xvdWRmbGFyZWFjY2Vzcy5jb20iLCJjZl9h
Y2Nlc3NfYXVkIjoiYzk4MGRhNjdlZDg4OGZmZjkzNWZjZDkxM2FlMWViNGRhNjNmOWQ2NTgwNWQ2ZTFjNWNhNzZmOTJmNTE3
MDgyYiIsImFsbG93ZWRfdmlkZW9fcm9vdCI6Ii90bXAvYXBwaG9tZS9vdXRwdXQiLCJtYXhfcGFydF9tYiI6MjR9fQ==' |
base64 -d > /tmp/apphome/config.json && export RALLY_APP_HOME=/tmp/apphome && exec python3 -m
backend.main --server --host 0.0.0.0 --port 8080"]
          ports:
            - containerPort: 8080
          resources:
            limits:
              cpu: "2"
              memory: 8Gi
          env:
            - name: DEPLOYMENT_PROFILE
              value: cloud-serving
            - name: CLOUD_RUN_REGION
              value: asia-southeast1
            - name: GOOGLE_CLOUD_PROJECT
              value: gen-lang-client-0306026826
            - name: RALLY_ALLOWED_HOSTS
              value: "*"
```

561. user (2026-07-05T14:10:46.313Z)

```
null
```

562. assistant (2026-07-05T14:11:03.390Z)

```
{
  "rule": [
    {
```

```
      "action": {"type": "Delete"},
      "condition": {"age": 2, "matchesPrefix": ["videos/uploads/"]}
    }
  ]
}
```

563. user (2026-07-05T14:11:03.439Z)

```
File created successfully at: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-
badminton-highlight-indexer\693b418f-8bf1-49e5-b443-b86a83235a63\scratchpad\uploads-
lifecycle.json (file state is current in your context ? no need to Read it back)
```

564. user (2026-07-05T14:11:22.849Z)

```
Updating gs://khelsutra-rally-serving/...
..................
{
  "lifecycle_config": {
    "rule": [
      {
        "action": {
          "type": "Delete"
        },
        "condition": {
          "age": 2,
          "matchesPrefix": [
            "videos/uploads/"
          ]
        }
      }
    ]
  }
}
```

565. assistant (2026-07-05T14:11:33.340Z)

```
Lifecycle rule live (§4.8 step 4 done). Checking build progress.
```

566. user (2026-07-05T14:11:35.451Z)

```
---
```

567. assistant (2026-07-05T14:11:51.950Z)

```
All pre-deploy steps are staged while the image builds:

- **#495 merged** ? master `257f469`; every P-row is in.
- **Manifests generated and verified from the clean clone** ? the decoded baked config carries
both rev-00011 pins (`wasb_hybrid`, `max_part_mb=24`) plus `memory: 8Gi`, maxScale 1 ? decision
D5 working as designed.
- **Bucket lifecycle rule applied**: `videos/uploads/**` staging objects auto-delete after 2
days (there was no pre-existing policy to merge).
- **Cloud Build running** (~10 min CUDA image from master `257f469`).

Once the build lands: re-pin the worker Job, `services replace` with the auth-ON manifest, then
a short verification battery ? posture checks, the #484 cache-hit proof against the existing
`outputs/` (no GPU spend), the 2026-07-05 restart-replay via a fresh instance + `?video=` re-
hydrate, and a synthetic streamed upload to prove P2 live ? then the deploy runlog. The official
?2.5 GB browser CUJ (§9's first checkbox) stays yours to run whenever you like on the new build.
```

568. user (2026-07-05T14:11:53.622Z)

```
pls update this tracker as you do P9 and complete https://github.com/Khelsutra/badminton-
highlight-indexer/pull/490 accordingly
```

569. assistant (2026-07-05T14:12:28.990Z)

Understood ? I'll keep the §7 tracker updated live as P9 progresses (via the runlog PR at the end, plus the ALPHA_LAUNCH_READINESS DoD row), and close out **#490's loop**: once the live battery proves streamed staging on the deployed build, I'll post the runtime evidence on #490 and close its issue #480 (and likewise #471/#484/#485 with their live proofs). Checking the build.

570. user (2026-07-05T14:12:32.246Z)

(Bash completed with no output)

571. user (2026-07-05T14:12:44.892Z)

47877373-e457-4df0-a155-1ccc443c818b   WORKING 2026-07-05T14:10:22+00:00

572. assistant (2026-07-05T14:12:57.612Z)

Build `47877373` is WORKING. While it runs, mapping exactly which alpha-tracker rows P9 updates.

573. user (2026-07-05T14:12:59.426Z)

11:## 0. TL;DR
17:## 1. Problem & goal
27:## 2. Decisions locked
39:## 3. Verified snapshot, 2026-07-03
63:## 4. Launch strategy
65:### 4.1 Alpha lane: make the product usable
69:### 4.2 Quality lane: make the 15-video corpus gate-ready
73:### 4.3 Flywheel lane: make tester feedback useful
77:## 5. Risks and guardrails
88:## 6. Honest limits
94:## 7. Deliverables & progress tracker
101:| A1 | Stand up alpha serving endpoint: Cloud Run/GPU or approved host, health check, edge auth ON, upload -> process -> jobs -> compile -> download verified | `khelsutra` + `badminton-highlight-indexer` | A0 | Owner spend / CF config | DONE | #462/#464/#465/#467/#468/#469/#470/#474/#476; e2e runlog `docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_a1-e2e_2026-07-04.md` |
115:## 8. Concrete next PR sequence
224:- **Wider/public exit criteria:** before any public signup, broad invite wave, commercial detector claim, or model/weight sharing, close #172 and #486 or keep copy strictly non-model, and also clear the operational gates recorded in the A13 review: reliable browser CUJ, least-privilege runtime service account, and retention/deletion cleanup.
226:## 9. Open questions
234:## 10. Definition of done
247:## 11. References
275:### Changelog

574. user (2026-07-05T14:13:08.275Z)

7. Deliverables & progress tracker

Legend: TODO = not started, IN PROGRESS = actively being worked, DONE = shipped/verified, GATED = blocked on owner/legal/platform decision. One PR per row unless the row explicitly names a cross-repo batch.

ID	Deliverable	Repo / owner	Depends on	Gated?	Status	PR
A0	Create this alpha readiness tracker and index it in docs	`badminton-highlight-indexer`	-	No	DONE	#451
A1	Stand up alpha serving endpoint: Cloud Run/GPU or approved host, health check, edge auth ON, upload -> process -> jobs -> compile -> download verified	`khelsutra` + `badminton-highlight-indexer`	A0	Owner spend / CF config	DONE	#462/#464/#465/#467/#468/#469/#470/#474/#476; e2e runlog

`docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_a1-e2e_2026-07-04.md` |
A2	Make the n=15 golden manifest portable/reproducible; add a path/corpus smoke check that fails loudly when labels or trajectories are missing	`badminton-highlight-indexer`	A0	No	DONE	#452
A3	Promote the 15-video source-video/MD5 records into the collector/vault path; reconcile collector's six-video source manifest with the 15-video eval corpus	`sports-data-collector` + vault	A2	Owner upload/storage scope	DONE	collector #62
A4	Refresh the n=15 nightly regression baseline intentionally and record the exact command/output; keep stale-baseline warnings until reviewed	`badminton-highlight-indexer`	A2	Reviewer sign-off	DONE	#453
A5	Recompute the heuristic n=15 floor and replace the stale `heuristic_lovo_n6` floor for future Gen-0 comparisons	`badminton-highlight-indexer`	A2	No	DONE	#454
A6	Fix `backend.eval.ablation` CLI circular import, then run the R2/tolF1 ablation on the n=15 corpus	`badminton-highlight-indexer`	A2	No	DONE	#455
A7	Regenerate or upconvert full-fusion feature JSONs for the newer 9 videos so fusion/person/audio analyses run over all 15	`badminton-highlight-indexer`	A2	GPU/data availability	DONE	#458
A8	Re-run `fusion_compare`, group ablation, and served contrast after A7; explicitly promote/reject person fusion, Gate A served-default, and any other R-series default flips	`badminton-highlight-indexer`	A6, A7	Reviewer sign-off	DONE	#458
A9	Run Gen-0/TemporalMaxer shadow training against the refreshed n=15 floor; produce a non-serving artifact bundle and gate verdict	`badminton-highlight-indexer`	A5, A8	Owner approval for M0.4 scope	DONE	#458
A10	Decide and record the alpha ship-gate target: metric, corpus slice, minimum threshold, significance rule, and what claims are allowed	Owner + `badminton-highlight-indexer`	A4, A5, A8	No for limited product alpha; model/public gate remains #172	DONE for limited alpha	Decision record below; #488; #172 remains open for model ship-gate calibration
A11	Resolve or explicitly defer WASB C3 provenance for alpha messaging; if deferred, document "no commercial model sharing" copy guardrail	Owner/legal	A10	No for limited product alpha; public/model gate remains #486	DONE for limited alpha	Decision record below; #488; #486 tracks provenance resolution
A12	Reopen or defer P10/P11 corpus flywheel: `PATCH /api/segments`, collector promotion, `train`/`reeval` job, delta-F1 surface, and alignment with `COMPUTE_DECOUPLED_SERVING` M11	`khelsutra` + `badminton-highlight-indexer` + collector	A1, A3	Owner D13 reopen (`khelsutra/docs/STUDIO_MEDIA_LIFECYCLE.md`)	GATED	-
A13	Alpha launch review: runbook, privacy/consent copy, observability/job-failure triage, known limits, rollback/disable steps, and owner GO/NO-GO	Cross-repo	A1, A4, A10, A11	Owner launch decision	GATED	review DELIVERED 2026-07-04 ? `docs/ALPHA_LAUNCH_REVIEW_2026-07-04.md`; GO/NO-GO awaiting owner

8. Concrete next PR sequence

10. Definition of done

- [ ] Invited tester can complete the hosted record/upload -> reel download CUJ without SSH.
- [x] Alpha service is gated behind the chosen auth posture and has a documented disable/rollback path. (Edge auth verified live ? 401 sans CF JWT, `api.khelsutra.guru` ? CF login 302: e2e runlog §6, re-verified 2026-07-04; disable/rollback documented in `docs/ALPHA_LAUNCH_REVIEW_2026-07-04.md` §5.6.)
- [x] n=15 corpus is portable enough that manifest-driven quality commands do not rely on stale local absolute paths.
- [x] n=15 nightly baseline and heuristic floor are refreshed with recorded commands.
- [x] R2/tolF1 gate is promoted or rejected with a recorded ablation result.
- [x] Fusion/person/Gate-A served-default decisions are explicitly promoted or rejected on the 15-video corpus.
- [x] Ship-gate target and allowed alpha claims are recorded for the limited product alpha. (Model/public/commercial ship-gate calibration remains open in #172.)
- [x] WASB C3 status is recorded and reflected in alpha messaging for the limited product alpha. (C3 is deferred with no model/weight sharing or commercial model claims; #486 remains open.)
- [ ] P10/P11 are either reopened with a committed sequence or explicitly deferred for alpha-plus.
- [ ] Owner records GO/NO-GO for the first alpha wave.

575. user (2026-07-05T14:13:26.041Z)

```
47877373-e457-4df0-a155-1ccc443c818b    WORKING
```

576. user (2026-07-05T14:13:38.851Z)

```
Command running in background with ID: b0z5cf50h. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\693b418f-8bf1-49e5-b443-b86a83235a63\tasks\b0z5cf50h.output. You will be notified when
it completes. To check interim output, use Read on that file path.
```

577. assistant (2026-07-05T14:13:51.437Z)

```
Build poller is watching `47877373` (still WORKING ? CUDA builds run 8?15 min). The moment it
goes green: worker re-pin ? `services replace` with the verified auth-ON manifest ? the battery,
with tracker updates and the #490/#480 close-out riding the runlog PR as you asked. I'll pick up
automatically when the build finishes.
```

578. user (2026-07-05T14:18:47.373Z)

```
<task-notification>
<task-id>bq5u3urv7</task-id>
<tool-use-id>toolu_01KTa1KsoJwEyLRh1AJeN1Tk</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-
highlight-indexer\693b418f-8bf1-49e5-b443-b86a83235a63\tasks\bq5u3urv7.output</output-file>
<status>completed</status>
<summary>Background command "Submit worker image build to Cloud Build" completed (exit code
0)</summary>
</task-notification>
```

579. user (2026-07-05T14:18:56.813Z)

```
26411e38de87: Pushed
latest: digest: sha256:3bfefaf6ad4ac96d9cb30c1005d49890c11a42a1dc7f543261699cb67671b964 size:
3889
DONE
--------------------------------------------------------------------------------
ID                                    CREATE_TIME                DURATION  SOURCE
IMAGES                                                                                  STATUS
47877373-e457-4df0-a155-1ccc443c818b  2026-07-05T14:10:22+00:00  7M33S     gs://gen-lang-
client-0306026826_cloudbuild/source/1783260615.531581-5507c881000f40988cc76b26a2a50df2.tgz
asia-southeast1-docker.pkg.dev/gen-lang-client-0306026826/rally/rally-worker (+1 more)  SUCCESS
```

580. assistant (2026-07-05T14:19:16.420Z)

```
**Build SUCCESS (7m33s)** ? new digest `3bfefaf6?` (moved from the live `31475f27?`, confirming
a fresh image from master `257f469`). Re-pinning the worker Job, then deploying the CP.
```

581. user (2026-07-05T14:19:26.421Z)

```
To execute this job, use:
gcloud run jobs execute rally-worker
rally-worker-x5pt2      asia-southeast1-docker.pkg.dev/gen-lang-client-0306026826/rally/rally-
worker:latest
```

582. assistant (2026-07-05T14:19:35.338Z)

```
Job re-pinned. Deploying the control plane with the auth-ON manifest (8Gi + baked pins).
```

583. user (2026-07-05T14:21:16.838Z)

```
Routing traffic.....done
Done.
New configuration has been applied to service [rally-control-plane].
URL: https://rally-control-plane-492532266377.asia-southeast1.run.app
```

584. user (2026-07-05T14:21:32.715Z)

```
rally-control-plane-00012-wxk
8Gi     asia-southeast1-docker.pkg.dev/gen-lang-client-0306026826/rally/rally-
worker@sha256:3bfefaf6ad4ac96d9cb30c1005d49890c11a42a1dc7f543261699cb67671b964
```

585. user (2026-07-05T14:21:53.942Z)

```
health: 200 {"status":"ok","sport":"badminton","default_segmenter":"wasb_hybrid"}
process sans CF JWT: 401
api.khelsutra.guru: 302 ->
```

586. user (2026-07-05T14:22:28.739Z)

```
gs://khelsutra-rally-serving/outputs/0ab01a738d5a8d8a2ecafce37bd1b6e3/
gs://khelsutra-rally-serving/outputs/915473794e0909c9f95a01e4511a5e98/
gs://khelsutra-rally-serving/outputs/c2d45319b761e8dfd199fc5b80f55369/
gs://khelsutra-rally-serving/outputs/d8335f51acd9b50c9f15bff3f5fcf4e7/
gs://khelsutra-rally-serving/outputs/f9009caebf65029c576929d7c19313af/
```

587. user (2026-07-05T14:23:00.452Z)

```
New configuration has been applied to service [rally-control-plane].
URL: https://rally-control-plane-492532266377.asia-southeast1.run.app
rally-control-plane-00013-rlc
```

588. user (2026-07-05T14:23:20.911Z)

```
replay segments: 29
{"id":1,"video_id":"c2d45319b761e8dfd199fc5b80f55369","start_time":3.337,"end_time":7.941,"durat
ion":4.603999999999999,"server":null,"receiver":null,"metadata":{"source":"cloud_run","ending_re
ason":""},"events":[]}
```

589. user (2026-07-05T14:23:42.132Z)

```
submit: 732 ms -> {"job_id":"f9283b583f6046a7b3047d7e041d2ef8","status":"queued","poll":"/api/jo
bs/f9283b583f6046a7b3047d7e041d2ef8"}
job: status=running cached= segments=0
rally-worker-x5pt2
```

590. user (2026-07-05T14:28:27.500Z)

```
        4
job: status=running cached= segments=0 computepath=
rally-worker-2mz8t
```

591. user (2026-07-05T14:29:25.016Z)

```
2026-07-05T14:20:57.475455Z     Starting new instance. Reason:
MANUAL_OR_CUSTOMER_MIN_INSTANCE - Instance
started because
of customer-configured min-instances or
manual scaling.
2026-07-05T14:21:07.965722Z     INFO:     Started server process [1]
2026-07-05T14:21:20.000181Z     Shutting down user disabled instance
2026-07-05T14:22:46.478811Z     Starting new instance. Reason:
MANUAL_OR_CUSTOMER_MIN_INSTANCE - Instance
started because
of customer-configured min-instances or
manual scaling.
2026-07-05T14:22:54.858099Z     INFO:     Started server process [1]
2026-07-05T14:23:00.507581Z     Shutting down user disabled instance
```

592. user (2026-07-05T14:30:02.482Z)

```
progress: 'segmenting (wasb_hybrid) on cloud GPU ? 5 min elapsed'
caps: max_upload_mb=4096 max_part_mb=24 default_segmenter=wasb_hybrid cloud_available=True
ui=0.1.0
```

593. user (2026-07-05T14:30:33.512Z)

```
init: id=738319931ea64477b7dfed9986acc324 part_mb=24
parts: 2 x 1572864 bytes total
complete: path=gs://khelsutra-rally-
serving/videos/uploads/738319931ea64477b7dfed9986acc324/p9-stream-proof.mp4
video_id=dc65ef78bd4638e9b21aab5a576faad8 size=1572864
```

594. assistant (2026-07-05T14:30:44.636Z)

```
Streamed staging works live. Verifying the staged object's server-side md5 matches the streamed
digest (the non-composite guarantee that keeps the instant cache path for browser uploads).
```

595. user (2026-07-05T14:30:54.161Z)

```
object md5(hex)=dc65ef78bd4638e9b21aab5a576faad8 size=1572864 -> matches streamed video_id: True
jobs list: 1 row(s); first: f9283b58 running 'segmenting (wasb_hybrid) on cloud GPU ? 6 min
elapsed'
```

596. user (2026-07-05T14:31:18.844Z)

```
Command running in background with ID: bhyf55n4w. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\693b418f-8bf1-49e5-b443-b86a83235a63\tasks\bhyf55n4w.output. You will be notified when
it completes. To check interim output, use Read on that file path.
```

597. user (2026-07-05T14:31:31.006Z)

```
HEAD is now at 257f469 chore(ui): re-vendor the SPA snapshot at khelsutra 90d1d90 (CP-rebuild
P8) + batch the §7 tracker updates (#495)
```

598. assistant (2026-07-05T14:32:22.022Z)

DEPLOY REPORT ? control-plane durable rebuild (CP-rebuild P9), 2026-07-05

```
> The §4.8/§9 execution record for `docs/CONTROL_PLANE_DURABLE_REBUILD.md` (spec #487, owner-
confirmed).
> Everything here ran live against project `gen-lang-client-0306026826`, region `asia-
southeast1`,
> on 2026-07-05 (14:10?14:5x UTC). Honest-limits in §6: what this run does NOT prove.
```

1. What shipped in the image

```
Master `257f469` ? the full P-row sweep, all merged on verified LGTMs the same day:
P1 #489 (generator pins + 8Gi) · P2 #490 (GCS-streamed upload staging) · P4 #492 (submit-time
MD5 + cache-reuse) · P5a #491 (poll budget 3900 s + capabilities caps) · P6 #493 (cloud-poll
heartbeat) · P3 #494 (restart-safe recovery + read-path re-hydrate) · P8 #495 (SPA re-vendored
at khelsutra `90d1d90` = #153/#155/#158/#162 baked; platform-stable `ui_version` hash).
khelsutra side: #162 (60-min SPA ceilings) on khelsutra master `90d1d90`.
```

2. Build ? pin ? deploy (§4.8 steps 1?4)

```
| Step | Command / object | Result |
|------|------------------|--------|
| Image build | `gcloud builds submit --config=deploy/cloudrun/cloudbuild.yaml` from a clean
worktree at `257f469` | build `47877373` **SUCCESS in 7m33s** ? digest
```

`sha256:3bfefaf6ad4a?7671b964` (moved from the prior live `31475f27?`) |
Worker re-pin	`gcloud run jobs update rally-worker --image ?/rally-worker:latest`	job resolves the new digest
CP deploy (auth ON)	`gcloud run services replace service-auth-on.yaml` (generated by `deploy/cloudrun/gen_service_yaml.py --edge-auth on` from the clean clone)	rev **`rally-control-plane-00012-wxk`** · **8Gi** / 2 vCPU · imageDigest == `3bfefaf6?`
Pins in the baked config `[verified pre-deploy]`	base64-decoded the manifest's wrapper payload	`indexing.default_segmenter="wasb_hybrid"` + `security.max_part_mb=24` + `edge_auth_enabled=true` present ? **D5 works from a clean clone**
Upload-staging lifecycle	`gcloud storage buckets update --lifecycle-file ?`	`videos/uploads/**` age-2d Delete rule live (bucket previously had NO lifecycle policy)

3. Posture (§9, on the final auth-ON revision 00012)

- `GET /api/health` (identity token) ? **200** `{"status":"ok","sport":"badminton","default_segmenter":"wasb_hybrid"}` ? the segmenter pin is visible in the health payload itself.
- `POST /api/process` without a CF JWT ? **401** (edge-auth ON).
- `https://api.khelsutra.guru/api/health` ? **302** (Cloudflare Access intercept).

4. Verification battery (short auth-OFF window, rev `00013-rlc` ? the precedented A1-e2e pattern)

The auth-off deploy replaced the instance ? **fresh, empty SQLite: exactly the 2026-07-05 post-restart state** the spec's DoD demands be replayed.

Proof	What ran	Result
Restart replay (#471/#485 ? THE loss shape)	`POST /api/query {"video_id":"c2d45319?f55369"}` against the wiped DB	**29 play_segments returned** ? re-hydrated from `outputs/<md5>/` on demand (`metadata.source=cloud_run` rows). The July-5 morning failure shape now recovers by itself
Streamed upload staging (P2/#480)	2×768 KiB parts ? `/api/upload/init ? part/0 ? part/1 ? complete`	`path=gs://?/videos/uploads/7383?/p9-stream-proof.mp4`, `video_id=dc65ef78?`; **server-side object `md5Hash` (hex) == the streamed video_id EXACTLY**, `component_count` empty (non-composite) ? browser-staged uploads stay md5-addressable, so the instant cache path holds for the tester flow
Capabilities (P5a/#480 item 5)	`GET /api/capabilities`	`max_upload_mb=4096`, `max_part_mb=24`, `default_segmenter=wasb_hybrid`, `deployment.cloud_available=true`
Heartbeat (P6/#482 CP slice)	`GET /api/jobs/{id}` during the live detection	`progress = "segmenting (wasb_hybrid) on cloud GPU ? 5 min elapsed"` ? later ticks 6, 7? ? the silent 22-minute spinner is dead
Jobs queue (#481)	`GET /api/jobs?status=all`	lists the running job with its live progress
Fresh detection on the new image	the same submit dispatched `rally-worker-2mz8t` (see §5 for WHY it dispatched)	<!--JOB_RESULT-->

5. Findings (honest ? two real ones)

1. **Composite originals bypass the submit-time cache (documented P4 boundary, now measured).** `videos/MahadevpuraSingles.MP4` was bucket-uploaded as a **composite object** (`component_count=4`) ? GCS exposes **no `md5Hash`** ? `submit_time_video_id` returns None ? the instant-done path can't fire. By itself that would only degrade to the drain-tier DB check; combined with (2) it re-dispatched.
2. **Fresh-DB + composite-original ? the bucket cache is never consulted on the drain path.** Cloud Run replaced the rev-00013 instance between the replay query and the submit (`MANUAL_OR_CUSTOMER_MIN_INSTANCE` churn in the logs) ? the drain hashed on an empty DB, found no video row, and dispatched ? even though `outputs/<md5>/report.json` existed. The drain checks only the DB after hashing; the bucket tier lives on the submit path (md5-known) and the read path. **Residual gap, bounded:** it needs BOTH a composite/md5-less source AND an instance swap; browser-staged uploads (the tester flow) always carry `md5Hash` and take the instant path. Follow-up filed on #484: consult `cached_process_result` on the drain after hashing. Cost of the encounter here: one L4 re-detection (~US$0.4), harvested as the fresh-detection proof below.

6. What this run does NOT prove (stays on the owner's §1.1 CUJ)

```
- The **?2.5 GB browser upload** through Cloudflare Access on the real domain (the §9 headline
  memory-survival proof under load) ? the synthetic upload proves the staging PATH, not the
  4 GiB-scale memory profile. The owner's official 5-minute CUJ covers it on this build.
- SPA-rendered UX (progress copy, medium picker, pickable rallies) ? API-level proofs only here;
  the vendored snapshot provenance (`ui_version.spa_commit=90d1d90`, content-hash guard in CI)
  is what ties the served UI to the verified khelsutra master.
- Concurrent-duplicate submit collapse (#481) was regression-proven in CI (12-thread test) but
  not re-driven live in this window.
```

7. Restore + final posture

```
<!--RESTORE-->
```

8. Tracker updates riding this PR

```
- `docs/CONTROL_PLANE_DURABLE_REBUILD.md` §7: **P9 ? ?** (this runlog); status header note.
- `docs/ALPHA_LAUNCH_READINESS.md`: DoD item 1 annotated ? the infra blocker behind the failed
  2026-07-05 owner CUJ is deployed-fixed (rev 00012-wxk); the official CUJ pass remains the
  owner's checkbox. A1 row cross-references this runlog.
- Issues: #480 + #471 + #485 closed with live evidence; #484 kept open (composite-gap follow-
up);
  #482 stays open (worker-side heartbeat + per-job runlog).
```

599. user (2026-07-05T14:32:22.050Z)

```
File created successfully at: C:\Users\avidu\Projects\khelsutra-
guru\wt-p9-runlog\docs\COMPUTE_DECOUPLED_SERVING\runlogs\DEPLOY_REPORT_cloud-serving_cp-
rebuild_2026-07-05.md (file state is current in your context ? no need to Read it back)
```

600. user (2026-07-05T14:32:50.124Z)

```
api_version=1 spa_commit=90d1d90cebee057b98ec4adf165b036257f14978 built_at=07/05/2026 13:22:19
```

601. assistant (2026-07-05T14:33:02.626Z)

```
Served-UI provenance confirmed: `/api/version` ? `spa_commit=90d1d90?`, exactly the P8 snapshot.
Now the tracker edits that ride the runlog PR.
```

602. user (2026-07-05T14:33:35.084Z)

```
spec tracker updated
```

603. user (2026-07-05T14:33:58.291Z)

```
alpha tracker updated
[check-md-links] clean ? no broken internal links in live docs.
warning: in the working copy of 'docs/ALPHA_LAUNCH_READINESS.md', LF will be replaced by CRLF
the next time Git touches it
warning: in the working copy of 'docs/CONTROL_PLANE_DURABLE_REBUILD.md', LF will be replaced by
CRLF the next time Git touches it
DOCS_OK
```

604. user (2026-07-05T14:34:08.606Z)

```
poll 3: running | segmenting (wasb_hybrid) on cloud GPU ? 8 min elapsed
poll 4: running | segmenting (wasb_hybrid) on cloud GPU ? 8 min elapsed
poll 5: running | segmenting (wasb_hybrid) on cloud GPU ? 9 min elapsed
poll 6: running | segmenting (wasb_hybrid) on cloud GPU ? 9 min elapsed
```

605. assistant (2026-07-05T14:34:24.760Z)

Trackers updated and link-checked in the runlog worktree; the detection is at ~9 minutes with the heartbeat ticking cleanly (the poller output itself is a nice demo ? the progress string advances every poll). When the job lands: fill the runlog's last two sections, restore auth-ON, then the runlog PR + issue close-outs (#480 via #490 as you asked, #471, #485) and the CUJ handover to you. The poller will wake me at completion.

606. user (2026-07-05T14:37:48.358Z)

<task-notification>
<task-id>bhyf55n4w</task-id>
<tool-use-id>toolu_012zEisDGCiJKFuycoxEKU12</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\693b418f-8bf1-49e5-b443-b86a83235a63\tasks\bhyf55n4w.output</output-file>
<status>completed</status>
<summary>Background command "Poll the verification job to completion" completed (exit code 0)</summary>
</task-notification>

607. user (2026-07-05T14:37:57.249Z)

poll 11: running | segmenting (wasb_hybrid) on cloud GPU ? 12 min elapsed
poll 12: running | segmenting (wasb_hybrid) on cloud GPU ? 13 min elapsed
poll 13: done | finished

608. user (2026-07-05T14:38:10.224Z)

status=done segments=48 path=cloud_run exec=projects/gen-lang-client-0306026826/locations/asia-southeast1/jobs/rally-worker/executions/rally-worker-2mz8t
video_id=915473794e0909c9f95a01e4511a5e98
started=2026-07-05 14:23:41 finished=2026-07-05 14:37:45

609. user (2026-07-05T14:39:01.575Z)

submit2: {"job_id":"d6eac2b493824a118e43326db61b4cb7","status":"queued","poll":"/api/jobs/d6eac2b493824a118e43326db61b4cb7"}
job2: status=done cached=True segments=48 path=cached in 11s
rally-worker-2mz8t

610. user (2026-07-05T14:40:00.554Z)

New configuration has been applied to service [rally-control-plane].
URL: https://rally-control-plane-492532266377.asia-southeast1.run.app
rally-control-plane-00014-7hk
health: 200
process sans CF JWT: 401
....

611. user (2026-07-05T14:40:45.194Z)

runlog finalized
[check-md-links] clean ? no broken internal links in live docs.
[leak-scan] clean ? no MD5 / personal-path / private-name leaks in the code surface.
warning: in the working copy of 'docs/ALPHA_LAUNCH_READINESS.md', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'docs/CONTROL_PLANE_DURABLE_REBUILD.md', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_cp-rebuild_2026-07-05.md', LF will be replaced by CRLF the next time Git touches it
[docs/p9-cp-rebuild-runlog d19f2af] docs(runlog): CP-rebuild P9 executed ? rev 00012/00014, battery green, trackers updated
 3 files changed, 126 insertions(+), 20 deletions(-)
 create mode 100644 docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_cp-rebuild_2026-07-05.md
 * [new branch]   docs/p9-cp-rebuild-runlog -> docs/p9-cp-rebuild-runlog

612. user (2026-07-05T14:41:11.703Z)

https://github.com/Khelsutra/badminton-highlight-indexer/pull/496

613. user (2026-07-05T14:41:35.469Z)

https://github.com/Khelsutra/badminton-highlight-indexer/pull/490#issuecomment-4886420798

614. user (2026-07-05T14:41:45.394Z)

Command running in background with ID: brgylnpau. Output is being written to: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\693b418f-8bf1-49e5-b443-b86a83235a63\tasks\brgylnpau.output. You will be notified when it completes. To check interim output, use Read on that file path.

615. assistant (2026-07-05T14:41:53.958Z)

Memory checkpoint for the P9 execution, then the summary.

616. user (2026-07-05T14:42:10.807Z)

current-state checkpointed

617. user (2026-07-05T14:42:29.990Z)

Exit code 1
Traceback (most recent call last):
  File "<stdin>", line 6, in <module>
AssertionError

618. user (2026-07-05T14:42:39.881Z)

14:## ? YOU ARE HERE ? P8 MERGED; P9 REBUILD/DEPLOY VALIDATION NEXT
11:> Last refreshed: 2026-07-05 late night. **Current monitor rule:** review/LGTM only; do not merge unless Avi explicitly asks in-thread. **MERGED ? spec + P1/P2/P3/P4/P5a/P5b/P6/P8:** spec #487 (`cc5e83a`) · P1 #489 (`e80bca8`) · sibling #488 (`f1b60d0`) · P5a #491 (`97feff4`) · P4 #492 (`851f21d`) · P6 CP slice #493 (`4cd36fd`) · P2 #490 (`6acddce`) · **P3 #494 (`f6864f1`)** · **P8 #495 (`257f469`)** · **P5b kh#162 (khelsutra master `90d1d90`)**. **OPEN:** none in `Khelsutra/badminton-highlight-indexer` as of the last poll. **NEXT:** P9 is fully unblocked (owner already authorized via the spec confirm): consolidated rebuild/redeploy + holistic §9 validation and runlog. Earlier: #481 (`0a7b3ff`); khelsutra #158/#155. Prior: 2026-07-04 ? **A1 DONE** (#477, `122f174`).

619. user (2026-07-05T14:43:02.030Z)

handoff refreshed

620. assistant (2026-07-05T14:43:26.063Z)

The rebuild is live. P9 executed end-to-end tonight:

What's deployed

CP revision **`00014-7hk`** (via 00012): the new image (digest `3bfefaf6?`, master `257f469` = all eight P-rows), **8 GiB**, both rev-00011 pins baked from code, edge-auth ON, maxScale 1, plus an age-2-day lifecycle rule on the upload-staging prefix. Worker Job re-pinned to the same image.

Battery results (full record in [PR #496](https://github.com/Khelsutra/badminton-highlight-indexer/pull/496))

- **Your exact loss shape, replayed and healed:** fresh wiped SQLite ? `?video=<md5>` query ? **29 rallies re-hydrated from the bucket**.
- **Streamed staging live:** an upload landed at `gs://?/videos/uploads/<id>/?` with the object's server md5 **exactly equal** to the streamed video_id ? never touched tmpfs.
- **Cache proof, the literal DoD box:** re-submitting the same video returned `done cached=true`

in **11 seconds with zero GPU** (execution list unchanged).
- **Heartbeat live:** `"segmenting (wasb_hybrid) on cloud GPU ? N min elapsed"` ticked through a real fresh detection on the new image (**48 segments, 14m04s**) ? which also proved the segmenter pin in worker reality, not just config.
- Capabilities (`4096/24/wasb_hybrid`), served SPA provenance (`spa_commit=90d1d90` ? the medium-picker trap is dead), and the 200/401/302 auth posture all verified, with auth restored and re-checked after the test window.

Two honest findings (runlog §5)

Composite-uploaded originals (like the bucket's `MahadevpuraSingles.MP4`, which turns out to be the 48-rally e2e file, not your morning upload) carry no `md5Hash`, so they skip the instant cache path; and a fresh-DB drain doesn't consult the bucket cache ? bounded to that composite+instance-churn combination, follow-up noted on #484. Cost of hitting it: one 14-minute L4 run, harvested as the fresh-detection proof.

What's left

#496 (docs-only) awaits review ? on LGTM I merge and close **#480/#471/#485** with the evidence (the #490 completion comment is already [posted](https://github.com/Khelsutra/badminton-highlight-indexer/pull/490#issuecomment-4886420798) as you asked). Then the last three §9 boxes are yours: **the official five-minute browser CUJ on this build** ? big upload, pickable rallies, reel download. Memory and trackers are all checkpointed; the watcher is on #496.

621. user (2026-07-05T14:50:52.744Z)

```
<task-notification>
<task-id>brgylnpau</task-id>
<tool-use-id>toolu_019BFQGUEQioK32eAcPYkt7E</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\693b418f-8bf1-49e5-b443-b86a83235a63\tasks\brgylnpau.output</output-file>
<status>completed</status>
<summary>Background command "Point watcher at 496 and launch" completed (exit code 0)</summary>
</task-notification>
```

622. user (2026-07-05T14:51:03.620Z)

```
CHANGE DETECTED (poll 3):
  was: engine#496:OPEN/MERGEABLE/c0/r0 kh#162:MERGED/UNKNOWN/c2/r0
  now: engine#496:OPEN/MERGEABLE/c1/r0 kh#162:MERGED/UNKNOWN/c2/r0
LGTM!
```

623. assistant (2026-07-05T14:51:12.319Z)

LGTM on #496 ? merging, then closing the issue chain with the runlog evidence.

624. user (2026-07-05T14:51:25.983Z)

```
MERGEABLE SUCCESS

From https://github.com/Khelsutra/badminton-highlight-indexer
   257f469..6ab5a71  master     -> origin/master
6ab5a71 docs(runlog): CP-rebuild P9 executed ? rev 00012/00014, battery green, trackers updated (#496)
Deleted branch docs/p9-cp-rebuild-runlog (was d19f2af).
done
```

625. user (2026-07-05T14:52:01.329Z)

```
? Closed issue Khelsutra/badminton-highlight-indexer#480 (Hosted alpha: browser chunked upload 413s (Cloud Run HTTP/1 ~32MiB request cap vs 95MB parts) + 4Gi tmpfs ceiling)
? Closed issue Khelsutra/badminton-highlight-indexer#471 (cloud-serving: control-plane state (SQLite jobs/videos DB) is per-instance ephemeral ? maxScale>1 splits the queue and /api/jobs polls)
? Closed issue Khelsutra/badminton-highlight-indexer#485 (Durable resumable link: re-hydrate
```

session state from the bucket (source of truth) on ?video=<md5>, surviving CP restarts (#471))
https://github.com/Khelsutra/badminton-highlight-indexer/issues/484#issuecomment-4886453972

626. user (2026-07-05T14:52:11.784Z)

Updated task #9 status

627. user (2026-07-05T14:52:17.438Z)

handoff final update ok

628. assistant (2026-07-05T14:52:38.526Z)

That's a wrap on the rebuild. [#496](https://github.com/Khelsutra/badminton-highlight-indexer/pull/496) merged (`6ab5a71`), and the issue chain is closed out with live evidence:

- **#480** (upload 413 + tmpfs ceiling) ? closed: bucket-streamed staging + baked 24 MB pin + 8 GiB, proven live.
- **#471** (ephemeral state) ? closed: bucket-grounded state, proven by the wiped-DB ? 29-rallies replay.
- **#485** (durable `?video=` link) ? closed: read-path re-hydrate live, plus `video_uri` now recorded in worker reports.
- **#484** ? kept open per the bounded drain-tier/composite follow-up, with the 11-second zero-GPU live proof recorded; **#482** stays open for the worker-side heartbeat; **#483** sits in your warm backlog.

The full arc of this session: your spec confirmation this morning ? 10 PRs written, review-hardened (7 substantive reviewer findings, all real, all fixed), and merged on verified LGTMs ? the image rebuilt and deployed ? the exact failure that blocked your CUJ this morning replayed against the live service and healed. The stack now running at `api.khelsutra.guru`: rev `00014-7hk`, 8 GiB, streamed uploads, durable state, instant re-runs, live progress, honest timeouts, and the SPA with all four merged fixes.

One thing remains, and it's yours: the official five-minute browser CUJ ? upload a real match video, watch the progress tick, pick rallies, download the reel. The three unchecked §9 boxes all collapse into it, and it's the last gate before your GO/NO-GO on the alpha wave. Everything is checkpointed for whichever session picks up next.

629. user (2026-07-05T15:03:37.284Z)

I just started a run, can yu watch it?

630. assistant (2026-07-05T15:04:08.427Z)

Watching it now ? first an immediate look at where the run is (upload part traffic, any dispatch, instance health), then I'll keep a live watch running.

631. user (2026-07-05T15:04:18.489Z)

```
=== executions ===
rally-worker-2mz8t      2026-07-05T14:24:14.137333Z     2026-07-05T14:37:49.067936Z
rally-worker-x5pt2      2026-07-05T08:37:19.897451Z     2026-07-05T08:59:50.869301Z
```

632. assistant (2026-07-05T15:04:31.802Z)

No traffic visible yet under that filter ? widening the look and checking clock alignment.

633. user (2026-07-05T15:04:38.713Z)

```
now UTC: 2026-07-05T15:04:35Z
2026-07-05T15:04:31.512071Z     PUT
https://api.khelsutra.guru/api/upload/702636eed76145e88aa59c3000c7e415/part/16  200
2026-07-05T15:04:28.325506Z     PUT
```

```
https://api.khelsutra.guru/api/upload/702636eed76145e88aa59c3000c7e415/part/15  200
2026-07-05T15:04:24.300288Z     PUT
https://api.khelsutra.guru/api/upload/702636eed76145e88aa59c3000c7e415/part/14  200
2026-07-05T15:04:20.565768Z     PUT
https://api.khelsutra.guru/api/upload/702636eed76145e88aa59c3000c7e415/part/13  200
2026-07-05T15:04:17.026699Z     PUT
https://api.khelsutra.guru/api/upload/702636eed76145e88aa59c3000c7e415/part/12  200
2026-07-05T15:04:12.742229Z     PUT
https://api.khelsutra.guru/api/upload/702636eed76145e88aa59c3000c7e415/part/11  200
2026-07-05T15:04:09.012946Z     PUT
https://api.khelsutra.guru/api/upload/702636eed76145e88aa59c3000c7e415/part/10  200
2026-07-05T15:04:05.091809Z     PUT
https://api.khelsutra.guru/api/upload/702636eed76145e88aa59c3000c7e415/part/9   200
2026-07-05T15:04:01.272985Z     PUT
https://api.khelsutra.guru/api/upload/702636eed76145e88aa59c3000c7e415/part/8   200
2026-07-05T15:03:57.109435Z     PUT
https://api.khelsutra.guru/api/upload/702636eed76145e88aa59c3000c7e415/part/7   200
2026-07-05T15:03:53.011813Z     PUT
https://api.khelsutra.guru/api/upload/702636eed76145e88aa59c3000c7e415/part/6   200
2026-07-05T15:03:49.321698Z     PUT
https://api.khelsutra.guru/api/upload/702636eed76145e88aa59c3000c7e415/part/5   200
```

634. user (2026-07-05T15:05:14.040Z)

```
Command running in background with ID: bayuqoilj. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\693b418f-8bf1-49e5-b443-b86a83235a63\tasks\bayuqoilj.output. You will be notified when
it completes. To check interim output, use Read on that file path.
```

635. assistant (2026-07-05T15:05:31.289Z)

Watching. Here's what I can see and what to expect:

Now: your upload is streaming through the real domain ? Cloudflare Access ? edge-auth ? the CP ? with every 24 MB part returning 200 in ~3.7 s (?6.5 MB/s uplink). Each part feeds the GCS resumable writer directly; the control plane is holding only a chunk buffer, not your video. If this is the 2.68 GB file, expect roughly 112 parts, so the upload phase should finish in about 6?8 more minutes.

One thing worth knowing in advance: if this is the *same file* as this morning (md5 `c2d45319?`), its 29-rally detection already lives in the bucket ? so the moment the upload completes and the SPA submits, the new submit-time cache should recognize it from the streamed md5 and return **done with all 29 rallies in seconds, zero GPU**. That would be the #484 flow working exactly as designed on the real tester path. If it's a new video, you'll see the dispatch instead, with the progress line ticking ("segmenting (wasb_hybrid) on cloud GPU ? N min elapsed") for the ~15?25 minutes of detection.

The watcher will wake me at each phase change ? upload complete, process submit, dispatch-or-instant-cache, compile, reel download ? and I'm also watching for any memory-limit or instance-restart events (the thing that killed this flow this morning; on 8 GiB with streamed staging it shouldn't appear). I'll report as milestones land.

636. user (2026-07-05T15:07:08.632Z)

```
<task-notification>
<task-id>bayuqoilj</task-id>
<tool-use-id>toolu_018PghSLZ1Zy1znCmDLiAhhn</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-
highlight-indexer\693b418f-8bf1-49e5-b443-b86a83235a63\tasks\bayuqoilj.output</output-file>
<status>completed</status>
<summary>Background command "Watch the live CUJ for phase transitions" completed (exit code
0)</summary>
</task-notification>
```

637. user (2026-07-05T15:07:18.489Z)

```
poll 2: uploading? latest=part/37
<string>:1: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for
removal in a future version. Use timezone-aware objects to represent datetimes in UTC:
datetime.datetime.now(datetime.UTC).
poll 3: uploading? latest=part/48
<string>:1: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for
removal in a future version. Use timezone-aware objects to represent datetimes in UTC:
datetime.datetime.now(datetime.UTC).
=== MILESTONE: PROCESS_SUBMITTED (poll 4) ===
--- recent requests ---
2026-07-05T15:06:58.028436Z     GET
https://api.khelsutra.guru/api/jobs/c1be435f7e0a4941b53fa2786bee76a9    200
2026-07-05T15:06:56.041582Z     GET
https://api.khelsutra.guru/api/jobs/c1be435f7e0a4941b53fa2786bee76a9    200
2026-07-05T15:06:53.841985Z     POST    https://api.khelsutra.guru/api/process  202
2026-07-05T15:06:53.627321Z     POST
https://api.khelsutra.guru/api/upload/702636eed76145e88aa59c3000c7e415/complete 200
2026-07-05T15:06:53.527550Z     PUT
https://api.khelsutra.guru/api/upload/702636eed76145e88aa59c3000c7e415/part/60  200
2026-07-05T15:06:52.371667Z     PUT
https://api.khelsutra.guru/api/upload/702636eed76145e88aa59c3000c7e415/part/59  200
2026-07-05T15:06:48.947729Z     PUT
https://api.khelsutra.guru/api/upload/702636eed76145e88aa59c3000c7e415/part/58  200
2026-07-05T15:06:46.318620Z     PUT
https://api.khelsutra.guru/api/upload/702636eed76145e88aa59c3000c7e415/part/57  200
2026-07-05T15:06:42.921110Z     PUT
https://api.khelsutra.guru/api/upload/702636eed76145e88aa59c3000c7e415/part/56  200
2026-07-05T15:06:40.169638Z     PUT
https://api.khelsutra.guru/api/upload/702636eed76145e88aa59c3000c7e415/part/55  200
2026-07-05T15:06:37.114070Z     PUT
https://api.khelsutra.guru/api/upload/702636eed76145e88aa59c3000c7e415/part/54  200
2026-07-05T15:06:33.961916Z     PUT
https://api.khelsutra.guru/api/upload/702636eed76145e88aa59c3000c7e415/part/53  200
```

638. user (2026-07-05T15:07:43.825Z)

```
=== latest exec ===
rally-worker-2mz8t      2026-07-05T14:24:14.137333Z
```

639. user (2026-07-05T15:08:05.233Z)

```
2026-07-05T15:06:52.371667Z
2026-07-05T15:06:53.294850Z     INFO:     169.254.169.126:38094 - "PUT
/api/upload/702636eed76145e88aa59c3000c7e415/part/59 HTTP/1.1" 200 OK
2026-07-05T15:06:53.527550Z
2026-07-05T15:06:53.535311Z     INFO:     169.254.169.126:38108 - "PUT
/api/upload/702636eed76145e88aa59c3000c7e415/part/60 HTTP/1.1" 200 OK
2026-07-05T15:06:53.627321Z
2026-07-05T15:06:53.751331Z     INFO:     169.254.169.126:38120 - "POST
/api/upload/702636eed76145e88aa59c3000c7e415/complete HTTP/1.1" 200 OK
2026-07-05T15:06:53.841985Z
2026-07-05T15:06:54.728080Z     INFO:     169.254.169.126:38134 - "POST /api/process HTTP/1.1"
202 Accepted
2026-07-05T15:06:56.041582Z
2026-07-05T15:06:56.057767Z     INFO:     169.254.169.126:38148 - "GET
/api/jobs/c1be435f7e0a4941b53fa2786bee76a9 HTTP/1.1" 200 OK
2026-07-05T15:06:58.028436Z
2026-07-05T15:06:58.045894Z     INFO:     169.254.169.126:38164 - "GET
/api/jobs/c1be435f7e0a4941b53fa2786bee76a9 HTTP/1.1" 200 OK
2026-07-05T15:07:01.040114Z
2026-07-05T15:07:01.050423Z     INFO:     169.254.169.126:57904 - "GET
/api/jobs/c1be435f7e0a4941b53fa2786bee76a9 HTTP/1.1" 200 OK
2026-07-05T15:07:05.035475Z
```

640. user (2026-07-05T15:08:26.121Z)

<tool_use_error>Blocked: Start-Sleep 45 followed by: gcloud run jobs executions list --job rally-worker --region asia-southeast1 --limit 1 --format="value(name, metadata.creationTimestamp)" 2>$null; gcloud logging read 'resource.type="cloud_run_revision" AND resource.labels.service_name="rally-control-plane" AND timestamp>="2026-07-05T15:07:00Z" AND (textPayload:"dispatched" OR textPayload:"cloud-run" OR textPayload:"Reusing" OR textPayload:"COMPUTE")' --project gen-lang-client-0306026826 --limit 6 --format="value(timestamp, textPayload)" --order=asc 2>$null | Select-Object -First 6. To wait for a condition, use Monitor with an until-loop (e.g. `until <check>; do sleep 2; done` ? Monitor runs bash). To wait for a command you started, use run_in_background: true. Do not chain shorter sleeps to work around this block.</tool_use_error>

641. user (2026-07-05T15:08:41.208Z)

rally-worker-2mz8t 2026-07-05T14:24:14.137333Z

642. assistant (2026-07-05T15:09:00.931Z)

No dispatch yet (~2 min post-submit) ? reading the unfiltered app log to see where the drain actually is.

643. user (2026-07-05T15:09:07.468Z)

2026-07-05T15:06:54.728080Z INFO: 169.254.169.126:38134 - "POST /api/process HTTP/1.1" 202 Accepted
2026-07-05T15:07:19.155259Z 15:07:19 [INFO] backend.orchestration: Running validations for gs://khelsutra-rally-serving/videos/uploads/702636eed76145e88aa59c3000c7e415/Video 1 - Badminton.MP4...

644. user (2026-07-05T15:09:26.051Z)

Command running in background with ID: bw0ghgxh2. Output is being written to: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\693b418f-8bf1-49e5-b443-b86a83235a63\tasks\bw0ghgxh2.output. You will be notified when it completes. To check interim output, use Read on that file path.

645. assistant (2026-07-05T15:09:40.422Z)

Status so far, all healthy:

Phase	Result
Upload (browser ? CF ? CP)	**60 parts × 24 MB ? 1.4 GB, all 200s**, streamed into the bucket ? no tmpfs, no 413, no restart
Complete ? submit	`/complete` 200 ? `POST /api/process` 202 in **200 ms**; staged at `videos/uploads/702636ee?/Video 1 - Badminton.MP4`
Cache check	New video (fresh md5) ? correctly no cache hit
Now	Drain acquired the staged file in ~25 s and is running the CPU validators since 15:07:19 ? normal for this size

Next expected: dispatch to the L4 (a new `rally-worker-*` execution), then the heartbeat ticking in your UI for the ~10?20 minute detection. The watcher will wake me at the dispatch or on anything anomalous (memory events, instance churn). So far this is the morning's failed journey replaying step-for-step ? except everything is holding.

646. user (2026-07-05T15:09:42.010Z)

<task-notification>
<task-id>bw0ghgxh2</task-id>
<tool-use-id>toolu_01TJTPuZhorzz4fjp14nRC1W</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\693b418f-8bf1-49e5-b443-b86a83235a63\tasks\bw0ghgxh2.output</output-file>
<status>completed</status>

<summary>Background command "Watch for dispatch and detection milestones" completed (exit code 0)</summary>
</task-notification>

647. user (2026-07-05T15:09:52.258Z)

```
2026-07-05T15:09:20.039800Z     GET
https://api.khelsutra.guru/api/jobs/c1be435f7e0a4941b53fa2786bee76a9    200
2026-07-05T15:09:14.106094Z     GET
https://api.khelsutra.guru/api/jobs/c1be435f7e0a4941b53fa2786bee76a9    200
2026-07-05T15:09:08.054299Z     GET
https://api.khelsutra.guru/api/jobs/c1be435f7e0a4941b53fa2786bee76a9    200
2026-07-05T15:09:02.041373Z     GET
https://api.khelsutra.guru/api/jobs/c1be435f7e0a4941b53fa2786bee76a9    200
2026-07-05T15:08:56.039075Z     GET
https://api.khelsutra.guru/api/jobs/c1be435f7e0a4941b53fa2786bee76a9    200
2026-07-05T15:08:50.044112Z     GET
https://api.khelsutra.guru/api/jobs/c1be435f7e0a4941b53fa2786bee76a9    200
2026-07-05T15:08:44.435224Z     GET
https://api.khelsutra.guru/api/jobs/c1be435f7e0a4941b53fa2786bee76a9    200
2026-07-05T15:08:39.324120Z     GET
https://api.khelsutra.guru/api/jobs/c1be435f7e0a4941b53fa2786bee76a9    200
2026-07-05T15:08:34.212041Z     GET
https://api.khelsutra.guru/api/jobs/c1be435f7e0a4941b53fa2786bee76a9    200
2026-07-05T15:08:29.091317Z     GET
https://api.khelsutra.guru/api/jobs/c1be435f7e0a4941b53fa2786bee76a9    200
2026-07-05T15:08:23.982567Z     GET
https://api.khelsutra.guru/api/jobs/c1be435f7e0a4941b53fa2786bee76a9    200
2026-07-05T15:08:18.884416Z     GET
https://api.khelsutra.guru/api/jobs/c1be435f7e0a4941b53fa2786bee76a9    200
```

648. user (2026-07-05T15:10:05.390Z)

```
=== MILESTONE: PROCESS_SUBMITTED (poll 1) ===
```

649. user (2026-07-05T15:10:29.414Z)

```
Command running in background with ID: bpxw4yz1v. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\693b418f-8bf1-49e5-b443-b86a83235a63\tasks\bpxw4yz1v.output. You will be notified when
it completes. To check interim output, use Read on that file path.
```

650. assistant (2026-07-05T15:10:38.689Z)

```
Phase-2 watcher armed ? it fires only on forward milestones now: the dispatch (new `rally-
worker-*` execution), rallies loading, compile, reel download, or any memory/error event. Your
run is in the validator phase on the CP; I'll report the moment it dispatches to the GPU.
```

651. user (2026-07-05T15:15:38.985Z)

```
"Video failed validation: blur_check: Video quality rejected: Focus check failed. Average
sharpness score of 11.9 is below the minimum required threshold of 20.0.

"
```

652. assistant (2026-07-05T15:16:22.972Z)

```
That's the blur validator doing its job on a soft-scoring video ? but 11.9 vs threshold 20 on
real footage deserves a quick look before deciding it's the video's fault. Checking the
validator's implementation, its config knob, and how the morning's video scored.
```

653. user (2026-07-05T15:16:26.655Z)

```
backend/pipeline/validators/__init__.py:2:from backend.pipeline.validators.blur import
```

```
BlurValidator
backend/pipeline/validators/blur.py:11:        return "blur_check"
backend/pipeline/validators/blur.py:15:        return "Samples frames across the video and
checks if the footage is excessively blurry or out of focus."
backend/pipeline/validators/blur.py:25:                message="Failed to open video file for
blur analysis.",
backend/pipeline/validators/blur.py:59:                message="Could not read frames to perform
blur verification.",
backend/pipeline/validators/blur.py:72:        min_blur_threshold = val_cfg.min_blur_threshold
backend/pipeline/validators/blur.py:74:        if avg_variance < min_blur_threshold:
backend/pipeline/validators/blur.py:78:                message=f"Video quality rejected: Focus
check failed. Average sharpness score of {round(avg_variance, 1)} is below the minimum required
threshold of {min_blur_threshold}.",
backend/pipeline/validators/blur.py:85:            message="Video sharpness meets focus
requirements.",
28:    min_blur_threshold: float = 20.0
```

654. user (2026-07-05T15:16:34.627Z)

```
1       from typing import Any
2
3       import cv2
4
5       from backend.pipeline.validators.base import ValidationResult, VideoValidator
6
7
8       class BlurValidator(VideoValidator):
9           @property
10          def name(self) -> str:
11              return "blur_check"
12
13          @property
14          def description(self) -> str:
15              return "Samples frames across the video and checks if the footage is excessively
blurry or out of focus."
16
17          def validate(
18              self, video_path: str, config: Any, context: dict[str, Any]
19          ) -> ValidationResult:
20              cap = cv2.VideoCapture(video_path)
21              if not cap.isOpened():
22                  return ValidationResult(
23                      validator_name=self.name,
24                      passed=False,
25                      message="Failed to open video file for blur analysis.",
26                  )
27
28              total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
29              if total_frames <= 0:
30                  cap.release()
31                  return ValidationResult(
32                      validator_name=self.name, passed=False, message="Invalid frame count."
33                  )
34
35              # Sample 15 frames evenly distributed across the video
36              num_samples = 15
37              step = max(1, total_frames // num_samples)
38
39              variances = []
40
41              for i in range(num_samples):
42                  frame_idx = i * step
43                  cap.set(cv2.CAP_PROP_POS_FRAMES, frame_idx)
44                  ret, frame = cap.read()
45                  if not ret:
```

```
46                      continue
47
48                  gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
49                  # Calculate the Laplacian variance (standard focus metric)
50                  laplacian_var = cv2.Laplacian(gray, cv2.CV_64F).var()
51                  variances.append(laplacian_var)
52
53              cap.release()
54
55              if not variances:
56                  return ValidationResult(
57                      validator_name=self.name,
58                      passed=False,
59                      message="Could not read frames to perform blur verification.",
60                  )
61
62              avg_variance = sum(variances) / len(variances)
63
64              details = {
65                  "avg_laplacian_variance": round(avg_variance, 2),
66                  "min_laplacian_variance": round(min(variances), 2),
67                  "max_laplacian_variance": round(max(variances), 2),
68                  "sampled_frames_count": len(variances),
69              }
70
71              val_cfg = config.validation
72              min_blur_threshold = val_cfg.min_blur_threshold
73
74              if avg_variance < min_blur_threshold:
75                  return ValidationResult(
76                      validator_name=self.name,
77                      passed=False,
78                      message=f"Video quality rejected: Focus check failed. Average sharpness
score of {round(avg_variance, 1)} is below the minimum required threshold of
{min_blur_threshold}.",
79                      details=details,
80                  )
81
82              return ValidationResult(
83                  validator_name=self.name,
84                  passed=True,
85                  message="Video sharpness meets focus requirements.",
86                  details=details,
87              )
88
```

655. user (2026-07-05T15:17:24.148Z)

Command running in background with ID: bpcwrmf6u. Output is being written to: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\693b418f-8bf1-49e5-b443-b86a83235a63\tasks\bpcwrmf6u.output. You will be notified when it completes. To check interim output, use Read on that file path.

656. assistant (2026-07-05T15:17:43.730Z)

While it downloads, here's the decision framing:

What just worked, for the record: the whole pre-detection pipeline behaved exactly as rebuilt ? 1.4 GB streamed to the bucket with zero instance stress, instant submit, drain acquire in 25 s, and the honest failure verdict surfaced verbatim in your UI (that message rendering is the khelsutra `b0f70a5` feature). The gate also saved a paid GPU run on footage *it* considered unusable ? that's its purpose.

The question is whether it judged correctly. Two candidate responses once I've measured:

1. **Quick unblock (config-only, ~2 min):** patch the live baked config with a lower `validation.min_blur_threshold` (or disable `blur_check` for the alpha) and `services replace` ? same mechanism as rev-00011, folded back into the generator later per the D5 discipline. Then you can re-submit without re-uploading: your file is still staged at `gs://khelsutra-rally-serving/videos/uploads/702636ee?/Video 1 - Badminton.MP4`, and the SPA's cloud-link form accepts a `gs://` paste directly.
2. **Durable fix (small review-gated PR):** change the metric from *mean* to a high percentile (or max) of the sampled variances ? "is even the sharpest sampled frame soft?" is the right focus question; motion-blurred frames stop polluting the answer. Plus recalibrate the threshold against footage that has actually passed detection.

My measurement will tell us which world we're in ? if the video's stationary frames score high, it's option 1 now + option 2 as the real fix; if everything scores ~12 flat, the video is genuinely soft and the gate did its job. One question meanwhile: **does the footage look sharp to your eye?**

657. user (2026-07-05T15:18:33.386Z)

not the best but manageable

658. assistant (2026-07-05T15:19:00.840Z)

"Manageable" is the product verdict ? a tester's usable video should process, so this is a false reject for the alpha's purpose. Plan: measure the distribution (data picks the new threshold), config-patch the live service (rev-00011-style), then you re-submit **without re-uploading** by pasting the staged `gs://` link. Checking the download.

659. user (2026-07-05T15:19:03.444Z)

```
Average throughput: 20.8MiB/s
drwxr-xr-x 1 avidu 197609          0 Jul  5 20:47 ..
-rw-r--r-- 1 avidu 197609 1510427943 Jul  5 20:48 video1.mp4
```

660. assistant (2026-07-05T15:19:13.899Z)

Download done (1.51 GB). Running the validator's exact metric plus a denser distribution.

661. user (2026-07-05T15:20:49.896Z)

```
video: 5312x2988 @ 29.97fps, 3019 frames (~101s)
validator replica (15 frames): mean=25.1 min=23.6 max=27.0
dense (n=60): mean=25.3 p25=23.7 median=25.7 p75=26.7 p90=27.7 max=28.7
```

662. user (2026-07-05T15:21:31.142Z)

```
24:class ValidationConfig(BaseModel):
28:    min_blur_threshold: float = 20.0
520:    validation: ValidationConfig = Field(default_factory=ValidationConfig)
```

663. user (2026-07-05T15:21:45.246Z)

patched config validation: {'min_blur_threshold': 8.0} | pins intact: wasb_hybrid 24 | auth: True

664. user (2026-07-05T15:22:21.392Z)

```
New configuration has been applied to service [rally-control-plane].
URL: https://rally-control-plane-492532266377.asia-southeast1.run.app
rally-control-plane-00015-kp9
```

665. user (2026-07-05T15:22:57.944Z)

https://github.com/Khelsutra/badminton-highlight-indexer/issues/497

666. user (2026-07-05T15:23:08.547Z)

```
Command running in background with ID: bzivm2051. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\693b418f-8bf1-49e5-b443-b86a83235a63\tasks\bzivm2051.output. You will be notified when
it completes. To check interim output, use Read on that file path.
```

667. assistant (2026-07-05T15:23:26.867Z)

Summary of the last ten minutes:

The reject was wrong, and provably so. Your 5.3K video measures **25.1 locally with the validator's own exact sampling** (threshold 20) ? the deployed stack scored the identical bytes **11.9**. Two real defects: the metric isn't portable across decode stacks (~2× drift), and it penalizes high-resolution footage structurally. Both are documented with the full measurements in [#497](https://github.com/Khelsutra/badminton-highlight-indexer/issues/497), along with the durable fix shape (canonical downscale before Laplacian, median instead of mean, recalibration on the deployed stack, and a CI portability pin ? plus folding the threshold into the generator per the D5 rule).

You're unblocked now: rev `00015-kp9` is serving with the threshold at 8.0 (every pin and the auth posture untouched). Paste the staged link into the Studio's cloud-source field and it goes straight to detection ? no re-upload:

```
gs://khelsutra-rally-serving/videos/uploads/702636eed76145e88aa59c3000c7e415/Video 1 -
Badminton.MP4
```

Watcher's armed for your re-submit ? I'll report the dispatch and follow the detection through.

668. user (2026-07-05T15:28:21.968Z)

check

669. user (2026-07-05T15:28:32.832Z)

```
poll 3: exec=rally-worker-2mz8t (waiting)
poll 4: exec=rally-worker-2mz8t (waiting)
poll 5: exec=rally-worker-2mz8t (waiting)
poll 6: exec=rally-worker-2mz8t (waiting)
poll 7: exec=rally-worker-2mz8t (waiting)
poll 8: exec=rally-worker-2mz8t (waiting)
```

670. user (2026-07-05T15:28:47.562Z)

```
2026-07-05T15:28:41.205301Z     INFO:     169.254.169.126:57676 - "GET
/api/jobs/e0a58416af3f4e8ab189f061a1635303 HTTP/1.1" 200 OK
2026-07-05T15:28:41.198140Z             GET
https://api.khelsutra.guru/api/jobs/e0a58416af3f4e8ab189f061a1635303    200
2026-07-05T15:28:36.082744Z     INFO:     169.254.169.126:57662 - "GET
/api/jobs/e0a58416af3f4e8ab189f061a1635303 HTTP/1.1" 200 OK
2026-07-05T15:28:36.075492Z             GET
https://api.khelsutra.guru/api/jobs/e0a58416af3f4e8ab189f061a1635303    200
2026-07-05T15:28:30.062626Z     INFO:     169.254.169.126:60872 - "GET
/api/jobs/e0a58416af3f4e8ab189f061a1635303 HTTP/1.1" 200 OK
2026-07-05T15:28:30.053733Z             GET
https://api.khelsutra.guru/api/jobs/e0a58416af3f4e8ab189f061a1635303    200
2026-07-05T15:28:24.062100Z     INFO:     169.254.169.126:44182 - "GET
/api/jobs/e0a58416af3f4e8ab189f061a1635303 HTTP/1.1" 200 OK
2026-07-05T15:28:24.054165Z             GET
https://api.khelsutra.guru/api/jobs/e0a58416af3f4e8ab189f061a1635303    200
2026-07-05T15:28:18.058592Z     INFO:     169.254.169.126:44174 - "GET
/api/jobs/e0a58416af3f4e8ab189f061a1635303 HTTP/1.1" 200 OK
```

```
2026-07-05T15:28:18.051280Z             GET
https://api.khelsutra.guru/api/jobs/e0a58416af3f4e8ab189f061a1635303    200
2026-07-05T15:28:12.725198Z     INFO:     169.254.169.126:3052 - "GET
/api/jobs/e0a58416af3f4e8ab189f061a1635303 HTTP/1.1" 200 OK
2026-07-05T15:28:12.718203Z             GET
https://api.khelsutra.guru/api/jobs/e0a58416af3f4e8ab189f061a1635303    200
2026-07-05T15:28:07.611267Z     INFO:     169.254.169.126:3036 - "GET
/api/jobs/e0a58416af3f4e8ab189f061a1635303 HTTP/1.1" 200 OK
2026-07-05T15:28:07.603708Z             GET
https://api.khelsutra.guru/api/jobs/e0a58416af3f4e8ab189f061a1635303    200
```

671. assistant (2026-07-05T15:29:01.768Z)

A new job is live ? `e0a58416?` (your re-submit), SPA polling steadily. Getting its phase.

672. user (2026-07-05T15:29:09.781Z)

```
2026-07-05T15:27:32.137455Z             https://api.khelsutra.guru/api/process  202
2026-07-05T15:27:33.269222Z     INFO:     169.254.169.126:9206 - "POST /api/process HTTP/1.1"
202 Accepted
2026-07-05T15:27:44.941668Z     15:27:44 [INFO] backend.orchestration: Running validations for
gs://khelsutra-rally-serving/videos/uploads/3e7979010109459e888971a7dc2115a4/vlc-
record-2026-07-05-20h52m53s-Adarsh and Avi (last game).MP4-.mp4...
```

673. assistant (2026-07-05T15:29:28.963Z)

Status: your re-run is past the gate that killed the last one and is in the validator phase. Detail worth noting ? this is a **new upload**, not the old link: `vlc-record-?-Adarsh and Avi (last game).MP4` (your freshly-trimmed game), staged under a new upload id. Which means a second streamed browser upload just went through cleanly without ceremony.

Timeline: submit 202 at 15:27:32 ? drain acquired the staged file in **12 s** ? validators running since 15:27:44. With the threshold now at 8.0, blur should clear (the CP scored the previous 5.3K file 11.9; a VLC re-encode typically scores similar or higher). Next milestone is the L4 dispatch ? the watcher fires the moment the execution appears, and then your UI should show the heartbeat ticking through detection. I'll report at dispatch.

674. user (2026-07-05T15:30:22.036Z)

why such delay?

675. assistant (2026-07-05T15:30:45.546Z)

Checking exactly where it's stuck and how big this file is ? then I'll give you the honest why.

676. user (2026-07-05T15:30:57.590Z)

```
 648542264  2026-07-05T15:27:31Z  gs://khelsutra-rally-
serving/videos/uploads/3e7979010109459e888971a7dc2115a4/vlc-record-2026-07-05-20h52m53s-Adarsh
and Avi (last game).MP4-.mp4
TOTAL: 1 objects, 648542264 bytes (618.50MiB)
rally-worker-2mz8t
```

677. assistant (2026-07-05T15:31:28.586Z)

Found it. The file is **618 MB** and the drain has been inside the **blur validator** since 15:27:44 ? and this particular file is the pathological case for how that validator samples:

- Validation runs **on the control plane** (2 vCPU, no GPU) by design ? it's the fail-fast gate so a bad video never burns a paid GPU run. Your previous video took ~2.5 min there.
- The blur check grabs 15 frames by **random seeking** (`CAP_PROP_POS_FRAMES`). On a clean camera file, each seek lands near a keyframe and is quick. But this is a **`vlc-record-?` capture** ? VLC's record/trim cuts mid-GOP and produces files with sparse, irregular keyframes and messy seek indexes. Each of the 15 seeks can force the decoder to grind through thousands of

frames in software to reach the target. Fifteen of those on 2 vCPUs = the minutes you're watching.

It's bounded ? each seek does terminate ? so expect it to break through to the L4 dispatch rather than hang; the SPA meanwhile shows the "validating" stage with no movement, which is itself part of the #482 worker-side gap (no progress within a validator). The durable fixes are exactly what's already queued: [#497](https://github.com/Khelsutra/badminton-highlight-indexer/issues/497)'s blur rework (I'll add this VLC-seek pathology to it ? sequential sampling instead of random seeks kills this cost), and longer-term the #483 proxy makes all of this trivial. Watcher's still armed; I'll report the dispatch the moment it fires.

678. user (2026-07-05T15:31:57.126Z)

<task-notification>
<task-id>bpxw4yz1v</task-id>
<tool-use-id>toolu_01VHERPAcvKn2AnVB6rjtMDG</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\693b418f-8bf1-49e5-b443-b86a83235a63\tasks\bpxw4yz1v.output</output-file>
<status>completed</status>
<summary>Background command "Phase-2 watch: dispatch through completion" completed (exit code 0)</summary>
</task-notification>

679. user (2026-07-05T15:32:08.302Z)

```
=== MILESTONE: DISPATCHED:rally-worker-v9gn5 (poll 33) ===
2026-07-05T15:31:21.052126Z     GET
https://api.khelsutra.guru/api/jobs/e0a58416af3f4e8ab189f061a1635303    200
2026-07-05T15:31:15.045049Z     GET
https://api.khelsutra.guru/api/jobs/e0a58416af3f4e8ab189f061a1635303    200
2026-07-05T15:31:09.065562Z     GET
https://api.khelsutra.guru/api/jobs/e0a58416af3f4e8ab189f061a1635303    200
2026-07-05T15:31:03.595497Z     GET
https://api.khelsutra.guru/api/jobs/e0a58416af3f4e8ab189f061a1635303    200
```

680. user (2026-07-05T15:32:32.524Z)

```
2026-07-05T15:31:26.533392Z
2026-07-05T15:31:26.770801Z     15:31:26 [INFO] compute.cloud_run: cloud-run: dispatched
job=rally-worker exec=projects/gen-lang-client-0306026826/locations/asia-southeast1/jobs/rally-
worker/executions/rally-worker-v9gn5; bucket-polling gs://khelsutra-rally-
serving/_dispatch/fc1543335ddd42c99496045d9dbdc519.done
2026-07-05T15:31:26.770835Z     15:31:26 [INFO] compute.cloud_run: cloud-run-infer: waiting
(poll_every=10s, max_wait=3900s)
```

681. user (2026-07-05T15:32:55.808Z)

Command running in background with ID: bk8jzbk6v. Output is being written to: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\693b418f-8bf1-49e5-b443-b86a83235a63\tasks\bk8jzbk6v.output. You will be notified when it completes. To check interim output, use Read on that file path.

682. assistant (2026-07-05T15:33:09.391Z)

Your run is now exactly where it should be:

Upload #2 (618 MB VLC capture)	? streamed to the bucket, no drama
Blur gate at the patched threshold	? passed (validators 3m42s ? the VLC seek pathology, now documented on #497)
Dispatch	? `rally-worker-v9gn5` at 15:31:26, poll budget showing the new `max_wait=3900s`
Now	L4 detecting; your UI ticks "segmenting (wasb_hybrid) on cloud GPU ? N min elapsed"

Expected detection time depends on the clip's length and resolution ? for a ~10-minute game at 30 fps, roughly 10?15 minutes. The watcher fires when the rallies land (or on anything anomalous), and after that it's your pick-and-compile moment ? the last untested stretch of the CUJ.

683. user (2026-07-05T15:35:27.786Z)

can you tail some logs fro the GPU regarding progress?

684. user (2026-07-05T15:36:55.338Z)

2026-07-05T15:31:35.912293Z GCSFuse is mounted with bucket khelsutra-rally-serving. Enable GCSFuse logging with mount options for more logs
2026-07-05T15:31:35.967473Z time="05/07/2026 03:31:35.965850" severity=INFO message="GCSFuse Config" mount-id=khelsutra-rally-serving-b4be9605 "CLI Flags"="map[app-name:serverless dir-mode:777 file-mode:666 foreground:true gid:1000 ignore-interrupts:true implicit-dirs:true log-format:text o:[allow_other] uid:1000]"
2026-07-05T15:31:35.967895Z time="05/07/2026 03:31:35.965898" severity=INFO message="GCSFuse Config" mount-id=khelsutra-rally-serving-b4be9605 "Full Config"="&{AppName:serverless CacheDir: CloudProfiler:{AllocatedHeap:true Cpu:true Enabled:false Goroutines:false Heap:true Label:gcsfuse-0.0.0 Mutex:false ServiceName:gcsfuse} Debug:{ExitOnInvariantViolation:false Fuse:false Gcs:false LogMutex:false} DisableAutoconfig:false DisableListAccessCheck:true DummyIo:{Enable:false PerMbLatency:0s ReaderLatency:0s} EnableAtomicRenameObject:true EnableGoogleLibAuth:true EnableHns:true EnableNewReader:true EnableStandardSymlinks:true EnableTypeCacheDeprecation:true EnableUnsupportedPathSupport:true FileCache:{CacheFileForRangeRead:false DownloadChunkSizeMb:200 EnableCrc:false EnableExperimentalSharedChunkCache:false EnableODirect:false EnableParallelDownloads:false ExcludeRegex: ExperimentalDisableSizeCalculationFix:false ExperimentalEnableChunkCache:false ExperimentalParallelDownloadsDefaultOn:true IncludeRegex: MaxParallelDownloads:16 MaxSizeMb:-1 ParallelDownloadsPerFile:16 SharedCacheChunkSizeMb:8 WriteBufferSize:4194304} FileSystem:{CongestionThreshold:0 DirMode:511 DisableParallelDirops:false EnableKernelReader:false ExperimentalEnableDentryCache:false ExperimentalEnablePirlo:false ExperimentalEnableReaddirplus:false ExperimentalODirect:false FileMode:438 FuseOptions:[allow_other] Gid:1000 IgnoreInterrupts:true InactiveMrdCacheSize:1000 KernelListCacheTtlSecs:0 KernelParamsFile: MaxBackground:0 MaxReadAheadKb:0 PreconditionErrors:true RenameDirLimit:0 TempDir: Uid:1000} Foreground:true GcsAuth:{AnonymousAccess:false KeyFile: ReuseTokenFromUrl:true TokenUrl:} GcsConnection:{BillingProject: ClientProtocol:http1 CustomEndpoint: EnableHttpDnsCache:true ExperimentalEnableJsonRead:false ExperimentalLocalSocketAddress: GrpcConnPoolSize:1 GrpcPathStrategy:direct-path-with-fallback HttpClientTimeout:0s LimitBytesPerSec:-1 LimitOpsPerSec:-1 MaxConnsPerHost:0 MaxIdleConnsPerHost:100 SequentialReadSizeMb:200} GcsRetries:{ChunkRetryDeadlineSecs:120 ChunkTransferTimeoutSecs:10 ExperimentalNonrapidFolderApiStallRetry:false MaxRetryAttempts:9223372036854775807 MaxRetrySleep:30s Multiplier:2 ReadStall:{Enable:true InitialReqTimeout:20s MaxReqTimeout:20m0s MinReqTimeout:1.5s ReqIncreaseRate:15 ReqTargetPercentile:0.99}} ImplicitDirs:true List:{EnableEmptyManagedFolders:false} Logging:{FilePath: Format:text LogRotate:{BackupFileCount:10 Compress:true MaxFileSizeMb:512} Severity:INFO WireLog:} MachineType: MetadataCache:{DeprecatedStatCacheCapacity:20460 DeprecatedStatCacheTtl:1m0s DeprecatedTypeCacheTtl:1m0s EnableMetadataPrefetch:false EnableNonexistentTypeCache:false ExperimentalMetadataPrefetchOnMount:disabled MetadataPrefetchEntriesLimit:5000 MetadataPrefetchMaxWorkers:10 NegativeTtlSecs:5 StatCacheMaxSizeMb:34 TtlSecs:60 TypeCacheMaxSizeMb:4} Metrics:{BufferSize:256 CloudMetricsExportIntervalSecs:0 ExperimentalEnableGrpcMetrics:true PrometheusPort:0 StackdriverExportInterval:0s UseNewNames:false Workers:3} Mrd:{PoolSize:4} OnlyDir: Profile: Read:{BlockSizeMb:16 EnableBufferedRead:false GlobalMaxBlocks:40 InactiveStreamTimeout:10s MaxBlocksPerHandle:20 MinBlocksPerHandle:4 RandomSeekThreshold:3 StartBlocksPerHandle:1} Trace:{Exporters:[gcpexporter] ProjectId: SamplingRatio:0} WorkloadInsight:{ForwardMergeThresholdMb:0 OutputFile: Visualize:false} Write:{BlockSizeMb:32 CreateEmptyFile:false EnableRapidAppends:true EnableStreamingWrites:true FinalizeFileForRapid:false GlobalMaxBlocks:4 MaxBlocksPerFile:1}}"
2026-07-05T15:31:36.300992Z time="05/07/2026 03:31:35.970082" severity=INFO message="Creating Storage handle..." mount-id=khelsutra-rally-serving-b4be9605
2026-07-05T15:31:36.301261Z time="05/07/2026 03:31:35.970118" severity=INFO message="UserAgent = gcsfuse/3.9.2 (Go version go1.26.4) (GPN:gcsfuse-serverless) (Cfg:0:0:0:1:0:0) (mount-id:khelsutra-rally-serving-b4be9605)\n" mount-id=khelsutra-rally-

```
serving-b4be9605
2026-07-05T15:31:36.635084Z     time="05/07/2026 03:31:35.970187" severity=INFO message="Calling
cred.UniverseDomain request for \"credentials\" with deadline=30s" mount-id=khelsutra-rally-
serving-b4be9605
2026-07-05T15:31:36.635480Z     time="05/07/2026 03:31:35.970494" severity=INFO message="Success
in fetching cred.UniverseDomain" mount-id=khelsutra-rally-serving-b4be9605
2026-07-05T15:31:36.635495Z     time="05/07/2026 03:31:35.971003" severity=INFO
message="Creating a new server...\n" mount-id=khelsutra-rally-serving-b4be9605
2026-07-05T15:31:36.635504Z     time="05/07/2026 03:31:35.971026" severity=INFO message="MRD
cache (LRU-based) enabled with maxSize=1000 (pools closed-file MRDs)" mount-id=khelsutra-rally-
serving-b4be9605
2026-07-05T15:31:36.635510Z     time="05/07/2026 03:31:35.971030" severity=INFO message="Set up
root directory for bucket khelsutra-rally-serving" mount-id=khelsutra-rally-serving-b4be9605
2026-07-05T15:31:36.635804Z     time="05/07/2026 03:31:35.971044" severity=INFO
message="GetStorageLayout <- (khelsutra-rally-serving)" mount-id=khelsutra-rally-
serving-b4be9605
2026-07-05T15:31:36.635818Z     time="05/07/2026 03:31:35.971066" severity=INFO message="Calling
GetStorageLayout request for \"projects/_/buckets/khelsutra-rally-serving/storageLayout\" with
deadline=30s" mount-id=khelsutra-rally-serving-b4be9605
2026-07-05T15:31:36.636150Z     time="05/07/2026 03:31:36.052010" severity=INFO
message="GetStorageLayout -> (khelsutra-rally-serving) 80 msec" mount-id=khelsutra-rally-
serving-b4be9605
2026-07-05T15:31:36.636158Z     time="05/07/2026 03:31:36.052096" severity=INFO message="Calling
cred.UniverseDomain request for \"credentials\" with deadline=30s" mount-id=khelsutra-rally-
serving-b4be9605
2026-07-05T15:31:36.636375Z     time="05/07/2026 03:31:36.052482" severity=INFO message="Success
in fetching cred.UniverseDomain" mount-id=khelsutra-rally-serving-b4be9605
2026-07-05T15:31:36.636381Z     time="05/07/2026 03:31:36.056022" severity=INFO
message="Mounting file system \"khelsutra-rally-serving\"..." mount-id=khelsutra-rally-
serving-b4be9605
2026-07-05T15:31:36.636636Z     time="05/07/2026 03:31:36.056910" severity=INFO message="File
system has been successfully mounted." mount-id=khelsutra-rally-serving-b4be9605
2026-07-05T15:31:45.224974Z     /usr/local/lib/python3.10/dist-
packages/google/api_core/_python_version_support.py:255: FutureWarning: You are using a Python
version (3.10.12) which Google will stop supporting in new releases of google.api_core once it
reaches its end of life (2026-10-04). Please upgrade to the latest Python version, or at least
Python 3.11, to continue receiving updates for google.api_core past that date.
2026-07-05T15:31:45.225007Z       warnings.warn(message, FutureWarning)
2026-07-05T15:31:54.718204Z     [worker] pulled gs://khelsutra-rally-
serving/videos/uploads/3e7979010109459e888971a7dc2115a4/vlc-record-2026-07-05-20h52m53s-Adarsh
and Avi (last game).MP4-.mp4 -> /tmp/rally-worker-mr30qnkg/in.mp4
2026-07-05T15:34:03.685749Z     15:34:03 [INFO] backend.pipeline.segmenters.trajectory_hybrid:
WASB in FULLY-LOCAL mode (skip_ai_handoff=True): candidate windows will be emitted as rallies;
no gemini handoff, no API key needed.
2026-07-05T15:34:03.766485Z     15:34:03 [INFO] backend.pipeline.segmenters.trajectory_hybrid:
WASB: detector_impl=native ? Linux-native runner (no WSL).
2026-07-05T15:34:14.346803Z     15:34:14 [INFO] backend.pipeline.segmenters.trajectory_hybrid:
WASB detector env OK: torch 1.11.0+cu113 cuda True
2026-07-05T15:34:14.353997Z     15:34:14 [INFO] backend.pipeline.detectors.native_wasb_runner:
Running native WASB inference (device=cuda) on in.mp4 ...
```

685. assistant (2026-07-05T15:37:25.088Z)

Here's the GPU's story so far ? including an honest limitation you're about to recognize:

```
15:31:35  GCS FUSE mounted (bucket ? /gcs)
15:31:54  [worker] pulled ?/vlc-record-?(last game).MP4 ? /tmp (618 MB in ~9 s)
15:34:03  WASB in FULLY-LOCAL mode (skip_ai_handoff=True) ? pin honored
15:34:03  detector_impl=native ? Linux-native runner
15:34:14  WASB detector env OK: torch 1.11.0+cu113 cuda True
15:34:14  Running native WASB inference (device=cuda) on in.mp4 ...
          ?? silence since ??
```

That silence is **exactly the #482 worker-side gap** we scoped out of tonight's P6 slice: the native runner announces the GPU pass once and says nothing until it finishes. There is genuinely nothing more to tail from the GPU right now ? the per-frame "N% (~M min left)" line is the follow-up that needs a real-GPU verification pass before it ships. Your live signals in the meantime are the CP heartbeat in the UI (elapsed minutes, updating every poll) and the two-minute mark between pull and inference start (that's the worker's own validation+prep on the clip).

Rough ETA math: WASB runs ~25 fps on the L4, so a ~10-minute 30 fps game ? 12 minutes of GPU; ~15 minutes of footage ? 18. Inference started at 15:34, so expect the done-marker somewhere in the 15:46?15:52 window if the clip is in that length range. The watcher's on the marker ? I'll tell you the moment your rallies land.

686. user (2026-07-05T15:39:22.757Z)

check, it found 5 rallies

687. assistant (2026-07-05T15:39:39.221Z)

Checking the completion from my side ? timings, the ingest, and whether 5 rallies squares with the clip's actual length.

688. user (2026-07-05T15:39:46.227Z)

```
2026-07-05T15:39:05.530663Z
2026-07-05T15:39:02.004630Z     Container called exit(0).
2026-07-05T15:39:00.843411Z     15:39:00 [INFO] backend.worker: [worker] wrote completion marker
-> gs://khelsutra-rally-serving/_dispatch/fc1543335ddd42c99496045d9dbdc519.done
2026-07-05T15:39:00.627805Z         gs://khelsutra-rally-
serving/outputs/1f072ae8129deca25fa2a6ed6e21e34d/report.json
2026-07-05T15:39:00.627799Z         gs://khelsutra-rally-
serving/outputs/1f072ae8129deca25fa2a6ed6e21e34d/intervals.csv
2026-07-05T15:39:00.627764Z     [worker] infer done: 5 segment(s); pushed 2 artifact(s):
2026-07-05T15:39:00.216575Z     15:39:00 [INFO] backend.pipeline.segmenters.trajectory_hybrid:
Phase 3 SKIPPED (fully-local): emitted 5 candidate windows as rallies (no AI handoff).
2026-07-05T15:39:00.213079Z     15:39:00 [INFO] backend.pipeline.segmenters.trajectory_hybrid:
[WASB rally] 00:01:17-00:01:20 (3.0s) | frames=169 peak_v=1.023 mean_v=0.294
2026-07-05T15:39:00.213073Z     15:39:00 [INFO] backend.pipeline.segmenters.trajectory_hybrid:
[WASB rally] 00:01:08-00:01:11 (3.3s) | frames=156 peak_v=4.564 mean_v=0.266
2026-07-05T15:39:00.213031Z     15:39:00 [INFO] backend.pipeline.segmenters.trajectory_hybrid:
[WASB rally] 00:00:52-00:00:58 (6.5s) | frames=354 peak_v=2.191 mean_v=0.255
2026-07-05T15:39:00.213003Z     15:39:00 [INFO] backend.pipeline.segmenters.trajectory_hybrid:
[WASB rally] 00:00:38-00:00:43 (4.4s) | frames=239 peak_v=3.271 mean_v=0.325
2026-07-05T15:39:00.212997Z     15:39:00 [INFO] backend.pipeline.segmenters.trajectory_hybrid:
[WASB rally] 00:00:10-00:00:14 (4.9s) | frames=276 peak_v=1.112 mean_v=0.303
```

689. user (2026-07-05T15:40:17.598Z)

status: success | segments: 5 | video_uri: gs://khelsutra-rally-serving/videos/uploads/3e7979010109459e888971a7dc2115a4/vlc-record-2026-07-05-2

690. user (2026-07-05T15:40:36.258Z)

Command running in background with ID: b2uuvw8jp. Output is being written to: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\693b418f-8bf1-49e5-b443-b86a83235a63\tasks\b2uuvw8jp.output. You will be notified when it completes. To check interim output, use Read on that file path.

691. assistant (2026-07-05T15:40:50.592Z)

Watching for your compile. The scoreboard for this CUJ so far ? every failure from this morning, now passing on the same journey:

| This morning (old build) | Tonight (rev 00015) |

|---|---|
95 MB parts ? 413 at the edge	60+24 parts × 24 MB, all 200s, two uploads
2.68 GB assembled in RAM ? **OOM, restart, results lost**	Streamed to the bucket; instance untouched
Silent 10-min spinner ? SPA gave up	Heartbeat ticking; SPA stayed with it
Wrong segmenter risk	`wasb_hybrid` pinned ? visible in health, logs, and the run
Results unrecoverable after restart	`report.json` carries `video_uri`; everything re-hydrates
(new tonight) blur gate false-rejected real footage	Measured, patched live (threshold 8.0), root cause filed as #497

Pick your five rallies and compile ? reel download is the final checkbox.

692. user (2026-07-05T15:40:52.175Z)

<task-notification>
<task-id>b2uuvw8jp</task-id>
<tool-use-id>toolu_01EcJNMLf3KV899jSA6Ro4TT</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\693b418f-8bf1-49e5-b443-b86a83235a63\tasks\b2uuvw8jp.output</output-file>
<status>completed</status>
<summary>Background command "Watch for compile and reel download" completed (exit code 0)</summary>
</task-notification>

693. user (2026-07-05T15:41:03.838Z)

=== MILESTONE: COMPILE_SUBMITTED (poll 1) ===
2026-07-05T15:40:14.497223Z POST https://api.khelsutra.guru/api/compile 202

694. user (2026-07-05T15:41:18.300Z)

Command running in background with ID: bl713ctx4. Output is being written to: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\693b418f-8bf1-49e5-b443-b86a83235a63\tasks\bl713ctx4.output. You will be notified when it completes. To check interim output, use Read on that file path.

695. assistant (2026-07-05T15:41:27.055Z)

Stitch in progress. When the reel lands you'll get the download link in the UI (it 307s to a signed URL straight from the bucket ? the >32 MiB ceiling died back in #463). I'll confirm from the logs when the download fires.

696. user (2026-07-05T15:41:28.762Z)

<task-notification>
<task-id>bl713ctx4</task-id>
<tool-use-id>toolu_017kaufPtrcjUrDvBJJ9dQTj</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\693b418f-8bf1-49e5-b443-b86a83235a63\tasks\bl713ctx4.output</output-file>
<status>completed</status>
<summary>Background command "Watch for reel download" completed (exit code 0)</summary>
</task-notification>

697. user (2026-07-05T15:41:39.626Z)

=== MILESTONE: COMPILE_SUBMITTED (poll 1) ===

698. assistant (2026-07-05T15:41:50.231Z)

Watcher re-fired on the still-recent compile POST ? the stitch is ongoing. Checking its actual state, then re-arming for the download only.

699. user (2026-07-05T15:41:57.371Z)

```
2026-07-05T15:41:47.229292Z     15:41:47 [INFO] backend.pipeline.stitcher.compiler: Dynamic
highlight reel saved: /tmp/apphome/output/first-e2e-working-reel.mp4 (60.1 MB)
2026-07-05T15:41:46.822394Z     15:41:46 [INFO] backend.pipeline.stitcher.compiler: Stitching
clips together...
2026-07-05T15:41:46.821822Z     15:41:46 [INFO] backend.pipeline.stitcher.compiler: Successfully
trimmed 5/5 clips. Preparing to stitch...
2026-07-05T15:41:36.558937Z     15:41:36 [INFO] backend.pipeline.stitcher.compiler: Trimming
Segment #5/5 [77.47s - 80.45s] -> [76.975s - 80.464s] (duration: 3.49s)
2026-07-05T15:41:36.558908Z     15:41:36 [INFO] backend.pipeline.stitcher.compiler: Segment #5/5
[77.47s - 80.45s]: Padded end (80.95s) exceeds video duration. Clamping to 80.46s (end padding
partially applied).
2026-07-05T15:41:22.828335Z     15:41:22 [INFO] backend.pipeline.stitcher.compiler: Trimming
Segment #4/5 [68.28s - 71.60s] -> [67.776s - 72.100s] (duration: 4.32s)
2026-07-05T15:41:00.458633Z     15:41:00 [INFO] backend.pipeline.stitcher.compiler: Trimming
Segment #3/5 [52.14s - 58.67s] -> [51.643s - 59.173s] (duration: 7.53s)
2026-07-05T15:40:42.954983Z     15:40:42 [INFO] backend.pipeline.stitcher.compiler: Trimming
Segment #2/5 [38.90s - 43.30s] -> [38.397s - 43.796s] (duration: 5.40s)
```

700. user (2026-07-05T15:42:24.789Z)

```
(PowerShell completed with no output)
```

701. user (2026-07-05T15:42:43.992Z)

```
2026-07-05T15:41:47.229292Z     15:41:47 [INFO] backend.pipeline.stitcher.compiler: Dynamic
highlight reel saved: /tmp/apphome/output/first-e2e-working-reel.mp4 (60.1 MB)
  63046928  2026-07-05T15:41:48Z  gs://khelsutra-rally-
serving/highlights/1f072ae8129deca25fa2a6ed6e21e34d/first-e2e-working-reel.mp4
TOTAL: 1 objects, 63046928 bytes (60.13MiB)
```

702. user (2026-07-05T15:43:23.307Z)

```
HEAD is now at 6ab5a71 docs(runlog): CP-rebuild P9 executed ? rev 00012/00014, battery green,
trackers updated (#496)
```

703. user (2026-07-05T15:44:04.197Z)

```
spec §9 + changelog updated
alpha DoD box 1 ticked with evidence
[check-md-links] clean ? no broken internal links in live docs.
warning: in the working copy of 'docs/ALPHA_LAUNCH_READINESS.md', LF will be replaced by CRLF
the next time Git touches it
warning: in the working copy of 'docs/CONTROL_PLANE_DURABLE_REBUILD.md', LF will be replaced by
CRLF the next time Git touches it
warning: in the working copy of 'docs/ALPHA_LAUNCH_READINESS.md', LF will be replaced by CRLF
the next time Git touches it
warning: in the working copy of 'docs/CONTROL_PLANE_DURABLE_REBUILD.md', LF will be replaced by
CRLF the next time Git touches it
[docs/cuj-pass-evidence 1f6e2dd] docs: record the owner's §1.1 browser CUJ PASS ? DoD #1
satisfied (rev 00015-kp9)
 2 files changed, 19 insertions(+), 9 deletions(-)
 * [new branch]      docs/cuj-pass-evidence -> docs/cuj-pass-evidence
```

704. user (2026-07-05T15:44:24.142Z)

```
https://github.com/Khelsutra/badminton-highlight-indexer/pull/498
```

705. user (2026-07-05T15:44:52.049Z)

```
current-state checkpointed
handoff heading updated
```

706. assistant (2026-07-05T15:45:10.425Z)

Everything's recorded. [PR #498](https://github.com/Khelsutra/badminton-highlight-indexer/pull/498) carries the evidence into the trackers (spec §9 + alpha DoD #1 ticked with the journey record, the ?2.5 GB box honestly annotated), and memory is checkpointed with the full story ? including the one piece of live debt the next session must not lose: rev `00015-kp9`'s blur threshold is a config patch that the #497 fix PR needs to fold into the generator, per the same D5 rule that saved the segmenter pin.

The day in one line: this morning your CUJ died at the first upload; tonight the same journey ? recorded on your phone, trimmed in VLC, uploaded from your browser ? ended with `first-e2e-working-reel.mp4` sitting durably in the bucket, and even the one new bug found along the way (the blur false-reject) was measured, patched live mid-run, and filed with its durable fix.

#498 awaits review, and then the board is down to one item that's always been yours: **GO/NO-GO on the first alpha wave.** Congratulations on the reel. ?

707. user (2026-07-05T16:01:00.407Z)

Check for LGTM and merge

708. user (2026-07-05T16:01:14.441Z)

```
mergeable=MERGEABLE checks=SUCCESS comments=1
LGTM
```

709. user (2026-07-05T16:02:03.070Z)

```
From https://github.com/Khelsutra/badminton-highlight-indexer
   6ab5a71..2a8a931  master     -> origin/master
2a8a931 docs: record the owner's §1.1 browser CUJ PASS ? DoD #1 satisfied (rev 00015-kp9) (#498)
Deleted branch docs/cuj-pass-evidence (was 1f6e2dd).
```

710. user (2026-07-05T16:02:41.485Z)

```
35:| D5 | **P10/P11 corpus promotion and train/re-eval are alpha-plus unless owner reopens
D13.** | sibling `khelsutra/docs/STUDIO_MEDIA_LIFECYCLE.md`; verified 2026-07-03 | First alpha
can use manual/offline corpus promotion; automated flywheel follows after serving and gates are
stable. |
85:| P10/P11 expands scope before hosted alpha is usable | Corpus flywheel is valuable but can
consume the alpha launch window | Keep P10/P11 behind explicit owner reopen unless alpha lane is
already stable |
112:| A12 | Reopen or defer P10/P11 corpus flywheel: `PATCH /api/segments`, collector promotion,
`train`/`reeval` job, delta-F1 surface, and alignment with `COMPUTE_DECOUPLED_SERVING` M11 |
`khelsutra` + `badminton-highlight-indexer` + collector | A1, A3 | Owner D13 reopen
(`khelsutra/docs/STUDIO_MEDIA_LIFECYCLE.md`) | GATED | - |
228:1. **Owner:** Is the first alpha allowed to use manual/offline corpus promotion, or should
P10/P11 be reopened before any tester wave?
244:- [ ] P10/P11 are either reopened with a committed sequence or explicitly deferred for
alpha-plus.
9:> end?to?end. ? REMAINING = owner?gated only: **P10/P11** (corpus promotion + train/re?eval,
DEFERRED D13) + the **`L`**
65:| D13 | **Annotate?in?UI (P8) is APPROVED** (owner 2026?07?01) ? build the P7?P8 track now.
Ships behind a **prod feature flag** (default?OFF) until consented / no?minors footage is in
hand; demos run on the mock/synthetic clip. **Contribute?train (P10/P11) is DEFERRED** by the
owner. | Unblocks the marquee annotation flow while keeping real?footage exposure
owner?controlled; the training loop waits |
71:> here remains open ? only owner?gated P10/P11 + the `L` beta follow?ups.*
161:| P8 | **Annotate?in?Studio**: SPA `AnnotatePanel` over the vendored core (mark start/end ·
reason · shots · save/undo · loud save?state), flagged `?annotate=1` (D13); screencast `03`.
**P8a REAL engine endpoints DONE** ? `POST /api/annotations` writes the verbatim CSV to
`output/labels/<video_id>.rallies.csv` (MD5?keyed) + `GET /api/source/{id}` (Range?capable)
serves a **frame?preserving proxy** (D4, [engine #406](https://github.com/Khelsutra/badminton-
```

highlight-indexer/pull/406) ? `scale=-2:720`, timeline/duration?parity verified, warmed in background; labels stay keyed to the master MD5), source fallback. | khelsutra + engine | P6,P7 | ? APPROVED (D13; prod flag) | ? | SPA [#128](https://github.com/avidullu/khelsutra/pull/128) · engine [#405](https://github.com/Khelsutra/badminton-highlight-indexer/pull/405) · proxy [#406](https://github.com/Khelsutra/badminton-highlight-indexer/pull/406) |
162:| P9 | **Collector library + fields (DONE):** MD5 in export manifest ([col #51](https://github.com/Khelsutra/sports-data-collector/pull/51) ? the deterministic join key) · callable `import_annotations` + **maturity gate** ([col #52](https://github.com/Khelsutra/sports-data-collector/pull/52) ? `status=STAGING`=needs?review, no auto?promote) · **write?safety**: vault atomic manifest write ([vault #3](https://github.com/Khelsutra/rally-corpus-vault/pull/3)) + collector SQLite `busy_timeout` ([col #53](https://github.com/Khelsutra/sports-data-collector/pull/53)). *`labeled_window` DEFERRED ? premature: nothing captures a reviewed window yet (annotator records rallies, not a span) & the eval that'd consume it is deferred (P10/P11).* | sports?data?collector + vault | ? | no | ? | #51 · #52 · #53 · vault #3 |
163:| P10 | **Corpus promotion glue**: collector export ? `storagectl add`; cross?repo `video_id==compute_video_id` guard test | khelsutra/deploy + vault | P9 | ? DEFERRED (D13) | ? | ? |

711. user (2026-07-05T16:03:37.623Z)

1.5GB scratch video deleted

712. user (2026-07-05T16:03:49.684Z)

```
    71572       0       0       6036  ?              0 21:33:49 C:\Program
Files\Git\bin\bash.exe
     1912       1    1912      30756  ?         197609 21:33:49 /usr/bin/bash
   101220       0       0      35684  ?              0 21:33:50 C:\Program
Files\Git\usr\bin\bash.exe
--- harness task check: all completed per notifications ---
```

713. assistant (2026-07-05T16:03:59.970Z)

No stray processes ? every watcher already exited on its own milestone (all completion notifications came through). Now the repo clean-slate.

714. user (2026-07-05T16:04:11.700Z)

```
error: failed to delete '.git/worktrees/a1-master-worktree': Permission denied
error: failed to delete '.git/worktrees/fix-worktree': Permission denied
error: failed to delete '.git/worktrees/master-compare': Permission denied
error: failed to delete '.git/worktrees/wt-droplet-docrot': Permission denied
error: failed to delete '.git/worktrees/wt-p5b-ceilings': Permission denied
wt-157-before
wt-157-ingest-ux
wt-465fix
wt-a1-runlog
wt-cloudrun-dispatch
wt-cov-base
wt-dockerfile-auth
wt-drain-concurrency
wt-input-jail
wt-pr158-screenshot-assets
wt-pr481-review
wt-signed-download
wt-upload-staging
wt-yaml-gen
```

715. user (2026-07-05T16:04:47.805Z)

```
all worktree dirs gone
5
```

716. user (2026-07-05T16:04:59.538Z)

```
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer
6ab5a71 [master]
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/cfg-wt        20b2d5a [claude/wasb-
config-validation]
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/compile-wt    b09a1d4 [claude/compile-
reliability]
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/pr458-pre-wt  355dad6 (detached HEAD)
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/pr458-wt      5de1ef2 (detached HEAD)
---
```

master...origin/master [behind 1]

feat/processing-queue-and-dedup...origin/feat/processing-queue-and-dedup

717. user (2026-07-05T16:05:21.175Z)

```
2a8a931 docs: record the owner's §1.1 browser CUJ PASS ? DoD #1 satisfied (rev 00015-kp9) (#498)
+ claude/compile-reliability
  claude/serving-loud-config-failure
+ claude/wasb-config-validation
  claude/wasb-gpu-batching
  codex/a2-golden-manifest-portability
  codex/a4-n15-nightly-baseline
  codex/a5-heuristic-n15-floor
  codex/a6-ablation-cli
---
90d1d90 Raise SPA poll ceilings 10 -> 60 min for honest cloud job lengths (CP-rebuild P5b)
(#162)
  assets-pr-158-screenshots
+ claude/zealous-pike-7d0a85
  feat/processing-queue-and-dedup
  fix/157-ingest-ux
* master
```

718. user (2026-07-05T16:05:33.941Z)

```
error: branch 'codex/a10-a11-alpha-guardrails' not found
Deleted branch codex/a2-golden-manifest-portability (was 7f6f7c5).
Deleted branch codex/a4-n15-nightly-baseline (was 60506b2).
Deleted branch codex/a5-heuristic-n15-floor (was c27ef81).
Deleted branch codex/a6-ablation-cli (was 8ac71bb).
Deleted branch claude/serving-loud-config-failure (was bcf47bf).
Deleted branch claude/wasb-gpu-batching (was 66e3dc4).
--- remaining engine branches ---
+ claude/compile-reliability
+ claude/wasb-config-validation
  codex/a7-a9-quality-chain
  docs/a1-deploy-runlog
  docs/cp-durable-rebuild-spec
  fix/cloudrun-dispatch-client
  fix/dockerfile-auth-extra
  fix/drain-concurrency
  fix/highlights-signed-url
  fix/upload-gcs-staging
Deleted branch fix/157-ingest-ux (was 5ddb81b).
--- remaining khelsutra branches ---
  assets-pr-158-screenshots
+ claude/zealous-pike-7d0a85
  feat/processing-queue-and-dedup
* master
```

719. user (2026-07-05T16:05:48.273Z)

```
error: branch 'codex/a7-a9-quality-chain' not found
error: branch 'docs/a1-deploy-runlog' not found
error: branch 'docs/cp-durable-rebuild-spec' not found
error: branch 'fix/cloudrun-dispatch-client' not found
error: branch 'fix/dockerfile-auth-extra' not found
error: branch 'fix/drain-concurrency' not found
error: branch 'fix/highlights-signed-url' not found
error: branch 'fix/upload-gcs-staging' not found
error: branch 'fix/allowed-video-root-uri' not found
error: branch 'feat/cloudrun-service-manifest' not found
--- final engine branches ---
  assets-pr-158-screenshots
+ claude/zealous-pike-7d0a85
  feat/processing-queue-and-dedup
* master
```

720. user (2026-07-05T16:06:07.783Z)

```
Deleted branch codex/a7-a9-quality-chain (was f49a110).
Deleted branch docs/a1-deploy-runlog (was 5bff264).
Deleted branch docs/cp-durable-rebuild-spec (was 9bad982).
--- final engine branches ---
+ claude/compile-reliability
+ claude/wasb-config-validation
  fix/cloudrun-dispatch-client
  fix/dockerfile-auth-extra
  fix/drain-concurrency
  fix/highlights-signed-url
  fix/upload-gcs-staging
* master
--- final states ---
```

master...origin/master

master...origin/master

721. user (2026-07-05T16:06:17.462Z)

```
Deleted branch fix/cloudrun-dispatch-client (was ac6cf85).
hint: Disable this message with "git config advice.forceDeleteBranch false"
hint: Disable this message with "git config advice.forceDeleteBranch false"
hint: Disable this message with "git config advice.forceDeleteBranch false"
hint: Disable this message with "git config advice.forceDeleteBranch false"
+ claude/compile-reliability
+ claude/wasb-config-validation
  fix/dockerfile-auth-extra
  fix/drain-concurrency
  fix/highlights-signed-url
  fix/upload-gcs-staging
* master
```

722. assistant (2026-07-05T16:06:33.889Z)

```
Those four were squash-merged (PRs #467?#470, all merged 2026-07-04) so `-d` can't see the
ancestry ? force-deleting with that verification on record.
```

723. user (2026-07-05T16:06:36.591Z)

```
Deleted branch fix/dockerfile-auth-extra (was 9ed5d42).
Deleted branch fix/drain-concurrency (was 8534077).
Deleted branch fix/highlights-signed-url (was d3b0e64).
Deleted branch fix/upload-gcs-staging (was d3a9629).
--- FINAL ---
```

```
+ claude/compile-reliability
+ claude/wasb-config-validation
* master
  assets-pr-158-screenshots
+ claude/zealous-pike-7d0a85
  feat/processing-queue-and-dedup
* master
```

724. user (2026-07-05T16:07:04.522Z)

```
9       # Session handoff ? khelsutra-guru
10
11      > Last refreshed: 2026-07-05 (post-P9). **THE REBUILD IS DEPLOYED**: CP rev
**00014-7hk** (via 00012-wxk) ? 8Gi, digest `3bfefaf6?` (master `257f469`), pins live
(`/api/health` shows `wasb_hybrid`), edge-auth ON, uploads-lifecycle rule set. **Battery GREEN**
(runlog in PR [#496](https://github.com/Khelsutra/badminton-highlight-indexer/pull/496), in
review): wiped-DB replay ? 29 rallies re-hydrated · streamed staging md5-identity · cache re-
submit done-cached in 11 s zero-GPU · heartbeat live through a real 48-segment/14m04s detection
(exec `rally-worker-2mz8t`) · `/api/version` spa_commit `90d1d90` · posture 200/401/302. ALL
P-rows merged same-day on verified LGTMs (P1 #489 · P2 #490 · P3 #494 · P4 #492 · P5a #491 · P5b
kh#162 · P6 #493 · P8 #495). Protocol: listen?address?**merge on LGTM** ([[working-agreement-
merging]]). **#496 MERGED (`6ab5a71`) ? runlog + trackers on master; #480/#471/#485 CLOSED with
evidence; #484 kept open (drain-tier composite follow-up). THE ONLY REMAINING STEP: owner's §1.1
five-minute browser CUJ on this build** (the last three §9 boxes) ? GO/NO-GO. Two honest
findings in runlog §5 (composite originals lack md5Hash; fresh-DB drain skips the bucket cache ?
follow-up on #484). Warm backlog: #483/P7 (owner-declared) · #482 worker-side.
12      > See [[current-state]] for the blow-by-blow.
13
14      ## ? YOU ARE HERE ? **? CUJ PASSED (DoD #1): the owner completed upload?rallies?reel on
rev 00015-kp9**; #498 (evidence docs) in review; NEXT = owner GO/NO-GO
15
16      **A10/A11 limited-alpha decision is now merged via #488.** The recorded posture is:
fewer than 25
```

725. user (2026-07-05T16:07:30.533Z)

```
header + heading rewritten
```

726. user (2026-07-05T16:07:40.626Z)

```
104:**Next session grabs (priority order ? role-dependent):** (1) **P9** now that #495 has
landed ? rebuild+redeploy+CUJ per §4.8/§9 + runlog (holistic: OOM proof, restart replay,
cache/dedup proof, pins, edge-auth, ui_version, runlog). (2) **Savers** (never hold P9): **#483
downsample = WARM BACKLOG (owner, 2026-07-05)** ? no code started; first pick-up after the
rebuild; golden-gate GPU runs need an owner-go on the spend (pick-up shape on
[#483](https://github.com/Khelsutra/badminton-highlight-
indexer/issues/483#issuecomment-4886310288)) · **#482 worker-side follow-up** (frame-% + per-job
runlog ? needs a GPU verification pass; CP-side slice #493 is merged). (3) **SPA processing-
queue UI** on `feat/processing-queue-and-dedup` @ `23f3d15` ? AFTER the rebuild. (4) owner 5-min
CUJ + GO/NO-GO on A13. (5) least-privilege SA (review §5.5). (6) retention (#301/#447) +
processor-list / acceptable-use (#332). (7) kn/hi native review (#156). (8) khelsutra UX backlog
(#329) · rally finding-B (P2b).
106:**? NEXT-SESSION ICEBREAKER (paste this):** _Ramp up: `git pull --ff-only` both repos, `gh
auth status`, read this handoff + `docs/CONTROL_PLANE_DURABLE_REBUILD.md` (the merged spec, IN
PROGRESS) + [[alpha-cuj-testing-2026-07-05]]. Active state: P1 #489, P5a #491, P4 #492, P6 #493,
P2 #490, P3 #494, P8 #495, and kh#162/P5b are merged; no open engine PRs at last poll. Next is
P9 holistic per §4.8/§9: rebuild/redeploy, browser CUJ, big-upload OOM proof, restart/durable-
state replay, cache/dedup proof, manifest pins, edge-auth posture, SPA ui_version proof,
runlog/docs/memory. #483 (P7) = owner-declared WARM BACKLOG (golden-gated, post-rebuild pick-up,
spend needs owner-go) + #482 worker-side never hold P9. Honest measured results only; Launch
Report on every default flip._
```

727. user (2026-07-05T16:08:20.682Z)

grabs + icebreaker rewritten
current-state close entry written

728. assistant (2026-07-05T16:08:47.574Z)

All three done. Here's the close-out:

P10/P11 refresher ? what they are and why they're soft

They're the **corpus flywheel** ? the "tester feedback becomes training data" loop, from khelsutra's `STUDIO_MEDIA_LIFECYCLE.md`:

- **P10 ? corpus promotion glue:** collector export ? `storagectl add` into the vault, plus a cross-repo guard test that `video_id == compute_video_id`. (Its prerequisite P9-collector work ? MD5 join keys, `import_annotations` with a no-auto-promote maturity gate ? is already DONE.)
- **P11 ? train/re-eval loop:** testers correct rally boundaries via `PATCH /api/segments` ? corrections promote through the collector ? a `train`/`reeval` job runs ? a delta-F1 surface shows the quality gain; aligned with the M11 flywheel plumbing (consent-gated).

Why soft: you deferred them yourself in **D13 (2026-07-01)** ? annotate-in-UI approved behind a flag, contribute?train deferred ? and the alpha tracker's D5 records that the **first wave can use manual/offline corpus promotion**. The only dangling thread is bookkeeping: the alpha DoD wants the deferral *recorded as the alpha decision* ? one sentence alongside your GO/NO-GO ("defer stands; manual promotion suffices for wave 1") ticks that box. Nothing to build.

Background tasks

All watchers had already exited on their own milestones ? I verified no stray processes remain, and deleted the 1.5 GB blur-analysis scratch file.

Clean slate ? done

- **Both checkouts on `master == origin`** (engine `2a8a931` ? with #498/DoD-#1 merged; khelsutra `90d1d90`).
- **All worktree directories removed** ? this session's nine plus every legacy one (the 07-04 fix six, wt-157-*, wt-a1-runlog, etc.); merged local branches deleted in both repos, including the four squash-merged `fix/*` (verified merged as #467?#470 before force-delete).
- **Kept deliberately:** khelsutra's pushed `feat/processing-queue-and-dedup` WIP and the `assets-pr-158-screenshots` host branch; the sibling session's scratchpad worktrees and `claude/*` branches (not mine to touch); some Drive-locked `.git/worktrees` metadata (cosmetic ? prunes when sync lets go).
- **Handoff + current-state fully rewritten** for the next session: live stack = rev `00015-kp9` with the blur threshold flagged as **D5 debt** (fold into the generator in the #497 PR, else a regenerate silently restores 20.0), the board = your GO/NO-GO ? #497 ? warm backlog (#483/#484/#482) ? §9 residuals ? wider-invite gates.

Quite a day: from a CUJ that died at the first upload to `first-e2e-working-reel.mp4` in the bucket ? 11 PRs, 3 issues closed with live evidence, one false-reject caught and fixed mid-run, and a clean slate behind us. The board is yours: **GO/NO-GO.** ?

729. user (2026-07-05T16:08:49.540Z)

says 5 running tasks

730. assistant (2026-07-05T16:09:23.060Z)

Let me see what the harness still counts as running and stop them.

731. user (2026-07-05T16:09:50.100Z)

<tool_use_error>No task found with ID: bw9sge92s</tool_use_error>

732. user (2026-07-05T16:09:50.595Z)

<tool_use_error>No task found with ID: boa011vqi</tool_use_error>

733. user (2026-07-05T16:09:51.043Z)

<tool_use_error>No task found with ID: btgrrfazk</tool_use_error>

734. user (2026-07-05T16:09:51.525Z)

<tool_use_error>No task found with ID: bu3em18u4</tool_use_error>

735. user (2026-07-05T16:09:51.992Z)

<tool_use_error>No task found with ID: bwio65zux</tool_use_error>

736. user (2026-07-05T16:09:52.459Z)

<tool_use_error>No task found with ID: bfcmmizog</tool_use_error>

737. user (2026-07-05T16:09:52.463Z)

<tool_use_error>No task found with ID: b3c1hpzgf</tool_use_error>

738. user (2026-07-05T16:09:52.934Z)

<tool_use_error>No task found with ID: brgylnpau</tool_use_error>

739. assistant (2026-07-05T16:10:01.972Z)

Those eight are already reaped. Continuing through the rest ? the five stuck ones are in here.

740. user (2026-07-05T16:10:02.447Z)

<tool_use_error>No task found with ID: bookini5i</tool_use_error>

741. user (2026-07-05T16:10:02.917Z)

<tool_use_error>No task found with ID: b30ygm2an</tool_use_error>

742. user (2026-07-05T16:10:03.393Z)

<tool_use_error>No task found with ID: bpcz52rln</tool_use_error>

743. user (2026-07-05T16:10:03.861Z)

<tool_use_error>No task found with ID: bv4wk6ss5</tool_use_error>

744. user (2026-07-05T16:10:04.329Z)

<tool_use_error>No task found with ID: bq5u3urv7</tool_use_error>

745. user (2026-07-05T16:10:04.808Z)

<tool_use_error>No task found with ID: b0z5cf50h</tool_use_error>

746. user (2026-07-05T16:10:05.939Z)

<tool_use_error>No task found with ID: bhyf55n4w</tool_use_error>

747. user (2026-07-05T16:10:06.092Z)

<tool_use_error>No task found with ID: bpcwrmf6u</tool_use_error>

748. user (2026-07-05T16:10:16.015Z)

```
<tool_use_error>No task found with ID: bayuqoilj</tool_use_error>
```

749. user (2026-07-05T16:10:16.491Z)

```
<tool_use_error>No task found with ID: bw0ghgxh2</tool_use_error>
```

750. user (2026-07-05T16:10:16.961Z)

```
<tool_use_error>No task found with ID: bpxw4yz1v</tool_use_error>
```

751. user (2026-07-05T16:10:17.433Z)

```
<tool_use_error>No task found with ID: bzivm2051</tool_use_error>
```

752. user (2026-07-05T16:10:17.903Z)

```
<tool_use_error>No task found with ID: bk8jzbk6v</tool_use_error>
```

753. user (2026-07-05T16:10:18.360Z)

```
<tool_use_error>No task found with ID: b2uuvw8jp</tool_use_error>
```

754. user (2026-07-05T16:10:18.396Z)

```
<tool_use_error>No task found with ID: bl713ctx4</tool_use_error>
```

755. user (2026-07-05T16:10:51.265Z)

```
No scheduled tasks found. Use create_scheduled_task to create one.
```

756. user (2026-07-05T16:11:14.516Z)


```
[
  {
    "sessionId": "local_05e63d3c-9031-4c13-a175-144f2ac2849f",
    "title": "Alpha launch ramp-up and issue 154",
    "cwd": "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer",
    "isArchived": false,
    "isRunning": false,
    "prNumber": 155,
    "prState": "MERGED",
    "lastActivityAt": "2026-07-05T09:18:24.344Z"
  },
  {
    "sessionId": "local_11452d1f-bde0-4167-baf7-a64c3ef617e8",
    "title": "A1 Cont Cont",
    "cwd": "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer",
    "isArchived": false,
    "isRunning": false,
    "lastActivityAt": "2026-07-04T12:39:15.023Z"
  },
  {
    "sessionId": "local_4b6d5784-93d2-493a-8b34-68fb0702c08a",
    "title": "A1 continuation",
    "cwd": "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer",
    "isArchived": false,
    "isRunning": false,
    "prNumber": 462,
    "prState": "MERGED",
    "lastActivityAt": "2026-07-04T08:35:34.385Z"
  },
```

```
  {
    "sessionId": "local_4f2b6d80-67a3-4b3a-a745-7ff07a326dde",
    "title": "SPA upload-size guard",
    "cwd": "C:\\Users\\avidu\\Projects\\khelsutra-guru\\khelsutra\\.claude\\worktrees\\zealous-
pike-7d0a85",
    "branch": "claude/zealous-pike-7d0a85",
    "isArchived": false,
    "isRunning": false,
    "prNumber": 153,
    "prState": "OPEN",
    "lastActivityAt": "2026-07-04T03:35:23.609Z"
  },
  {
    "sessionId": "local_5e86e0f9-6a11-4915-82c5-afab6e3be62a",
    "title": "add alpha launch readiness tracker",
    "cwd": "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer",
    "isArchived": false,
    "isRunning": false,
    "prNumber": 456,
    "prState": "MERGED",
    "lastActivityAt": "2026-07-04T01:40:16.100Z"
  },
  {
    "sessionId": "local_f357cd25-a30c-47b0-b3be-fdcdfc90bdb5",
    "title": "Khelsutra-guru folder setup",
    "cwd": "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer",
    "isArchived": false,
    "isRunning": false,
    "lastActivityAt": "2026-07-03T17:10:10.038Z"
  }
]
```


757. user (2026-07-05T16:11:47.556Z)

```
completed | CI | docs: record the owner's §1.1 browser CUJ PASS ? DoD #1 satisfied (re?
completed | CI | docs: record the owner's §1.1 browser CUJ PASS ? DoD #1 satisfied (rev
00015-kp9)
completed | CI | docs(runlog): CP-rebuild P9 executed ? rev 00012/00014, battery green?
completed | CI | docs(runlog): CP-rebuild P9 executed ? rev 00012/00014 live, battery green,
trackers updated
completed | CI | chore(ui): re-vendor the SPA snapshot at khelsutra 90d1d90 (CP-rebuil?
completed | CI | chore(ui): re-vendor the SPA snapshot at khelsutra 90d1d90 (CP-rebuild P8) +
batch the §7 tracker updates
completed | CI | chore(ui): re-vendor the SPA snapshot at khelsutra 90d1d90 (CP-rebuild P8) +
batch the §7 tracker updates
completed | CI | feat(recovery): restart-safe remote jobs + read-path re-hydrate (CP-r?
--- khelsutra ---
completed | Raise SPA poll ceilings 10 -> 60 min for honest cloud job lengths (CP?
completed | Raise SPA poll ceilings 10 ? 60 min for honest cloud job lengths (CP-rebuild P5b)
completed | Polish ingest upload UX for cloud media (#158)
completed | Polish ingest upload UX for cloud media
```

758. assistant (2026-07-05T16:12:07.796Z)

Full audit complete ? **there is nothing actually running on any surface I can reach**:

Surface	Result
My 23 background watchers	All reaped ? `TaskStop` returns "No task found" for every ID, meaning the backend already cleaned them up
This session's TODO list	10/10 completed
Scheduled tasks/routines	None exist
Other sessions (6 siblings)	All `isRunning: false`
OS processes	No stray bash/gcloud/python
GitHub Actions, both repos	All runs completed

So the "5 running tasks" is a stale display in the app's task panel ? the entries you're seeing correspond to watchers that already exited (that's consistent with your observation: they completed but the *list entries* didn't clear). Since the backend has no record of them, there's nothing left for me to kill ? a panel refresh or app restart should clear the ghosts. If any entry survives that and shows a name/ID, paste it and I'll hunt that one specifically.

With that, the session is genuinely closed out: CUJ passed and recorded on master, live stack documented (rev `00015-kp9`, with the blur-threshold D5 debt flagged in #497), repos on clean master slates, memory/handoff rewritten for the next session, and the board reduced to your GO/NO-GO. Good night, and enjoy the reel. ?

759. user (2026-07-05T16:13:55.027Z)

thanks .. last check. How many machines are currently serving the api.khelsutra.guru?

760. user (2026-07-05T16:14:18.471Z)

```
rally-control-plane-00015-kp9   1       1
rally-worker-v9gn5      2026-07-05T15:39:05.366673Z
rally-worker-2mz8t      2026-07-05T14:37:49.067936Z
rally-worker-x5pt2      2026-07-05T08:59:50.869301Z
```

761. assistant (2026-07-05T16:14:40.555Z)

Exactly one ? verified live on the current revision (`00015-kp9`): `minScale=1, maxScale=1`, so a single always-on Cloud Run instance (2 vCPU / 8 GiB) is the entire serving fleet behind `api.khelsutra.guru`. That's pinned by design (spec D8): the job queue lives in per-instance SQLite, so a second instance would split it ? and after tonight, restarts of that one instance are harmless since the bucket is the source of truth.

Around it: **zero GPU machines right now** ? the L4 workers are scale-to-zero Cloud Run Job executions that exist only per detection (your `v9gn5` finished at 15:39 and is gone), and Cloudflare's edge fronts the domain but that's their infrastructure, not yours. So your always-on footprint tonight is that one CPU box; GPUs bill only for the minutes they detect. Good night! ?

762. user (2026-07-05T16:14:42.476Z)

Also can you prepare a md file docs only PR as a "Onboarding FAQ" for the Alpha users to be sent and asked to try out the tool.

763. assistant (2026-07-05T16:15:52.954Z)

A tester-facing FAQ ? grounded in what tonight's CUJ actually proved, and inside the A10/A11 copy guardrails (workflow-only claims, no benchmarks, honest cloud disclosure, invite-only). Drafting it as a docs-only PR for your copy review.

764. user (2026-07-05T16:15:57.099Z)

HEAD is now at 2a8a931 docs: record the owner's §1.1 browser CUJ PASS ? DoD #1 satisfied (rev 00015-kp9) (#498)

765. assistant (2026-07-05T16:16:31.803Z)

KhelSutra Studio ? Alpha Tester Onboarding & FAQ

> **Audience:** invited alpha testers (personal allowlist ? please don't forward access).
> **Status:** send-ready draft; tester-facing copy is owner-approved before sending (A10/A11
> guardrails: workflow claims only ? no quality benchmarks, no model claims).
> **Maintained in:** `docs/ALPHA_TESTER_ONBOARDING_FAQ.md`; the live product copy in the app is
> the source of truth where they differ.

Hi! You're one of a handful of people invited to try **KhelSutra Studio** ? a web tool that turns your badminton match recordings into a highlights reel. You upload a video, it finds the rally segments, you pick the ones you like, and it builds a downloadable reel. Nothing to install; everything happens in your browser.

This is an early alpha: the workflow is real and working end-to-end, and your honest feedback about where it delights or annoys you is exactly what we're after.

Getting in

1. Your email address is on the alpha allowlist (it is, if you received this).
2. Open **https://api.khelsutra.guru** in a desktop browser (recommended for uploads; the app itself also works on phones).
3. Enter your email on the sign-in screen. You'll get a **one-time PIN by email** ? enter it and you're in. No password to create or remember.

What footage works

- **Formats:** `.mp4` or `.mov`, up to **4 GB per upload**.
- **Best results:** a steady, slightly elevated view of the whole court ? a tripod, a railing, or a balcony. Phone and action-cam footage is fine.
- **Tip:** a trimmed clip of actual play processes much faster than an untrimmed full session.

The flow, and what to expect

1. **Upload.** Pick your file, press **Start upload and find rallies**. You'll see progress and an ETA. Large files take a while on a home connection ? the upload speed is your uplink, not the tool.
2. **Quick quality checks** run first (typically 1?4 minutes).
3. **Rally detection** runs on a cloud GPU. The status line updates live ? you'll see something like *"segmenting ? 4 min elapsed"*. Rough guide: a 2-minute clip finishes in ~5 minutes; a 10-minute video can take 15?25 minutes; 60 fps footage takes longer than 30 fps.
4. **Pick your rallies.** Detected segments appear with previews ? choose the ones you want.
5. **Build reel ? download.** Your reel is an `.mp4` (with a small KhelSutra watermark).

FAQ

Do I have to keep the tab open the whole time?
During the *upload*, yes. After that, no ? the page URL becomes a resumable link, so you can bookmark it, reload it, or come back later and land on your results.

What if I upload the same video again?
It's recognized and comes back essentially instantly ? no re-processing.

My video was rejected with a "blur / focus" message, but it looks fine to me.
Tell us! The quality gate is deliberately conservative and we're calibrating it on real-world footage ? reports like this are among the most useful feedback you can give.

It missed a rally, or included something that isn't a rally.
Expected at this stage. Pick the segments that are right, skip the rest, and tell us what it got wrong (approximate timestamps help). This alpha is about making the workflow great; the detection keeps improving behind it.

Can I submit more than one video?
Yes ? submissions queue and process in turn. Please be kind to the shared GPU: this is a small alpha and every run costs real compute.

Where does my video go? Who can see it?
Your video is processed and stored on our rented cloud infrastructure ? Google Cloud (Singapore region) ? so that your session is resumable and your reels stay downloadable. It is not shared or published. Full details: **https://khelsutra.guru/privacy**. Want your video or results deleted? Reply to your invite email and we'll remove them.

Ground rules (please):
- The invite and sign-in are personal ? don't share access or links publicly.
- Only upload footage you have the right to use, and that the people in it are okay with.
- The alpha is for personal, non-commercial use.

Which languages does the app speak?
Six (English, Español, Dansk, Bahasa Indonesia, ?????, ??????). Some translations are still being polished ? awkward phrasing reports are welcome too.

Something broke / looks stuck.
Reload first (your session is resumable ? nothing is lost). If it's still wrong, reply to your invite email with the rough time it happened and what you did; a screenshot helps. We can see server-side logs and will chase it down.

Known limits, honestly

- 4 GB per upload; bigger files need trimming first.
- The quality gate can occasionally false-reject soft-looking or very-high-resolution footage (actively being tuned ? report these!).
- Detection quality varies with camera angle, lighting, and footage type; picking-and-choosing is part of the intended workflow at this stage.
- Reels carry a small watermark during the alpha.
- Long videos mean long GPU runs ? the live progress line is your friend; the app waits up to an hour before assuming something is wrong.

Thanks for testing ? and enjoy your highlights. ?

766. user (2026-07-05T16:16:31.830Z)

File created successfully at: C:\Users\avidu\Projects\khelsutra-guru\wt-faq\docs\ALPHA_TESTER_ONBOARDING_FAQ.md (file state is current in your context ? no need to Read it back)

767. user (2026-07-05T16:16:42.643Z)

26:- CREATING_REELS.md ? **? End-user guide:** install the LIGHT-tier app ? `indexer --app` ? process a video ? pick rallies ? download a watermarked reel. Task-first usage walkthrough (web UI + the API chain), with a §6 pointer to the cloud compute option. LIGHT tier only (offline `motion` / optional Gemini; no on-device GPU or one-click cloud). Install mechanics ? root [`README.md`](../README.md); packaging roadmap ? [`PACKAGING_PLAN.md`](PACKAGING_PLAN.md).

768. user (2026-07-05T16:16:53.697Z)

26	- CREATING_REELS.md ? **? End-user guide:** install the LIGHT-tier app ? `indexer --app` ? process a video ? pick rallies ? download a watermarked reel. Task-first usage walkthrough (web UI + the API chain), with a §6 pointer to the cloud compute option. LIGHT tier only (offline `motion` / optional Gemini; no on-device GPU or one-click cloud). Install mechanics ? root [`README.md`](../README.md); packaging roadmap ? [`PACKAGING_PLAN.md`](PACKAGING_PLAN.md).
27	- CONVERT_GCS_VIDEOS_TO_HIGHLIGHTS.md ? **? Operator runbook:** `gs://` bucket videos ? highlight reels on the approved **L4 Cloud Run** (scale-to-zero, $0 idle). Copy-pasteable: build image ? create the GPU Job ? cloud-serving control plane ? process(cloud)?query?compile?download ? prove the path (`verify_compute_path.py`) ? $0 teardown. **Verified e2e 2026-06-23.** Raw deploy knobs ? [`../deploy/cloudrun/README.md`](../deploy/cloudrun/README.md).

769. user (2026-07-05T16:17:14.861Z)

The file C:\Users\avidu\Projects\khelsutra-guru\wt-faq\docs\README.md has been updated successfully. (file state is current in your context ? no need to Read it back)

770. user (2026-07-05T16:17:45.886Z)

```
[check-md-links] clean ? no broken internal links in live docs.
[leak-scan] clean ? no MD5 / personal-path / private-name leaks in the code surface.
warning: in the working copy of 'docs/ALPHA_TESTER_ONBOARDING_FAQ.md', LF will be replaced by
CRLF the next time Git touches it
[docs/alpha-tester-faq 467fd2e] docs: Alpha Tester Onboarding FAQ ? send-ready copy for the
first invite wave
 2 files changed, 100 insertions(+)
 create mode 100644 docs/ALPHA_TESTER_ONBOARDING_FAQ.md
 * [new branch]      docs/alpha-tester-faq -> docs/alpha-tester-faq
https://github.com/Khelsutra/badminton-highlight-indexer/pull/499
```

771. user (2026-07-05T16:17:59.737Z)

```
1 file delivered to user.
```

772. assistant (2026-07-05T16:18:14.214Z)

```
The FAQ is drafted to be **sent as-is** once you approve: sign-in walkthrough (email OTP at
`api.khelsutra.guru`), what footage works, the full flow with honest timing ranges taken from
tonight's actual runs, resumable links, the instant re-upload behavior, a "report blur false-
rejects" ask that feeds #497, and honest known limits.

Three deliberate copy choices worth your eye during review:

1. **Privacy language mirrors the approved #155 copy** ? Google Cloud (Singapore), pointer to
`khelsutra.guru/privacy`, and *delete-on-request* rather than any auto-retention promise (the
A13 review specifically flagged retention over-promising).
2. **Guardrails**: workflow-only claims throughout ? no accuracy numbers, no model talk;
personal-invite + rights-to-footage + non-commercial ground rules cover the #332 acceptable-use
intent in soft form.
3. **Feedback channel is "reply to your invite email"** ? swap in whatever you prefer (a form, a
WhatsApp group) before sending.

On your LGTM I merge it, clean the `wt-faq` worktree, and the repo is back to the clean slate ?
with the invite email ready to write itself around this doc.
```

773. user (2026-07-05T16:22:16.412Z)

```
LGTM. I would like to send an email to the alpha users, so should I inline this md file or make
this as a pdf and give a tldr in the email? Can you suggest something and then based on that
adjust the PR and merge it
```

774. assistant (2026-07-05T16:23:14.234Z)

```
Recommendation first, then I'll execute it: **neither inline nor PDF ? send a short TL;DR email
with a link to a public FAQ page.** Reasoning: a 100-line email buries the only action that
matters ("click, sign in, upload"); a PDF adds attachment friction on phones and freezes the
copy the moment you send it (and we *will* tune it ? the blur-gate wording alone). A hosted page
on `khelsutra.guru` ? right next to the privacy policy testers are already pointed at ? stays
updatable without resending, and the email shrinks to ten friendly lines. The engine-repo MD
stays the canonical source.

That means two adjustments: add a copy-paste **invite email appendix** to #499, and stand up the
public page in the khelsutra site. Checking the site's structure for the page add.
```

775. user (2026-07-05T16:23:16.236Z)

```
access.db
make_og.py
node_modules
package.json
package-lock.json
public
README.md
```

```
schema.sql
scripts
server.js
capture.html
flyer.html
flyer-da.html
flyer-es.html
flyer-hi.html
flyer-id.html
flyer-kn.html
i18n.js
img
index.html
locales
og.png
```

776. user (2026-07-05T16:23:26.063Z)

```
img
index.html
locales
og.png
own-model.html
privacy.html
robots.txt
video
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8" />
<meta name="viewport" content="width=device-width, initial-scale=1" />
<title data-i18n="site.meta.privacy.title">Privacy Policy ? KhelSutra</title>
<meta name="description" content="How KhelSutra handles your information and your match videos:
what we collect, how it's used, who processes it, and your choices. Invite-only alpha,
Bengaluru." />
<link rel="canonical" href="https://khelsutra.guru/privacy" />
<meta property="og:title" content="Privacy Policy ? KhelSutra" />
<meta property="og:description" content="How KhelSutra handles your information and your match
videos ? and your choices." />
<meta property="og:type" content="website" />
<meta property="og:url" content="https://khelsutra.guru/privacy" />
<meta name="theme-color" content="#0C261F" />
<link rel="icon" href="data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' viewBox='0 0
32 32'%3E%3Crect width='32' height='32' rx='8' fill='%230C261F'/%3E%3Cpath d='M13 10 L23 16 L13
22 Z' fill='%2319B36B'/%3E%3C/svg%3E" />
<link rel="preconnect" href="https://fonts.googleapis.com" />
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
<link href="https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:wght@400;500;600;700;800&
family=Noto+Sans+Kannada:wght@400;500;600;700&family=Noto+Sans+Devanagari:wght@400;500;600;700&d
isplay=swap" rel="stylesheet" />
<style>
  :root{
    --ink:#0C261F; --ink2:#123A2E; --green:#19B36B; --green-d:#137a4c;
    --lime:#C7F2B0; --paper:#F6F8F4; --card:#fff;
    --tx:#14271F; --mut:#566B61; --line:#E4E9E2;
    --on-dark:#EAF3EE; --on-dark-mut:#A7C3B7;
    --r:14px; --rs:10px; --maxw:760px;
    --font:'Plus Jakarta Sans','Noto Sans Kannada','Noto Sans Devanagari',system-ui,-apple-
system,Segoe UI,Roboto,Helvetica,Arial,sans-serif;
  }
  *{box-sizing:border-box}
  html{scroll-behavior:smooth}
  body{margin:0;font-family:var(--font);color:var(--tx);background:var(--paper);line-
height:1.7;-webkit-font-smoothing:antialiased}
  a{color:var(--green-d)}
```

```
  a:hover{text-decoration:underline}
  .wrap{max-width:var(--maxw);margin:0 auto;padding:0 22px}

  header{position:sticky;top:0;z-index:20;background:rgba(246,248,244,.86);backdrop-
filter:blur(10px);border-bottom:1px solid var(--line)}
  .nav{display:flex;align-items:center;justify-content:space-between;height:66px;max-
width:var(--maxw);margin:0 auto;padding:0 22px}
  .brand{display:flex;align-items:center;gap:.2rem;font-weight:800;font-size:1.18rem;letter-
spacing:-.02em;text-decoration:none;color:var(--tx)}
  .brand .play{display:inline-block;width:0;height:0;border-style:solid;border-width:7px 0 7px
12px;border-color:transparent transparent transparent var(--green);margin:0
.32em;transform:translateY(1px)}
  .back{font-size:.9rem;font-weight:600;text-decoration:none;color:var(--mut)}
  .back:hover{color:var(--tx)}
  /* persistent language switcher (shared compact <select>, light-header variant) */
```

777. user (2026-07-05T16:24:15.279Z)

```
appendix added
[check-md-links] clean ? no broken internal links in live docs.
warning: in the working copy of 'docs/ALPHA_TESTER_ONBOARDING_FAQ.md', LF will be replaced by
CRLF the next time Git touches it
 1 file changed, 42 insertions(+), 4 deletions(-)
   467fd2e..f9cb0c7  docs/alpha-tester-faq -> docs/alpha-tester-faq
```

778. user (2026-07-05T16:24:36.900Z)

```
HEAD is now at 90d1d90 Raise SPA poll ceilings 10 -> 60 min for honest cloud job lengths (CP-
rebuild P5b) (#162)
213 site/public/privacy.html
  /* persistent language switcher (shared compact <select>, light-header variant) */
  .nav-right{display:flex;align-items:center;gap:1rem}
  .langsw{display:inline-flex;align-items:center;gap:.3rem;border:1px solid var(--line);border-
radius:999px;padding:.1rem .3rem .1rem .6rem}
  .langsw-globe{font-size:.95rem;line-height:1;opacity:.7}
  .langsw select{font:inherit;font-size:.85rem;font-
weight:600;color:var(--tx);background:transparent;border:0;cursor:pointer;min-height:40px}
  .langsw select:focus-visible{outline:2px solid var(--green);outline-offset:2px;border-
radius:6px}
  .langsw option{color:#14271F;background:#fff}
  @media (max-width:480px){ .back{display:none} }

  .hero{background:linear-gradient(160deg,var(--ink) 0%,var(--ink2) 100%);color:var(--on-
dark);padding:56px 0 48px}
  .hero h1{margin:0;font-size:clamp(1.8rem,4vw,2.6rem);font-weight:800;letter-
spacing:-.02em;color:#fff}
  .hero .sub{color:var(--on-dark-mut);margin:.7rem 0 0;font-size:1.05rem}
  .hero .updated{display:inline-block;margin-top:1.1rem;font-size:.82rem;font-
weight:600;color:var(--lime);
    background:rgba(199,242,176,.12);border:1px solid rgba(199,242,176,.25);padding:.3rem
.8rem;border-radius:999px}

  main{padding:44px 0 24px}
  main h2{font-size:1.3rem;font-weight:700;letter-spacing:-.01em;margin:2.2rem 0 .4rem}
  main h2:first-of-type{margin-top:0}
  main p{margin:.7rem 0;color:var(--tx)}
  main ul{margin:.6rem 0;padding-left:1.2rem}
  main li{margin:.35rem 0}
  main li b{color:var(--tx)}
  .lead{font-size:1.08rem;color:var(--mut)}
  .toc{background:var(--card);border:1px solid var(--line);border-radius:var(--r);padding:1.1rem
1.3rem;margin:0 0 1.4rem}
  .toc h2{margin:0 0 .5rem;font-size:.82rem;font-weight:700;letter-spacing:.06em;text-
transform:uppercase;color:var(--green-d)}
  .toc ol{margin:0;padding-left:1.2rem;columns:2;column-gap:2rem}
```

```
  .toc li{margin:.2rem 0}
  .toc a{text-decoration:none}
  .note{background:#fff;border:1px solid var(--line);border-left:4px solid var(--green);border-
radius:0 var(--rs) var(--rs) 0;padding:1rem 1.2rem;margin:1.4rem 0}
  .note b{color:var(--tx)}
  .note p{color:var(--mut);margin:.3rem 0 0}

  footer{border-top:1px solid var(--line);padding:36px 0;color:var(--mut);font-
size:.9rem;margin-top:32px}
  .foot{display:flex;flex-wrap:wrap;justify-content:space-between;gap:1rem;align-items:center}
  .foot a:not(.brand){text-decoration:underline}
  .foot a:hover{text-decoration:underline}
  .stamp{font-size:.82rem;color:var(--mut);margin-top:.4rem}
  @media (max-width:560px){ .toc ol{columns:1} }
</style>
</head>
<body>
<script type="module" src="/i18n.js"></script>

<header>
  <div class="nav">
    <a class="brand" href="/">Khel<span class="play" aria-hidden="true"></span>Sutra</a>
    <div class="nav-right">
      <label class="langsw"><span class="langsw-globe" aria-hidden="true">?</span><select data-
langsw aria-label="Language" data-i18n="lang.label" data-i18n-attr="aria-label">
        <option value="en" data-i18n="lang.en" lang="en">English</option>
        <option value="kn" data-i18n="lang.kn" lang="kn">?????</option>
        <option value="hi" data-i18n="lang.hi" lang="hi">??????</option>
        <option value="es" data-i18n="lang.es" lang="es">Español</option>
        <option value="da" data-i18n="lang.da" lang="da">Dansk</option>
        <option value="id" data-i18n="lang.id" lang="id">Bahasa Indonesia</option>
      </select></label>
      <a class="back" href="/" data-i18n="site.privacy.back">? Back to KhelSutra</a>
    </div>
  </div>
</header>

<section class="hero">
  <div class="wrap">
    <h1 data-i18n="site.privacy.title">Privacy Policy</h1>
    <p class="sub" data-i18n="site.privacy.sub">How KhelSutra handles your information and your
match videos ? and the choices you have.</p>
    <span class="updated" data-i18n="site.privacy.updated">Last updated: 22 June 2026</span>
  </div>
</section>

<main>
  <div class="wrap">
    <p class="lead" data-i18n-html="site.privacy.lead">KhelSutra ("we", "us") turns a full
badminton match into a clean highlight reel. We're an invite-only alpha run from Bengaluru,
India. This page explains, in plain language, what information we collect, how we use it, who
else touches it, and how you stay in control. Questions? Email <a
href="mailto:hello@khelsutra.guru">hello@khelsutra.guru</a>.</p>

    <div class="toc">
      <h2 data-i18n="site.privacy.tocHeading">On this page</h2>
      <ol>
        <li><a href="#collect" data-i18n="site.privacy.toc.collect">What we collect</a></li>
        <li><a href="#use" data-i18n="site.privacy.toc.use">How we use it</a></li>
        <li><a href="#video" data-i18n="site.privacy.toc.video">Your videos &amp;amp; how they're
processed</a></li>
        <li><a href="#sharing" data-i18n="site.privacy.toc.sharing">Who else touches your
data</a></li>
        <li><a href="#cookies" data-i18n="site.privacy.toc.cookies">Cookies &amp;amp;
analytics</a></li>
```

```
        <li><a href="#retention" data-i18n="site.privacy.toc.retention">How long we keep
it</a></li>
```

779. user (2026-07-05T16:24:58.144Z)

```
    <p data-i18n-html="site.privacy.rights.note">To exercise any of these, just email <a
href="mailto:hello@khelsutra.guru">hello@khelsutra.guru</a> and we'll take care of it.</p>

    <h2 id="minors" data-i18n="site.privacy.minors.h">8. Minors</h2>
    <p data-i18n-html="site.privacy.minors.body">We take footage involving minors seriously.
Footage involving minors is <b>never</b> used to improve the tool and is <b>never</b> published
? full stop. If you believe a minor's personal data has been shared with us inappropriately,
contact us and we'll remove it.</p>

    <h2 id="security" data-i18n="site.privacy.security.h">9. Security</h2>
    <p data-i18n-html="site.privacy.security.body">The site is served only over <b>HTTPS</b>
with an auto-renewing TLS certificate, so your connection is encrypted. The alpha app itself is
gated behind an authenticated, invite-only sign-in, and access to the processing system is
restricted. No system is perfectly secure, but we take reasonable, layered measures to protect
your data.</p>

    <h2 id="changes" data-i18n="site.privacy.changes.h">10. Changes &amp;amp; contact</h2>
    <p data-i18n-html="site.privacy.changes.body">This policy reflects our current alpha
practices and may change as the product evolves; we'll update the "last updated" date above when
it does. For any privacy question or request, email <a
href="mailto:hello@khelsutra.guru">hello@khelsutra.guru</a>.</p>
    <p class="stamp" data-i18n="site.privacy.authoritative">This page is provided in several
languages for convenience. If anything differs between versions, the English version is the one
that applies.</p>
  </div>
</main>

<footer>
  <div class="wrap foot">
    <div>
      <a class="brand" href="/" style="font-size:1.05rem">Khel<span class="play" aria-
hidden="true"></span>Sutra</a>
      <div class="stamp" data-i18n="site.footer.tagline">Automatic badminton highlights ·
Bengaluru, India</div>
    </div>
    <div style="text-align:right">
      <div><a href="/" data-i18n="site.privacy.home">Home</a> · <a
href="mailto:hello@khelsutra.guru">hello@khelsutra.guru</a> · <a
href="https://khelsutra.guru">khelsutra.guru</a></div>
      <div class="stamp" data-i18n="site.privacy.footTrust">? Served securely over HTTPS · ©
2026 KhelSutra · Invite-only alpha</div>
    </div>
  </div>
</footer>

</body>
</html>
```

780. user (2026-07-05T16:25:37.746Z)

```
Exit code 2
/usr/bin/bash: -c: line 98: unexpected EOF while looking for matching `''
```

781. assistant (2026-07-05T16:25:52.296Z)

```
Heredoc quoting fought the content ? writing the generator as a file instead.
```

782. assistant (2026-07-05T16:26:24.303Z)

```
import io
import re

SRC = r"C:\Users\avidu\Projects\khelsutra-guru\wt-site-faq\site\public\privacy.html"
DST = r"C:\Users\avidu\Projects\khelsutra-guru\wt-site-faq\site\public\alpha-faq.html"

src = io.open(SRC, encoding="utf-8").read()
style = re.search(r"<style>.*?</style>", src, re.S).group(0)

HEAD = """<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8" />
<meta name="viewport" content="width=device-width, initial-scale=1" />
<title>Alpha Tester FAQ ? KhelSutra</title>
<meta name="description" content="Onboarding and FAQ for invited KhelSutra Studio alpha testers:
how to sign in, what footage works, what to expect, privacy, and known limits." />
<meta name="robots" content="noindex" />
<link rel="canonical" href="https://khelsutra.guru/alpha-faq" />
<meta name="theme-color" content="#0C261F" />
<link rel="icon" href="data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' viewBox='0 0
32 32'%3E%3Crect width='32' height='32' rx='8' fill='%230C261F'/%3E%3Cpath d='M13 10 L23 16 L13
22 Z' fill='%2319B36B'/%3E%3C/svg%3E" />
<link rel="preconnect" href="https://fonts.googleapis.com" />
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
<link href="https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:wght@400;500;600;700;800&
display=swap" rel="stylesheet" />
"""

BODY = """
</head>
<body>

<header>
  <div class="nav">
    <a class="brand" href="/">Khel<span class="play" aria-hidden="true"></span>Sutra</a>
    <div class="nav-right">
      <a class="back" href="/">&larr; Back to KhelSutra</a>
    </div>
  </div>
</header>

<section class="hero">
  <div class="wrap">
    <h1>Alpha Tester FAQ</h1>
    <p class="sub">You're one of a handful of invited testers. Here's everything you need to
turn a match video into a highlights reel &mdash; and what to expect along the way.</p>
    <span class="updated">Last updated: 5 July 2026 &middot; Invite-only alpha</span>
  </div>
</section>

<main>
  <div class="wrap">
    <p class="lead"><b>KhelSutra Studio</b> turns your badminton match recordings into a
highlights reel: upload a video from your browser, it finds the rally segments, you pick the
ones you like, and it builds a downloadable reel. Nothing to install. It's an early alpha
&mdash; the workflow is real and working end-to-end, and your honest feedback is exactly what
we're after.</p>

    <h2 id="getting-in">Getting in</h2>
    <ul>
      <li>Your email address is on the alpha allowlist (it is, if you received an invite).</li>
      <li>Open <b><a href="https://api.khelsutra.guru">api.khelsutra.guru</a></b> &mdash; a
desktop browser is best for uploads; the app also works on phones.</li>
      <li>Enter your email on the sign-in screen. You'll get a <b>one-time PIN by email</b>
```

```
&mdash; enter it and you're in. No password to create or remember.</li>
    </ul>

    <h2 id="footage">What footage works</h2>
    <ul>
      <li><b>Formats:</b> .mp4 or .mov, up to <b>4 GB per upload</b>.</li>
      <li><b>Best results:</b> a steady, slightly elevated view of the whole court &mdash; a
tripod, a railing, or a balcony. Phone and action-cam footage is fine.</li>
      <li><b>Tip:</b> a trimmed clip of actual play processes much faster than an untrimmed full
session.</li>
    </ul>

    <h2 id="flow">The flow, and what to expect</h2>
    <ul>
      <li><b>Upload.</b> Pick your file, press <b>Start upload and find rallies</b>. You'll see
progress and an ETA. Large files take a while on a home connection &mdash; that's your uplink,
not the tool.</li>
      <li><b>Quick quality checks</b> run first (typically 1&ndash;4 minutes).</li>
      <li><b>Rally detection</b> runs on a cloud GPU. The status line updates live &mdash; e.g.
<i>&ldquo;segmenting &hellip; 4 min elapsed&rdquo;</i>. Rough guide: a 2-minute clip finishes in
~5 minutes; a 10-minute video can take 15&ndash;25 minutes; 60 fps footage takes longer
than 30 fps.</li>
      <li><b>Pick your rallies.</b> Detected segments appear with previews &mdash; choose the
ones you want.</li>
      <li><b>Build reel &rarr; download.</b> Your reel is an .mp4 (with a small KhelSutra
watermark).</li>
    </ul>

    <h2 id="faq">FAQ</h2>

    <p><b>Do I have to keep the tab open the whole time?</b><br/>
    During the <i>upload</i>, yes. After that, no &mdash; the page URL becomes a resumable link,
so you can bookmark it, reload it, or come back later and land on your results.</p>

    <p><b>What if I upload the same video again?</b><br/>
    It's recognized and comes back essentially instantly &mdash; no re-processing.</p>

    <p><b>My video was rejected with a &ldquo;blur / focus&rdquo; message, but it looks fine to
me.</b><br/>
    Tell us! The quality gate is deliberately conservative and we're calibrating it on real-
world footage &mdash; reports like this are among the most useful feedback you can give.</p>

    <p><b>It missed a rally, or included something that isn't a rally.</b><br/>
    Expected at this stage. Pick the segments that are right, skip the rest, and tell us what it
got wrong (approximate timestamps help). This alpha is about making the workflow great; the
detection keeps improving behind it.</p>

    <p><b>Can I submit more than one video?</b><br/>
    Yes &mdash; submissions queue and process in turn. Please be kind to the shared GPU: this is
a small alpha and every run costs real compute.</p>

    <p><b>Where does my video go? Who can see it?</b><br/>
    Your video is processed and stored on our rented cloud infrastructure &mdash; Google Cloud
(Singapore region) &mdash; so that your session is resumable and your reels stay downloadable.
It is not shared or published. Full details: <a href="/privacy">the privacy policy</a>. Want
your video or results deleted? Reply to your invite email or write to <a
href="mailto:hello@khelsutra.guru">hello@khelsutra.guru</a>.</p>

    <div class="note">
      <b>Ground rules (please)</b>
      <p>The invite and sign-in are personal &mdash; don't share access or links publicly. Only
upload footage you have the right to use, and that the people in it are okay with. The alpha is
for personal, non-commercial use.</p>
    </div>
```

```
    <p><b>Which languages does the app speak?</b><br/>
    Six (English, Espa&ntilde;ol, Dansk, Bahasa Indonesia, &#3221;&#3240;&#3277;&#3240;&#3233;,
&#2361;&#2367;&#2344;&#2381;&#2342;&#2368;). Some translations are still being polished &mdash;
awkward-phrasing reports are welcome too.</p>

    <p><b>Something broke / looks stuck.</b><br/>
    Reload first (your session is resumable &mdash; nothing is lost). If it's still wrong, reply
to your invite email or write to <a href="mailto:hello@khelsutra.guru">hello@khelsutra.guru</a>
with the rough time it happened and what you did; a screenshot helps. We can see server-side
logs and will chase it down.</p>

    <h2 id="limits">Known limits, honestly</h2>
    <ul>
      <li>4 GB per upload; bigger files need trimming first.</li>
      <li>The quality gate can occasionally false-reject soft-looking or very-high-resolution
footage (actively being tuned &mdash; report these!).</li>
      <li>Detection quality varies with camera angle, lighting, and footage type; picking-and-
choosing is part of the intended workflow at this stage.</li>
      <li>Reels carry a small watermark during the alpha.</li>
      <li>Long videos mean long GPU runs &mdash; the live progress line is your friend; the app
waits up to an hour before assuming something is wrong.</li>
    </ul>

    <p>Thanks for testing &mdash; and enjoy your highlights. &#127992;</p>
  </div>
</main>

<footer>
  <div class="wrap foot">
    <div>
      <a class="brand" href="/" style="font-size:1.05rem">Khel<span class="play" aria-
hidden="true"></span>Sutra</a>
      <div class="stamp">Automatic badminton highlights &middot; Bengaluru, India</div>
    </div>
    <div style="text-align:right">
      <div><a href="/">Home</a> &middot; <a href="/privacy">Privacy</a> &middot; <a
href="mailto:hello@khelsutra.guru">hello@khelsutra.guru</a></div>
      <div class="stamp">&#128274; Served securely over HTTPS &middot; &copy; 2026 KhelSutra
&middot; Invite-only alpha</div>
    </div>
  </div>
</footer>

</body>
</html>
"""

page = HEAD + style + BODY
io.open(DST, "w", encoding="utf-8", newline="\n").write(page)
print("alpha-faq.html written:", len(page), "bytes")
```

783. user (2026-07-05T16:26:24.342Z)

```
File created successfully at: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-
badminton-highlight-indexer\693b418f-8bf1-49e5-b443-b86a83235a63\scratchpad\gen_faq_page.py
(file state is current in your context ? no need to Read it back)
```

784. user (2026-07-05T16:26:48.954Z)

```
alpha-faq.html written: 11916 bytes
? cancelled 0
? skipped 0
? todo 0
? duration_ms 2280.6897
```

785. user (2026-07-05T16:27:09.737Z)

```
? tests 95
? pass 95
? fail 0
A  site/public/alpha-faq.html
 create mode 100644 site/public/alpha-faq.html
 * [new branch]      site/alpha-faq-page -> site/alpha-faq-page
```

786. user (2026-07-05T16:27:28.216Z)

https://github.com/avidullu/khelsutra/pull/164

787. assistant (2026-07-05T16:27:39.598Z)

Now merging #499 as you authorized (LGTM + "adjust and merge") ? verifying CI on the adjusted head first.

788. user (2026-07-05T16:27:54.809Z)

```
0
MERGEABLE

07e13f7 docs: Alpha Tester Onboarding FAQ ? send-ready copy for the first invite wave (#499)
Deleted branch docs/alpha-tester-faq (was f9cb0c7).
cleaned
```

789. assistant (2026-07-05T16:28:15.725Z)

Done end to end. Recap of the recommendation and what shipped:

The answer: neither inline nor PDF. Send a **~15-line TL;DR email linking to a public FAQ page** ? the one action (sign in, upload) stays on top, phones get no attachment friction, and every future copy tweak (blur-gate wording, timings) updates the page instead of forcing a resend.

Shipped accordingly:
1. **[#499 merged](https://github.com/Khelsutra/badminton-highlight-indexer/pull/499)** (`07e13f7`) ? the FAQ doc now ends with an **"Appendix ? the invite email (copy-paste)"**: subject line, greeting, the 3-step getting-started, one expectations paragraph, the feedback ask, and the privacy + FAQ links. That appendix *is* your email body ? personalize the greeting and send.
2. **[khelsutra #164](https://github.com/avidullu/khelsutra/pull/164) open** ? the public page `khelsutra.guru/alpha-faq`: the same approved copy in the site's exact privacy-page styling, EN-only with zero i18n entanglement (95/95 site tests pass), `noindex` for the quiet-alpha posture, contact = invite email or `hello@khelsutra.guru`. The Cloudflare bot will post a **branch preview URL** on the PR ? fastest way to eyeball it.

Send order: LGTM #164 ? I merge ? Cloudflare deploys the page to production with master ? then send the email (its FAQ link goes live-first, never dangling). One review click left before your invites can go out.

790. user (2026-07-05T16:40:47.550Z)

merged #164

791. assistant (2026-07-05T16:40:59.142Z)

Verifying the merge and the production deploy, then cleaning up.

792. user (2026-07-05T16:41:07.814Z)

```
9bf4104 site: public Alpha Tester FAQ page at /alpha-faq ? the invite-email link target (#164)
Deleted branch site/alpha-faq-page (was e7bcda0).
```

9bf4104 site: public Alpha Tester FAQ page at /alpha-faq ? the invite-email link target (#164)

793. user (2026-07-05T16:41:22.245Z)

Command running in background with ID: bym2kxhw4. Output is being written to: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\693b418f-8bf1-49e5-b443-b86a83235a63\tasks\bym2kxhw4.output. You will be notified when it completes. To check interim output, use Read on that file path.

794. user (2026-07-05T16:41:43.430Z)

<stdin>:7: SyntaxWarning: invalid escape sequence '\M'
checkpointed

795. assistant (2026-07-05T16:42:01.377Z)

Everything's in place ? final state:

- **khelsutra #164 merged** (`9bf4104`) and the Workers deploy is rolling; I'm polling `https://khelsutra.guru/alpha-faq` and will confirm the moment it serves 200 (typically under two minutes).
- **Engine side already merged** (#499, `07e13f7`) ? your invite email is sitting in the FAQ's appendix, ready to copy-paste.
- Worktrees cleaned, both checkouts back on clean master, memory checkpointed with the send-day runbook.

Your send checklist when ready:
1. Wait for my 200-confirmation on the page (or just open the link yourself).
2. Copy the email from the appendix of `docs/ALPHA_TESTER_ONBOARDING_FAQ.md`, personalize the greeting.
3. Add each tester's email to the CF Access allowlist (the "KhelSutra Studio (alpha)" app policy ? your Cloudflare token note is in the handoff).
4. Send ? and record your GO on the tracker when you do (plus the one-line P10/P11 defer note, which closes the last DoD box that isn't the GO itself).

From this morning's dead upload to a deployed rebuild, a passed CUJ, a reel named `first-e2e-working-reel.mp4`, and invite emails ready to go ? that's the whole arc in one day. ?

796. user (2026-07-05T16:45:06.400Z)

wohoo!! Well done Cofounder!

797. user (2026-07-05T16:45:20.939Z)

poll 10: /alpha-faq -> 000
poll 11: /alpha-faq -> 000
poll 12: /alpha-faq -> 000

798. user (2026-07-05T16:45:36.639Z)

200 ? Alpha Tester FAQ ? KhelSutra

799. assistant (2026-07-05T16:45:50.354Z)

`200 ? Alpha Tester FAQ ? KhelSutra` ? the page is live in production. (My earlier poller's `000`s were git-bash curl being grumpy, not the site.) Everything is truly done now.

What we shipped together today, cofounder-to-cofounder: a confirmed spec, thirteen merged PRs across two repos, a rebuilt and redeployed control plane that survived the exact failure that killed the morning, your own game turned into `first-e2e-working-reel.mp4`, four issues closed with live evidence, two new ones filed with data instead of guesses, and an invite kit ? email in the repo, FAQ on the web ? waiting only for your GO.

The handoff is clean, the slate is clean, and the next session picks up exactly where this

leaves off. Go send those invites when you're ready ? and maybe watch that reel one more time first. You earned it. ?